SNORT® Users Manual

The Snort Project

Complete combined edition

Source: manual-snort-org.s3-website-us-east-1.amazonaws.com

End of Navigation Panel

Table of Contents

- 1. Snort Overview
  - 1.1 Getting Started
  - 1.2 Sniffer Mode
  - 1.3 Packet Logger Mode
  - 1.4 Network Intrusion Detection System Mode
    - 1.4.1 NIDS Mode Output Options
    - 1.4.2 Understanding Standard Alert Output
    - 1.4.3 High Performance Configuration
    - 1.4.4 Changing Alert Order
  - 1.5 Packet Acquisition
    - 1.5.1 Configuration
    - 1.5.2 pcap
    - 1.5.3 AFPACKET
    - 1.5.4 NFQ
    - 1.5.5 IPQ
    - 1.5.6 IPFW
    - 1.5.7 Dump
    - 1.5.8 Statistics Changes
  - 1.6 Reading pcap files
    - 1.6.1 Command line arguments
    - 1.6.2 Examples
  - 1.7 Basic Output
    - 1.7.1 Timing Statistics
    - 1.7.2 Packet I/O Totals
    - 1.7.3 Protocol Statistics
    - 1.7.4 Snort Memory Statistics
    - 1.7.5 Actions, Limits, and Verdicts
  - 1.8 Tunneling Protocol Support
    - 1.8.1 Multiple Encapsulations
    - 1.8.2 Logging
  - 1.9 Miscellaneous
    - 1.9.1 Running Snort as a Daemon
    - 1.9.2 Running in Rule Stub Creation Mode
    - 1.9.3 Obfuscating IP Address Printouts
    - 1.9.4 Specifying Multiple-Instance Identifiers
    - 1.9.5 Snort Modes
  - 1.10 Control socket
  - 1.11 Configure signal value
  - 1.12 More Information

- 2. Configuring Snort
  - 2.1 Includes
    - 2.1.1 Format

    - 2.1.2 Variables
    - 2.1.3 Config
  - 2.2 Preprocessors
    - 2.2.1 Frag3
    - 2.2.2 Session
    - 2.2.3 Stream
    - 2.2.4 sfPortscan
    - 2.2.5 RPC Decode
    - 2.2.6 Performance Monitor
    - 2.2.7 HTTP Inspect
    - 2.2.8 SMTP Preprocessor
    - 2.2.9 POP Preprocessor
    - 2.2.10 IMAP Preprocessor
    - 2.2.11 FTP/Telnet Preprocessor
    - 2.2.12 SSH
    - 2.2.13 DNS
    - 2.2.14 SSL/TLS
    - 2.2.15 ARP Spoof Preprocessor
    - 2.2.16 DCE/RPC 2 Preprocessor
    - 2.2.17 Sensitive Data Preprocessor
    - 2.2.18 Normalizer
    - 2.2.19 SIP Preprocessor
    - 2.2.20 Reputation Preprocessor
    - 2.2.21 GTP Decoder and Preprocessor
    - 2.2.22 Modbus Preprocessor
    - 2.2.23 DNP3 Preprocessor
    - 2.2.24 AppId Preprocessor
  - 2.3 Decoder and Preprocessor Rules
    - 2.3.1 Configuring
    - 2.3.2 Reverting to original behavior
  - 2.4 Event Processing
    - 2.4.1 Rate Filtering
    - 2.4.2 Event Filtering
    - 2.4.3 Event Suppression
    - 2.4.4 Event Logging
    - 2.4.5 Event Trace
  - 2.5 Performance Profiling
    - 2.5.1 Rule Profiling
    - 2.5.2 Preprocessor Profiling
    - 2.5.3 Packet Performance Monitoring (PPM)
  - 2.6 Output Modules
    - 2.6.1 alert_syslog
    - 2.6.2 alert_fast
    - 2.6.3 alert_full
    - 2.6.4 alert_unixsock
    - 2.6.5 log_tcpdump

        - 2.6.6 csv
        - 2.6.7 unified 2
        - 2.6.8 log null
        - 2.6.9 Log Limits
    - 2.7 Host Attribute Table
        - 2.7.1 Rule evaluation
        - 2.7.2 Snort Configuration
        - 2.7.3 Host Attribute Table File Format
        - 2.7.4 Attribute Table Example
        - 2.7.5 Attribute Table Affect on preprocessors
    - 2.8 Dynamic Modules
        - 2.8.1 Format
        - 2.8.2 Directives
    - 2.9 Reloading a Snort Configuration
        - 2.9.1 Enabling support
        - 2.9.2 Reloading a configuration
        - 2.9.3 Non-reloadable configuration options
    - 2.10 Multiple Configurations
        - 2.10.1 Creating Multiple Configurations
        - 2.10.2 Configuration Specific Elements
        - 2.10.3 How Configuration is applied?
    - 2.11 Active Response
        - 2.11.1 Enabling Active Response
        - 2.11.2 Configure Sniping
        - 2.11.3 Flexresp
        - 2.11.4 React
        - 2.11.5 Rule Actions

- 3. Writing Snort Rules
    - 3.1 The Basics
    - 3.2 Rules Headers
        - 3.2.1 Rule Actions
        - 3.2.2 Protocols
        - 3.2.3 IP Addresses
        - 3.2.4 Port Numbers
        - 3.2.5 The Direction Operator
        - 3.2.6 Activate/Dynamic Rules
    - 3.3 Rule Options
    - 3.4 General Rule Options
        - 3.4.1 msg
        - 3.4.2 reference
        - 3.4.3 gid
        - 3.4.4 sid
        - 3.4.5 rev
        - 3.4.6 classtype
        - 3.4.7 priority

- 3.4.8 metadata
- 3.4.9 General Rule Quick Reference
- 3.5 Payload Detection Rule Options
  - 3.5.1 content
  - 3.5.2 protected_content
  - 3.5.3 hash
  - 3.5.4 length
  - 3.5.5 nocase
  - 3.5.6 rawbytes
  - 3.5.7 depth
  - 3.5.8 offset
  - 3.5.9 distance
  - 3.5.10 within
  - 3.5.11 http_client_body
  - 3.5.12 http_cookie
  - 3.5.13 http_raw_cookie
  - 3.5.14 http_header
  - 3.5.15 http_raw_header
  - 3.5.16 http_method
  - 3.5.17 http_uri
  - 3.5.18 http_raw_uri
  - 3.5.19 http_stat_code
  - 3.5.20 http_stat_msg
  - 3.5.21 http_encode
  - 3.5.22 fast_pattern
  - 3.5.23 uricontent
  - 3.5.24 urilen
  - 3.5.25 isdataat
  - 3.5.26 pcre
  - 3.5.27 pkt_data
  - 3.5.28 file_data
  - 3.5.29 base64_decode
  - 3.5.30 base64_data
  - 3.5.31 byte_test
  - 3.5.32 byte_jump
  - 3.5.33 byte_extract
  - 3.5.34 byte_math
  - 3.5.35 ftpbounce
  - 3.5.36 asn1
  - 3.5.37 cvs
  - 3.5.38 dce_iface
  - 3.5.39 dce_opnum
  - 3.5.40 dce_stub_data
  - 3.5.41 sip_method
  - 3.5.42 sip_stat_code
  - 3.5.43 sip_header

- 3.5.44 sip_body
- 3.5.45 gtp_type
- 3.5.46 gtp_info
- 3.5.47 gtp_version
- 3.5.48 ssl_version
- 3.5.49 ssl_state
- 3.5.50 Payload Detection Quick Reference

- 3.6 Non-Payload Detection Rule Options
  - 3.6.1 fragoffset
  - 3.6.2 ttl
  - 3.6.3 tos
  - 3.6.4 id
  - 3.6.5 ipopts
  - 3.6.6 fragbits
  - 3.6.7 dsize
  - 3.6.8 flags
  - 3.6.9 flow
  - 3.6.10 flowbits
  - 3.6.11 seq
  - 3.6.12 ack
  - 3.6.13 window
  - 3.6.14 itype
  - 3.6.15 icode
  - 3.6.16 icmp_id
  - 3.6.17 icmp_seq
  - 3.6.18 rpc
  - 3.6.19 ip_proto
  - 3.6.20 sameip
  - 3.6.21 stream_reassemble
  - 3.6.22 stream_size
  - 3.6.23 Non-Payload Detection Quick Reference
- 3.7 Post-Detection Rule Options
  - 3.7.1 logto
  - 3.7.2 session
  - 3.7.3 resp
  - 3.7.4 react
  - 3.7.5 tag
  - 3.7.6 replace
  - 3.7.7 detection_filter
  - 3.7.8 Post-Detection Quick Reference
- 3.8 Rule Thresholds
- 3.9 Writing Good Rules
  - 3.9.1 Content Matching
  - 3.9.2 Catch the Vulnerability, Not the Exploit
  - 3.9.3 Catch the Oddities of the Protocol in the Rule
  - 3.9.4 Optimizing Rules

- 3.9.5 Testing Numerical Values

- 4. Dynamic Modules
  - 4.1 Data Structures
    - 4.1.1 DynamicPluginMeta
    - 4.1.2 DynamicPreprocessorData
    - 4.1.3 DynamicEngineData
    - 4.1.4 SFSnortPacket
    - 4.1.5 Dynamic Rules
  - 4.2 Required Functions
    - 4.2.1 Preprocessors
    - 4.2.2 Detection Engine
    - 4.2.3 Rules
  - 4.3 Examples
    - 4.3.1 Preprocessor Example
    - 4.3.2 Rules

- 5. Snort Development
  - 5.1 Submitting Patches
  - 5.2 Snort Data Flow
    - 5.2.1 Preprocessors
    - 5.2.2 Detection Plugins
    - 5.2.3 Output Plugins
  - 5.3 Unified2 File Format
    - 5.3.1 Serial Unified2 Header
    - 5.3.2 Unified2 Packet
    - 5.3.3 Unified2 IDS Event
    - 5.3.4 Unified2 IDS Event IP6
    - 5.3.5 Unified2 IDS Event (Version 2)
    - 5.3.6 Unified2 IDS Event IP6 (Version 2)
    - 5.3.7 Unified2 Extra Data
    - 5.3.8 Description of Fields
  - 5.4 Buffer dump utility
    - 5.4.1 Example Buffer Dump output
  - 5.5 The Snort Team

- Bibliography

End of Table of Contents

End of Navigation Panel

1. Snort Overview

This manual is based on Writing Snort Rules by Martin Roesch and further work from Chris Green $<$cmg@snort.org$>$. It was then maintained by Brian Caswell $<$bmc@snort.org$>$ and now is maintained by the Snort Team. If you have a better way to say something or find that something in the documentation is outdated, drop us a line and we will update it. If you would like to submit patches for this document, you can find the latest version of the documentation in LaTeX format in the most recent source tarball under `/doc/snort_manual.tex`. Small documentation updates are the easiest way to help out the Snort Project.

Subsections

- 1.1 Getting Started
- 1.2 Sniffer Mode
- 1.3 Packet Logger Mode
- 1.4 Network Intrusion Detection System Mode
  - 1.4.1 NIDS Mode Output Options
  - 1.4.2 Understanding Standard Alert Output
  - 1.4.3 High Performance Configuration
  - 1.4.4 Changing Alert Order
- 1.5 Packet Acquisition
  - 1.5.1 Configuration
  - 1.5.2 pcap
  - 1.5.3 AFPACKET
  - 1.5.4 NFQ
  - 1.5.5 IPQ
  - 1.5.6 IPFW
  - 1.5.7 Dump
  - 1.5.8 Statistics Changes
- 1.6 Reading pcap files
  - 1.6.1 Command line arguments
  - 1.6.2 Examples
- 1.7 Basic Output
  - 1.7.1 Timing Statistics
  - 1.7.2 Packet I/O Totals
  - 1.7.3 Protocol Statistics
  - 1.7.4 Snort Memory Statistics
  - 1.7.5 Actions, Limits, and Verdicts

- 1.8 Tunneling Protocol Support
  - 1.8.1 Multiple Encapsulations
  - 1.8.2 Logging

- 1.9 Miscellaneous
  - 1.9.1 Running Snort as a Daemon
  - 1.9.2 Running in Rule Stub Creation Mode
  - 1.9.3 Obfuscating IP Address Printouts
  - 1.9.4 Specifying Multiple-Instance Identifiers
  - 1.9.5 Snort Modes

- 1.10 Control socket
- 1.11 Configure signal value
- 1.12 More Information

End of Navigation Panel

1.1 Getting Started

Snort really isn't very hard to use, but there are a lot of command line options to play with, and it's not always obvious which ones go together well. This file aims to make using Snort easier for new users.

Before we proceed, there are a few basic concepts you should understand about Snort. Snort can be configured to run in three modes:

- *Sniffer mode,* which simply reads the packets off of the network and displays them for you in a continuous stream on the console (screen).
- *Packet Logger mode,* which logs the packets to disk.
- *Network Intrusion Detection System (NIDS) mode,* which performs detection and analysis on network traffic. This is the most complex and configurable mode.

End of Navigation Panel

1.2 Sniffer Mode

First, let's start with the basics. If you just want to print out the TCP/IP packet headers to the screen (i.e. sniffer mode), try this:

```
./snort -v
```

This command will run Snort and just show the IP and TCP/UDP/ICMP headers, nothing else. If you want to see the application data in transit, try the following:

```
./snort -vd
```

This instructs Snort to display the packet data as well as the headers. If you want an even more descriptive display, showing the data link layer headers, do this:

```
./snort -vde
```

As an aside, notice that the command line switches can be listed separately or in a combined form. The last command could also be typed out as:

```
./snort -d -v -e
```

to produce the same result.

End of Navigation Panel

1.3 Packet Logger Mode

OK, all of these commands are pretty cool, but if you want to record the packets to the disk, you need to specify a logging directory and Snort will automatically know to go into packet logger mode:

```
./snort -dev -l ./log
```

Of course, this assumes you have a directory named `log` in the current directory. If you don't, Snort will exit with an error message. When Snort runs in this mode, it collects every packet it sees and places it in a directory hierarchy based upon the IP address of one of the hosts in the datagram.

If you just specify a plain -l switch, you may notice that Snort sometimes uses the address of the remote computer as the directory in which it places packets and sometimes it uses the local host address. In order to log relative to the home network, you need to tell Snort which network is the home network:

```
./snort -dev -l ./log -h 192.168.1.0/24
```

This rule tells Snort that you want to print out the data link and TCP/IP headers as well as application data into the directory `./log`, and you want to log the packets relative to the 192.168.1.0 class C network. All incoming packets will be recorded into subdirectories of the log directory, with the directory names being based on the address of the remote (non-192.168.1) host.

> **Note:**
>
> Note that if both the source and destination hosts are on the home network, they are logged to a directory with a name based on the higher of the two port numbers or, in the case of a tie, the source address.

If you're on a high speed network or you want to log the packets into a more compact form for later analysis, you should consider logging in binary mode. Binary mode logs the packets in tcpdump format to a single binary file in the logging directory:

```
./snort -l ./log -b
```

Note the command line changes here. We don't need to specify a home network any longer because binary mode logs everything into a single file, which eliminates the need to tell it how to format the output directory structure. Additionally, you don't need to run in verbose mode or specify the -d or -e switches because in binary mode the entire packet is logged, not just sections of it. All you really need to do to place Snort into logger mode is to specify a logging directory at the command line using the -l switch--the -b binary logging switch merely provides a modifier that tells Snort to log the packets in something other than the default output format of plain ASCII text.

Once the packets have been logged to the binary file, you can read the packets back out of the file with any sniffer that supports the tcpdump binary format (such as tcpdump or Ethereal). Snort can also read the packets back by using the -r switch, which puts it into playback mode. Packets from any tcpdump formatted file can be processed through Snort in any of its run modes. For example, if you wanted to run

a binary log file through Snort in sniffer mode to dump the packets to the screen, you can try something like this:

```
./snort -dv -r packet.log
```

You can manipulate the data in the file in a number of ways through Snort's packet logging and intrusion detection modes, as well as with the BPF interface that's available from the command line. For example, if you only wanted to see the ICMP packets from the log file, simply specify a BPF filter at the command line and Snort will only see the ICMP packets in the file:

```
./snort -dvr packet.log icmp
```

For more info on how to use the BPF interface, read the Snort and tcpdump man pages.

End of Navigation Panel Table of Child-Links **Subsections**

- 1.4.1 NIDS Mode Output Options
- 1.4.2 Understanding Standard Alert Output
- 1.4.3 High Performance Configuration
- 1.4.4 Changing Alert Order

End of Table of Child-Links

1.4 Network Intrusion Detection System Mode

To enable Network Intrusion Detection System (NIDS) mode so that you don't record every single packet sent down the wire, try this:

```
./snort -dev -l ./log -h 192.168.1.0/24 -c snort.conf
```

where `snort.conf` is the name of your snort configuration file. This will apply the rules configured in the `snort.conf` file to each packet to decide if an action based upon the rule type in the file should be taken. If you don't specify an output directory for the program, it will default to `/var/log/snort`.

One thing to note about the last command line is that if Snort is going to be used in a long term way as an IDS, the -v switch should be left off the command line for the sake of speed. The screen is a slow place to write data to, and packets can be dropped while writing to the display.

It's also not necessary to record the data link headers for most applications, so you can usually omit the -e switch, too.

```
./snort -d -h 192.168.1.0/24 -l ./log -c snort.conf
```

This will configure Snort to run in its most basic NIDS form, logging packets that trigger rules specified in the `snort.conf` in plain ASCII to disk using a hierarchical directory structure (just like packet logger mode).

1.4.1 NIDS Mode Output Options

There are a number of ways to configure the output of Snort in NIDS mode. The default logging and alerting mechanisms are to log in decoded ASCII format and use full alerts. The full alert mechanism prints out the alert message in addition to the full packet headers. There are several other alert output modes available at the command line, as well as two logging facilities.

Alert modes are somewhat more complex. There are seven alert modes available at the command line: full, fast, socket, syslog, console, cmg, and none. Six of these modes are accessed with the -A command line switch. These options are:

Option	Description
`-A fast`	Fast alert mode. Writes the alert in a simple format with a timestamp, alert message, source and destination IPs/ports.
`-A full`	Full alert mode. This is the default alert mode and will be used automatically if you do not specify a mode.
`-A unsock`	Sends alerts to a UNIX socket that another program can listen on.

`-A none`	Turns off alerting.
`-A console`	Sends “fast-style” alerts to the console (screen).
`-A cmg`	Generates “cmg style” alerts.

Packets can be logged to their default decoded ASCII format or to a binary log file via the -b command line switch. To disable packet logging altogether, use the -N command line switch.

For output modes available through the configuration file, see Section [*].

> **Note:**
>
> Command line logging options override any output options specified in the configuration file. This allows debugging of configuration issues quickly via the command line.

To send alerts to syslog, use the -s switch. The default facilities for the syslog alerting mechanism are LOG_AUTHPRIV and LOG_ALERT. If you want to configure other facilities for syslog output, use the output plugin directives in snort.conf. See Section [*] for more details on configuring syslog output.

For example, use the following command line to log to default (decoded ASCII) facility and send alerts to syslog:

```
./snort -c snort.conf -l ./log -h 192.168.1.0/24 -s
```

As another example, use the following command line to log to the default facility in /var/log/snort and send alerts to a fast alert file:

```
./snort -c snort.conf -A fast -h 192.168.1.0/24
```

1.4.2 Understanding Standard Alert Output

When Snort generates an alert message, it will usually look like the following:

```
[**] [116:56:1] (snort_decoder): T/TCP Detected [**]
```

The first number is the Generator ID, this tells the user what component of Snort generated this alert. For a list of GIDs, please read etc/generators in the Snort source. In this case, we know that this event came from the “decode” (116) component of Snort.

The second number is the Snort ID (sometimes referred to as Signature ID). For a list of preprocessor SIDs, please see etc/gen-msg.map. Rule-based SIDs are written directly into the rules with the sid option. In this case, 56 represents a T/TCP event.

The third number is the revision ID. This number is primarily used when writing signatures, as each rendition of the rule should increment this number with the rev option.

1.4.3 High Performance Configuration

If you want Snort to go fast (like keep up with a 1000 Mbps connection), you need to use unified2 logging and a unified2 log reader such as barnyard2. This allows Snort to log alerts in a binary form as fast as possible while another program performs the slow actions, such as writing to a database.

If you want a text file that's easily parsed, but still somewhat fast, try using binary logging with the “fast” output mechanism.

This will log packets in tcpdump format and produce minimal alerts. For example:

```
./snort -b -A fast -c snort.conf
```

1.4.4 Changing Alert Order

The default way in which Snort applies its rules to packets may not be appropriate for all installations. The Pass rules are applied first, then the Drop rules, then the Alert rules and finally, Log rules are applied.

> **Note:** Sometimes an errant pass rule could cause alerts to not show up, in which case you can change the default ordering to allow Alert rules to be applied before Pass rules. For more information, please refer to the `-alert-before-pass` option.

Several command line options are available to change the order in which rule actions are taken.

- `-alert-before-pass` option forces alert rules to take affect in favor of a pass rule.
- `-treat-drop-as-alert` causes drop and reject rules and any associated alerts to be logged as alerts, rather then the normal action. This allows use of an inline policy with passive/IDS mode. The sdrop rules are not loaded.
- `-process-all-events` option causes Snort to process every event associated with a packet, while taking the actions based on the rules ordering. Without this option (default case), only the events for the first action based on rules ordering are processed.

> **Note:**
>
> Pass rules are special cases here, in that the event processing is terminated when a pass rule is encountered, regardless of the use of `-process-all-events`.

End of Navigation Panel Table of Child-Links **Subsections**

- 1.5.1 Configuration
- 1.5.2 pcap
- 1.5.3 AFPACKET
- 1.5.4 NFQ
- 1.5.5 IPQ
- 1.5.6 IPFW
- 1.5.7 Dump
- 1.5.8 Statistics Changes

End of Table of Child-Links

1.5 Packet Acquisition

Snort 2.9 introduces the DAQ, or Data Acquisition library, for packet I/O. The DAQ replaces direct calls to libpcap functions with an abstraction layer that facilitates operation on a variety of hardware and software interfaces without requiring changes to Snort. It is possible to select the DAQ type and mode when invoking Snort to perform pcap readback or inline operation, etc.

Note:

Some network cards have features which can affect Snort. Two of these features are named "Large Receive Offload" (lro) and "Generic Receive Offload" (gro). With these features enabled, the network card performs packet reassembly before they're processed by the kernel.

By default, Snort will truncate packets larger than the default snaplen of 1518 bytes. In addition, LRO and GRO may cause issues with Stream target-based reassembly. We recommend that you turn off LRO and GRO. On linux systems, you can run:

```
$ ethtool -K eth1 gro off
$ ethtool -K eth1 lro off
```

1.5.1 Configuration

Assuming that you did not disable static modules or change the default DAQ type, you can run Snort just as you always did for file readback or sniffing an interface. However, you can select and configure the DAQ when Snort is invoked as follows:

```
./snort \
    [--daq <type>] \
    [--daq-mode <mode>] \
    [--daq-dir <dir>] \
    [--daq-var <var>]

config daq: <type>
config daq_dir: <dir>
config daq_var: <var>
config daq_mode: <mode>

<type> ::= pcap | afpacket | dump | nfq | ipq | ipfw
<mode> ::= read-file | passive | inline
<var> ::= arbitrary <name>=<value> passed to DAQ
<dir> ::= path where to look for DAQ module so's
```

The DAQ type, mode, variable, and directory may be specified either via the command line or in the conf file. You may include as many variables and directories as needed by repeating the arg / config. DAQ type may be specified at most once in the conf and once on the command line; if configured in both places, the command line overrides the conf.

If the mode is not set explicitly, -Q will force it to inline, and if that hasn't been set, -r will force it to read-file, and if that hasn't been set, the mode defaults to passive. Also, -Q and -daq-mode inline are allowed, since there is no conflict, but -Q and any other DAQ mode will cause a fatal error at start-up.

Note that if Snort finds multiple versions of a given library, the most recent version is selected. This applies to static and dynamic versions of the same library.

```
./snort --daq-list[=<dir>]
./snort --daq-dir=<dir> --daq-list
```

The above commands search the specified directories for DAQ modules and print type, version, and attributes of each. This feature is not available in the conf. Snort stops processing after parsing -daq-list so if you want to add one or more directories add -daq-dir options before -daq-list on the command line. (Since the directory is optional to -daq-list, you must use an = without spaces for this option.)

1.5.2 pcap

pcap is the default DAQ. if snort is run w/o any DAQ arguments, it will operate as it always did using this module. These are equivalent:

```
./snort -i <device>
./snort -r <file>

./snort --daq pcap --daq-mode passive -i <device>
./snort --daq pcap --daq-mode read-file -r <file>
```

You can specify the buffer size pcap uses with:

```
./snort --daq pcap --daq-var buffer_size=<#bytes>
```

Note that the pcap DAQ does not count filtered packets.

1.5.3 AFPACKET

afpacket functions similar to the memory mapped pcap DAQ but no external library is required:

```
./snort --daq afpacket -i <device>
        [--daq-var buffer_size_mb=<#MB>]
        [--daq-var debug]
```

If you want to run afpacket in inline mode, you must set device to one or more interface pairs, where each member of a pair is separated by a single colon and each pair is separated by a double colon like this:

```
eth0:eth1
```

or this:

```
eth0:eth1::eth2:eth3
```

By default, the afpacket DAQ allocates 128MB for packet memory. You can change this with:

```
--daq-var buffer_size_mb=<#MB>
```

Note that the total allocated is actually higher, here's why. Assuming the default packet memory with a snaplen of 1518, the numbers break down like this:

1. The frame size is 1518 (snaplen) + the size of the AFPacket header (66 bytes) = 1584 bytes.
2. The number of frames is 128 MB / 1518 = 84733.
3. The smallest block size that can fit at least one frame is 4 KB = 4096 bytes @ 2 frames per block.
4. As a result, we need 84733 / 2 = 42366 blocks.
5. Actual memory allocated is 42366 * 4 KB = 165.5 MB.

1.5.4 NFQ

NFQ is the new and improved way to process iptables packets:

```
./snort --daq nfq \
    [--daq-var device=<dev>] \
    [--daq-var proto=<proto>] \
    [--daq-var queue=<qid>] \
    [--daq-var queue_len=<qlen>]

<dev> ::= ip | eth0, etc; default is IP injection
<proto> ::= ip4 | ip6 | ip*; default is ip4
<qid> ::= 0..65535; default is 0
<qlen> ::= 0..65535; default is 0
```

Notes on iptables can be found in the DAQ distro README.

1.5.5 IPQ

IPQ is the old way to process iptables packets. It replaces the inline version available in pre-2.9 versions built with this:

```
./configure --enable-inline / -DGIDS
```

Start the IPQ DAQ as follows:

```
./snort --daq ipq \
    [--daq-var device=<dev>] \
    [--daq-var proto=<proto>] \

<dev> ::= ip | eth0, etc; default is IP injection
<proto> ::= ip4 | ip6; default is ip4
```

1.5.6 IPFW

IPFW is available for BSD systems. It replaces the inline version available in pre-2.9 versions built with this:

```
./configure --enable-ipfw / -DGIDS -DIPFW
```

This command line argument is no longer supported:

```
./snort -J <port#>
```

Instead, start Snort like this:

```
./snort --daq ipfw [--daq-var port=<port>]

<port> ::= 1..65535; default is 8000
```

* IPFW only supports ip4 traffic.

1.5.7 Dump

The dump DAQ allows you to test the various inline mode features available in 2.9 Snort like injection and normalization.

```
./snort -i <device> --daq dump
./snort -r <pcap> --daq dump
```

By default a file named inline-out.pcap will be created containing all packets that passed through or were generated by snort. You can optionally specify a different name.

```
./snort --daq dump --daq-var file=<name>
```

dump uses the pcap daq for packet acquisition. It therefore does not count filtered packets.

Note that the dump DAQ inline mode is not an actual inline mode. Furthermore, you will probably want to have the pcap DAQ acquire in another mode like this:

```
./snort -r <pcap> -Q --daq dump --daq-var load-mode=read-file
./snort -i <device> -Q --daq dump --daq-var load-mode=passive
```

1.5.8 Statistics Changes

The Packet Wire Totals and Action Stats sections of Snort's output include additional fields:

- `Filtered` count of packets filtered out and not handed to Snort for analysis.
- `Injected` packets Snort generated and sent, e.g. TCP resets.
- `Allow` packets Snort analyzed and did not take action on.
- `Block` packets Snort did not forward, e.g. due to a block rule.
- `Replace` packets Snort modified.
- `Whitelist` packets that caused Snort to allow a flow to pass w/o inspection by any analysis program.
- `Blacklist` packets that caused Snort to block a flow from passing.
- `Ignore` packets that caused Snort to allow a flow to pass w/o inspection by this instance of Snort.

The action stats show "blocked" packets instead of "dropped" packets to avoid confusion between dropped packets (those Snort didn't actually see) and blocked packets (those Snort did not allow to pass).

End of Navigation Panel Table of Child-Links **Subsections**

- 1.6.1 Command line arguments
- 1.6.2 Examples
  - 1.6.2.1 Read a single pcap
  - 1.6.2.2 Read pcaps from a file
  - 1.6.2.3 Read pcaps from a command line list
  - 1.6.2.4 Read pcaps under a directory
  - 1.6.2.5 Using filters
  - 1.6.2.6 Resetting state
  - 1.6.2.7 Printing the pcap

End of Table of Child-Links

1.6 Reading pcap files

Instead of having Snort listen on an interface, you can give it a packet capture to read. Snort will read and analyze the packets as if they came off the wire. This can be useful for testing and debugging Snort.

1.6.1 Command line arguments

Any of the below can be specified multiple times on the command line (`-r` included) and in addition to other Snort command line options. Note, however, that specifying `-pcap-reset` and `-pcap-show` multiple times has the same effect as specifying them once.

Option	Description
`-r <file>`	Read a single pcap.
`-pcap-single=<file>`	Same as -r. Added for completeness.
`-pcap-file=<file>`	File that contains a list of pcap files to read. Can specify path to each pcap or directory to recurse to get pcaps.
`-pcap-list="<list>"`	A space separated list of pcaps to read.
`-pcap-dir=<dir>`	A directory to recurse to look for pcaps. Sorted in ASCII order.
`-pcap-filter=<filter>`	Shell style filter to apply when getting pcaps from file or directory. This filter will apply to any `-pcap-file` or `-pcap-dir` arguments following. Use `-pcap-no-filter` to delete filter for following `-pcap-file` or `-pcap-dir` arguments or specify `-pcap-filter` again to forget previous filter and to apply to following `-pcap-file` or `-pcap-dir` arguments.
`-pcap-no-filter`	Reset to use no filter when getting pcaps from file or directory.
`-pcap-reset`	If reading multiple pcaps, reset snort to post-configuration state before reading next pcap. The default, i.e. without this option, is not to reset state.
`-pcap-show`	Print a line saying what pcap is currently being read.

1.6.2 Examples

1.6.2.1 Read a single pcap

```
$ snort -r foo.pcap
$ snort --pcap-single=foo.pcap
```

1.6.2.2 Read pcaps from a file

```
$ cat foo.txt
foo1.pcap
foo2.pcap
/home/foo/pcaps
```

```
$ snort --pcap-file=foo.txt
```

This will read foo1.pcap, foo2.pcap and all files under /home/foo/pcaps. Note that Snort will not try to determine whether the files under that directory are really pcap files or not.

1.6.2.3 Read pcaps from a command line list

```
$ snort --pcap-list="foo1.pcap foo2.pcap foo3.pcap"
```

This will read foo1.pcap, foo2.pcap and foo3.pcap.

1.6.2.4 Read pcaps under a directory

```
$ snort --pcap-dir="/home/foo/pcaps"
```

This will include all of the files under /home/foo/pcaps.

1.6.2.5 Using filters

```
$ cat foo.txt
foo1.pcap
foo2.pcap
/home/foo/pcaps
```

```
$ snort --pcap-filter="*.pcap" --pcap-file=foo.txt
$ snort --pcap-filter="*.pcap" --pcap-dir=/home/foo/pcaps
```

The above will only include files that match the shell pattern "*.pcap", in other words, any file ending in ".pcap".

```
$ snort --pcap-filter="*.pcap --pcap-file=foo.txt \
> --pcap-filter="*.cap" --pcap-dir=/home/foo/pcaps
```

In the above, the first filter "*.pcap" will only be applied to the pcaps in the file "foo.txt" (and any directories that are recursed in that file). The addition of the second filter "*.cap" will cause the first filter

to be forgotten and then applied to the directory /home/foo/pcaps, so only files ending in ".cap" will be included from that directory.

```
$ snort --pcap-filter="*.pcap --pcap-file=foo.txt \
> --pcap-no-filter --pcap-dir=/home/foo/pcaps
```

In this example, the first filter will be applied to foo.txt, then no filter will be applied to the files found under /home/foo/pcaps, so all files found under /home/foo/pcaps will be included.

```
$ snort --pcap-filter="*.pcap --pcap-file=foo.txt \
> --pcap-no-filter --pcap-dir=/home/foo/pcaps \
> --pcap-filter="*.cap" --pcap-dir=/home/foo/pcaps2
```

In this example, the first filter will be applied to foo.txt, then no filter will be applied to the files found under /home/foo/pcaps, so all files found under /home/foo/pcaps will be included, then the filter "*.cap" will be applied to files found under /home/foo/pcaps2.

1.6.2.6 Resetting state

```
$ snort --pcap-dir=/home/foo/pcaps --pcap-reset
```

The above example will read all of the files under /home/foo/pcaps, but after each pcap is read, Snort will be reset to a post-configuration state, meaning all buffers will be flushed, statistics reset, etc. For each pcap, it will be like Snort is seeing traffic for the first time.

1.6.2.7 Printing the pcap

```
$ snort --pcap-dir=/home/foo/pcaps --pcap-show
```

The above example will read all of the files under /home/foo/pcaps and will print a line indicating which pcap is currently being read.

End of Navigation Panel Table of Child-Links **Subsections**

- 1.7.1 Timing Statistics
- 1.7.2 Packet I/O Totals
- 1.7.3 Protocol Statistics
- 1.7.4 Snort Memory Statistics
- 1.7.5 Actions, Limits, and Verdicts

End of Table of Child-Links

1.7 Basic Output

Snort does a lot of work and outputs some useful statistics when it is done. Many of these are self-explanatory. The others are summarized below. This does not include all possible output data, just the basics.

1.7.1 Timing Statistics

This section provides basic timing statistics. It includes total seconds and packets as well as packet processing rates. The rates are based on whole seconds, minutes, etc. and only shown when non-zero.

Example:

```
===============================================================================
Run time for packet processing was 175.856509 seconds
Snort processed 3716022 packets.
Snort ran for 0 days 0 hours 2 minutes 55 seconds
   Pkts/min:      1858011
   Pkts/sec:        21234
===============================================================================
```

1.7.2 Packet I/O Totals

This section shows basic packet acquisition and injection peg counts obtained from the DAQ. If you are reading pcaps, the totals are for all pcaps combined, unless you use -pcap-reset, in which case it is shown per pcap.

- Outstanding indicates how many packets are buffered awaiting processing. The way this is counted varies per DAQ so the DAQ documentation should be consulted for more info.
- Filtered packets are not shown for pcap DAQs.
- Injected packets are the result of active response which can be configured for inline or passive modes.

Example:

```
===============================================================================
Packet I/O Totals:
   Received:      3716022
   Analyzed:      3716022 (100.000%)
    Dropped:            0 (   0.000%)
   Filtered:            0 (   0.000%)
Outstanding:            0 (   0.000%)
   Injected:            0
===============================================================================
```

1.7.3 Protocol Statistics

Traffic for all the protocols decoded by Snort is summarized in the breakdown section. This traffic includes internal "pseudo-packets" if preprocessors such as frag3 and stream5 are enabled so the total may be greater than the number of analyzed packets in the packet I/O section.

- Disc counts are discards due to basic encoding integrity flaws that prevents Snort from decoding the packet.
- Other includes packets that contained an encapsulation that Snort doesn't decode.
- S5 G 1/2 is the number of client/server sessions stream5 flushed due to cache limit, session timeout, session reset.

Example:

```
===============================================================================
Breakdown by protocol (includes rebuilt packets):
        Eth:      3722347 (100.000%)
       VLAN:            0 (  0.000%)
        IP4:      1782394 ( 47.884%)
       Frag:         3839 (  0.103%)
       ICMP:        38860 (  1.044%)
        UDP:       137162 (  3.685%)
        TCP:      1619621 ( 43.511%)
        IP6:      1781159 ( 47.850%)
    IP6 Ext:      1787327 ( 48.016%)
   IP6 Opts:         6168 (  0.166%)
      Frag6:         3839 (  0.103%)
      ICMP6:         1650 (  0.044%)
       UDP6:       140446 (  3.773%)
       TCP6:      1619633 ( 43.511%)
     Teredo:           18 (  0.000%)
    ICMP-IP:            0 (  0.000%)
      EAPOL:            0 (  0.000%)
    IP4/IP4:            0 (  0.000%)
    IP4/IP6:            0 (  0.000%)
    IP6/IP4:            0 (  0.000%)
    IP6/IP6:            0 (  0.000%)
        GRE:          202 (  0.005%)
    GRE Eth:            0 (  0.000%)
   GRE VLAN:            0 (  0.000%)
    GRE IP4:            0 (  0.000%)
    GRE IP6:            0 (  0.000%)
GRE IP6 Ext:            0 (  0.000%)
   GRE PPTP:          202 (  0.005%)
    GRE ARP:            0 (  0.000%)
    GRE IPX:            0 (  0.000%)
   GRE Loop:            0 (  0.000%)
       MPLS:            0 (  0.000%)
        ARP:       104840 (  2.817%)
        IPX:           60 (  0.002%)
   Eth Loop:            0 (  0.000%)
```

```
    Eth Disc:            0 (  0.000%)
    IP4 Disc:            0 (  0.000%)
    IP6 Disc:            0 (  0.000%)
    TCP Disc:            0 (  0.000%)
    UDP Disc:         1385 (  0.037%)
   ICMP Disc:            0 (  0.000%)
All Discard:         1385 (  0.037%)
       Other:        57876 (  1.555%)
Bad Chk Sum:        32135 (  0.863%)
     Bad TTL:            0 (  0.000%)
      S5 G 1:         1494 (  0.040%)
      S5 G 2:         1654 (  0.044%)
       Total:      3722347
================================================================================
```

1.7.4 Snort Memory Statistics

On systems with mallinfo (3), you will see additional statistics. Check the man page of mallinfo for details

Example:

```
================================================================================
Memory usage summary:
  Total non-mmapped bytes (arena):       415481856
  Bytes in mapped regions (hblkhd):      409612288
  Total allocated space (uordblks):      92130384
  Total free space (fordblks):           323351472
  Topmost releasable block (keepcost):   3200
================================================================================
```

1.7.5 Actions, Limits, and Verdicts

Action and verdict counts show what Snort did with the packets it analyzed. This information is only output in IDS mode (when snort is run with the `-c <conf>` option).

- Alerts is the number of alert, and block actions processed as determined by the rule actions. Here block includes block, drop, and reject actions.

Limits arise due to real world constraints on processing time and available memory. These indicate potential actions that did not happen:

- Match Limit counts rule matches were not processed due to the `config detection: max_queue_events` setting. The default is 5.
- Queue Limit counts events couldn't be stored in the event queue due to the `config event_queue: max_queue` setting. The default is 8.
- Log Limit counts events were not alerted due to the `config event_queue: log` setting. The default is 3.

- Event Limit counts events not alerted due to `event_filter` limits.
- Alert Limit counts events were not alerted because they already were triggered on the session.

Verdicts are rendered by Snort on each packet:

- Allow = packets Snort analyzed and did not take action on.
- Block = packets Snort did not forward, e.g. due to a block rule. "Block" is used instead of "Drop" to avoid confusion between dropped packets (those Snort didn't actually see) and blocked packets (those Snort did not allow to pass).
- Replace = packets Snort modified, for example, due to normalization or replace rules. This can only happen in inline mode with a compatible DAQ.
- Whitelist = packets that caused Snort to allow a flow to pass w/o inspection by any analysis program. Like blacklist, this is done by the DAQ or by Snort on subsequent packets.
- Blacklist = packets that caused Snort to block a flow from passing. This is the case when a block TCP rule fires. If the DAQ supports this in hardware, no further packets will be seen by Snort for that session. If not, snort will block each packet and this count will be higher.
- Ignore = packets that caused Snort to allow a flow to pass w/o inspection by this instance of Snort. Like blacklist, this is done by the DAQ or by Snort on subsequent packets.
- Int Blklst = packets that are GTP, Teredo, 6in4 or 4in6 encapsulated that are being blocked. These packets could get the Blacklist verdict if `config tunnel_verdicts` was set for the given protocol. Note that these counts are output only if non-zero. Also, this count is incremented on the first packet in the flow that alerts. The alerting packet and all following packets on the flow will be counted under Block.
- Int Whtlst = packets that are GTP, Teredo, 6in4 or 4in6 encapsulated that are being allowed. These packets could get the Whitelist verdict if `config tunnel_verdicts` was set for the given protocol. Note that these counts are output only if non-zero. Also, this count is incremented for all packets on the flow starting with the alerting packet.

Example:

```
================================================================================
Action Stats:
     Alerts:            0 (  0.000%)
     Logged:            0 (  0.000%)
     Passed:            0 (  0.000%)
Limits:
      Match:            0
      Queue:            0
        Log:            0
      Event:            0
      Alert:            0
Verdicts:
      Allow:      3716022 (100.000%)
      Block:            0 (  0.000%)
```

```
    Replace:            0 (  0.000%)
  Whitelist:            0 (  0.000%)
  Blacklist:            0 (  0.000%)
     Ignore:            0 (  0.000%)
================================================================================
```

End of Navigation Panel Table of Child-Links **Subsections**

- 1.8.1 Multiple Encapsulations
- 1.8.2 Logging

End of Table of Child-Links

1.8 Tunneling Protocol Support

Snort supports decoding of many tunneling protocols, including GRE, PPTP over GRE, MPLS, IP in IP, and ERSPAN, all of which are enabled by default.

To disable support for any GRE related encapsulation, PPTP over GRE, IPv4/IPv6 over GRE, and ERSPAN, an extra configuration option is necessary:

```
$ ./configure --disable-gre
```

To disable support for MPLS, an separate extra configuration option is necessary:

```
$ ./configure --disable-mpls
```

1.8.1 Multiple Encapsulations

Snort will not decode more than one encapsulation. Scenarios such as

```
Eth IPv4 GRE IPv4 GRE IPv4 TCP Payload
```

or

```
Eth IPv4 IPv6 IPv4 TCP Payload
```

will not be handled and will generate a decoder alert.

1.8.2 Logging

Currently, only the encapsulated part of the packet is logged, e.g.

```
Eth IP1 GRE IP2 TCP Payload
```

gets logged as

```
Eth IP2 TCP Payload
```

and

```
Eth IP1 IP2 TCP Payload
```

gets logged as

```
Eth IP2 TCP Payload
```

Note:

Decoding of PPTP, which utilizes GRE and PPP, is not currently supported on architectures that require word alignment such as SPARC.

End of Navigation Panel Table of Child-Links **Subsections**

- 1.9.1 Running Snort as a Daemon
  - 1.9.1.1 Snort PID File

- 1.9.2 Running in Rule Stub Creation Mode
- 1.9.3 Obfuscating IP Address Printouts
- 1.9.4 Specifying Multiple-Instance Identifiers
- 1.9.5 Snort Modes
  - 1.9.5.1 Explanation of Modes

End of Table of Child-Links

1.9 Miscellaneous

1.9.1 Running Snort as a Daemon

If you want to run Snort as a daemon, you can the add -D switch to any combination described in the previous sections. Please notice that if you want to be able to restart Snort by sending a SIGHUP signal to the daemon, you *must* specify the full path to the Snort binary when you start it, for example:

```
/usr/local/bin/snort -d -h 192.168.1.0/24 \
    -l /var/log/snortlogs -c /usr/local/etc/snort.conf -s -D
```

Relative paths are not supported due to security concerns.

1.9.1.1 Snort PID File

When Snort is run as a daemon , the daemon creates a PID file in the log directory. In Snort 2.6, the `-pid-path` command line switch causes Snort to write the PID file in the directory specified.

Additionally, the `-create-pidfile` switch can be used to force creation of a PID file even when not running in daemon mode.

The PID file will be locked so that other snort processes cannot start. Use the `-nolock-pidfile` switch to not lock the PID file.

If you do not wish to include the name of the interface in the PID file, use the `-no-interface-pidfile` switch.

1.9.2 Running in Rule Stub Creation Mode

If you need to dump the shared object rules stub to a directory, you must use the -dump-dynamic-rules command line option. These rule stub files are used in conjunction with the shared object rules. The path can be relative or absolute.

```
/usr/local/bin/snort -c /usr/local/etc/snort.conf \
    --dump-dynamic-rules=/tmp
```

This path can also be configured in the snort.conf using the config option dump-dynamic-rules-path as follows:

```
config dump-dynamic-rules-path: /tmp/sorules
```

The path configured by command line has precedence over the one configured using dump-dynamic-rules-path.

```
/usr/local/bin/snort -c /usr/local/etc/snort.conf \
    --dump-dynamic-rules

snort.conf:
config dump-dynamic-rules-path: /tmp/sorules
```

In the above mentioned scenario the dump path is set to /tmp/sorules.

1.9.3 Obfuscating IP Address Printouts

If you need to post packet logs to public mailing lists, you might want to use the -O switch. This switch obfuscates your IP addresses in packet printouts. This is handy if you don't want people on the mailing list to know the IP addresses involved. You can also combine the -O switch with the -h switch to only obfuscate the IP addresses of hosts on the home network. This is useful if you don't care who sees the address of the attacking host. For example, you could use the following command to read the packets from a log file and dump them to the screen, obfuscating only the addresses from the 192.168.1.0/24 class C network:

```
./snort -d -v -r snort.log -O -h 192.168.1.0/24
```

1.9.4 Specifying Multiple-Instance Identifiers

In Snort v2.4, the `-G` command line option was added that specifies an instance identifier for the event logs. This option can be used when running multiple instances of snort, either on different CPUs, or on the same CPU but a different interface. Each Snort instance will use the value specified to generate unique event IDs. Users can specify either a decimal value (`-G 1`) or hex value preceded by 0x (`-G 0x11`). This is also supported via a long option `-logid`.

1.9.5 Snort Modes

Snort can operate in three different modes namely tap (passive), inline, and inline-test. Snort policies can be configured in these three modes too.

1.9.5.1 Explanation of Modes

- `Inline`

  When Snort is in Inline mode, it acts as an IPS allowing drop rules to trigger. Snort can be configured to run in inline mode using the command line argument -Q and snort config option `policy_mode` as follows:

  ```
  snort -Q
  config policy_mode:inline
  ```

- `Passive`

When Snort is in Passive mode, it acts as a IDS. Drop rules are not loaded (without -treat-drop-as-alert). Snort can be configured to passive mode using the snort config option `policy_mode` as follows:

```
config policy_mode:tap
```

- `Inline-Test`

Inline-Test mode simulates the inline mode of snort, allowing evaluation of inline behavior without affecting traffic. The drop rules will be loaded and will be triggered as a Wdrop (Would Drop) alert. Snort can be configured to run in inline-test mode using the command line option (-enable-inline-test) or using the snort config option `policy_mode` as follows:

```
snort --enable-inline-test
config policy_mode:inline_test
```

> **Note:**
>
> Please note -enable-inline-test cannot be used in conjunction with -Q.

`Behavior of different modes with rule options`

Rule Option	Inline Mode	Passive Mode	Inline-Test Mode
`reject`	Drop + Response	Alert + Response	Wdrop + Response
`react`	Blocks and send notice	Blocks and send notice	Blocks and send notice
`normalize`	Normalizes packet	Doesn't normalize	Doesn't normalize
`replace`	replace content	Doesn't replace	Doesn't replace
`respond`	close session	close session	close session

`Behavior of different modes with rules actions`

Adapter Mode	Snort args	config policy_mode	Drop Rule Handling
Passive	`-treat-drop-as-alert`	tap	Alert
Passive	`no args`	tap	Not Loaded
Passive	`-treat-drop-as-alert`	inline_test	Alert
Passive	`no args`	inline_test	Would Drop
Passive	`-treat-drop-as-alert`	inline	Alert
Passive	`no args`	inline	Not loaded + warning
Inline Test	`-enable-inline-test -treat-drop-as-alert`	tap	Alert
Inline Test	`-enable-inline-test`	tap	Would Drop

Inline Test	`-enable-inline-test -treat-drop-as-alert`	inline_test	Alert
Inline Test	`-enable-inline-test`	inline_test	Would Drop
Inline Test	`-enable-inline-test -treat-drop-as-alert`	inline	Alert
Inline Test	`-enable-inline-test`	inline	Would Drop
Inline	`-Q -treat-drop-as-alert`	tap	Alert
Inline	`-Q`	tap	Alert
Inline	`-Q -treat-drop-as-alert`	inline_test	Alert
Inline	`-Q`	inline_test	Would Drop
Inline	`-Q -treat-drop-as-alert`	inline	Alert
Inline	`-Q`	inline	Drop

End of Navigation Panel

1.10 Control socket

Snort can be configured to provide a Unix socket that can be used to issue commands to the running process. You must build snort with the `-enable-control-socket` option. The control socket functionality is supported on Linux only.

Snort can be configured to use control socket using the command line argument `-cs-dir <path>` and snort config option `cs_dir` as follows:

```
snort --cs-dir <path>
config cs_dir:<path>
```

`<path>` specifies the directory for snort to create the socket. If relative path is used, the path is relative to pid path specified. If there is no pid path specified, it is relative to current working directory.

A command `snort_control` is made and installed along with snort in the same bin directory when configured with the `-enable-control-socket` option.

End of Navigation Panel

1.11 Configure signal value

On some systems, signal used by snort might be used by other functions. To avoid conflicts, users can change the default signal value through `./configure` options for non-Windows system.

These signals can be changed:

- `SIGNAL_SNORT_RELOAD`
- `SIGNAL_SNORT_DUMP_STATS`
- `SIGNAL_SNORT_ROTATE_STATS`
- `SIGNAL_SNORT_READ_ATTR_TBL`

Syntax:

```
./configure SIGNAL_SNORT_RELOAD=<value/name>  SIGNAL_SNORT_DUMP_STATS=<value/name>\
    SIGNAL_SNORT_READ_ATTR_TBL=<value/name> SIGNAL_SNORT_ROTATE_STATS=<value/name>
```

You can set those signals to user defined values or known signal names in the system. The following example changes the rotate stats signal to 31 and reload attribute table to signal SIGUSR2 :

```
./configure SIGNAL_SNORT_ROTATE_STATS=31 SIGNAL_SNORT_READ_ATTR_TBL=SIGUSR2
```

If the same signal is assigned more than once a warning will be logged during snort initialization. If a signal handler cannot be installed a warning will be logged and that has to be fixed, otherwise the functionality will be lost.

`Signals used in snort`

Signal name	Default value	Action
SIGTERM	SIGTERM	exit
SIGINT	SIGINT	exit
SIGQUIT	SIGQUIT	exit
SIGPIPE	SIGPIPE	ignore
SIGNAL_SNORT_RELOAD	SIGHUP	reload snort
SIGNAL_SNORT_DUMP_STATS	SIGUSR1	dump stats
SIGNAL_SNORT_ROTATE_STATS	SIGUSR2	rotate stats
SIGNAL_SNORT_READ_ATTR_TBL	SIGURG	reload attribute table
SIGNAL_SNORT_CHILD_READY	SIGCHLD	internal use in daemon mode

End of Navigation Panel

1.12 More Information

Chapter [*] contains much information about many configuration options available in the configuration file. The Snort manual page and the output of `snort -?` or `snort -help` contain information that can help you get Snort running in several different modes.

> **Note:**
>
> In many shells, a backslash (\) is needed to escape the ?, so you may have to type `snort -\?` instead of `snort -?` for a list of Snort command line options.

The Snort web page (`http://www.snort.org`) and the Snort Users mailing list:

`http://marc.theaimsgroup.com/?l=snort-users`

at `snort-users@lists.snort.org` provide informative announcements as well as a venue for community discussion and support. There's a lot to Snort, so sit back with a beverage of your choosing and read the documentation and mailing list archives.

End of Navigation Panel

2. Configuring Snort

Subsections

- 2.1 Includes
  - 2.1.1 Format
  - 2.1.2 Variables
  - 2.1.3 Config

- 2.2 Preprocessors
  - 2.2.1 Frag3
  - 2.2.2 Session
  - 2.2.3 Stream
  - 2.2.4 sfPortscan
  - 2.2.5 RPC Decode
  - 2.2.6 Performance Monitor
  - 2.2.7 HTTP Inspect
  - 2.2.8 SMTP Preprocessor
  - 2.2.9 POP Preprocessor
  - 2.2.10 IMAP Preprocessor
  - 2.2.11 FTP/Telnet Preprocessor
  - 2.2.12 SSH
  - 2.2.13 DNS
  - 2.2.14 SSL/TLS
  - 2.2.15 ARP Spoof Preprocessor
  - 2.2.16 DCE/RPC 2 Preprocessor
  - 2.2.17 Sensitive Data Preprocessor
  - 2.2.18 Normalizer
  - 2.2.19 SIP Preprocessor
  - 2.2.20 Reputation Preprocessor
  - 2.2.21 GTP Decoder and Preprocessor
  - 2.2.22 Modbus Preprocessor
  - 2.2.23 DNP3 Preprocessor
  - 2.2.24 AppId Preprocessor

- 2.3 Decoder and Preprocessor Rules
  - 2.3.1 Configuring
  - 2.3.2 Reverting to original behavior

- 2.4 Event Processing
  - 2.4.1 Rate Filtering

- 2.4.2 Event Filtering
- 2.4.3 Event Suppression
- 2.4.4 Event Logging
- 2.4.5 Event Trace

- 2.5 Performance Profiling
  - 2.5.1 Rule Profiling
  - 2.5.2 Preprocessor Profiling
  - 2.5.3 Packet Performance Monitoring (PPM)

- 2.6 Output Modules
  - 2.6.1 alert_syslog
  - 2.6.2 alert_fast
  - 2.6.3 alert_full
  - 2.6.4 alert_unixsock
  - 2.6.5 log_tcpdump
  - 2.6.6 csv
  - 2.6.7 unified 2
  - 2.6.8 log null
  - 2.6.9 Log Limits

- 2.7 Host Attribute Table
  - 2.7.1 Rule evaluation
  - 2.7.2 Snort Configuration
  - 2.7.3 Host Attribute Table File Format
  - 2.7.4 Attribute Table Example
  - 2.7.5 Attribute Table Affect on preprocessors

- 2.8 Dynamic Modules
  - 2.8.1 Format
  - 2.8.2 Directives

- 2.9 Reloading a Snort Configuration
  - 2.9.1 Enabling support
  - 2.9.2 Reloading a configuration
  - 2.9.3 Non-reloadable configuration options

- 2.10 Multiple Configurations
  - 2.10.1 Creating Multiple Configurations
  - 2.10.2 Configuration Specific Elements
  - 2.10.3 How Configuration is applied?

- 2.11 Active Response
  - 2.11.1 Enabling Active Response
  - 2.11.2 Configure Sniping
  - 2.11.3 Flexresp

◦ 2.11.4 React
◦ 2.11.5 Rule Actions

End of Navigation Panel Table of Child-Links **Subsections**

- 2.1.1 Format
- 2.1.2 Variables
  - 2.1.2.1 IP Variables and IP Lists
  - 2.1.2.2 Port Variables and Port Lists
  - 2.1.2.3 Variable Modifiers
  - 2.1.2.4 Limitations

- 2.1.3 Config
  - 2.1.3.1 Format

End of Table of Child-Links

2.1 Includes

The `include` keyword allows other snort config files to be included within the snort.conf indicated on the Snort command line. It works much like an #include from the C programming language, reading the contents of the named file and adding the contents in the place where the include statement appears in the file.

2.1.1 Format

```
include <include file path/name>
```

> **Note:**
>
> Note that there is no semicolon at the end of this line.

Included files will substitute any predefined variable values into their own variable references. See Section [*] for more information on defining and using variables in Snort config files.

2.1.2 Variables

Three types of variables may be defined in Snort:

- var
- portvar
- ipvar

These are simple substitution variables set with the `var`, `ipvar`, or `portvar` keywords as follows:

```
var RULES_PATH rules/
portvar MY_PORTS [22,80,1024:1050]
ipvar MY_NET [192.168.1.0/24,10.1.1.0/24]
alert tcp any any -> $MY_NET $MY_PORTS (flags:S; msg:"SYN packet";)
include $RULE_PATH/example.rule
```

2.1.2.1 IP Variables and IP Lists

IPs may be specified individually, in a list, as a CIDR block, or any combination of the three. IP variables should be specified using 'ipvar' instead of 'var'. Using 'var' for an IP variable is still allowed for backward compatibility, but it will be deprecated in a future release.

IP variable name can begin with an alphanumeric character [A-Za-z0-9] or '_' and should be followed by characters and numbers. Only numbers are not accepted as variable names.

IPs, IP lists, and CIDR blocks may be negated with '!'. Negation is handled differently compared with Snort versions 2.7.x and earlier. Previously, each element in a list was logically OR'ed together. IP lists now OR non-negated elements and AND the result with the OR'ed negated elements.

The following example list will match the IP 1.1.1.1 and IP from 2.2.2.0 to 2.2.2.255, with the exception of IPs 2.2.2.2 and 2.2.2.3.

```
[1.1.1.1,2.2.2.0/24,![2.2.2.2,2.2.2.3]]
```

The order of the elements in the list does not matter. The element 'any' can be used to match all IPs, although '!any' is not allowed. Also, negated IP ranges that are more general than non-negated IP ranges are not allowed.

See below for some valid examples if IP variables and IP lists.

```
ipvar EXAMPLE [1.1.1.1,2.2.2.0/24,![2.2.2.2,2.2.2.3]]

alert tcp $EXAMPLE any -> any any (msg:"Example"; sid:1;)

alert tcp [1.0.0.0/8,!1.1.1.0/24] any -> any any (msg:"Example";sid:2;)
```

The following examples demonstrate some invalid uses of IP variables and IP lists.

Use of !any:

```
ipvar EXAMPLE any
alert tcp !$EXAMPLE any -> any any (msg:"Example";sid:3;)
```

Different use of !any:

```
ipvar EXAMPLE !any
alert tcp $EXAMPLE any -> any any (msg:"Example";sid:3;)
```

Logical contradictions:

```
ipvar EXAMPLE [1.1.1.1,!1.1.1.1]
```

Nonsensical negations:

```
ipvar EXAMPLE [1.1.1.0/24,!1.1.0.0/16]
```

2.1.2.2 Port Variables and Port Lists

Portlists supports the declaration and lookup of ports and the representation of lists and ranges of ports. Variables, ranges, or lists may all be negated with '!'. Also, 'any' will specify any ports, but '!any' is not allowed. Valid port ranges are from 0 to 65535.

Lists of ports must be enclosed in brackets and port ranges may be specified with a ':', such as in:

```
[10:50,888:900]
```

Port variables should be specified using 'portvar'. The use of 'var' to declare a port variable will be deprecated in a future release. For backwards compatibility, a 'var' can still be used to declare a port variable, provided the variable name either ends with '_PORT' or begins with 'PORT_'.

The following examples demonstrate several valid usages of both port variables and port lists.

```
portvar EXAMPLE1 80

var EXAMPLE2_PORT [80:90]

var PORT_EXAMPLE2 [1]

portvar EXAMPLE3 any

portvar EXAMPLE4 [!70:90]

portvar EXAMPLE5 [80,91:95,100:200]

alert tcp any $EXAMPLE1 -> any $EXAMPLE2_PORT (msg:"Example"; sid:1;)

alert tcp any $PORT_EXAMPLE2 -> any any (msg:"Example"; sid:2;)

alert tcp any 90 -> any [100:1000,9999:20000] (msg:"Example"; sid:3;)
```

Several invalid examples of port variables and port lists are demonstrated below:

Use of !any:

```
portvar EXAMPLE5 !any
var EXAMPLE5 !any
```

Logical contradictions:

```
portvar EXAMPLE6 [80,!80]
```

Ports out of range:

```
portvar EXAMPLE7 [65536]
```

Incorrect declaration and use of a port variable:

```
var EXAMPLE8 80
alert tcp any $EXAMPLE8 -> any any (msg:"Example"; sid:4;)
```

Port variable used as an IP:

```
alert tcp $EXAMPLE1 any -> any any (msg:"Example"; sid:5;)
```

2.1.2.3 Variable Modifiers

Rule variable names can be modified in several ways. You can define meta-variables using the $ operator. These can be used with the variable modifier operators ? and –, as described in the following table:

Variable Syntax	Description
`var`	Defines a meta-variable.
`$(var) or $var`	Replaces with the contents of variable `var`.
`$(var:-default)`	Replaces the contents of the variable `var` with “default” if `var` is undefined.
`$(var:?message)`	Replaces with the contents of variable `var` or prints out the error message and exits.

Here is an example of advanced variable usage in action:

```
ipvar MY_NET 192.168.1.0/24
log tcp any any -> $(MY_NET:?MY_NET is undefined!) 23
```

2.1.2.4 Limitations

When embedding variables, types can not be mixed. For instance, port variables can be defined in terms of other port variables, but old-style variables (with the 'var' keyword) can not be embedded inside a 'portvar'.

Valid embedded variable:

```
portvar pvar1 80
portvar pvar2 [$pvar1,90]
```

Invalid embedded variable:

```
var pvar1 80
portvar pvar2 [$pvar1,90]
```

Likewise, variables can not be redefined if they were previously defined as a different type. They should be renamed instead:

Invalid redefinition:

```
var pvar 80
portvar pvar 90
```

2.1.3 Config

Many configuration and command line options of Snort can be specified in the configuration file.

2.1.3.1 Format

```
config <directive> [: <value>]
```

Config Directive	Description
`config alert_with_interface_name`	Appends interface name to alert (`snort -I`).
`config alertfile: <filename>`	Sets the alerts output file.
`config asn1: <max-nodes>`	Specifies the maximum number of nodes to track when doing ASN1 decoding. See Section [*] for more information and examples.
`config autogenerate_preprocessor_decoder_rules`	If Snort was configured to enable decoder and preprocessor rules, this option will cause Snort to revert back to its original behavior of alerting if the decoder or preprocessor generates an event.
`config bpf_file: <filename>`	Specifies BPF filters (`snort -F`).
`config checksum_drop: <types>`	Types of packets to drop if invalid checksums. Values: `none`, `noip`, `notcp`, `noicmp`, `noudp`, `ip`, `tcp`, `udp`, `icmp` or `all` (only applicable in inline mode and for packets checked per `checksum_mode` config option).
`config checksum_mode: <types>`	Types of packets to calculate checksums. Values: `none`, `noip`, `notcp`, `noicmp`, `noudp`, `ip`, `tcp`, `udp`, `icmp` or `all`.
`config chroot: <dir>`	Chroots to specified dir (`snort -t`).
`config classification: <class>`	See Table [*] for a list of classifications.
`config cs_dir: <path>`	configure snort to provide a Unix socket in the path that can be used to issue commands to the running process. See Section [*] for more details.

`config daemon`	Forks as a daemon (`snort -D`).
`config decode_data_link`	Decodes Layer2 headers (`snort -e`).
`config default_rule_state: <state>`	Global configuration directive to enable or disable the loading of rules into the detection engine. Default (with or without directive) is enabled. Specify `disabled` to disable loading rules.
`config daq: <type>`	Selects the type of DAQ to instantiate. The DAQ with the highest version of the given type is selected if there are multiple of the same type (this includes any built-in DAQs).
`config daq_mode: <mode>`	Select the DAQ mode: passive, inline, or read-file. Not all DAQs support modes. See the DAQ distro README for possible DAQ modes or list DAQ capabilities for a brief summary.
`config daq_var: <name=value>`	Set a DAQ specific variable. Snort just passes this information down to the DAQ. See the DAQ distro README for possible DAQ variables.
`config daq_dir: <dir>`	Tell Snort where to look for available dynamic DAQ modules. This can be repeated. The selected DAQ will be the one with the latest version.
`config daq_list: [<dir>]`	Tell Snort to dump basic DAQ capabilities and exit. You can optionally specify a directory to include any dynamic DAQs from that directory. You can also precede this option with extra DAQ directory options to look in multiple directories.
`config decode_esp: [enable | disable]`	Enable or disable the decoding of Encapsulated Security Protocol (ESP). This is disabled by default. Some networks use ESP for authentication without encryption, allowing their content to be inspected. Encrypted ESP may cause some false positives if this option is enabled.
`config detection: [search-method <method>]`	Select type of fast pattern matcher algorithm to use. • `search-method <method>` ◦ Queued match search methods - Matches are queued until the fast pattern matcher is finished with the payload, then evaluated. This was found to generally increase performance through fewer cache misses (evaluating each rule would generally blow away the fast pattern matcher state in the cache). ▪ `ac` and `ac-q` - Aho-Corasick Full (high memory, best performance). ▪ `ac-bnfa` and `ac-bnfa-q` - Aho-Corasick Binary NFA (low memory, high performance) ▪ `lowmem` and `lowmem-q` - Low Memory Keyword Trie (low memory, moderate performance)

	▪ `ac-split` - Aho-Corasick Full with ANY-ANY port group evaluated separately (low memory, high performance). Note this is shorthand for `search-method ac, split-any-any` ▪ `intel-cpm` - Intel CPM library (must have compiled Snort with location of libraries to enable this) ◦ No queue search methods - The "nq" option specifies that matches should not be queued and evaluated as they are found. ▪ `ac-nq` - Aho-Corasick Full (high memory, best performance). ▪ `ac-bnfa-nq` - Aho-Corasick Binary NFA (low memory, high performance). This is the default search method if none is specified. ▪ `lowmem-nq` - Low Memory Keyword Trie (low memory, moderate performance) ◦ Other search methods (the above are considered superior to these) ▪ `ac-std` - Aho-Corasick Standard (high memory, high performance) ▪ `acs` - Aho-Corasick Sparse (high memory, moderate performance) ▪ `ac-banded` - Aho-Corasick Banded (high memory, moderate performance) ▪ `ac-sparsebands` - Aho-Corasick Sparse-Banded (high memory, moderate performance)
`config detection: [split-any-any] [search-optimize] [max-pattern-len <int>]`	Other options that affect fast pattern matching. • `split-any-any` ◦ A memory/performance tradeoff. By default, ANY-ANY port rules are added to every non ANY-ANY port group so that only one port group rule evaluation needs to be done per packet. Not putting the ANY-ANY port rule group into every other port group can significantly reduce the memory footprint of the fast pattern matchers if there are many ANY-ANY port rules. But doing so may require two port group evaluations per packet - one for the specific port group and one for the ANY-ANY port group, thus potentially reducing performance. This option is generic and can be used with any `search-method` but was specifically intended for use with the `ac search-method` where the memory

	footprint is significantly reduced though overall fast pattern performance is better than `ac-bnfa`. Of note is that the lower memory footprint can also increase performance through fewer cache misses. Default is not to split the ANY-ANY port group. • `search-optimize` ◦ Optimizes fast pattern memory when used with `search-method ac` or `ac-split` by dynamically determining the size of a state based on the total number of states. When used with `ac-bnfa`, some fail-state resolution will be attempted, potentially increasing performance. Default is not to optimize. • `max-pattern-len <integer>` ◦ This is a memory optimization that specifies the maximum length of a pattern that will be put in the fast pattern matcher. Patterns longer than this length will be truncated to this length before inserting into the pattern matcher. Useful when there are very long contents being used and truncating the pattern won't diminish the uniqueness of the patterns. Note that this may cause more false positive rule evaluations, i.e. rules that will be evaluated because a fast pattern was matched, but eventually fail, however CPU cache can play a part in performance so a smaller memory footprint of the fast pattern matcher can potentially increase performance. Default is to not set a maximum pattern length.
`config detection: [no_stream_inserts] [max_queue_events <int>] [enable-single-rule-group] [bleedover-port-limit]`	Other detection engine options. • `no_stream_inserts` ◦ Specifies that stream inserted packets should not be evaluated against the detection engine. This is a potential performance improvement with the idea that the stream rebuilt packet will contain the payload in the inserted one so the stream inserted packet doesn't need to be evaluated. Default is to inspect stream inserts. • `max_queue_events <integer>` ◦ Specifies the maximum number of matching fast-pattern states to queue per packet. Default is 5 events. • `enable-single-rule-group` ◦ Put all rules into one port group. Not recommended. Default is not to do this.

	• `bleedover-port-limit` ◦ The maximum number of source or destination ports designated in a rule before the rule is considered an ANY-ANY port group rule. Default is 1024.

`config detection: [debug] [debug-print-nocontent-rule-tests] [debug-print-rule-group-build-details] [debug-print-rule-groups-uncompiled] [debug-print-rule-groups-compiled] [debug-print-fast-pattern] [bleedover-warnings-enabled]`	Options for detection engine debugging.
	• `debug`
	  ◦ Prints fast pattern information for a particular port group.
	• `debug-print-nocontent-rule-tests`
	  ◦ Prints port group information during packet evaluation.
	• `debug-print-rule-group-build-details`
	  ◦ Prints port group information during port group compilation.
	• `debug-print-rule-groups-uncompiled`
	  ◦ Prints uncompiled port group information.
	• `debug-print-rule-groups-compiled`
	  ◦ Prints compiled port group information.
	• `debug-print-fast-pattern`
	  ◦ For each rule with fast pattern content, prints information about the content being used for the fast pattern matcher.
	• `bleedover-warnings-enabled`
	  ◦ Prints a warning if the number of source or destination ports used in a rule exceed the `bleedover-port-limit` forcing the rule to be moved into the ANY-ANY port group.

`config disable_decode_alerts`	Turns off the alerts generated by the decode phase of Snort.
`config disable_inline_init_failopen`	Disables failopen thread that allows inline traffic to pass while Snort is starting up. Only useful if Snort was configured with -enable-inline-init-failopen. (`snort -disable-inline-init-failopen`)
`config disable_ipopt_alerts`	Disables IP option length validation alerts.
`config disable_tcpopt_alerts`	Disables option length validation alerts.
`config disable_tcpopt_experimental_alerts`	Turns off alerts generated by experimental TCP options.
`config disable_tcpopt_obsolete_alerts`	Turns off alerts generated by obsolete TCP options.
`config disable_tcpopt_ttcp_alerts`	Turns off alerts generated by T/TCP options.
`config disable_ttcp_alerts`	Turns off alerts generated by T/TCP options.
`config dump_chars_only`	Turns on character dumps (`snort -C`).
`config dump_payload`	Dumps application layer (`snort -d`).
`config dump_payload_verbose`	Dumps raw packet starting at link layer (`snort -X`).

`config enable_decode_drops`	Enables the dropping of bad packets identified by decoder (only applicable in inline mode).
`config enable_decode_oversized_alerts`	Enable alerting on packets that have headers containing length fields for which the value is greater than the length of the packet.
`config enable_decode_oversized_drops`	Enable dropping packets that have headers containing length fields for which the value is greater than the length of the packet. `enable_decode_oversized_alerts` must also be enabled for this to be effective (only applicable in inline mode).
`config enable_deep_teredo_inspection`	Snort's packet decoder only decodes Teredo (IPv6 over UDP over IPv4) traffic on UDP port 3544. This option makes Snort decode Teredo traffic on all UDP ports.
`config enable_ipopt_drops`	Enables the dropping of bad packets with bad/truncated IP options (only applicable in inline mode).
`config enable_mpls_multicast`	Enables support for MPLS multicast. This option is needed when the network allows MPLS multicast traffic. When this option is off and MPLS multicast traffic is detected, Snort will generate an alert. By default, it is off.
`config enable_mpls_overlapping_ip`	Enables support for overlapping IP addresses in an MPLS network. In a normal situation, where there are no overlapping IP addresses, this configuration option should not be turned on. However, there could be situations where two private networks share the same IP space and different MPLS labels are used to differentiate traffic from the two VPNs. In such a situation, this configuration option should be turned on. By default, it is off.
`config enable_tcpopt_drops`	Enables the dropping of bad packets with bad/truncated TCP option (only applicable in inline mode).
`config enable_tcpopt_experimental_drops`	Enables the dropping of bad packets with experimental TCP option. (only applicable in inline mode).
`config enable_tcpopt_obsolete_drops`	Enables the dropping of bad packets with obsolete TCP option. (only applicable in inline mode).
`config enable_tcpopt_ttcp_drops`	Enables the dropping of bad packets with T/TCP option. (only applicable in inline mode).
`config enable_ttcp_drops`	Enables the dropping of bad packets with T/TCP option. (only applicable in inline mode).
`config event_filter: memcap <bytes>`	Set global memcap in bytes for thresholding. Default is 1048576 bytes (1 megabyte).
`config event_queue: [max_queue <num>] [log <num>] [order_events <order>]`	Specifies conditions about Snort's event queue. You can use the following options: • `max_queue $<$integer$>$` (max events supported) • `log $<$integer$>$` (number of events to log)

	• order_events [priority$ \vert$content_length] (how to order events within the queue) See Section [*] for more information and examples.
config flowbits_size: <num-bits>	Specifies the maximum number of flowbit tags that can be used within a rule set. The default is 1024 bits and maximum is 2048.
config ignore_ports: <proto> <port-list>	Specifies ports to ignore (useful for ignoring noisy NFS traffic). Specify the protocol (TCP, UDP, IP, or ICMP), followed by a list of ports. Port ranges are supported.
config interface: <iface>	Sets the network interface (snort -i).
config ipv6_frag: [bsd_icmp_frag_alert on|off] [, bad_ipv6_frag_alert on|off] [, frag_timeout <secs>] [, max_frag_sessions <max-track>]	The following options can be used: • bsd_icmp_frag_alert on|off (Specify whether or not to alert. Default is on) • bad_ipv6_frag_alert on|off (Specify whether or not to alert. Default is on) • frag_timeout $<$integer$>$ (Specify amount of time in seconds to timeout first frag in hash table) • max_frag_sessions $<$integer$>$ (Specify the number of fragments to track in the hash table)
config logdir: <dir>	Sets the logdir (snort -l).
config log_ipv6_extra_data	Set Snort to log IPv6 source and destination addresses as unified2 extra data events.
config max_attribute_hosts: <hosts>	Sets a limit on the maximum number of hosts to read from the attribute table. Minimum value is 32 and the maximum is 524288 (512k). The default is 10000. If the number of hosts in the attribute table exceeds this value, an error is logged and the remainder of the hosts are ignored. This option is only supported with a Host Attribute Table (see section [*]).
config max_attribute_services_per_host: <hosts>	Sets a per host limit on the maximum number of services to read from the attribute table. Minimum value is 1 and the maximum is 65535. The default is 100. For a given host, if the number of services in the attribute table exceeds this value, an error is logged and the remainder of the services for that host are ignored. This option is only supported with a Host Attribute Table (see section [*]).
config max_mpls_labelchain_len: <num-hdrs>	Sets a Snort-wide limit on the number of MPLS headers a packet can have. Its default value is -1, which means that there is no limit on label chain length.
config max_ip6_extensions: <num-extensions>	Sets the maximum number of IPv6 extension headers that Snort will decode. Default is 8.

`config min_ttl: <ttl>`	Sets a Snort-wide minimum ttl to ignore all traffic.
`config mpls_payload_type: ipv4|ipv6|ethernet`	Sets a Snort-wide MPLS payload type. In addition to ipv4, ipv6 and ethernet are also valid options. The default MPLS payload type is ipv4
`config no_promisc`	Disables promiscuous mode (`snort -p`).
`config nolog`	Disables logging. Note: Alerts will still occur. (`snort -N`).
`config nopcre`	Disables pcre pattern matching.
`config obfuscate`	Obfuscates IP Addresses (`snort -0`).
`config order: <order>`	Changes the order that rules are evaluated, e.g.: pass alert log activation.
`config pcre_match_limit: $<$integer$>$`	Restricts the amount of backtracking a given PCRE option. For example, it will limit the number of nested repeats within a pattern. A value of -1 allows for unlimited PCRE, up to the PCRE library compiled limit (around 10 million). A value of 0 results in no PCRE evaluation. The snort default value is 1500.
`config pcre_match_limit_recursion: $<$integer$>$`	Restricts the amount of stack used by a given PCRE option. A value of -1 allows for unlimited PCRE, up to the PCRE library compiled limit (around 10 million). A value of 0 results in no PCRE evaluation. The snort default value is 1500. This option is only useful if the value is less than the `pcre_match_limit`
`config pkt_count: <N>`	Exits after N packets (`snort -n`).
`config policy_version: $<$base-version-string$>$ [$<$binding-version-string$>$]`	Supply versioning information to configuration files. Base version should be a string in all configuration files including included ones. In addition, binding version must be in any file configured with `config binding`. This option is used to avoid race conditions when modifying and loading a configuration within a short time span - before Snort has had a chance to load a previous configuration.
`config profile_preprocs`	Print statistics on preprocessor performance. See Section [*] for more details.
`config profile_rules`	Print statistics on rule performance. See Section [*] for more details.
`config protected_content: md5|sha256|sha512`	Specifies a default algorithm to use for protected_content rules.
`config quiet`	Disables banner and status reports (`snort -q`). NOTE: The command line switch -q takes effect immediately after processing the command line parameters, whereas using `config quiet` in snort.conf takes effect when the configuration line in snort.conf is parsed. That may occur after other configuration settings that result in output to console or syslog.
`config reference: <ref>`	

	Adds a new reference system to Snort, e.g.: myref http://myurl.com/?id=
`config reference_net <cidr>`	For IP obfuscation, the obfuscated net will be used if the packet contains an IP address in the reference net. Also used to determine how to set up the logging directory structure for the `session` post detection rule option and ASCII output plugin - an attempt is made to name the log directories after the IP address that is not in the reference net.
`config response: [attempts <count>] [, device <dev>]`	Set the number of strafing attempts per injected response and/or the device, such as eth0, from which to send responses. These options may appear in any order but must be comma separated. The are intended for passive mode.
`config set_gid: <gid>`	Changes GID to specified GID (`snort -g`).
`config set_uid: <uid>`	Sets UID to $<$id$>$ (`snort -u`).
`config show_year`	Shows year in timestamps (`snort -y`).
`config snaplen: <bytes>`	Set the snaplength of packet, same effect as -P `$<$snaplen$>$` or `-snaplen $<$snaplen$>$` options.
`config so_rule_memcap: <bytes>`	Set global memcap in bytes for so rules that dynamically allocate memory for storing session data in the stream preprocessor. A value of 0 disables the memcap. Default is 0. Maximum value is the maximum value an unsigned 32 bit integer can hold which is 4294967295 or 4GB.
`config stateful`	Sets assurance mode for stream (stream is established).
`config tagged_packet_limit: <max-tag>`	When a metric other than `packets` is used in a tag option in a rule, this option sets the maximum number of packets to be tagged regardless of the amount defined by the other metric. See Section [*] on using the tag option when writing rules for more details. The default value when this option is not configured is 256 packets. Setting this option to a value of 0 will disable the packet limit.
`config threshold: memcap <bytes>`	Set global memcap in bytes for thresholding. Default is 1048576 bytes (1 megabyte). (This is deprecated. Use config event_filter instead.)
`config umask: <umask>`	Sets umask when running (`snort -m`).
`config utc`	Uses UTC instead of local time for timestamps (`snort -U`).
`config verbose`	Uses verbose logging to STDOUT (`snort -v`).
`config vlan_agnostic`	Causes Snort to ignore vlan headers for the purposes of connection and frag tracking. This option is only valid in the base configuration when using multiple configurations, and the default is off.
`config address_space_agnostic`	Causes Snort to ignore DAQ address space ID for the purposes of connection and frag tracking. This option is

	only valid in the base configuration when using multiple configurations, and the default is off.
`config policy_mode: tap|inline|inline_test`	Sets the policy mode to either `passive`, `inline` or `inline_test`.
`config disable_replace`	Disables content replace option. Default behaviour is to replace content.
`config tunnel_verdicts: gtp|teredo|6in4|4in6`	By default, whitelist and blacklist verdicts are handled internally by Snort for GTP, Teredo, 6in4 and 4in6 encapsulated traffic. This means Snort actually gives the DAQ a pass or block verdict instead. This is to workaround cases where the DAQ would apply the verdict to the whole tunnel instead of the individual session within the tunnel. If your DAQ decodes GTP, Teredo, 6in4 or 4in6 correctly, setting this config will allow the whitelist or blacklist verdict to go to the DAQ. There is a modest performance boost by doing this where possible since Snort won't see the remaining packets on the session.

End of Navigation Panel Table of Child-Links **Subsections**

- 2.2.1 Frag3
  - 2.2.1.1 Frag 3 Configuration
  - 2.2.1.2 Format
  - 2.2.1.3 Frag 3 Alert Output

- 2.2.2 Session
  - 2.2.2.1 Session API
  - 2.2.2.2 Session Global Configuration
  - 2.2.2.3 Session HA Configuration
  - 2.2.2.4 Example Configurations

- 2.2.3 Stream
  - 2.2.3.1 Transport Protocols
  - 2.2.3.2 Target-Based
  - 2.2.3.3 Stream API
  - 2.2.3.4 Anomaly Detection
  - 2.2.3.5 Protocol Aware Flushing
  - 2.2.3.6 Stream TCP Configuration
  - 2.2.3.7 Stream UDP Configuration
  - 2.2.3.8 Stream ICMP Configuration
  - 2.2.3.9 Stream IP Configuration
  - 2.2.3.10 Example Configurations

- 2.2.4 sfPortscan
  - 2.2.4.1 sfPortscan Configuration
  - 2.2.4.2 Format
  - 2.2.4.3 Example
  - 2.2.4.4 sfPortscan Alert Output
  - 2.2.4.5 Tuning sfPortscan

- 2.2.5 RPC Decode
  - 2.2.5.1 Format

- 2.2.6 Performance Monitor
  - 2.2.6.1 Examples

- 2.2.7 HTTP Inspect
  - 2.2.7.1 Global Configuration
  - 2.2.7.2 Format
  - 2.2.7.3 Example Global Configuration
  - 2.2.7.4 Server Configuration
  - 2.2.7.5 Example Default Configuration
  - 2.2.7.6 Example IP Configuration
  - 2.2.7.7 Example Multiple IP Configuration
  - 2.2.7.8 Server Configuration Options

- 2.2.7.9 Example
- 2.2.7.10 Examples

- 2.2.8 SMTP Preprocessor
  - 2.2.8.1 Configuration
  - 2.2.8.2 Example
  - 2.2.8.3 Default
  - 2.2.8.4 Note

- 2.2.9 POP Preprocessor
  - 2.2.9.1 Configuration
  - 2.2.9.2 Example
  - 2.2.9.3 Default

- 2.2.10 IMAP Preprocessor
  - 2.2.10.1 Configuration
  - 2.2.10.2 Example
  - 2.2.10.3 Default

- 2.2.11 FTP/Telnet Preprocessor
  - 2.2.11.1 Global Configuration
  - 2.2.11.2 Format
  - 2.2.11.3 Example Global Configuration
  - 2.2.11.4 Telnet Configuration
  - 2.2.11.5 Format
  - 2.2.11.6 Example Telnet Configuration
  - 2.2.11.7 FTP Server Configuration
  - 2.2.11.8 Example Default FTP Server Configuration
  - 2.2.11.9 Example IP specific FTP Server Configuration
  - 2.2.11.10 FTP Server Configuration Options
  - 2.2.11.11 FTP Server Base Configuration Options
  - 2.2.11.12 FTP Client Configuration
  - 2.2.11.13 Example Default FTP Client Configuration
  - 2.2.11.14 Example IP specific FTP Client Configuration
  - 2.2.11.15 FTP Client Configuration Options
  - 2.2.11.16 Examples/Default Configuration from snort.conf

- 2.2.12 SSH
  - 2.2.12.1 Configuration
  - 2.2.12.2 Example Configuration from snort.conf

- 2.2.13 DNS
  - 2.2.13.1 Configuration
  - 2.2.13.2 Examples/Default Configuration from snort.conf

- 2.2.14 SSL/TLS
  - 2.2.14.1 Configuration
  - 2.2.14.2 Examples/Default Configuration from snort.conf
  - 2.2.14.3 Rule Options

- 2.2.15 ARP Spoof Preprocessor
  - 2.2.15.1 Format
  - 2.2.15.2 Example Configuration

- 2.2.16 DCE/RPC 2 Preprocessor
  - 2.2.16.1 Dependency Requirements
  - 2.2.16.2 Target Based
  - 2.2.16.3 Configuration
  - 2.2.16.4 Events
  - 2.2.16.5 Rule Options

- 2.2.17 Sensitive Data Preprocessor
  - 2.2.17.1 Dependencies
  - 2.2.17.2 Preprocessor Configuration
  - 2.2.17.3 Rule Options

- 2.2.18 Normalizer
  - 2.2.18.1 IP4 Normalizations
  - 2.2.18.2 IP6 Normalizations
  - 2.2.18.3 ICMP4/6 Normalizations
  - 2.2.18.4 TCP Normalizations
  - 2.2.18.5 TTL Normalization

- 2.2.19 SIP Preprocessor
  - 2.2.19.1 Dependency Requirements
  - 2.2.19.2 Configuration
  - 2.2.19.3 Events
  - 2.2.19.4 Rule Options

- 2.2.20 Reputation Preprocessor
  - 2.2.20.1 Configuration
  - 2.2.20.2 Events
  - 2.2.20.3 Shared memory support

- 2.2.21 GTP Decoder and Preprocessor
  - 2.2.21.1 Dependency Requirements
  - 2.2.21.2 GTP Data Channel Decoder Configuration
  - 2.2.21.3 GTP Control Channel Preprocessor Configuration
  - 2.2.21.4 GTP Decoder Events
  - 2.2.21.5 GTP Preprocessor Events
  - 2.2.21.6 Rule Options

- 2.2.22 Modbus Preprocessor
  - 2.2.22.1 Dependency Requirements
  - 2.2.22.2 Preprocessor Configuration
  - 2.2.22.3 Rule Options
  - 2.2.22.4 Preprocessor Events

- 2.2.23 DNP3 Preprocessor
  - 2.2.23.1 Dependency Requirements
  - 2.2.23.2 Preprocessor Configuration
  - 2.2.23.3 Rule Options
  - 2.2.23.4 Preprocessor Events

- 2.2.24 AppId Preprocessor
  - 2.2.24.1 Dependency Requirements
  - 2.2.24.2 Preprocessor Configuration
  - 2.2.24.3 Rule Options
  - 2.2.24.4 Application Rule Events
  - 2.2.24.5 Application Usage Statistics
  - 2.2.24.6 Open Detector Package (ODP) Installation
  - 2.2.24.7 User Created Application Detectors

End of Table of Child-Links

2.2 Preprocessors

Preprocessors were introduced in version 1.5 of Snort. They allow the functionality of Snort to be extended by allowing users and programmers to drop modular plugins into Snort fairly easily. Preprocessor code is run before the detection engine is called, but after the packet has been decoded. The packet can be modified or analyzed in an out-of-band manner using this mechanism.

Preprocessors are loaded and configured using the `preprocessor` keyword. The format of the preprocessor directive in the Snort config file is:

```
preprocessor <name>: <options>
```

2.2.1 Frag3

The frag3 preprocessor is a target-based IP defragmentation module for Snort. Frag3 is designed with the following goals:

6. Fast execution with less complex data management.
7. Target-based host modeling anti-evasion techniques.

Frag3 uses the sfxhash data structure and linked lists for data handling internally which allows it to have much more predictable and deterministic performance in any environment which should aid us in managing heavily fragmented environments.

Target-based analysis is a relatively new concept in network-based intrusion detection. The idea of a target-based system is to model the actual targets on the network instead of merely modeling the protocols and looking for attacks within them. When IP stacks are written for different operating systems, they are usually implemented by people who read the RFCs and then write their interpretation of what the RFC outlines into code. Unfortunately, there are ambiguities in the way that the RFCs define some of the edge conditions that may occur and when this happens different people implement certain aspects of their IP stacks differently. For an IDS this is a big problem.

In an environment where the attacker can determine what style of IP defragmentation is being used on a particular target, the attacker can try to fragment packets such that the target will put them back together in a specific manner while any passive systems trying to model the host traffic have to guess which way the target OS is going to handle the overlaps and retransmits. As I like to say, if the attacker has more information about the targets on a network than the IDS does, it is possible to evade the IDS. This is where the idea for “target-based IDS” came from. For more detail on this issue and how it affects IDS, check out the famous Ptacek & Newsham paper at `http://www.snort.org/docs/idspaper/`.

The basic idea behind target-based IDS is that we tell the IDS information about hosts on the network so that it can avoid Ptacek & Newsham style evasion attacks based on information about how an individual target IP stack operates. Vern Paxson and Umesh Shankar did a great paper on this very topic in 2003

that detailed mapping the hosts on a network and determining how their various IP stack implementations handled the types of problems seen in IP defragmentation and TCP stream reassembly. Check it out at http://www.icir.org/vern/papers/activemap-oak03.pdf.

We can also present the IDS with topology information to avoid TTL-based evasions and a variety of other issues, but that's a topic for another day. Once we have this information we can start to really change the game for these complex modeling problems.

Frag3 was implemented to showcase and prototype a target-based module within Snort to test this idea.

2.2.1.1 Frag 3 Configuration

There are at least two preprocessor directives required to activate frag3, a global configuration directive and an engine instantiation. There can be an arbitrary number of engines defined at startup with their own configuration, but only one global configuration.

Global Configuration

- Preprocessor name: `frag3_global`
- Available options: NOTE: Global configuration options are comma separated.
  - `max_frags $<$number$>$` - Maximum simultaneous fragments to track. Default is 8192.
  - `memcap $<$bytes$>$` - Memory cap for self preservation. Default is 4MB.
  - `prealloc_memcap $<$bytes$>$` - alternate memory management mode, use preallocated fragment nodes based on a memory cap (faster in some situations).
  - `prealloc_frags $<$number$>$` - Alternate memory management mode, use preallocated fragment nodes (faster in some situations).
  - `disabled` - This optional keyword is allowed with any policy to avoid packet processing. This option disables the preprocessor for this config, but not for other instances of multiple configurations. Use the disable keyword in the base configuration to specify values for the options `memcap`, `prealloc_memcap`, and `prealloc_frags` without having the preprocessor inspect traffic for traffic applying to the base configuration. The other options are parsed but not used. Any valid configuration may have "disabled" added to it.

Engine Configuration

- Preprocessor name: `frag3_engine`
- Available options: NOTE: Engine configuration options are space separated.
  - `timeout $<$seconds$>$` - Timeout for fragments. Fragments in the engine for longer than this period will be automatically dropped. Default is 60 seconds.
  - `min_ttl $<$value$>$` - Minimum acceptable TTL value for a fragment packet. Default is 1. The accepted range for this option is 1 - 255.
  - `detect_anomalies` - Detect fragment anomalies.

- `bind_to $<$ip_list$>$` - IP List to bind this engine to. This engine will only run for packets with destination addresses contained within the IP List. Default value is `all`.
- `overlap_limit <number>` - Limits the number of overlapping fragments per packet. The default is "0" (unlimited). This config option takes values equal to or greater than zero. This is an optional parameter. detect_anomalies option must be configured for this option to take effect.
- `min_fragment_length <number>` - Defines smallest fragment size (payload size) that should be considered valid. Fragments smaller than or equal to this limit are considered malicious and an event is raised, if detect_anomalies is also configured. The default is "0" (unlimited), the minimum is "0". This is an optional parameter. detect_anomalies option must be configured for this option to take effect.
- `policy $<$type$>$` - Select a target-based defragmentation mode. Available types are first, last, bsd, bsd-right, linux, windows and solaris. Default type is bsd.

  The Paxson Active Mapping paper introduced the terminology frag3 is using to describe policy types. The known mappings are as follows. Anyone who develops more mappings and would like to add to this list please feel free to send us an email!

Platform	Type
AIX 2	BSD
AIX 4.3 8.9.3	BSD
Cisco IOS	Last
FreeBSD	BSD
HP JetDirect (printer)	BSD-right
HP-UX B.10.20	BSD
HP-UX 11.00	First
IRIX 4.0.5F	BSD
IRIX 6.2	BSD
IRIX 6.3	BSD
IRIX64 6.4	BSD
Linux 2.2.10	linux
Linux 2.2.14-5.0	linux
Linux 2.2.16-3	linux
Linux 2.2.19-6.2.10smp	linux
Linux 2.4.7-10	linux
Linux 2.4.9-31SGI 1.0.2smp	linux
Linux 2.4 (RedHat 7.1-7.3)	linux
MacOS (version unknown)	First

NCD Thin Clients	BSD
OpenBSD (version unknown)	linux
OpenBSD (version unknown)	linux
OpenVMS 7.1	BSD
OS/2 (version unknown)	BSD
OSF1 V3.0	BSD
OSF1 V3.2	BSD
OSF1 V4.0,5.0,5.1	BSD
SunOS 4.1.4	BSD
SunOS 5.5.1,5.6,5.7,5.8	First
Tru64 Unix V5.0A,V5.1	BSD
Vax/VMS	BSD
Windows (95/98/NT4/W2K/XP)	Windows

2.2.1.2 Format

Note in the advanced configuration below that there are three engines specified running with Linux, `first` and `last` policies assigned. The first two engines are bound to specific IP address ranges and the last one applies to all other traffic. Packets that don't fall within the address requirements of the first two engines automatically fall through to the third one.

2.2.1.2.1 Basic Configuration

```
preprocessor frag3_global
preprocessor frag3_engine
```

2.2.1.2.2 Advanced Configuration

```
preprocessor frag3_global: prealloc_nodes 8192
preprocessor frag3_engine: policy linux bind_to 192.168.1.0/24
preprocessor frag3_engine: policy first bind_to [10.1.47.0/24,172.16.8.0/24]
preprocessor frag3_engine: policy last detect_anomalies
```

2.2.1.3 Frag 3 Alert Output

Frag3 is capable of detecting eight different types of anomalies. Its event output is packet-based so it will work with all output modes of Snort. Read the documentation in the `doc/signatures` directory with filenames that begin with “123-” for information on the different event types.

2.2.2 Session

The Session preprocessor is a global stream session management module for Snort. It is derived from the session management functions that were part of the Stream5 preprocessor.

Since Session implements part of the functionality and API that was previously in Stream5 it cannot be used with Stream5 but must be used in conjunction with the new Stream preprocessor. Similarly, due to the API changes, the other preprocessors in Snort 2.9.7 work only with the new Session and Stream preprocessers.

2.2.2.1 Session API

Session provides an API to enable the creation and management of the session control block for a flow and the management of data and state that may be associated with that flow by service and application preprocessors (most of these functions were previously supported by the Stream5 API). These methods are called to identify sessions that may be ignored (large data transfers, etc), and update the identifying information about the session (application protocol, direction, etc) that can later be used by rules. API methods to enable preprocessors to register for dispatch for ports and services for which they should be called to process the packet have been added to the Session API. Session is required for the use of the 'flow' and 'flowbits' keywords.

2.2.2.2 Session Global Configuration

Global settings for the Session preprocessor.

```
    preprocessor stream5_global: \
        [track_tcp <yes|no>], [max_tcp <number>], \
        [memcap <number bytes>], \
        [track_udp <yes|no>], [max_udp <number>], \
        [track_icmp <yes|no>], [max_icmp <number>], \
        [track_ip <yes|no>], [max_ip <number>], \
        [flush_on_alert], [show_rebuilt_packets], \
        [prune_log_max <number bytes>], [disabled], \
        [enable_ha]
```

Option	Description
`track_tcp <yes|no>`	Track sessions for TCP. The default is "yes".
`max_tcp <num sessions>`	Maximum simultaneous TCP sessions tracked. The default is "262144", maximum is "1048576", minimum is "2".
`memcap <num bytes>`	Memcap for TCP packet storage. The default is "8388608" (8MB), maximum is "1073741824" (1GB), minimum is "32768" (32KB).

`track_udp <yes|no>`	Track sessions for UDP. The default is "yes".
`max_udp <num sessions>`	Maximum simultaneous UDP sessions tracked. The default is "131072", maximum is "1048576", minimum is "1".
`track_icmp <yes|no>`	Track sessions for ICMP. The default is "no".
`max_icmp <num sessions>`	Maximum simultaneous ICMP sessions tracked. The default is "65536", maximum is "1048576", minimum is "1".
`track_ip <yes|no>`	Track sessions for IP. The default is "no". Note that "IP" includes all non-TCP/UDP traffic over IP including ICMP if ICMP not otherwise configured.
`max_ip <num sessions>`	Maximum simultaneous IP sessions tracked. The default is "16384", maximum is "1048576", minimum is "1".
`disabled`	Option to disable the stream5 tracking. By default this option is turned off. When the preprocessor is disabled only the options memcap, max_tcp, max_udp and max_icmp are applied when specified with the configuration.
`flush_on_alert`	Backwards compatibility. Flush a TCP stream when an alert is generated on that stream. The default is set to off.
`show_rebuilt_packets`	Print/display packet after rebuilt (for debugging). The default is set to off.
`prune_log_max <num bytes>`	Print a message when a session terminates that was consuming more than the specified number of bytes. The default is "1048576" (1MB), minimum can be either "0" (disabled) or if not disabled the minimum is "1024" and maximum is "1073741824".
`enable_ha`	Enable High Availability state sharing. The default is set to off.

2.2.2.3 Session HA Configuration

Configuration for HA session state sharing.

```
preprocessor stream5_ha: [min_session_lifetime <num millisecs>], \
    [min_sync_interval <num millisecs>], [startup_input_file <filename>], \
    [runtime_output_file <filename>], [use_side_channel]
```

Option	Description
`min_session_lifetime <num millisecs>`	Minimum session liftime in milliseconds. HA update messages will only be generated once a session has existed for at least this long. The default is 0, the minimum is 0, and the maximum is 65535.
`min_sync_interval <num millisecs>`	Minimum synchronization interval in milliseconds. HA update messages will not be generated more often than once per interval on a given session. The default is 0, the minimum is 0, and the maximum is 65535.
`startup_input_file <filename>`	The name of a file for snort to read HA messages from at startup.
`runtime_output_file <filename>`	The name of a file to which Snort will write all HA messages that are generated while it is running.
`use_side_channel`	Indicates that all HA messages should also be sent to the side channel for processing.

2.2.2.4 Example Configurations

1. This example configuration sets a maximum number of TCP session control blocks to 8192, enables tracking of TCP and UPD sessions, and disables tracking of ICMP sessions. The number of UDP session control blocks will be set to the compiled default.

```
preprocessor stream5_global: \
    max_tcp 8192, track_tcp yes, track_udp yes, track_icmp no

preprocessor stream5_tcp: \
    policy first, use_static_footprint_sizes

preprocessor stream5_udp: \
    ignore_any_rules
```

2.2.3 Stream

The Stream preprocessor is a target-based TCP reassembly module for Snort. It is capable of tracking sessions for both TCP and UDP.

2.2.3.1 Transport Protocols

TCP sessions are identified via the classic TCP "connection". UDP sessions are established as the result of a series of UDP packets from two end points via the same set of ports. ICMP messages are tracked

for the purposes of checking for unreachable and service unavailable messages, which effectively terminate a TCP or UDP session.

2.2.3.2 Target-Based

Stream, like Frag3, introduces target-based actions for handling of overlapping data and other TCP anomalies. The methods for handling overlapping data, TCP Timestamps, Data on SYN, FIN and Reset sequence numbers, etc. and the policies supported by Stream are the results of extensive research with many target operating systems.

2.2.3.3 Stream API

Stream supports the modified Stream API that is now focused on functions specific to reassembly and protocol aware flushing operations. Session management functions have been moved to the Session API. The remaining API functions enable other protocol normalizers/preprocessors to dynamically configure reassembly behavior as required by the application layer protocol.

2.2.3.4 Anomaly Detection

TCP protocol anomalies, such as data on SYN packets, data received outside the TCP window, etc are configured via the `detect_anomalies` option to the TCP configuration. Some of these anomalies are detected on a per-target basis. For example, a few operating systems allow data in TCP SYN packets, while others do not.

2.2.3.5 Protocol Aware Flushing

Protocol aware flushing of HTTP, SMB and DCE/RPC can be enabled with this option:

```
config paf_max: <max-pdu>
```

where `<max-pdu>` is between zero (off) and 63780. This allows Snort to statefully scan a stream and reassemble a complete PDU regardless of segmentation. For example, multiple PDUs within a single TCP segment, as well as one PDU spanning multiple TCP segments will be reassembled into one PDU per packet for each PDU. PDUs larger than the configured maximum will be split into multiple packets.

2.2.3.6 Stream TCP Configuration

Provides a means on a per IP address target to configure TCP policy. This can have multiple occurrences, per policy that is bound to an IP address or network. One default policy must be specified, and that policy is not bound to an IP address or network.

```
    preprocessor stream5_tcp: \
        [log_asymmetric_traffic <yes|no>], \
        [bind_to <ip_addr>], \
        [timeout <number secs>], [policy <policy_id>], \
        [overlap_limit <number>], [max_window <number>], \
        [require_3whs [<number secs>]], [detect_anomalies], \
        [check_session_hijacking], [use_static_footprint_sizes], \
```

```
        [dont_store_large_packets], [dont_reassemble_async], \
        [max_queued_bytes <bytes>], [max_queued_segs <number segs>], \
        [small_segments <number> bytes <number> [ignore_ports number [number]*]],  \
        [ports <client|server|both> <all|number|!number [number]* [!number]*>], \
        [protocol <client|server|both> <all|service name [service name]*>], \
        [ignore_any_rules], [flush_factor <number segs>]
```

<table>
<tr><th>Option</th><th>Description</th></tr>
<tr><td>bind_to <ip_addr></td><td>IP address or network for this policy. The default is set to any.</td></tr>
<tr><td>timeout <num seconds></td><td>Session timeout. The default is "30", the minimum is "1", and the maximum is "86400" (approximately 1 day).</td></tr>
<tr><td>policy <policy_id></td><td>The Operating System policy for the target OS. The policy_id can be one of the following:
<table>
<tr><td>Policy Name</td><td>Operating Systems.</td></tr>
<tr><td>first</td><td>Favor first overlapped segment.</td></tr>
<tr><td>last</td><td>Favor last overlapped segment.</td></tr>
<tr><td>bsd</td><td>FresBSD 4.x and newer, NetBSD 2.x and newer, OpenBSD 3.x and newer</td></tr>
<tr><td>linux</td><td>Linux 2.4 and newer</td></tr>
<tr><td>old-linux</td><td>Linux 2.2 and earlier</td></tr>
<tr><td>windows</td><td>Windows 2000, Windows XP, Windows 95/98/ME</td></tr>
<tr><td>win2003</td><td>Windows 2003 Server</td></tr>
<tr><td>vista</td><td>Windows Vista</td></tr>
<tr><td>solaris</td><td>Solaris 9.x and newer</td></tr>
<tr><td>hpux</td><td>HPUX 11 and newer</td></tr>
<tr><td>hpux10</td><td>HPUX 10</td></tr>
<tr><td>irix</td><td>IRIX 6 and newer</td></tr>
<tr><td>macos</td><td>MacOS 10.3 and newer</td></tr>
</table></td></tr>
<tr><td>overlap_limit <number></td><td>Limits the number of overlapping packets per session. The default is "0" (unlimited), the minimum is "0", and the maximum is "255".</td></tr>
<tr><td>max_window <number></td><td>Maximum TCP window allowed. The default is "0" (unlimited), the minimum is "0", and the maximum is "1073725440" (65535 left shift 14). That is the highest possible TCP window per RFCs. This option is</td></tr>
</table>

	intended to prevent a DoS against Stream by an attacker using an abnormally large window, so using a value near the maximum is discouraged.
require_3whs [<number seconds>]	Establish sessions only on completion of a SYN/SYN-ACK/ACK handshake. The default is set to off. The optional number of seconds specifies a startup timeout. This allows a grace period for existing sessions to be considered established during that interval immediately after Snort is started. The default is "0" (don't consider existing sessions established), the minimum is "0", and the maximum is "86400" (approximately 1 day).
detect_anomalies	Detect and alert on TCP protocol anomalies. The default is set to off.
check_session_hijacking	Check for TCP session hijacking. This check validates the hardware (MAC) address from both sides of the connect - as established on the 3-way handshake against subsequent packets received on the session. If an ethernet layer is not part of the protocol stack received by Snort, there are no checks performed. Alerts are generated (per 'detect_anomalies' option) for either the client or server when the MAC address for one side or the other does not match. The default is set to off.
use_static_footprint_sizes	Use static values for determining when to build a reassembled packet to allow for repeatable tests. This option should not be used production environments. The default is set to off.
dont_store_large_packets	Performance improvement to not queue large packets in reassembly buffer. The default is set to off. Using this option may result in missed attacks.
dont_reassemble_async	Don't queue packets for reassembly if traffic has not been seen in both directions. The default is set to queue packets.
max_queued_bytes <bytes>	Limit the number of bytes queued for reassembly on a given TCP session to bytes. Default is "1048576" (1MB). A value of "0" means unlimited, with a non-zero minimum of "1024", and a maximum of "1073741824" (1GB). A message is written to console/syslog when this limit is enforced.
max_queued_segs <num>	Limit the number of segments queued for reassembly on a given TCP session. The default is "2621", derived based on an average size of 400 bytes. A value of "0" means unlimited, with a non-zero minimum of "2",

	and a maximum of "1073741824" (1GB). A message is written to console/syslog when this limit is enforced.
`small_segments <number> bytes <number> [ignore_ports <number(s)> ]`	Configure the maximum small segments queued. This feature requires that detect_anomalies be enabled. The first number is the number of consecutive segments that will trigger the detection rule. The default value is "0" (disabled), with a maximum of "2048". The second number is the minimum bytes for a segment to be considered "small". The default value is "0" (disabled), with a maximum of "2048". ignore_ports is optional, defines the list of ports in which will be ignored for this rule. The number of ports can be up to "65535". A message is written to console/syslog when this limit is enforced.
`ports <client|server|both> <all|number(s)|!number(s)>`	Specify the client, server, or both and list of ports in which to perform reassembly. This can appear more than once in a given config. The default settings are `ports client 21 23 25 42 53 80 110 111 135 136 137 139 143 445 513 514 1433 1521 2401 3306`. The minimum port allowed is "1" and the maximum allowed is "65535". To disable reassembly for a port specifiy the port number preceeded by an '!', e.g. !8080 !25
`protocol <client|server|both> <all|service name(s)>`	Specify the client, server, or both and list of services in which to perform reassembly. This can appear more than once in a given config. The default settings are `ports client ftp telnet smtp nameserver dns http pop3 sunrpc dcerpc netbios-ssn imap login shell mssql oracle cvs mysql`. The service names can be any of those used in the host attribute table (see [*]), including any of the internal defaults (see [*]) or others specific to the network.
`ignore_any_rules`	Don't process any -> any (ports) rules for TCP that attempt to match payload if there are no port specific rules for the src or destination port. Rules that have flow or flowbits will never be ignored. This is a performance improvement and may result in missed attacks. Using this does not affect rules that look at protocol headers, only those with content, PCRE, or byte test options. The default is "off". This option can be used only in default policy.
`flush_factor`	Useful in ips mode to flush upon seeing a drop in segment size after N segments of non-decreasing size. The drop in size often indicates an end of request or response.

Note:

If no options are specified for a given TCP policy, that is the default TCP policy. If only a bind_to option is used with no other options that TCP policy uses all of the default values.

2.2.3.7 Stream UDP Configuration

Configuration for UDP session tracking. Since there is no target based binding, there should be only one occurrence of the UDP configuration.

```
preprocessor stream5_udp: [timeout <number secs>], [ignore_any_rules]
```

Option	Description
`timeout <num seconds>`	Session timeout. The default is "30", the minimum is "1", and the maximum is "86400" (approximately 1 day).
`ignore_any_rules`	Don't process any –> any (ports) rules for UDP that attempt to match payload if there are no port specific rules for the src or destination port. Rules that have flow or flowbits will never be ignored. This is a performance improvement and may result in missed attacks. Using this does not affect rules that look at protocol headers, only those with content, PCRE, or byte test options. The default is "off".

Note:

With the ignore_any_rules option, a UDP rule will be ignored except when there is another port specific rule that may be applied to the traffic. For example, if a UDP rule specifies destination port 53, the 'ignored' any –> any rule will be applied to traffic to/from port 53, but NOT to any other source or destination port. A list of rule SIDs affected by this option are printed at Snort's startup.

Note:

With the ignore_any_rules option, if a UDP rule that uses any –> any ports includes either flow or flowbits, the ignore_any_rules option is effectively pointless. Because of the potential impact of disabling a flowbits rule, the ignore_any_rules option will be disabled in this case.

2.2.3.8 Stream ICMP Configuration

Configuration for ICMP session tracking. Since there is no target based binding, there should be only one occurrence of the ICMP configuration.

Note:

ICMP is currently untested, in minimal code form and is NOT ready for use in production networks. It is not turned on by default.

```
preprocessor stream5_icmp: [timeout <number secs>]
```

Option	Description

`timeout <num seconds>`	Session timeout. The default is "30", the minimum is "1", and the maximum is "86400" (approximately 1 day).

2.2.3.9 Stream IP Configuration

Configuration for IP session tracking. Since there is no target based binding, there should be only one occurrence of the IP configuration.

> **Note:**
>
> "IP" includes all non-TCP/UDP traffic over IP including ICMP if ICMP not otherwise configured. It is not turned on by default.

```
preprocessor stream5_ip: [timeout <number secs>]
```

Option	Description
`timeout <num seconds>`	Session timeout. The default is "30", the minimum is "1", and the maximum is "86400" (approximately 1 day).

2.2.3.10 Example Configurations

1. This example configuration is the default configuration in snort.conf and can be used for repeatable tests of stream reassembly in readback mode.

```
preprocessor stream5_global: \
    max_tcp 8192, track_tcp yes, track_udp yes, track_icmp no

preprocessor stream5_tcp: \
    policy first, use_static_footprint_sizes

preprocessor stream5_udp: \
    ignore_any_rules
```

2. This configuration maps two network segments to different OS policies, one for Windows and one for Linux, with all other traffic going to the default policy of Solaris.

```
preprocessor stream5_global: track_tcp yes
preprocessor stream5_tcp: bind_to 192.168.1.0/24, policy windows
preprocessor stream5_tcp: bind_to 10.1.1.0/24, policy linux
preprocessor stream5_tcp: policy solaris
```

2.2.4 sfPortscan

The sfPortscan module, developed by Sourcefire, is designed to detect the first phase in a network attack: Reconnaissance. In the Reconnaissance phase, an attacker determines what types of network protocols or services a host supports. This is the traditional place where a portscan takes place. This phase assumes the attacking host has no prior knowledge of what protocols or services are supported by the target; otherwise, this phase would not be necessary.

As the attacker has no beforehand knowledge of its intended target, most queries sent by the attacker will be negative (meaning that the service ports are closed). In the nature of legitimate network communications, negative responses from hosts are rare, and rarer still are multiple negative responses within a given amount of time. Our primary objective in detecting portscans is to detect and track these negative responses.

One of the most common portscanning tools in use today is Nmap. Nmap encompasses many, if not all, of the current portscanning techniques. sfPortscan was designed to be able to detect the different types of scans Nmap can produce.

sfPortscan will currently alert for the following types of Nmap scans:

- TCP Portscan
- UDP Portscan
- IP Portscan

These alerts are for one$\rightarrow$one portscans, which are the traditional types of scans; one host scans multiple ports on another host. Most of the port queries will be negative, since most hosts have relatively few services available.

sfPortscan also alerts for the following types of decoy portscans:

- TCP Decoy Portscan
- UDP Decoy Portscan
- IP Decoy Portscan

Decoy portscans are much like the Nmap portscans described above, only the attacker has a spoofed source address inter-mixed with the real scanning address. This tactic helps hide the true identity of the attacker.

sfPortscan alerts for the following types of distributed portscans:

- TCP Distributed Portscan
- UDP Distributed Portscan
- IP Distributed Portscan

These are many$\rightarrow$one portscans. Distributed portscans occur when multiple hosts query one host for open services. This is used to evade an IDS and obfuscate command and control hosts.

Note:

Negative queries will be distributed among scanning hosts, so we track this type of scan through the scanned host.

sfPortscan alerts for the following types of portsweeps:

- TCP Portsweep
- UDP Portsweep
- IP Portsweep
- ICMP Portsweep

These alerts are for one$\rightarrow$many portsweeps. One host scans a single port on multiple hosts. This usually occurs when a new exploit comes out and the attacker is looking for a specific service.

Note:

The characteristics of a portsweep scan may not result in many negative responses. For example, if an attacker portsweeps a web farm for port 80, we will most likely not see many negative responses.

sfPortscan alerts on the following filtered portscans and portsweeps:

- TCP Filtered Portscan
- UDP Filtered Portscan
- IP Filtered Portscan

- TCP Filtered Decoy Portscan
- UDP Filtered Decoy Portscan
- IP Filtered Decoy Portscan

- TCP Filtered Portsweep
- UDP Filtered Portsweep
- IP Filtered Portsweep
- ICMP Filtered Portsweep

- TCP Filtered Distributed Portscan
- UDP Filtered Distributed Portscan
- IP Filtered Distributed Portscan

“Filtered” alerts indicate that there were no network errors (ICMP unreachables or TCP RSTs) or responses on closed ports have been suppressed. It's also a good indicator of whether the alert is just a very active legitimate host. Active hosts, such as NATs, can trigger these alerts because they can send out many connection attempts within a very small amount of time. A filtered alert may go off before responses from the remote hosts are received.

sfPortscan only generates one alert for each host pair in question during the time window (more on windows below). On TCP scan alerts, sfPortscan will also display any open ports that were scanned. On TCP sweep alerts however, sfPortscan will only track open ports after the alert has been triggered. Open port events are not individual alerts, but tags based on the original scan alert.

2.2.4.1 sfPortscan Configuration

Use of the Stream preprocessor is required for sfPortscan. Stream gives portscan direction in the case of connectionless protocols like ICMP and UDP. You should enable the Stream preprocessor in your `snort.conf`, as described in Section [*].

The parameters you can use to configure the portscan module are:

8.

proto $<$protocol$>$

Available options:

- `TCP`
- `UDP`
- `ICMP`
- `ip_proto`
- `all`

9.

scan_type $<$scan_type$>$

Available options:

- `portscan`
- `portsweep`
- `decoy_portscan`
- `distributed_portscan`
- `all`

10.

sense_level $<$level$>$

Available options:

- `low` - “Low” alerts are only generated on error packets sent from the target host, and because of the nature of error responses, this setting should see very few false positives. However, this setting will never trigger a Filtered Scan alert because of a lack of error responses. This setting is based on a static time window of 60 seconds, after which this window is reset.

- `medium` - “Medium” alerts track connection counts, and so will generate filtered scan alerts. This setting may false positive on active hosts (NATs, proxies, DNS caches, etc), so the user may need to deploy the use of Ignore directives to properly tune this directive.

- `high` - “High” alerts continuously track hosts on a network using a time window to evaluate portscan statistics for that host. A "High" setting will catch some slow scans because of the continuous monitoring, but is very sensitive to active hosts. This most definitely will require the user to tune sfPortscan.

11.

watch_ip $<$ip1$\vert$ip2/cidr[[port$\vert$port2-port3]]$>$

Defines which IPs, networks, and specific ports on those hosts to watch. The list is a comma separated list of IP addresses, IP address using CIDR notation. Optionally, ports are specified after the IP address/CIDR using a space and can be either a single port or a range denoted by a dash. IPs or networks not falling into this range are ignored if this option is used.

12. ignore_scanners $<$ip1$\vert$ip2/cidr[[port$\vert$port2-port3]]$>$

    Ignores the source of scan alerts. The parameter is the same format as that of `watch_ip`.

13. ignore_scanned $<$ip1$\vert$ip2/cidr[[port$\vert$port2-port3]]$>$

    Ignores the destination of scan alerts. The parameter is the same format as that of `watch_ip`.

14. logfile $<$file$>$

    This option will output portscan events to the file specified. If `file` does not contain a leading slash, this file will be placed in the Snort config dir.

15. include_midstream

    This option will include sessions picked up in midstream by Stream. This can lead to false alerts, especially under heavy load with dropped packets; which is why the option is off by default.

16. detect_ack_scans

    This option will include sessions picked up in midstream by the stream module, which is necessary to detect ACK scans. However, this can lead to false alerts, especially under heavy load with dropped packets; which is why the option is off by default.

17. disabled

    This optional keyword is allowed with any policy to avoid packet processing. This option disables the preprocessor. When the preprocessor is disabled only the memcap option is applied when specified with the configuration. The other options are parsed but not used. Any valid configuration may have "disabled" added to it.

2.2.4.2 Format

```
preprocessor sfportscan: proto <protocols> \
    scan_type <portscan|portsweep|decoy_portscan|distributed_portscan|all> \
    sense_level <low|medium|high> \
    watch_ip <IP or IP/CIDR> \
    ignore_scanners <IP list> \
```

```
        ignore_scanned <IP list> \
        logfile <path and filename> \
        disabled
```

2.2.4.3 Example

```
    preprocessor flow: stats_interval 0 hash 2
    preprocessor sfportscan:\
        proto { all } \
        scan_type { all } \
        sense_level { low }
```

2.2.4.4 sfPortscan Alert Output

2.2.4.4.1 Unified Output

In order to get all the portscan information logged with the alert, snort generates a pseudo-packet and uses the payload portion to store the additional portscan information of priority count, connection count, IP count, port count, IP range, and port range. The characteristics of the packet are:

```
    Src/Dst MAC Addr == MACDAD
    IP Protocol == 255
    IP TTL == 0
```

Other than that, the packet looks like the IP portion of the packet that caused the portscan alert to be generated. This includes any IP options, etc. The payload and payload size of the packet are equal to the length of the additional portscan information that is logged. The size tends to be around 100 - 200 bytes.

Open port alerts differ from the other portscan alerts, because open port alerts utilize the tagged packet output system. This means that if an output system that doesn't print tagged packets is used, then the user won't see open port alerts. The open port information is stored in the IP payload and contains the port that is open.

The sfPortscan alert output was designed to work with unified2 packet logging, so it is possible to extend favorite Snort GUIs to display portscan alerts and the additional information in the IP payload using the above packet characteristics.

2.2.4.4.2 Log File Output

Log file output is displayed in the following format, and explained further below:

```
    Time: 09/08-15:07:31.603880
    event_id: 2
    192.168.169.3 -> 192.168.169.5 (portscan) TCP Filtered Portscan
    Priority Count: 0
    Connection Count: 200
    IP Count: 2
    Scanner IP Range: 192.168.169.3:192.168.169.4
```

```
Port/Proto Count: 200
Port/Proto Range: 20:47557
```

If there are open ports on the target, one or more additional tagged packet(s) will be appended:

```
Time: 09/08-15:07:31.603881
event_ref: 2
192.168.169.3 -> 192.168.169.5 (portscan) Open Port
Open Port: 38458
```

18.

Event_id/Event_ref

These fields are used to link an alert with the corresponding `Open Port` tagged packet

19.

Priority Count

`Priority Count` keeps track of bad responses (resets, unreachables). The higher the priority count, the more bad responses have been received.

20.

Connection Count

`Connection Count` lists how many connections are active on the hosts (src or dst). This is accurate for connection-based protocols, and is more of an estimate for others. Whether or not a portscan was filtered is determined here. High connection count and low priority count would indicate filtered (no response received from target).

21.

IP Count

IP Count keeps track of the last IP to contact a host, and increments the count if the next IP is different. For one-to-one scans, this is a low number. For active hosts this number will be high regardless, and one-to-one scans may appear as a distributed scan.

22.

Scanned/Scanner IP Range

This field changes depending on the type of alert. Portsweep (one-to-many) scans display the scanned IP range; Portscans (one-to-one) display the scanner IP.

23.

Port Count

Port Count keeps track of the last port contacted and increments this number when that changes. We use this count (along with IP Count) to determine the difference between one-to-one portscans and one-to-one decoys.

2.2.4.5 Tuning sfPortscan

The most important aspect in detecting portscans is tuning the detection engine for your network(s). Here are some tuning tips:

24.

Use the watch_ip, ignore_scanners, and ignore_scanned options.

It's important to correctly set these options. The `watch_ip` option is easy to understand. The analyst should set this option to the list of CIDR blocks and IPs that they want to watch. If no `watch_ip` is defined, sfPortscan will watch all network traffic.

The `ignore_scanners` and `ignore_scanned` options come into play in weeding out legitimate hosts that are very active on your network. Some of the most common examples are NAT IPs, DNS cache servers, syslog servers, and nfs servers. sfPortscan may not generate false positives for these types of hosts, but be aware when first tuning sfPortscan for these IPs. Depending on the type of alert that the host generates, the analyst will know which to ignore it as. If the host is generating portsweep events, then add it to the `ignore_scanners` option. If the host is generating portscan alerts (and is the host that is being scanned), add it to the `ignore_scanned` option.

25.

Filtered scan alerts are much more prone to false positives.

When determining false positives, the alert type is very important. Most of the false positives that sfPortscan may generate are of the filtered scan alert type. So be much more suspicious of filtered portscans. Many times this just indicates that a host was very active during the time period in question. If the host continually generates these types of alerts, add it to the `ignore_scanners` list or use a lower sensitivity level.

26.

Make use of the Priority Count, Connection Count, IP Count, Port Count, IP Range, and Port Range to determine false positives.

The portscan alert details are vital in determining the scope of a portscan and also the confidence of the portscan. In the future, we hope to automate much of this analysis in assigning a scope level and confidence level, but for now the user must manually do this. The easiest way to determine false positives is through simple ratio estimations. The following is a list of ratios to estimate and the associated values that indicate a legitimate scan and not a false positive.

Connection Count / IP Count: This ratio indicates an estimated average of connections per IP. For portscans, this ratio should be high, the higher the better. For portsweeps, this ratio should be low.

Port Count / IP Count: This ratio indicates an estimated average of ports connected to per IP. For portscans, this ratio should be high and indicates that the scanned host's ports were connected to by fewer IPs. For portsweeps, this ratio should be low, indicating that the scanning host connected to few ports but on many hosts.

Connection Count / Port Count: This ratio indicates an estimated average of connections per port. For portscans, this ratio should be low. This indicates that each connection was to a different port. For portsweeps, this ratio should be high. This indicates that there were many connections to the same port.

The reason that `Priority Count` is not included, is because the priority count is included in the connection count and the above comparisons take that into consideration. The Priority Count play an important role in tuning because the higher the priority count the more likely it is a real portscan or portsweep (unless the host is firewalled).

27.

If all else fails, lower the sensitivity level.

If none of these other tuning techniques work or the analyst doesn't have the time for tuning, lower the sensitivity level. You get the best protection the higher the sensitivity level, but it's also important that the portscan detection engine generate alerts that the analyst will find informative. The low sensitivity level only generates alerts based on error responses. These responses indicate a portscan and the alerts generated by the low sensitivity level are highly accurate and require the least tuning. The low sensitivity level does not catch filtered scans; since these are more prone to false positives.

2.2.5 RPC Decode

The rpc_decode preprocessor normalizes RPC multiple fragmented records into a single un-fragmented record. It does this by normalizing the packet into the packet buffer. If stream5 is enabled, it will only process client-side traffic. By default, it runs against traffic on ports 111 and 32771.

2.2.5.1 Format

```
preprocessor rpc_decode: \
    <ports> [ alert_fragments ] \
    [no_alert_multiple_requests] \
    [no_alert_large_fragments] \
    [no_alert_incomplete]
```

Option	Description
`alert_fragments`	Alert on any fragmented RPC record.
`no_alert_multiple_requests`	Don't alert when there are multiple records in one packet.
`no_alert_large_fragments`	Don't alert when the sum of fragmented records exceeds one packet.

`no_alert_incomplete`	Don't alert when a single fragment record exceeds the size of one packet.

2.2.6 Performance Monitor

This preprocessor measures Snort's real-time and theoretical maximum performance. Whenever this preprocessor is turned on, it should have an output mode enabled, either “console” which prints statistics to the console window or “file” with a file name, where statistics get printed to the specified file name. By default, Snort's real-time statistics are processed. This includes:

- Time Stamp
- Drop Rate
- Mbits/Sec (wire) [duplicated below for easy comparison with other rates]
- Alerts/Sec
- K-Pkts/Sec (wire) [duplicated below for easy comparison with other rates]
- Avg Bytes/Pkt (wire) [duplicated below for easy comparison with other rates]
- Pat-Matched [percent of data received that Snort processes in pattern matching]
- Syns/Sec
- SynAcks/Sec
- New Sessions Cached/Sec
- Sessions Del fr Cache/Sec
- Current Cached Sessions
- Max Cached Sessions
- Stream Flushes/Sec
- Stream Session Cache Faults
- Stream Session Cache Timeouts
- New Frag Trackers/Sec
- Frag-Completes/Sec
- Frag-Inserts/Sec
- Frag-Deletes/Sec
- Frag-Auto Deletes/Sec [memory DoS protection]
- Frag-Flushes/Sec
- Frag-Current [number of current Frag Trackers]
- Frag-Max [max number of Frag Trackers at any time]
- Frag-Timeouts
- Frag-Faults
- Number of CPUs [*** Only if compiled with LINUX_SMP ***, the next three appear for each CPU]
- CPU usage (user)
- CPU usage (sys)
- CPU usage (Idle)
- Mbits/Sec (wire) [average mbits of total traffic]
- Mbits/Sec (ipfrag) [average mbits of IP fragmented traffic]
- Mbits/Sec (ipreass) [average mbits Snort injects after IP reassembly]

- Mbits/Sec (tcprebuilt) [average mbits Snort injects after TCP reassembly]
- Mbits/Sec (applayer) [average mbits seen by rules and protocol decoders]
- Avg Bytes/Pkt (wire)
- Avg Bytes/Pkt (ipfrag)
- Avg Bytes/Pkt (ipreass)
- Avg Bytes/Pkt (tcprebuilt)
- Avg Bytes/Pkt (applayer)
- K-Pkts/Sec (wire)
- K-Pkts/Sec (ipfrag)
- K-Pkts/Sec (ipreass)
- K-Pkts/Sec (tcprebuilt)
- K-Pkts/Sec (applayer)
- Total Packets Received
- Total Packets Dropped (not processed)
- Total Packets Blocked (inline)
- Percentage of Packets Dropped
- Total Filtered TCP Packets
- Total Filtered UDP Packets
- Midstream TCP Sessions/Sec
- Closed TCP Sessions/Sec
- Pruned TCP Sessions/Sec
- TimedOut TCP Sessions/Sec
- Dropped Async TCP Sessions/Sec
- TCP Sessions Initializing
- TCP Sessions Established
- TCP Sessions Closing
- Max TCP Sessions (interval)
- New Cached UDP Sessions/Sec
- Cached UDP Ssns Del/Sec
- Current Cached UDP Sessions
- Max Cached UDP Sessions
- Current Attribute Table Hosts (Target Based)
- Attribute Table Reloads (Target Based)
- Mbits/Sec (Snort)
- Mbits/Sec (sniffing)
- Mbits/Sec (combined)
- uSeconds/Pkt (Snort)
- uSeconds/Pkt (sniffing)
- uSeconds/Pkt (combined)
- KPkts/Sec (Snort)
- KPkts/Sec (sniffing)
- KPkts/Sec (combined)

There are over 100 individual statistics included. A header line is output at startup and rollover that labels each column.

The following options can be used with the performance monitor:

- `flow` - Prints out statistics about the type and amount of traffic and protocol distributions that Snort is seeing. This option can produce large amounts of output.

- `flow-file` - Prints `flow` statistics in a comma-delimited format to the file that is specified.
    - Timestamp
    - Total % TCP bytes
    - Total % UDP bytes
    - Total % ICMP bytes
    - Total % OTHER bytes
    - Number of Packet length entries
    - Packet length entries - bytes,%total
    - Number of TCP port flow entries
    - TCP port flow entries : port,%total,%src,%dst
    - % TCP high port to high port
    - Number of UDP port flow entries
    - UDP port flow entries : port,%total,%src,%dst
    - % UDP high port to high port
    - Number of ICMP type entries
    - ICMP type entries : type,%total

  Specifying this option implicitly enables `flow` statistics.

- `events` - Turns on event reporting. This prints out statistics as to the number of rules that were evaluated and didn't match (non-qualified events) vs. the number of rules that were evaluated and matched (qualified events). A high non-qualified event to qualified event ratio can indicate there are many rules with either minimal content or no content that are being evaluated without success. The fast pattern matcher is used to select a set of rules for evaluation based on the longest `content` or a `content` modified with the `fast_pattern` rule option in a rule. Rules with short, generic contents are more likely to be selected for evaluation than those with longer, more unique contents. Rules without `content` are not filtered via the fast pattern matcher and are always evaluated, so if possible, adding a `content` rule option to those rules can decrease the number of times they need to be evaluated and improve performance.

- `max` - Turns on the theoretical maximum performance that Snort calculates given the processor speed and current performance. This is only valid for uniprocessor machines, since many operating systems don't keep accurate kernel statistics for multiple CPUs.

- `console` - Prints statistics at the console.

- `file` - Prints statistics in a comma-delimited format to the file that is specified. Not all statistics are output to this file. You may also use `snortfile` which will output into your defined Snort log directory. Both of these directives can be overridden on the command line with the `-z` or `-perfmon-file` options. At startup, Snort will log a distinctive line to this file with a timestamp to all readers to easily identify gaps in the stats caused by Snort not running.

- `pktcnt` - Adjusts the number of packets to process before checking for the time sample. This boosts performance, since checking the time sample reduces Snort's performance. By default, this is 10000.

- `time` - Represents the number of seconds between intervals.
- `accumulate` or `reset` - Defines which type of drop statistics are kept by the operating system. By default, `reset` is used.
- `atexitonly` - Dump stats for entire life of Snort. One or more of the following arguments can be given to specify specific statistic types to dump at exit:
    - `base-stats`
    - `flow-stats`
    - `flow-ip-stats`
    - `events-stats`

  Without any arguments, all enabled stats will be dumped only when Snort exits.
- `max_file_size` - Defines the maximum size of the comma-delimited file. Before the file exceeds this size, it will be rolled into a new date stamped file of the format YYYY-MM-DD, followed by YYYY-MM-DD.x, where x will be incremented each time the comma delimited file is rolled over. The minimum is 4096 bytes and the maximum is 2147483648 bytes (2GB). The default is the same as the maximum.
- `flow-ip` - Collects IP traffic distribution statistics based on host pairs. For each pair of hosts for which IP traffic has been seen, the following statistics are collected for both directions (A to B and B to A):
    - TCP Packets
    - TCP Traffic in Bytes
    - TCP Sessions Established
    - TCP Sessions Closed
    - UDP Packets
    - UDP Traffic in Bytes
    - UDP Sessions Created
    - Other IP Packets
    - Other IP Traffic in Bytes

  These statistics are printed and reset at the end of each interval.
- `flow-ip-file` - Prints the flow IP statistics in a comma-delimited format to the file that is specified. All of the statistics mentioned above, as well as the IP addresses of the host pairs in human-readable format, are included.

  Each line in the file will have its values correspond (in order) to those below:

    - IP Address A (String)
    - IP Address B (String)
    - TCP Packets from A to B
    - TCP Traffic in Bytes from A to B
    - TCP Packets from B to A
    - TCP Traffic in Bytes from B to A
    - UDP Packets from A to B
    - UDP Traffic in Bytes from A to B
    - UDP Packets from B to A
    - UDP Traffic in Bytes from B to A

    - Other IP Packets from A to B
    - Other IP Traffic in Bytes from A to B
    - Other IP Packets from B to A
    - Other IP Traffic in Bytes from B to A
    - TCP Sessions Established
    - TCP Sessions Closed
    - UDP Sessions Created

- `flow-ip-memcap` - Sets the memory cap on the hash table used to store IP traffic statistics for host pairs. Once the cap has been reached, the table will start to prune the statistics for the least recently seen host pairs to free memory. This value is in bytes and the default value is 52428800 (50MB).

2.2.6.1 Examples

```
    preprocessor perfmonitor: \
        time 30 events flow file stats.profile max console pktcnt 10000

    preprocessor perfmonitor: \
        time 300 file /var/tmp/snortstat pktcnt 10000

    preprocessor perfmonitor: \
        time 30 flow-ip flow-ip-file flow-ip-stats.csv pktcnt 1000

    preprocessor perfmonitor: \
        time 30 pktcnt 1000 snortfile base.csv flow-file flows.csv atexitonly flow-
stats

    preprocessor perfmonitor: \
        time 30 pktcnt 1000 flow events atexitonly base-stats flow-stats console
```

2.2.7 HTTP Inspect

HTTP Inspect is a generic HTTP decoder for user applications. Given a data buffer, HTTP Inspect will decode the buffer, find HTTP fields, and normalize the fields. HTTP Inspect works on both client requests and server responses.

HTTP Inspect has a very “rich” user configuration. Users can configure individual HTTP servers with a variety of options, which should allow the user to emulate any type of web server. Within HTTP Inspect, there are two areas of configuration: global and server.

2.2.7.1 Global Configuration

The global configuration deals with configuration options that determine the global functioning of HTTP Inspect. The following example gives the generic global configuration format:

2.2.7.2 Format

```
preprocessor http_inspect: \
    global \
    iis_unicode_map <map_filename> \
    codemap <integer> \
    [detect_anomalous_servers] \
    [proxy_alert] \
    [max_gzip_mem <num>] \
    [compress_depth <num>] [decompress_depth <num>] \
    [memcap <num>] \
    disabled
```

You can only have a single global configuration, you'll get an error if you try otherwise.

2.2.7.2.1 Configuration

28.

`iis_unicode_map $<$map_filename$>$ [codemap $<$integer$>$]`

This is the global `iis_unicode_map` file. The `iis_unicode_map` is a required configuration parameter. The map file can reside in the same directory as `snort.conf` or be specified via a fully-qualified path to the map file.

The `iis_unicode_map` file is a Unicode codepoint map which tells HTTP Inspect which codepage to use when decoding Unicode characters. For US servers, the codemap is usually 1252.

A Microsoft US Unicode codepoint map is provided in the Snort source `etc` directory by default. It is called `unicode.map` and should be used if no other codepoint map is available. A tool is supplied with Snort to generate custom Unicode `maps-ms_unicode_generator.c`, which is available at http://www.snort.org/dl/contrib/.

> **Note:**
>
> Remember that this configuration is for the global IIS Unicode map, individual servers can reference their own IIS Unicode map.

29.

`detect_anomalous_servers`

This global configuration option enables generic HTTP server traffic inspection on non-HTTP configured ports, and alerts if HTTP traffic is seen. Don't turn this on if you don't have a default server configuration that encompasses all of the HTTP server ports that your users might access. In the future, we want to limit this to specific networks so it's more useful, but for right now, this inspects all network traffic. This option is turned off by default.

30.

`proxy_alert`

This enables global alerting on HTTP server proxy usage. By configuring HTTP Inspect servers and enabling `allow_proxy_use`, you will only receive proxy use alerts for web users that aren't using the configured proxies or are using a rogue proxy server.

Please note that if users aren't required to configure web proxy use, then you may get a lot of proxy alerts. So, please only use this feature with traditional proxy environments. Blind firewall proxies don't count.

31.

`compress_depth $<$integer$>$` This option specifies the maximum amount of packet payload to decompress. This value can be set from 1 to 65535. The default for this option is 1460.

> **Note:**
>
> Please note, in case of multiple policies, the value specified in the default policy is used and this value overwrites the values specified in the other policies. In case of `unlimited_decompress` this should be set to its max value. This value should be specified in the default policy even when the HTTP inspect preprocessor is turned off using the `disabled` keyword.

32.

`decompress_depth $<$integer$>$` This option specifies the maximum amount of decompressed data to obtain from the compressed packet payload. This value can be set from 1 to 65535. The default for this option is 2920.

> **Note:**
>
> Please note, in case of multiple policies, the value specified in the default policy is used and this value overwrites the values specified in the other policies. In case of `unlimited_decompress` this should be set to its max value. This value should be specified in the default policy even when the HTTP inspect preprocessor is turned off using the `disabled` keyword.

33.

`max_gzip_mem $<$integer$>$`

This option determines (in bytes) the maximum amount of memory the HTTP Inspect preprocessor will use for decompression. The minimum allowed value for this option is 3276 bytes. This option determines the number of concurrent sessions that can be decompressed at any given instant. The default value for this option is 838860.

This value is also used for the optional SWF/PDF file decompression. If these modes are enabled this same value sets the maximum about of memory used for file decompression session state information.

> **Note:**
>
> This value should be specified in the default policy even when the HTTP inspect preprocessor is turned off using the `disabled` keyword.

34.

```
memcap $<$integer$>$
```

This option determines (in bytes) the maximum amount of memory the HTTP Inspect preprocessor will use for logging the URI and Hostname data. This value can be set from 2304 to 603979776 (576 MB). This option along with the maximum uri and hostname logging size (which is defined in snort) will determine the maximum HTTP sessions that will log the URI and hostname data at any given instant. The maximum size for logging URI data is 2048 and for hostname is 256. The default value for this option is 150994944 (144 MB).

> **Note:**
>
> This value should be specified in the default policy even when the HTTP inspect preprocessor is turned off using the `disabled` keyword. In case of multiple policies, the value specified in the default policy will overwrite the value specified in other policies.
>
> max http sessions logged = memcap /(max uri logging size + max hostname logging size) max uri logging size defined in snort : 2048 max hostname logging size defined in snort : 256

35.

```
normalize_random_nulls_in_text
```

This option normalizes the text content with randomly encoded null bytes in 16LE,16BE,32LE and 32BE UTF encodings to UTF8 in the server response. It relies on file preprocessor to determine if the content is text. Hence file preprocessor should be enabled and configured with prepackaged file magics wihtout which this option is not effective.

> **Note:** This opton relies on file prepreprocessor to determine if content can safely be considered as text before normalizing. However, it is possible that non text file types unknown to file preprocessor may get normalized as this option treats file types unknown to file preprocessor as text. Such cases may result in false positives or false negatives in detection.

36.

```
fast_blocking
```

This option enables inspecting http data before the data is flushed. This enables early IPS rule evaluation so that the block rules will take into effect and the connection is blocked at the earliest instead of blocking later after flushing the data. This config will be effective only when inline normalisation is enabled.

37.

```
disabled
```

This optional keyword is allowed with any policy to avoid packet processing. This option disables the preprocessor. When the preprocessor is disabled only the "memcap", "max_gzip_mem", "compress_depth" and "decompress_depth" options are applied when specified with the configuration. Other options are parsed but not used. Any valid configuration may have "disabled" added to it.

2.2.7.3 Example Global Configuration

```
preprocessor http_inspect: \
    global iis_unicode_map unicode.map 1252
```

2.2.7.4 Server Configuration

There are two types of server configurations: default and by IP address.

2.2.7.4.1 Default

This configuration supplies the default server configuration for any server that is not individually configured. Most of your web servers will most likely end up using the default configuration.

2.2.7.5 Example Default Configuration

```
preprocessor http_inspect_server: \
    server default profile all ports { 80 }
```

2.2.7.5.1 Configuration by IP Address

This format is very similar to “default”, the only difference being that specific IPs can be configured.

2.2.7.6 Example IP Configuration

```
preprocessor http_inspect_server: \
    server 10.1.1.1 profile all ports { 80 }
```

2.2.7.6.1 Configuration by Multiple IP Addresses

This format is very similar to “Configuration by IP Address”, the only difference being that multiple IPs can be specified via a space separated list. There is a limit of 40 IP addresses or CIDR notations per `http_inspect_server` line.

2.2.7.7 Example Multiple IP Configuration

```
preprocessor http_inspect_server: \
    server { 10.1.1.1 10.2.2.0/24 } profile all ports { 80 }
```

2.2.7.8 Server Configuration Options

Important: Some configuration options have an argument of `yes' or `no'. This argument specifies whether the user wants the configuration option to generate an HTTP Inspect alert or not. The `yes/no' argument does not specify whether the configuration option itself is on or off, only the alerting functionality. In other words, whether set to `yes' or 'no', HTTP normalization will still occur, and rules based on HTTP traffic will still trigger.

38.

```
profile $<$all$\vert$apache$\vert$iis$\vert$iis5_0$\vert$iis4_0$>$
```

Users can configure HTTP Inspect by using pre-defined HTTP server profiles. Profiles allow the user to easily configure the preprocessor for a certain type of server, but are not required for proper operation.

There are five profiles available: all, apache, iis, iis5_0, and iis4_0.

37-A.

```
all
```

The `all` profile is meant to normalize the URI using most of the common tricks available. We alert on the more serious forms of evasions. This is a great profile for detecting all types of attacks, regardless of the HTTP server. `profile all` sets the configuration options described in Table [*].

Table: Options for the “all” Profile

Option	Setting
server_flow_depth	300
client_flow_depth	300
post_depth	0
chunk encoding	alert on chunks larger than 500000 bytes
iis_unicode_map	codepoint map in the global configuration
ASCII decoding	on, alert off
multiple slash	on, alert off
directory normalization	on, alert off
apache whitespace	on, alert off
double decoding	on, alert on
%u decoding	on, alert on
bare byte decoding	on, alert on
iis unicode codepoints	on, alert on
iis backslash	on, alert off
iis delimiter	on, alert off
webroot	on, alert on
non_strict URL parsing	on
tab_uri_delimiter	is set
max_header_length	0, header length not checked
max_spaces	200

max_headers	0, number of headers not checked

37-B.

`apache`

The `apache` profile is used for Apache web servers. This differs from the `iis` profile by only accepting UTF-8 standard Unicode encoding and not accepting backslashes as legitimate slashes, like IIS does. Apache also accepts tabs as whitespace. `profile apache` sets the configuration options described in Table [*].

Table: Options for the `apache` Profile

Option	Setting
server_flow_depth	300
client_flow_depth	300
post_depth	0
chunk encoding	alert on chunks larger than 500000 bytes
ASCII decoding	on, alert off
multiple slash	on, alert off
directory normalization	on, alert off
webroot	on, alert on
apache whitespace	on, alert on
utf_8 encoding	on, alert off
non_strict url parsing	on
tab_uri_delimiter	is set
max_header_length	0, header length not checked
max_spaces	200
max_headers	0, number of headers not checked

37-C.

`iis`

The `iis` profile mimics IIS servers. So that means we use IIS Unicode codemaps for each server, %u encoding, bare-byte encoding, double decoding, backslashes, etc. `profile iis` sets the configuration options described in Table [*].

Table: Options for the `iis` Profile

Option	Setting
server_flow_depth	300
client_flow_depth	300
post_depth	-1
chunk encoding	alert on chunks larger than 500000 bytes
iis_unicode_map	codepoint map in the global configuration
ASCII decoding	on, alert off
multiple slash	on, alert off
directory normalization	on, alert off
webroot	on, alert on
double decoding	on, alert on
%u decoding	on, alert on
bare byte decoding	on, alert on
iis unicode codepoints	on, alert on
iis backslash	on, alert off
iis delimiter	on, alert on
apache whitespace	on, alert on
non_strict URL parsing	on
max_header_length	0, header length not checked
max_spaces	200
max_headers	0, number of headers not checked

37-D.

`iis4_0, iis5_0`

In IIS 4.0 and IIS 5.0, there was a double decoding vulnerability. These two profiles are identical to `iis`, except they will alert by default if a URL has a double encoding. Double decode is not supported in IIS 5.1 and beyond, so it's disabled by default.

37-E.

`default, no profile`

The default options used by HTTP Inspect do not use a profile and are described in Table [*].

Table: Default HTTP Inspect Options

Option	Setting
port	80

server_flow_depth	300
client_flow_depth	300
post_depth	-1
chunk encoding	alert on chunks larger than 500000 bytes
ASCII decoding	on, alert off
utf_8 encoding	on, alert off
multiple slash	on, alert off
directory normalization	on, alert off
webroot	on, alert on
iis backslash	on, alert off
apache whitespace	on, alert off
iis delimiter	on, alert off
non_strict URL parsing	on
max_header_length	0, header length not checked
max_spaces	200
max_headers	0, number of headers not checked

Profiles must be specified as the first server option and cannot be combined with any other options except:

- `ports`
- `iis_unicode_map`
- `allow_proxy_use`
- `server_flow_depth`
- `client_flow_depth`
- `post_depth`
- `no_alerts`
- `inspect_uri_only`
- `oversize_dir_length`
- `normalize_headers`
- `normalize_cookies`
- `normalize_utf`
- `max_header_length`
- `max_spaces`
- `max_headers`
- `extended_response_inspection`
- `enable_cookie`
- `inspect_gzip`
- `unlimited_decompress`
- `normalize_javascript`

- `max_javascript_whitespaces`
- `enable_xff`
- `http_methods`
- `log_uri`
- `log_hostname`
- `small_chunk_length`
- `decompress_swf`
- `decompress_pdf`
- `legacy_mode`

These options must be specified after the `profile` option.

2.2.7.9 Example

```
preprocessor http_inspect_server: \
    server 1.1.1.1 profile all ports { 80 3128 }
```

`ports $\{ <$port$> [<$port$> <...>] \}$`

This is how the user configures which ports to decode on the HTTP server. However, HTTPS traffic is encrypted and cannot be decoded with HTTP Inspect. To ignore HTTPS traffic, use the SSL preprocessor.

`iis_unicode_map $<$map_filename$>$ codemap $<$integer$>$`

The IIS Unicode map is generated by the program ms_unicode_generator.c. This program is located on the Snort.org web site at `http://www.snort.org/dl/contrib/` directory. Executing this program generates a Unicode map for the system that it was run on. So, to get the specific Unicode mappings for an IIS web server, you run this program on that server and use that Unicode map in this configuration.

When using this option, the user needs to specify the file that contains the IIS Unicode map and also specify the Unicode map to use. For US servers, this is usually 1252. But the ms_unicode_generator program tells you which codemap to use for you server; it's the ANSI code page. You can select the correct code page by looking at the available code pages that the ms_unicode_generator outputs.

`extended_response_inspection`

This enables the extended HTTP response inspection. The default http response inspection does not inspect the various fields of a HTTP response. By turning this option the HTTP response will be thoroughly inspected. The different fields of a HTTP response such as status code, status message, headers, cookie (when enable_cookie is configured) and body are extracted and saved into buffers. Different rule options are provided to inspect these buffers.

This option must be enabled to make use of the decompress_swf or decompress_pdf options.

Note:

When this option is turned on, if the HTTP response packet has a body then any content pattern matches (without http modifiers) will search the response body ((decompressed in case of gzip) and not the entire packet

payload. To search for patterns in the header of the response, one should use the http modifiers with content such as `http_header`, `http_stat_code`, `http_stat_msg` and `http_cookie`.

`enable_cookie`

This options turns on the cookie extraction from HTTP requests and HTTP response. By default the cookie inspection and extraction will be turned off. The cookie from the `Cookie` header line is extracted and stored in HTTP Cookie buffer for HTTP requests and cookie from the `Set-Cookie` is extracted and stored in HTTP Cookie buffer for HTTP responses. The `Cookie:` and `Set-Cookie:` header names itself along with leading spaces and the CRLF terminating the header line are stored in the HTTP header buffer and are not stored in the HTTP cookie buffer.

```
Ex: Set-Cookie: mycookie \r\n

In this case, Set-Cookie: \r\n will be in the HTTP header buffer and the pattern
mycookie will be in the HTTP cookie buffer.
```

`inspect_gzip`

This option specifies the HTTP inspect module to uncompress the compressed data(gzip/deflate) in HTTP response. You should select the config option "extended_response_inspection" before configuring this option. Decompression is done across packets. So the decompression will end when either the 'compress_depth' or 'decompress_depth' is reached or when the compressed data ends. When the compressed data is spanned across multiple packets, the state of the last decompressed packet is used to decompressed the data of the next packet. But the decompressed data are individually inspected. (i.e. the decompressed data from different packets are not combined while inspecting). Also the amount of decompressed data that will be inspected depends on the 'server_flow_depth' configured.

Http Inspect generates a preprocessor alert with gid 120 and sid 6 when the decompression fails. When the decompression fails due to a CRC error encountered by zlib, HTTP Inspect will also provide the detection module with the data that was decompressed by zlib.

`unlimited_decompress`

This option enables the user to decompress unlimited gzip data (across multiple packets).Decompression will stop when the compressed data ends or when a out of sequence packet is received. To ensure unlimited decompression, user should set the 'compress_depth' and 'decompress_depth' to its maximum values in the default policy. The decompression in a single packet is still limited by the 'compress_depth' and 'decompress_depth'.

`decompress_swf $\{ mode [mode] \}$`

This option will enable decompression of compressed SWF (Adobe Flash content) files encountered as the HTTP Response body in a GET transaction. The available decompression modes are 'deflate' and 'lzma'. A prerequisite is enabling extended_response_inspection (described above). When enabled, the preprocessor will examine the response body for the corresponding file signature. 'CWS' for Deflate/ZLIB compressed and 'ZWS' for LZMA compressed. Each decompression mode can be individually enabled. e.g. ... lzma or deflate or lzma deflate . The compressed content is decompressed 'in-place' with the

content made available to the detection/rules 'file_data' option. If enabled and located, the compressed SWF file signature is converted to 'FWS' to indicate an uncompressed file.

The 'decompress_depth', 'compress_depth', and 'unlimited_decompress' are optionally used to place limits on the decompression process. The semantics for SWF files are similar to the gzip decompression process.

During the decompression process, the preprocessor may generate alert 120:12 if Deflate decompression fails or alert 120:13 if LZMA decompression fails.

> **Note:** LZMA decompression is only available if Snort is built with the liblzma package present and functional. If the LZMA package is not present, then the lzma option will indicate a fatal parsing error. If the liblzma package IS present, but one desires to disable LZMA support, then the -disable-lzma option on configure will disable usage of the library.

`decompress_pdf $\{ mode [mode] \}$`

This option will enable decompression of the compressed portions of PDF files encountered as the HTTP Response body in a GET transaction. A prerequisite is enabling extended_response_inspection (described above).

When enabled, the preprocessor will examine the response body for the 'PDF files are then parsed, locating PDF 'streams' with a single '/FlateDecode' filter. These streams are decompressed in-place, replacing the compressed content.

The 'decompress_depth', 'compress_depth', and 'unlimited_decompress' are optionally used to place limits on the decompression process. The semantics for PDF files are similar to the gzip decompression process.

During the file parsing/decompression process, the preprocessor may generate several alerts:

Alert	Description
120:14	Deflate decompression failure
120:15	Located a 'stream' with an unsupported compression ('/Filter') algorithm
120:16	Located a 'stream' with unsupported cascaded '/FlateDecode' options, e.g.: `/Filter [ /FlateDecode /FlateDecode ]`
120:17	PDF File parsing error

`normalize_javascript` This option enables the normalization of Javascript within the HTTP response body. You should select the config option `extended_response_inspection` before configuring this option. When this option is turned on, Http Inspect searches for a Javascript within the HTTP response body by searching for the $<$script$>$ tags and starts normalizing it. When Http Inspect sees the $<$script$>$ tag without a type, it is considered as a javascript. The obfuscated data within the javascript functions such as unescape, String.fromCharCode, decodeURI, decodeURIComponent will be normalized. The different encodings handled within the unescape/ decodeURI/decodeURIComponent are `%XX`, `%uXXXX`, `XX` and

`uXXXXi`. Apart from these encodings, Http Inspect will also detect the consecutive whitespaces and normalize it to a single space. Http Inspect will also normalize the plus and concatenate the strings. The rule option `file_data` can be used to access this normalized buffer from the rule. A preprocessor alert with SID 9 and GID 120 is generated when the obfuscation levels within the Http Inspect is equal to or greater than 2.

```
Example:

HTTP/1.1 200 OK\r\n
Date: Wed, 29 Jul 2009 13:35:26 GMT\r\n
Server: Apache/2.2.3 (Debian) PHP/5.2.0-8+etch10 mod_ssl/2.2.3 OpenSSL/0.9.8c\r\n
Last-Modified: Sun, 20 Jan 2008 12:01:21 GMT\r\n
Accept-Ranges: bytes\r\n
Content-Length: 214\r\n
Keep-Alive: timeout=15, max=99\r\n
Connection: Keep-Alive\r\n
Content-Type: application/octet-stream\r\n\r\n
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>FIXME</title>
</head>
<body>
<script>document.write(unescape(unescape("%48%65%6C%6C%6F%2C%20%73%6E%6F%72%74%20%74%65
%61%6D%21")));
</script>
</body>
</html>
```

The above javascript will generate the preprocessor alert with SID 9 and GIDF 120 when `normalize_javascript` is turned on.

Http Inspect will also generate a preprocessor alert with GID 120 and SID 11 when there are more than one type of encodings within the escaped/encoded data.

```
For example:

unescape("%48\x65%6C%6C%6F%2C%20%73%6E%6F%72%74%20%74%65%61%6D%21");
String.fromCharCode(0x48, 0x65, 0x6c, 0x6c, 111, 44, 32, 115, 110, 111, 114, 116, 32,
116, 101, 97, 109, 33)

\\end{verbatim}

The above obfuscation will generate the preprocessor alert with GID 120 and SID 11.

This option is turned off by default in HTTP Inspect.

\item \texttt{max\_javascript\_whitespaces $<$positive integer up to 65535$>$}
This option takes an integer as an argument.  The integer determines the maximum number
of consecutive whitespaces allowed within the Javascript obfuscated data in a HTTP
response body. The config option \texttt{normalize\_javascript} should be turned on
before configuring
 this config option. When the whitespaces in the javascript obfuscated data is equal to
```

```
or more
than this value a preprocessor alert with GID 120 and SID 10 is generated. The default
value for
this option is 200.  To enable, specify an integer argument to
\texttt{max\_javascript\_spaces} of 1 to 65535.
Specifying a value of 0 is treated as disabling the alert.

\item \texttt{enable\_xff}

This option enables Snort to parse and log the original client IP present in the
X-Forwarded-For or True-Client-IP HTTP request headers along with the generated
events. The XFF/True-Client-IP Original client IP address is logged only with
unified2 output and is not logged with console (-A cmg) output.

\item \texttt{xff\_headers}

If/When the \texttt{enable\_xff} option is present, the \texttt{xff\_headers} option
specifies a set of custom 'xff'
headers.  This option allows the definition of up to six custom headers in addition to
the
two default (and always present) X-Forwarded-For and True-Client-IP headers.  The
option
permits both the custom and default headers to be prioritized.  The headers/priority
pairs
are specified as a list.  Lower numerical values imply a higher priority.  The headers
do
not need to be specified in priority order.  Nor do the priorities need to be
contiguous.
Priority values can range from 1 to 255.  The priority values and header names must be
unique.
The header names must not collide with known http headers such as 'host', 'cookie',
'content-length', etc.

A example of the \texttt{xff\_header} syntax is:
\begin{verbatim}
xff_headers { [ x-forwarded-highest-priority 1 ] [ x-forwarded-second-highest-priority
2 ] \
              [ x-forwarded-lowest-priority-custom 3 ] }
```

The default X-Forwarded-For and True-Client-IP headers are always present. They may be explicitly specified in the `xff_headers` config in order to determine their priority. If not specified, they will be automatically added to the xff list as the lowest priority headers.

For example, let us say that we have the following (abbreviated) HTTP request header:

```
...
Host: www.snort.org
X-Forwarded-For: 192.168.1.1
X-Was-Originally-Forwarded-From: 10.1.1.1
...
```

With the default xff behavior (no `xff_headers`), the 'X-Forwarded-For' header would be used to provide a 192.168.1.1 Original Client IP address in the unified2 log. Custom headers are not parsed.

With:

```
xff_headers { [ x-was-originally-forwarded-from 1 ] [ x-another-forwarding-header 2 ] \
              [ x-forwarded-for 3 ] }
```

The X-Was-Originally-Forwarded-From header is the highest priority present and its value of 10.1.1.1 will be logged as the Original Client IP in the unified2 log.

But with:

```
xff_headers { [ x-was-originally-forwarded-from 3 ] [ x-another-forwarding-header 2 ] \
              [ x-forwarded-for 1 ] }
```

Now the X-Forwarded-For header is the highest priority and its value of 192.168.1.1 is logged.

> **Note:**
>
> The original client IP from XFF/True-Client-IP in unified2 logs can be viewed using the tool u2spewfoo. This tool is present in the tools/u2spewfoo directory of snort source tree.

`server_flow_depth $<$integer$>$`

This specifies the amount of server response payload to inspect. When `extended_response_inspection` is turned on, it is applied to the HTTP response body (decompressed data when `inspect_gzip` is turned on) and not the HTTP headers. When `extended_response_inspection` is turned off the `server_flow_depth` is applied to the entire HTTP response (including headers). Unlike `client_flow_depth` this option is applied per TCP session. This option can be used to balance the needs of IDS performance and level of inspection of HTTP server response data. Snort rules are targeted at HTTP server response traffic and when used with a small flow_depth value may cause false negatives. Most of these rules target either the HTTP header, or the content that is likely to be in the first hundred or so bytes of non-header data. Headers are usually under 300 bytes long, but your mileage may vary. It is suggested to set the `server_flow_depth` to its maximum value.

This value can be set from -1 to 65535. A value of -1 causes Snort to ignore all server side traffic for ports defined in `ports` when `extended_response_inspection` is turned off. When the `extended_response_inspection` is turned on, value of -1 causes Snort to ignore the HTTP response body data and not the HTTP headers. Inversely, a value of 0 causes Snort to inspect all HTTP server payloads defined in "ports" (note that this will likely slow down IDS performance). Values above 0 tell Snort the number of bytes to inspect of the server response (excluding the HTTP headers when `extended_response_inspection` is turned on) in a given HTTP session. Only packets payloads starting with 'HTTP' will be considered as the first packet of a server response. If less than flow_depth bytes are in the payload of the HTTP response packets in a given session, the entire payload will be inspected. If more than flow_depth bytes are in the payload of the HTTP response packet in a session only flow_depth bytes of the payload will be inspected for that session. Rules that are meant to inspect data in the payload of the HTTP response packets in a session beyond 65535 bytes will be ineffective unless flow_depth is set to 0. The default value for `server_flow_depth` is 300. Note that the 65535 byte maximum flow_depth applies to stream reassembled packets as well. It is suggested to set the `server_flow_depth` to its maximum value.

> **Note:**
>
> `server_flow_depth` is the same as the old `flow_depth` option, which will be deprecated in a future release.

`client_flow_depth $<$integer$>$`

This specifies the amount of raw client request payload to inspect. This value can be set from -1 to 1460. Unlike `server_flow_depth` this value is applied to the first packet of the HTTP request. It is not a session based flow depth. It has a default value of 300. It primarily eliminates Snort from inspecting larger HTTP Cookies that appear at the end of many client request Headers.

A value of -1 causes Snort to ignore all client side traffic for ports defined in "ports." Inversely, a value of 0 causes Snort to inspect all HTTP client side traffic defined in "ports" (note that this will likely slow down IDS performance). Values above 0 tell Snort the number of bytes to inspect in the first packet of the client request. If less than flow_depth bytes are in the TCP payload (HTTP request) of the first packet, the entire payload will be inspected. If more than flow_depth bytes are in the payload of the first packet only flow_depth bytes of the payload will be inspected. Rules that are meant to inspect data in the payload of the first packet of a client request beyond 1460 bytes will be ineffective unless flow_depth is set to 0. Note that the 1460 byte maximum flow_depth applies to stream reassembled packets as well. It is suggested to set the `client_flow_depth` to its maximum value.

`post_depth $<$integer$>$`

This specifies the amount of data to inspect in a client post message. The value can be set from -1 to 65495. The default value is -1. A value of -1 causes Snort to ignore all the data in the post message. Inversely, a value of 0 causes Snort to inspect all the client post message. This increases the performance by inspecting only specified bytes in the post message.

`ascii $<$yes$\vert$no$>$`

The `ascii` decode option tells us whether to decode encoded ASCII chars, a.k.a %2f = /, %2e = ., etc. It is normal to see ASCII encoding usage in URLs, so it is recommended that you disable HTTP Inspect alerting for this option.

`extended_ascii_uri`

This option enables the support for extended ASCII codes in the HTTP request URI. This option is turned off by default and is not supported with any of the profiles.

`utf_8 $<$yes$\vert$no$>$`

The `utf-8` decode option tells HTTP Inspect to decode standard UTF-8 Unicode sequences that are in the URI. This abides by the Unicode standard and only uses % encoding. Apache uses this standard, so for any Apache servers, make sure you have this option turned on. As for alerting, you may be interested in knowing when you have a UTF-8 encoded URI, but this will be prone to false positives as legitimate web clients use this type of encoding. When `utf_8` is enabled, ASCII decoding is also enabled to enforce correct functioning.

`u_encode $<$yes$\vert$no$>$`

This option emulates the IIS %u encoding scheme. How the %u encoding scheme works is as follows: the encoding scheme is started by a %u followed by 4 characters, like %uxxxx. The xxxx is a hex-encoded value that correlates to an IIS Unicode codepoint. This value can most definitely be ASCII. An ASCII character is encoded like %u002f = /, %u002e = ., etc. If no iis_unicode_map is specified before or after this option, the default codemap is used.

You should alert on %u encodings, because we are not aware of any legitimate clients that use this encoding. So it is most likely someone trying to be covert.

`bare_byte $<$yes$\vert$no$>$`

Bare byte encoding is an IIS trick that uses non-ASCII characters as valid values when decoding UTF-8 values. This is not in the HTTP standard, as all non-ASCII values have to be encoded with a %. Bare byte encoding allows the user to emulate an IIS server and interpret non-standard encodings correctly.

The alert on this decoding should be enabled, because there are no legitimate clients that encode UTF-8 this way since it is non-standard.

`iis_unicode $<$yes$\vert$no$>$`

The `iis_unicode` option turns on the Unicode codepoint mapping. If there is no iis_unicode_map option specified with the server config, `iis_unicode` uses the default codemap. The `iis_unicode` option handles the mapping of non-ASCII codepoints that the IIS server accepts and decodes normal UTF-8 requests.

You should alert on the `iis_unicode option`, because it is seen mainly in attacks and evasion attempts. When `iis_unicode` is enabled, ASCII and UTF-8 decoding are also enabled to enforce correct decoding. To alert on UTF-8 decoding, you must enable also enable `utf_8 yes`.

`double_decode $<$yes$\vert$no$>$`

The `double_decode` option is once again IIS-specific and emulates IIS functionality. How this works is that IIS does two passes through the request URI, doing decodes in each one. In the first pass, it seems that all types of iis encoding is done: utf-8 unicode, ASCII, bare byte, and %u. In the second pass, the following encodings are done: ASCII, bare byte, and %u. We leave out utf-8 because I think how this works is that the % encoded utf-8 is decoded to the Unicode byte in the first pass, and then UTF-8 is decoded in the second stage. Anyway, this is really complex and adds tons of different encodings for one character. When `double_decode` is enabled, so ASCII is also enabled to enforce correct decoding.

`non_rfc_char $\{ <$byte$> [<$byte ...$>] \}$`

This option lets users receive an alert if certain non-RFC chars are used in a request URI. For instance, a user may not want to see null bytes in the request URI and we can alert on that. Please use this option with care, because you could configure it to say, alert on all `/' or something like that. It's flexible, so be careful.

`multi_slash $<$yes$\vert$no$>$`

This option normalizes multiple slashes in a row, so something like: “foo////////bar” get normalized to “foo/bar.”

If you want an alert when multiple slashes are seen, then configure with a `yes`; otherwise, use `no`.

`iis_backslash $<$yes$\vert$no$>$`

Normalizes backslashes to slashes. This is again an IIS emulation. So a request URI of "/foo$ \backslash$bar" gets normalized to "/foo/bar."

`directory $<$yes$\vert$no$>$`

This option normalizes directory traversals and self-referential directories.

The directory:

```
    /foo/fake\_dir/../bar
```

gets normalized to:

```
    /foo/bar
```

The directory:

```
    /foo/./bar
```

gets normalized to:

```
    /foo/bar
```

If you want to configure an alert, specify `yes`, otherwise, specify `no`. This alert may give false positives, since some web sites refer to files using directory traversals.

`apache_whitespace $<$yes$\vert$no$>$`

This option deals with the non-RFC standard of using tab for a space delimiter. Apache uses this, so if the emulated web server is Apache, enable this option. Alerts on this option may be interesting, but may also be false positive prone.

`iis_delimiter $<$yes$\vert$no$>$`

This started out being IIS-specific, but Apache takes this non-standard delimiter was well. Since this is common, we always take this as standard since the most popular web servers accept it. But you can still get an alert on this option.

`chunk_length $<$non-zero positive integer$>$`

This option is an anomaly detector for abnormally large chunk sizes. This picks up the Apache chunk encoding exploits, and may also alert on HTTP tunneling that uses chunk encoding.

`small_chunk_length { $<$chunk size$>$ $<$consecutive chunks$>$ }`

This option is an evasion detector for consecutive small chunk sizes when either the client or server use `Transfer-Encoding: chunked`. $<$chunk size$>$ specifies the maximum chunk size for which a chunk

will be considered small. $<$consecutive chunks$>$ specifies the number of consecutive small chunks $<$= $<$chunk size$>$ before an event will be generated. This option is turned off by default. Maximum values for each are 255 and a $<$chunk size$>$ of 0 disables. Events generated are gid:119, sid:26 for client small chunks and gid:120, sid:7 for server small chunks.

Example:

```
small_chunk_length { 10 5 }
```

Meaning alert if we see 5 consecutive chunk sizes of 10 or less.

`no_pipeline_req`

This option turns HTTP pipeline decoding off, and is a performance enhancement if needed. By default, pipeline requests are inspected for attacks, but when this option is enabled, pipeline requests are not decoded and analyzed per HTTP protocol field. It is only inspected with the generic pattern matching.

`non_strict`

This option turns on non-strict URI parsing for the broken way in which Apache servers will decode a URI. Only use this option on servers that will accept URIs like this: "get /index.html alsjdfk alsj lj aj la jsj s$\backslash$n". The non_strict option assumes the URI is between the first and second space even if there is no valid HTTP identifier after the second space.

`allow_proxy_use`

By specifying this keyword, the user is allowing proxy use on this server. This means that no alert will be generated if the `proxy_alert` global keyword has been used. If the proxy_alert keyword is not enabled, then this option does nothing. The `allow_proxy_use` keyword is just a way to suppress unauthorized proxy use for an authorized server.

`no_alerts`

This option turns off all alerts that are generated by the HTTP Inspect preprocessor module. This has no effect on HTTP rules in the rule set. No argument is specified.

`oversize_dir_length $<$non-zero positive integer$>$`

This option takes a non-zero positive integer as an argument. The argument specifies the max char directory length for URL directory. If a url directory is larger than this argument size, an alert is generated. A good argument value is 300 characters. This should limit the alerts to IDS evasion type attacks, like whisker -i 4.

`inspect_uri_only`

This is a performance optimization. When enabled, only the URI portion of HTTP requests will be inspected for attacks. As this field usually contains 90-95% of the web attacks, you'll catch most of the attacks. So if you need extra performance, enable this optimization. It's important to note that if this option is used without any `uricontent` rules, then no inspection will take place. This is obvious since the URI is only inspected with `uricontent` rules, and if there are none available, then there is nothing to inspect.

For example, if we have the following rule set:

```
alert tcp any any -> any 80 ( msg:"content"; content: "foo"; )
```

and the we inspect the following URI:

```
get /foo.htm http/1.0\r\n\r\n
```

No alert will be generated when `inspect_uri_only` is enabled. The `inspect_uri_only` configuration turns off all forms of detection except `uricontent` inspection.

`max_header_length $<$positive integer up to 65535$>$`

This option takes an integer as an argument. The integer is the maximum length allowed for an HTTP client request header field. Requests that exceed this length will cause a "Long Header" alert. This alert is off by default. To enable, specify an integer argument to max_header_length of 1 to 65535. Specifying a value of 0 is treated as disabling the alert.

`max_spaces $<$positive integer up to 65535$>$`

This option takes an integer as an argument. The integer determines the maximum number of whitespaces allowed with HTTP client request line folding. Requests headers folded with whitespaces equal to or more than this value will cause a "Space Saturation" alert with SID 26 and GID 119. The default value for this option is 200. To enable, specify an integer argument to `max_spaces` of 1 to 65535. Specifying a value of 0 is treated as disabling the alert.

`webroot $<$yes$\vert$no$>$`

This option generates an alert when a directory traversal traverses past the web server root directory. This generates much fewer false positives than the directory option, because it doesn't alert on directory traversals that stay within the web server directory structure. It only alerts when the directory traversals go past the web server root directory, which is associated with certain web attacks.

`tab_uri_delimiter`

This option turns on the use of the tab character (0x09) as a delimiter for a URI. Apache accepts tab as a delimiter; IIS does not. For IIS, a tab in the URI should be treated as any other character. Whether this option is on or not, a tab is treated as whitespace if a space character (0x20) precedes it. No argument is specified.

`normalize_headers`

This option turns on normalization for HTTP Header Fields, not including Cookies (using the same configuration parameters as the URI normalization (i.e., multi-slash, directory, etc.). It is useful for normalizing Referrer URIs that may appear in the HTTP Header.

`normalize_cookies`

This option turns on normalization for HTTP Cookie Fields (using the same configuration parameters as the URI normalization (i.e., multi-slash, directory, etc.). It is useful for normalizing data in HTTP Cookies that may be encoded.

`normalize_utf`

This option turns on normalization of HTTP response bodies where the Content-Type header lists the character set as "utf-16le", "utf-16be", "utf-32le", or "utf-32be". HTTP Inspect will attempt to normalize these back into 8-bit encoding, generating an alert if the extra bytes are non-zero.

`max_headers $<$positive integer up to 1024$>$`

This option takes an integer as an argument. The integer is the maximum number of HTTP client request header fields. Requests that contain more HTTP Headers than this value will cause a "Max Header" alert. The alert is off by default. To enable, specify an integer argument to max_headers of 1 to 1024. Specifying a value of 0 is treated as disabling the alert.

`http_methods $\{ cmd [cmd] \}$` This specifies additional HTTP Request Methods outside of those checked by default within the preprocessor (GET and POST). The list should be enclosed within braces and delimited by spaces, tabs, line feed or carriage return. The config option, braces and methods also needs to be separated by braces.

```
    http_methods { PUT CONNECT }
```

> **Note:**
>
> Please note the maximum length for a method name is 256.

`log_uri`

This option enables HTTP Inspect preprocessor to parse the URI data from the HTTP request and log it along with all the generated events for that session. Stream reassembly needs to be turned on HTTP ports to enable the logging. If there are multiple HTTP requests in the session, the URI data of the most recent HTTP request during the alert will be logged. The maximum URI logged is 2048.

> **Note:**
>
> Please note, this is logged only with the unified2 output and is not logged with console output (-A cmg). `u2spewfoo` can be used to read this data from the unified2.

`log_hostname`

This option enables HTTP Inspect preprocessor to parse the hostname data from the "Host" header of the HTTP request and log it along with all the generated events for that session. Stream reassembly needs to be turned on HTTP ports to enable the logging. If there are multiple HTTP requests in the session, the Hostname data of the most recent HTTP request during the alert will be logged. In case of multiple "Host" headers within one HTTP request, a preprocessor alert with sid 24 is generated. The maximum hostname length logged is 256.

> **Note:**
>
> Please note, this is logged only with the unified2 output and is not logged with console output (-A cmg). `u2spewfoo` can be used to read this data from the unified2.

```
##########################################
# HTTP2 SUPPORT IS STILL EXPERIMENTAL!
# DO NOT USE IN PRODUCTION ENVIRONMENTS.
# Please send any issues to the Snort team
##########################################
```

`legacy_mode` By default, HTTP2 traffic is not supported. You can use "legacy_mode no" to enable HTTP2 support. If http legacy mode is configured, HTTP2 inspection is disabled.

2.2.7.10 Examples

```
    preprocessor http_inspect_server: \
        server 10.1.1.1 \
        ports { 80 3128 8080 } \
        server_flow_depth 0 \
        ascii no \
        double_decode yes \
        non_rfc_char { 0x00 } \
        chunk_length 500000 \
        non_strict \
        no_alerts

    preprocessor http_inspect_server: \
        server default \
        ports  { 80 3128 }  \
        non_strict \
        non_rfc_char  { 0x00 }  \
        server_flow_depth 300  \
        apache_whitespace yes \
        directory no \
        iis_backslash no \
        u_encode yes \
        ascii no \
        chunk_length 500000 \
        bare_byte yes \
        double_decode yes \
        iis_unicode yes \
        iis_delimiter yes \
        multi_slash no

    preprocessor http_inspect_server: \
        server default \
        profile all \
        ports { 80 8080 }
```

2.2.8 SMTP Preprocessor

The SMTP preprocessor is an SMTP decoder for user applications. Given a data buffer, SMTP will decode the buffer and find SMTP commands and responses. It will also mark the command, data header data body sections, and TLS data.

SMTP handles stateless and stateful processing. It saves state between individual packets. However maintaining correct state is dependent on the reassembly of the client side of the stream (i.e., a loss of coherent stream data results in a loss of state).

2.2.8.1 Configuration

SMTP has the usual configuration items, such as `port` and `inspection_type`. Also, SMTP command lines can be normalized to remove extraneous spaces. TLS-encrypted traffic can be ignored, which improves performance. In addition, regular mail data can be ignored for an additional performance boost. Since so few (none in the current snort rule set) exploits are against mail data, this is relatively safe to do and can improve the performance of data inspection.

The configuration options are described below:

39.

```
ports { <port> [<port>] ... }
```

This specifies on what ports to check for SMTP data. Typically, this will include 25 and possibly 465, for encrypted SMTP.

40.

```
inspection_type <stateful | stateless>
```

Indicate whether to operate in stateful or stateless mode.

41.

```
normalize <all | none | cmds>
```

This turns on normalization. Normalization checks for more than one space character after a command. Space characters are defined as space (ASCII 0x20) or tab (ASCII 0x09).

`all` checks all commands

`none` turns off normalization for all commands.

`cmds` just checks commands listed with the `normalize_cmds` parameter.

42.

```
ignore_data
```

Ignore data section of mail (except for mail headers) when processing rules.

43.

```
ignore_tls_data
```

Ignore TLS-encrypted data when processing rules.

44.

```
max_command_line_len <int>
```

Alert if an SMTP command line is longer than this value. Absence of this option or a "0" means never alert on command line length. RFC 2821 recommends 512 as a maximum command line length.

45.

```
max_header_line_len <int>
```

Alert if an SMTP DATA header line is longer than this value. Absence of this option or a "0" means never alert on data header line length. RFC 2821 recommends 1024 as a maximum data header line length.

46.

```
max_response_line_len <int>
```

Alert if an SMTP response line is longer than this value. Absence of this option or a "0" means never alert on response line length. RFC 2821 recommends 512 as a maximum response line length.

47.

```
alt_max_command_line_len <int> { <cmd> [<cmd>] }
```

Overrides `max_command_line_len` for specific commands.

48.

```
no_alerts
```

Turn off all alerts for this preprocessor.

49.

```
invalid_cmds { <Space-delimited list of commands> }
```

Alert if this command is sent from client side. Default is an empty list.

50.

```
valid_cmds { <Space-delimited list of commands> }
```

List of valid commands. We do not alert on commands in this list. Default is an empty list, but preprocessor has this list hard-coded:

> { ATRN AUTH BDAT DATA DEBUG EHLO EMAL ESAM ESND ESOM ETRN EVFY EXPN HELO HELP IDENT MAIL NOOP QUIT RCPT RSET SAML SOML SEND ONEX QUEU STARTTLS TICK TIME TURN TURNME VERB VRFY X-EXPS X-LINK2STATE XADR XAUTH XCIR XEXCH50 XGEN XLICENSE XQUE XSTA XTRN XUSR }

51.

```
data_cmds { <Space-delimited list of commands> }
```

List of commands that initiate sending of data with an end of data delimiter the same as that of the DATA command per RFC 5321 - "`<CRLF>.<CRLF>`". Default is { DATA }.

52.

```
binary_data_cmds { <Space-delimited list of commands> }
```

List of commands that initiate sending of data and use a length value after the command to indicate the amount of data to be sent, similar to that of the BDAT command per RFC 3030. Default is { BDAT XEXCH50 }.

53.

```
auth_cmds { <Space-delimited list of commands> }
```

List of commands that initiate an authentication exchange between client and server. Default is { AUTH XAUTH X-EXPS }.

54.

```
alert_unknown_cmds
```

Alert if we don't recognize command. Default is off.

55.

```
normalize_cmds { <Space-delimited list of commands> }
```

Normalize this list of commands Default is { RCPT VRFY EXPN }.

56.

```
xlink2state { enable | disable [drop] }
```

Enable/disable xlink2state alert. Drop if alerted. Default is `enable`.

57.

```
print_cmds
```

List all commands understood by the preprocessor. This not normally printed out with the configuration because it can print so much data.

58.

```
disabled
```

Disables the SMTP preprocessor in a config. This is useful when specifying the decoding depths such as `b64_decode_depth`, `qp_decode_depth`, `uu_decode_depth`, `bitenc_decode_depth` or the memcap used for decoding `max_mime_mem` in default config without turning on the SMTP preprocessor.

59.

```
b64_decode_depth
```

This config option is used to turn off/on or set the base64 decoding depth used to decode the base64 encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the

base64 decoding of MIME attachments. The value of 0 sets the decoding of base64 encoded MIME attachments to unlimited. A value other than 0 or -1 restricts the decoding of base64 MIME attachments, and applies per attachment. A SMTP preprocessor alert with sid 10 is generated (if enabled) when the decoding fails.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the base64 encoded MIME attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

This option replaces the deprecated options, `enable_mime_decoding` and `max_mime_depth`. It is recommended that user inputs a value that is a multiple of 4. When the value specified is not a multiple of 4, the SMTP preprocessor will round it up to the next multiple of 4.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

60.

`qp_decode_depth`

This config option is used to turn off/on or set the Quoted-Printable decoding depth used to decode the Quoted-Printable(QP) encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the QP decoding of MIME attachments. The value of 0 sets the decoding of QP encoded MIME attachments to unlimited. A value other than 0 or -1 restricts the decoding of QP MIME attachments, and applies per attachment. A SMTP preprocessor alert with sid 11 is generated (if enabled) when the decoding fails.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the QP encoded MIME attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

61.

`bitenc_decode_depth`

This config option is used to turn off/on or set the non-encoded MIME extraction depth used to extract the non-encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the extraction of these MIME attachments. The value of 0 sets the extraction of these MIME attachments to unlimited. A value other than 0 or -1 restricts the extraction of these MIME attachments, and applies per attachment.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the non-encoded MIME attachments/data across multiple packets are extracted too.

The extracted data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

62.

`uu_decode_depth`

This config option is used to turn off/on or set the Unix-to-Unix decoding depth used to decode the Unix-to-Unix(UU) encoded attachments. The value ranges from -1 to 65535. A value of -1 turns off the UU decoding of SMTP attachments. The value of 0 sets the decoding of UU encoded SMTP attachments to unlimited. A value other than 0 or -1 restricts the decoding of UU SMTP attachments, and applies per attachment. A SMTP preprocessor alert with sid 13 is generated (if enabled) when the decoding fails.

Multiple UU attachments/data in one packet are pipelined. When stateful inspection is turned on the UU encoded SMTP attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

63.

`enable_mime_decoding`

Enables Base64 decoding of Mime attachments/data. Multiple base64 encoded MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the base64 encoded MIME attachments/data across multiple packets are decoded too. The decoding of base64 encoded attachments/data ends when either the `max_mime_depth` or maximum MIME sessions (calculated using `max_mime_depth` and `max_mime_mem`) is reached or when the encoded data ends. The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

This option is deprecated. Use the option `b64_decode_depth` to turn off or on the base64 decoding instead.

64.

`max_mime_depth <int>`

Specifies the maximum number of base64 encoded data to decode per SMTP attachment. The option take values ranging from 4 to 20480 bytes. The default value for this in snort in 1460 bytes.

It is recommended that user inputs a value that is a multiple of 4. When the value specified is not a multiple of 4, the SMTP preprocessor will round it up to the next multiple of 4.

This option is deprecated. Use the option `b64_decode_depth` to turn off or on the base64 decoding instead.

65.

`max_mime_mem <int>`

This option determines (in bytes) the maximum amount of memory the SMTP preprocessor will use for decoding base64 encoded/quoted-printable/non-encoded MIME attachments/data or Unix-to-Unix encoded attachments. This value can be set from 3276 bytes to 100MB.

This option along with the maximum of the decoding depths will determine the SMTP sessions that will be decoded at any given instant. The default value for this option is 838860.

Note: It is suggested to set this value such that the max smtp session calculated as follows is at least 1.

max smtp session = `max_mime_mem` /(2 * max of (`b64_decode_depth`, `uu_decode_depth`, `qp_decode_depth` or `bitenc_decode_depth`))

For example, if `b64_decode_depth` is 0 (indicates unlimited decoding) and `qp_decode_depth` is 100, then

max smtp session = `max_mime_mem`/2*65535 (max value for `b64_decode_depth`)

In case of multiple configs, the `max_mime_mem` of the non-default configs will be overwritten by the default config's value. Hence user needs to define it in the default config with the new keyword disabled (used to disable SMTP preprocessor in a config).

66.

`log_mailfrom` This option enables SMTP preprocessor to parse and log the sender's email address extracted from the "MAIL FROM" command along with all the generated events for that session. The maximum number of bytes logged for this option is 1024.

Please note, this is logged only with the unified2 output and is not logged with console output (-A cmg). u2spewfoo can be used to read this data from the unified2.

67.

`log_rcptto` This option enables SMTP preprocessor to parse and log the recipient's email addresses extracted from the "RCPT TO" command along with all the generated events for that session. Multiple recipients are appended with commas. The maximum number of bytes logged for this option is 1024.

Please note, this is logged only with the unified2 output and is not logged with console output (-A cmg). u2spewfoo can be used to read this data from the unified2.

68.

`log_filename` This option enables SMTP preprocessor to parse and log the MIME attachment filenames extracted from the Content-Disposition header within the MIME body along with all the generated events for that session. Multiple filenames are appended with commas. The maximum number of bytes logged for this option is 1024.

Please note, this is logged only with the unified2 output and is not logged with the console output (-A cmg). u2spewfoo can be used to read this data from the unified2.

69.

`log_email_hdrs` This option enables SMTP preprocessor to parse and log the SMTP email headers extracted from SMTP data along with all generated events for that session. The number of bytes extracted and logged depends upon the `email_hdrs_log_depth`.

Please note, this is logged only with the unified2 output and is not logged with the console output (-A cmg). u2spewfoo can be used to read this data from the unified2.

70.

`email_hdrs_log_depth <int>` This option specifies the depth for logging email headers. The allowed range for this option is 0 - 20480. A value of 0 will disable email headers logging. The default value for this option is 1464.

Please note, in case of multiple policies, the value specified in the default policy is used and the values specified in the targeted policies are overwritten by the default value. This option must be configured in the default policy even if the SMTP configuration is disabled.

71.

`memcap <int>` This option determines in bytes the maximum amount of memory the SMTP preprocessor will use for logging of filename, MAIL FROM addresses, RCPT TO addresses and email headers. This value along with the buffer size used to log MAIL FROM, RCPT TO, filenames and `email_hdrs_log_depth` will determine the maximum SMTP sessions that will log the email headers at any given time. When this memcap is reached SMTP will stop logging the filename, MAIL FROM address, RCPT TO addresses and email headers until memory becomes available.

Max SMTP sessions logging email headers at any given time = memcap/(1024 + 1024 + 1024 + `email_hdrs_log_depth`)

The size 1024 is the maximum buffer size used for logging filename, RCPTTO and MAIL FROM addresses.

Default value for this option is 838860. The allowed range for this option is 3276 to 104857600. The value specified in the default config is used when this option is specified in multiple configs. This option must be configured in the default config even if the SMTP configuration is disabled.

Please note, in case of multiple policies, the value specified in the default policy is used and the values specified in the targeted policies are overwritten by the default value. This option must be configured in the default policy even if the SMTP configuration is disabled.

2.2.8.2 Example

```
preprocessor SMTP: \
    ports { 25 } \
    inspection_type stateful \
    normalize cmds \
    normalize_cmds { EXPN VRFY RCPT } \
    ignore_data \
    ignore_tls_data \
    max_command_line_len  512 \
    max_header_line_len   1024 \
    max_response_line_len 512 \
```

```
        no_alerts \
        alt_max_command_line_len 300 { RCPT } \
        invalid_cmds { } \
        valid_cmds { } \
        xlink2state { disable } \
        print_cmds \
        log_filename \
        log_email_hdrs \
        log_mailfrom \
        log_rcptto \
        email_hdrs_log_depth 2920 \
        memcap 6000

    preprocessor SMTP: \
        b64_decode_depth 0\
        max_mime_mem 4000 \
        memcap 6000 \
        email_hdrs_log_depth 2920 \
        disabled
```

2.2.8.3 Default

```
    preprocessor SMTP: \
        ports { 25 } \
        inspection_type stateful \
        normalize cmds \
        normalize_cmds { EXPN VRFY RCPT } \
        alt_max_command_line_len 260 { MAIL } \
        alt_max_command_line_len 300 { RCPT } \
        alt_max_command_line_len 500 { HELP HELO ETRN } \
        alt_max_command_line_len 255 { EXPN VRFY }
```

2.2.8.4 Note

`RCPT TO:` and `MAIL FROM:` are SMTP commands. For the preprocessor configuration, they are referred to as RCPT and MAIL, respectively. Within the code, the preprocessor actually maps RCPT and MAIL to the correct command name.

2.2.9 POP Preprocessor

POP is an POP3 decoder for user applications. Given a data buffer, POP will decode the buffer and find POP3 commands and responses. It will also mark the command, data header data body sections and extract the POP3 attachments and decode it appropriately.

POP will handle stateful processing. It saves state between individual packets. However maintaining correct state is dependent on the reassembly of the server side of the stream (i.e., a loss of coherent stream data results in a loss of state).

Stream should be turned on for POP. Please ensure that the POP ports are added to the stream5 ports for proper reassembly.

The POP preprocessor uses GID 142 to register events.

2.2.9.1 Configuration

The configuration options are described below:

72.

```
ports { <port> [<port>] ... }
```

This specifies on what ports to check for POP data. Typically, this will include 110. Default ports if none are specified are 110 .

73.

```
disabled
```

Disables the POP preprocessor in a config. This is useful when specifying the decoding depths such as `b64_decode_depth`, `qp_decode_depth`, `uu_decode_depth`, `bitenc_decode_depth` or the memcap used for decoding `memcap` in default config without turning on the POP preprocessor.

74.

```
b64_decode_depth
```

This config option is used to turn off/on or set the base64 decoding depth used to decode the base64 encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the base64 decoding of MIME attachments. The value of 0 sets the decoding of base64 encoded MIME attachments to unlimited. A value other than 0 or -1 restricts the decoding of base64 MIME attachments, and applies per attachment. A POP preprocessor alert with sid 4 is generated (if enabled) when the decoding fails.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the base64 encoded MIME attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

It is recommended that user inputs a value that is a multiple of 4. When the value specified is not a multiple of 4, the POP preprocessor will round it up to the next multiple of 4.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

75.

```
qp_decode_depth
```

This config option is used to turn off/on or set the Quoted-Printable decoding depth used to decode the Quoted-Printable(QP) encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the QP decoding of MIME attachments. The value of 0 sets the decoding of QP encoded MIME attachments to unlimited. A value other than 0 or -1 restricts the

decoding of QP MIME attachments, and applies per attachment. A POP preprocessor alert with sid 5 is generated (if enabled) when the decoding fails.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the QP encoded MIME attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

76.

`bitenc_decode_depth`

This config option is used to turn off/on or set the non-encoded MIME extraction depth used to extract the non-encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the extraction of these MIME attachments. The value of 0 sets the extraction of these MIME attachments to unlimited. A value other than 0 or -1 restricts the extraction of these MIME attachments, and applies per attachment.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the non-encoded MIME attachments/data across multiple packets are extracted too.

The extracted data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

77.

`uu_decode_depth`

This config option is used to turn off/on or set the Unix-to-Unix decoding depth used to decode the Unix-to-Unix(UU) encoded attachments. The value ranges from -1 to 65535. A value of -1 turns off the UU decoding of POP attachments. The value of 0 sets the decoding of UU encoded POP attachments to unlimited. A value other than 0 or -1 restricts the decoding of UU POP attachments, and applies per attachment. A POP preprocessor alert with sid 7 is generated (if enabled) when the decoding fails.

Multiple UU attachments/data in one packet are pipelined. When stateful inspection is turned on the UU encoded POP attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

78.

`memcap <int>`

This option determines (in bytes) the maximum amount of memory the POP preprocessor will use for decoding base64 encoded/quoted-printable/non-encoded MIME attachments/data or Unix-to-Unix encoded attachments. This value can be set from 3276 bytes to 100MB.

This option along with the maximum of the decoding depths will determine the POP sessions that will be decoded at any given instant. The default value for this option is 838860.

Note: It is suggested to set this value such that the max pop session calculated as follows is at least 1.

max pop session = `memcap` /(2 * max of (`b64_decode_depth`, `uu_decode_depth`, `qp_decode_depth` or `bitenc_decode_depth`))

For example, if `b64_decode_depth` is 0 (indicates unlimited decoding) and `qp_decode_depth` is 100, then

max pop session = `memcap`/2*65535 (max value for `b64_decode_depth`)

In case of multiple configs, the `memcap` of the non-default configs will be overwritten by the default config's value. Hence user needs to define it in the default config with the new keyword disabled (used to disable POP preprocessor in a config).

When the memcap for decoding (`memcap`) is exceeded the POP preprocessor alert with sid 3 is generated (when enabled).

2.2.9.2 Example

```
preprocessor pop: \
  ports { 110 } \
  memcap 1310700 \
  qp_decode_depth -1 \
  b64_decode_depth 0 \
  bitenc_decode_depth 100


preprocessor pop: \
  memcap 1310700 \
  qp_decode_depth 0 \
  disabled
```

2.2.9.3 Default

```
preprocessor pop: \
  ports { 110 } \
  b64_decode_depth 1460 \
  qp_decode_depth 1460 \
  bitenc_decode_depth 1460 \
  uu_decode_depth 1460
```

2.2.10 IMAP Preprocessor

IMAP is an IMAP4 decoder for user applications. Given a data buffer, IMAP will decode the buffer and find IMAP4 commands and responses. It will also mark the command, data header data body sections and extract the IMAP4 attachments and decode it appropriately.

IMAP will handle stateful processing. It saves state between individual packets. However maintaining correct state is dependent on the reassembly of the server side of the stream (i.e., a loss of coherent stream data results in a loss of state).

Stream should be turned on for IMAP. Please ensure that the IMAP ports are added to the stream5 ports for proper reassembly.

The IMAP preprocessor uses GID 141 to register events.

2.2.10.1 Configuration

The configuration options are described below:

79.

```
ports { <port> [<port>] ... }
```

This specifies on what ports to check for IMAP data. Typically, this will include 143. Default ports if none are specified are 143 .

80.

```
disabled
```

Disables the IMAP preprocessor in a config. This is useful when specifying the decoding depths such as `b64_decode_depth`, `qp_decode_depth`, `uu_decode_depth`, `bitenc_decode_depth` or the memcap used for decoding `memcap` in default config without turning on the IMAP preprocessor.

81.

```
b64_decode_depth
```

This config option is used to turn off/on or set the base64 decoding depth used to decode the base64 encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the base64 decoding of MIME attachments. The value of 0 sets the decoding of base64 encoded MIME attachments to unlimited. A value other than 0 or -1 restricts the decoding of base64 MIME attachments, and applies per attachment. A IMAP preprocessor alert with sid 4 is generated (if enabled) when the decoding fails.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the base64 encoded MIME attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

It is recommended that user inputs a value that is a multiple of 4. When the value specified is not a multiple of 4, the IMAP preprocessor will round it up to the next multiple of 4.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

82.

`qp_decode_depth`

This config option is used to turn off/on or set the Quoted-Printable decoding depth used to decode the Quoted-Printable(QP) encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the QP decoding of MIME attachments. The value of 0 sets the decoding of QP encoded MIME attachments to unlimited. A value other than 0 or -1 restricts the decoding of QP MIME attachments, and applies per attachment. A IMAP preprocessor alert with sid 5 is generated (if enabled) when the decoding fails.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the QP encoded MIME attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

83.

`bitenc_decode_depth`

This config option is used to turn off/on or set the non-encoded MIME extraction depth used to extract the non-encoded MIME attachments. The value ranges from -1 to 65535. A value of -1 turns off the extraction of these MIME attachments. The value of 0 sets the extraction of these MIME attachments to unlimited. A value other than 0 or -1 restricts the extraction of these MIME attachments, and applies per attachment.

Multiple MIME attachments/data in one packet are pipelined. When stateful inspection is turned on the non-encoded MIME attachments/data across multiple packets are extracted too.

The extracted data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

84.

`uu_decode_depth`

This config option is used to turn off/on or set the Unix-to-Unix decoding depth used to decode the Unix-to-Unix(UU) encoded attachments. The value ranges from -1 to 65535. A value of -1 turns off the UU decoding of IMAP attachments. The value of 0 sets the decoding of UU encoded IMAP attachments to unlimited. A value other than 0 or -1 restricts the decoding of UU IMAP

attachments, and applies per attachment. A IMAP preprocessor alert with sid 7 is generated (if enabled) when the decoding fails.

Multiple UU attachments/data in one packet are pipelined. When stateful inspection is turned on the UU encoded IMAP attachments/data across multiple packets are decoded too.

The decoded data is available for detection using the rule option `file_data`. See [*] rule option for more details.

In case of multiple configs, the value specified in the non-default config cannot exceed the value specified in the default config.

85.

`memcap <int>`

This option determines (in bytes) the maximum amount of memory the IMAP preprocessor will use for decoding base64 encoded/quoted-printable/non-encoded MIME attachments/data or Unix-to-Unix encoded attachments. This value can be set from 3276 bytes to 100MB.

This option along with the maximum of the decoding depths will determine the IMAP sessions that will be decoded at any given instant. The default value for this option is 838860.

Note: It is suggested to set this value such that the max imap session calculated as follows is at least 1.

max imap session = `memcap` /(2 * max of (`b64_decode_depth`, `uu_decode_depth`, `qp_decode_depth` or `bitenc_decode_depth`))

For example, if `b64_decode_depth` is 0 (indicates unlimited decoding) and `qp_decode_depth` is 100, then

max imap session = `memcap`/2*65535 (max value for `b64_decode_depth`)

In case of multiple configs, the `memcap` of the non-default configs will be overwritten by the default config's value. Hence user needs to define it in the default config with the new keyword disabled (used to disable IMAP preprocessor in a config).

When the memcap for decoding (`memcap`) is exceeded the IMAP preprocessor alert with sid 3 is generated (when enabled).

2.2.10.2 Example

```
preprocessor imap: \
  ports { 110 } \
  memcap 1310700 \
  qp_decode_depth -1 \
  b64_decode_depth 0 \
  bitenc_decode_depth 100


preprocessor imap: \
  memcap 1310700 \
```

```
          qp_decode_depth 0 \
          disabled
```

2.2.10.3 Default

```
        preprocessor imap: \
          ports { 110 } \
          b64_decode_depth 1460 \
          qp_decode_depth 1460 \
          bitenc_decode_depth 1460 \
          uu_decode_depth 1460
```

2.2.11 FTP/Telnet Preprocessor

FTP/Telnet is an improvement to the Telnet decoder and provides stateful inspection capability for both FTP and Telnet data streams. FTP/Telnet will decode the stream, identifying FTP commands and responses and Telnet escape sequences and normalize the fields. FTP/Telnet works on both client requests and server responses.

FTP/Telnet has the capability to handle stateless processing, meaning it only looks for information on a packet-by-packet basis.

The default is to run FTP/Telnet in stateful inspection mode, meaning it looks for information and handles reassembled data correctly.

FTP/Telnet has a very “rich” user configuration, similar to that of HTTP Inspect (See [*]). Users can configure individual FTP servers and clients with a variety of options, which should allow the user to emulate any type of FTP server or FTP Client. Within FTP/Telnet, there are four areas of configuration: Global, Telnet, FTP Client, and FTP Server.

> **Note:**
>
> Some configuration options have an argument of `yes` or `no`. This argument specifies whether the user wants the configuration option to generate a ftptelnet alert or not. The presence of the option indicates the option itself is on, while the `yes/no` argument applies to the alerting functionality associated with that option.

2.2.11.1 Global Configuration

The global configuration deals with configuration options that determine the global functioning of FTP/Telnet. The following example gives the generic global configuration format:

2.2.11.2 Format

```
    preprocessor ftp_telnet: \
        global \
        inspection_type stateful \
```

```
        encrypted_traffic yes \
        check_encrypted
```

You can only have a single global configuration, you'll get an error if you try otherwise. The FTP/Telnet global configuration must appear before the other three areas of configuration.

2.2.11.2.1 Configuration

86.

`inspection_type`

This indicates whether to operate in stateful or stateless mode.

87.

`encrypted_traffic $<$yes|no$>$`

This option enables detection and alerting on encrypted Telnet and FTP command channels.

> **Note:**
>
> When `inspection_type` is in stateless mode, checks for encrypted traffic will occur on every packet, whereas in stateful mode, a particular session will be noted as encrypted and not inspected any further.

88.

`check_encrypted`

Instructs the preprocessor to continue to check an encrypted session for a subsequent command to cease encryption.

2.2.11.3 Example Global Configuration

```
    preprocessor ftp_telnet: \
        global inspection_type stateful encrypted_traffic no
```

2.2.11.4 Telnet Configuration

The telnet configuration deals with configuration options that determine the functioning of the Telnet portion of the preprocessor. The following example gives the generic telnet configuration format:

2.2.11.5 Format

```
    preprocessor ftp_telnet_protocol: \
        telnet \
        ports { 23 } \
        normalize \
        ayt_attack_thresh 6 \
        detect_anomalies
```

There should only be a single telnet configuration, and subsequent instances will override previously set values.

2.2.11.5.1 Configuration

89.

```
ports $\{ <$port$> [<$port$> <...>] \}$
```

This is how the user configures which ports to decode as telnet traffic. SSH tunnels cannot be decoded, so adding port 22 will only yield false positives. Typically port 23 will be included.

90.

```
normalize
```

This option tells the preprocessor to normalize the telnet traffic by eliminating the telnet escape sequences. It functions similarly to its predecessor, the telnet_decode preprocessor. Rules written with 'raw' content options will ignore the normalized buffer that is created when this option is in use.

91.

```
ayt_attack_thresh $<$ number $>$
```

This option causes the preprocessor to alert when the number of consecutive telnet Are You There (AYT) commands reaches the number specified. It is only applicable when the mode is stateful.

92.

```
detect_anomalies
```

In order to support certain options, Telnet supports subnegotiation. Per the Telnet RFC, subnegotiation begins with SB (subnegotiation begin) and must end with an SE (subnegotiation end). However, certain implementations of Telnet servers will ignore the SB without a corresponding SE. This is anomalous behavior which could be an evasion case. Being that FTP uses the Telnet protocol on the control connection, it is also susceptible to this behavior. The `detect_anomalies` option enables alerting on Telnet SB without the corresponding SE.

2.2.11.6 Example Telnet Configuration

```
preprocessor ftp_telnet_protocol: \
    telnet ports { 23 } normalize ayt_attack_thresh 6
```

2.2.11.7 FTP Server Configuration

There are two types of FTP server configurations: default and by IP address.

2.2.11.7.1 Default

This configuration supplies the default server configuration for any FTP server that is not individually configured. Most of your FTP servers will most likely end up using the default configuration.

2.2.11.8 Example Default FTP Server Configuration

```
preprocessor ftp_telnet_protocol: \
    ftp server default ports { 21 }
```

Refer to [*] for the list of options set in default ftp server configuration.

2.2.11.8.1 Configuration by IP Address

This format is very similar to “default”, the only difference being that specific IPs can be configured.

2.2.11.9 Example IP specific FTP Server Configuration

```
preprocessor _telnet_protocol: \
    ftp server 10.1.1.1 ports { 21 } ftp_cmds { XPWD XCWD }
```

2.2.11.10 FTP Server Configuration Options

93.

`ports $\{ <$port$> [<$port$> <...>] \}$`

This is how the user configures which ports to decode as FTP command channel traffic. Typically port 21 will be included.

94.

`print_cmds`

During initialization, this option causes the preprocessor to print the configuration for each of the FTP commands for this server.

95.

`ftp_cmds $\{ cmd [cmd] \}$`

The preprocessor is configured to alert when it sees an FTP command that is not allowed by the server.

This option specifies a list of additional commands allowed by this server, outside of the default FTP command set as specified in RFC 959. This may be used to allow the use of the 'X' commands identified in RFC 775, as well as any additional commands as needed.

For example:

```
ftp_cmds { XPWD XCWD XCUP XMKD XRMD }
```

96.

`def_max_param_len $<$number$>$`

This specifies the default maximum allowed parameter length for an FTP command. It can be used as a basic buffer overflow detection.

97.

```
alt_max_param_len $<$number$>$ $\{ cmd [cmd] \}$
```

This specifies the maximum allowed parameter length for the specified FTP command(s). It can be used as a more specific buffer overflow detection. For example the USER command - usernames may be no longer than 16 bytes, so the appropriate configuration would be:

```
    alt_max_param_len 16 { USER }
```

98.

```
chk_str_fmt $\{ cmd [cmd] \}$
```

This option causes a check for string format attacks in the specified commands.

99.

```
cmd_validity cmd $<$ fmt $>$
```

This option specifies the valid format for parameters of a given command.

fmt must be enclosed in $<>$'s and may contain the following:

<table>
<tr><th>Value</th><th>Description</th></tr>
<tr><td>int</td><td>Parameter must be an integer</td></tr>
<tr><td>number</td><td>Parameter must be an integer between 1 and 255</td></tr>
<tr><td>char $<$chars$>$</td><td>Parameter must be a single character, one of $<$chars$>$</td></tr>
<tr><td>date $<$datefmt$>$</td><td>Parameter follows format specified, where:
<table>
<tr><td>n</td><td>Number</td></tr>
<tr><td>C</td><td>Character</td></tr>
<tr><td>$[]$</td><td>optional format enclosed</td></tr>
<tr><td>$\vert$</td><td>OR</td></tr>
<tr><td>$\{\}$</td><td>choice of options</td></tr>
<tr><td>. + -</td><td>literal</td></tr>
</table></td></tr>
<tr><td>string</td><td>Parameter is a string (effectively unrestricted)</td></tr>
<tr><td>host_port</td><td>Parameter must be a host/port specified, per RFC 959</td></tr>
<tr><td>long_host_port</td><td>Parameter must be a long host port specified, per RFC 1639</td></tr>
<tr><td>extended_host_port</td><td>Parameter must be an extended host port specified, per RFC 2428</td></tr>
<tr><td>$\{\}$, $\vert$</td><td>One of choices enclosed within, separated by $\vert$</td></tr>
<tr><td>$\{\}$, $[]$</td><td>One of the choices enclosed within $\{\}$, optional value enclosed within $[]$</td></tr>
</table>

Examples of the cmd_validity option are shown below. These examples are the default checks, per RFC 959 and others performed by the preprocessor.

```
cmd_validity MODE <char SBC>
cmd_validity STRU <char FRP>
cmd_validity ALLO < int [ char R int ] >
cmd_validity TYPE < { char AE [ char NTC ] | char I | char L [ number ] } >
cmd_validity PORT < host_port >
```

A cmd_validity line can be used to override these defaults and/or add a check for other commands.

```
# This allows additional modes, including mode Z which allows for
# zip-style compression.
cmd_validity MODE < char ASBCZ >

# Allow for a date in the MDTM command.
cmd_validity MDTM < [ date nnnnnnnnnnnnnn[.n[n[n]]] ] string >
```

MDTM is an off case that is worth discussing. While not part of an established standard, certain FTP servers accept MDTM commands that set the modification time on a file. The most common among servers that do, accept a format using YYYYMMDDHHmmss[.uuu]. Some others accept a format using YYYYMMDDHHmmss[+|-]TZ format. The example above is for the first case (time format as specified in http://www.ietf.org/internet-drafts/draft-ietf-ftpext-mlst-16.txt)

To check validity for a server that uses the TZ format, use the following:

```
cmd_validity MDTM < [ date nnnnnnnnnnnnnn[{+|-}n[n]] ] string >
```

100.

`telnet_cmds $<$yes$\vert$no$>$`

This option turns on detection and alerting when telnet escape sequences are seen on the FTP command channel. Injection of telnet escape sequences could be used as an evasion attempt on an FTP command channel.

101.

`ignore_telnet_erase_cmds $<$yes|no$>$`

This option allows Snort to ignore telnet escape sequences for erase character (TNC EAC) and erase line (TNC EAL) when normalizing FTP command channel. Some FTP servers do not process those telnet escape sequences.

102.

`data_chan`

This option causes the rest of snort (rules, other preprocessors) to ignore FTP data channel connections. Using this option means that NO INSPECTION other than TCP state will be performed on FTP data transfers. It can be used to improve performance, especially with large file

transfers from a trusted source. If your rule set includes virus-type rules, it is recommended that this option not be used.

Use of the "data_chan" option is deprecated in favor of the "ignore_data_chan" option. "data_chan" will be removed in a future release.

103.

```
ignore_data_chan $<$yes$\vert$no$>$
```

This option causes the rest of Snort (rules, other preprocessors) to ignore FTP data channel connections. Setting this option to "yes" means that NO INSPECTION other than TCP state will be performed on FTP data transfers. It can be used to improve performance, especially with large file transfers from a trusted source. If your rule set includes virus-type rules, it is recommended that this option not be used.

2.2.11.11 FTP Server Base Configuration Options

The base FTP server configuration is as follows. Options specified in the configuration file will modify this set of options. FTP commands are added to the set of allowed commands. The other options will override those in the base configuration.

```
def_max_param_len 100
ftp_cmds { USER PASS ACCT CWD CDUP SMNT
           QUIT REIN TYPE STRU MODE RETR
           STOR STOU APPE ALLO REST RNFR
           RNTO ABOR DELE RMD MKD PWD LIST
          NLST SITE SYST STAT HELP NOOP }
ftp_cmds { AUTH ADAT PROT PBSZ CONF ENC }
ftp_cmds { PORT PASV LPRT LPSV EPRT EPSV }
ftp_cmds { FEAT OPTS }
ftp_cmds { MDTM REST SIZE MLST MLSD }
alt_max_param_len 0 { CDUP QUIT REIN PASV STOU ABOR PWD SYST NOOP }
cmd_validity MODE < char SBC >
cmd_validity STRU < char FRPO [ string ] >
cmd_validity ALLO < int [ char R int ] >
cmd_validity TYPE < { char AE [ char NTC ] | char I | char L [ number ] } >
cmd_validity PORT < host_port >
cmd_validity LPRT < long_host_port >
cmd_validity EPRT < extd_host_port >
cmd_validity EPSV < [ { '1' | '2' | 'ALL' } ] >
```

2.2.11.12 FTP Client Configuration

Similar to the FTP Server configuration, the FTP client configurations has two types: default, and by IP address.

2.2.11.12.1 Default

This configuration supplies the default client configuration for any FTP client that is not individually configured. Most of your FTP clients will most likely end up using the default configuration.

2.2.11.13 Example Default FTP Client Configuration

```
preprocessor ftp_telnet_protocol: \
    ftp client default bounce no max_resp_len 200
```

2.2.11.13.1 Configuration by IP Address

This format is very similar to “default”, the only difference being that specific IPs can be configured.

2.2.11.14 Example IP specific FTP Client Configuration

```
preprocessor ftp_telnet_protocol: \
    ftp client 10.1.1.1 bounce yes max_resp_len 500
```

2.2.11.15 FTP Client Configuration Options

104.

`max_resp_len $<$number$>$`

This specifies the maximum allowed response length to an FTP command accepted by the client. It can be used as a basic buffer overflow detection.

105.

`bounce $<$yes|no$>$`

This option turns on detection and alerting of FTP bounce attacks. An FTP bounce attack occurs when the FTP PORT command is issued and the specified host does not match the host of the client.

106.

`bounce_to $<$ CIDR,[port$\vert$portlow,porthi] $>$`

When the bounce option is turned on, this allows the PORT command to use the IP address (in CIDR format) and port (or inclusive port range) without generating an alert. It can be used to deal with proxied FTP connections where the FTP data channel is different from the client.

A few examples:

- Allow bounces to 192.162.1.1 port 20020 - i.e., the use of `PORT 192,168,1,1,78,52`.

  ```
  bounce_to { 192.168.1.1,20020 }
  ```

- Allow bounces to 192.162.1.1 ports 20020 through 20040 - i.e., the use of `PORT 192,168,1,1,78,xx`, where xx is 52 through 72 inclusive.

```
    bounce_to { 192.168.1.1,20020,20040 }
```

- Allow bounces to 192.162.1.1 port 20020 and 192.168.1.2 port 20030.

```
    bounce_to { 192.168.1.1,20020 192.168.1.2,20030 }
```

- Allows bounces to IPv6 address fe8::5 port 59340.

```
    bounce_to { fe8::5,59340 }
```

107.

`telnet_cmds $<$yes|no$>$`

This option turns on detection and alerting when telnet escape sequences are seen on the FTP command channel. Injection of telnet escape sequences could be used as an evasion attempt on an FTP command channel.

108.

`ignore_telnet_erase_cmds $<$yes|no$>$`

This option allows Snort to ignore telnet escape sequences for erase character (TNC EAC) and erase line (TNC EAL) when normalizing FTP command channel. Some FTP clients do not process those telnet escape sequences.

2.2.11.16 Examples/Default Configuration from snort.conf

```
preprocessor ftp_telnet: \
    global \
    encrypted_traffic yes \
    inspection_type stateful

preprocessor ftp_telnet_protocol:\
    telnet \
    normalize \
    ayt_attack_thresh 200

# This is consistent with the FTP rules as of 18 Sept 2004.
# Set CWD to allow parameter length of 200
# MODE has an additional mode of Z (compressed)
# Check for string formats in USER & PASS commands
# Check MDTM commands that set modification time on the file.

preprocessor ftp_telnet_protocol: \
    ftp server default \
    def_max_param_len 100 \
    alt_max_param_len 200 { CWD } \
    cmd_validity MODE < char ASBCZ > \
```

```
        cmd_validity MDTM < [ date nnnnnnnnnnnnnn[.n[n[n]]] ] string > \
        chk_str_fmt { USER PASS RNFR RNTO SITE MKD } \
        telnet_cmds yes \
        ignore_data_chan yes

    preprocessor ftp_telnet_protocol: \
        ftp client default \
        max_resp_len 256 \
        bounce yes \
        telnet_cmds yes
```

2.2.12 SSH

The SSH preprocessor detects the following exploits: Challenge-Response Buffer Overflow, CRC 32, Secure CRT, and the Protocol Mismatch exploit.

Both Challenge-Response Overflow and CRC 32 attacks occur after the key exchange, and are therefore encrypted. Both attacks involve sending a large payload (20kb+) to the server immediately after the authentication challenge. To detect the attacks, the SSH preprocessor counts the number of bytes transmitted to the server. If those bytes exceed a predefined limit within a predefined number of packets, an alert is generated. Since the Challenge-Response Overflow only effects SSHv2 and CRC 32 only effects SSHv1, the SSH version string exchange is used to distinguish the attacks.

The Secure CRT and protocol mismatch exploits are observable before the key exchange.

2.2.12.1 Configuration

By default, all alerts are disabled and the preprocessor checks traffic on port 22.

The available configuration options are described below.

109.

`server_ports $\{ <$port$> [<$port$> <...>] \}$`

This option specifies which ports the SSH preprocessor should inspect traffic to.

110.

`max_encrypted_packets $<$ number $>$`

The number of stream reassembled encrypted packets that Snort will inspect before ignoring a given SSH session. The SSH vulnerabilities that Snort can detect all happen at the very beginning of an SSH session. Once max_encrypted_packets packets have been seen, Snort ignores the session to increase performance. The default is set to 25. This value can be set from 0 to 65535.

111.

`max_client_bytes $<$ number $>$`

The number of unanswered bytes allowed to be transferred before alerting on Challenge-Response Overflow or CRC 32. This number must be hit before max_encrypted_packets packets

are sent, or else Snort will ignore the traffic. The default is set to 19600. This value can be set from 0 to 65535.

112.

```
max_server_version_len $<$ number $>$
```

The maximum number of bytes allowed in the SSH server version string before alerting on the Secure CRT server version string overflow. The default is set to 80. This value can be set from 0 to 255.

113.

```
autodetect
```

Attempt to automatically detect SSH.

114.

```
enable_respoverflow
```

Enables checking for the Challenge-Response Overflow exploit.

115.

```
enable_ssh1crc32
```

Enables checking for the CRC 32 exploit.

116.

```
enable_srvoverflow
```

Enables checking for the Secure CRT exploit.

117.

```
enable_protomismatch
```

Enables checking for the Protocol Mismatch exploit.

118.

```
enable_badmsgdir
```

Enable alerts for traffic flowing the wrong direction. For instance, if the presumed server generates client traffic, or if a client generates server traffic.

119.

```
enable_paysize
```

Enables alerts for invalid payload sizes.

120.

```
enable_recognition
```

Enable alerts for non-SSH traffic on SSH ports.

The SSH preprocessor should work by default. After max_encrypted_packets is reached, the preprocessor will stop processing traffic for a given session. If Challenge-Response Overflow or CRC 32 false positive, try increasing the number of required client bytes with max_client_bytes.

2.2.12.2 Example Configuration from snort.conf

Looks for attacks on SSH server port 22. Alerts at 19600 unacknowledged bytes within 20 encrypted packets for the Challenge-Response Overflow/CRC32 exploits.

```
preprocessor ssh: \
    server_ports { 22 } \
    max_client_bytes 19600 \
    max_encrypted_packets 20 \
    enable_respoverflow \
    enable_ssh1crc32
```

2.2.13 DNS

The DNS preprocessor decodes DNS Responses and can detect the following exploits: DNS Client RData Overflow, Obsolete Record Types, and Experimental Record Types.

DNS looks at DNS Response traffic over UDP and TCP and it requires Stream preprocessor to be enabled for TCP decoding.

2.2.13.1 Configuration

By default, all alerts are disabled and the preprocessor checks traffic on port 53.

The available configuration options are described below.

121.

`ports $\{ <$port$> [<$port$> <...>] \}$`

This option specifies the source ports that the DNS preprocessor should inspect traffic.

122.

`enable_obsolete_types`

Alert on Obsolete (per RFC 1035) Record Types

123.

`enable_experimental_types`

Alert on Experimental (per RFC 1035) Record Types

124.

`enable_rdata_overflow`

Check for DNS Client RData TXT Overflow

The DNS preprocessor does nothing if none of the 3 vulnerabilities it checks for are enabled. It will not operate on TCP sessions picked up midstream, and it will cease operation on a session if it loses state because of missing data (dropped packets).

2.2.13.2 Examples/Default Configuration from snort.conf

Looks for traffic on DNS server port 53. Check for the DNS Client RData overflow vulnerability. Do not alert on obsolete or experimental RData record types.

```
preprocessor dns: \
    ports { 53 } \
    enable_rdata_overflow
```

2.2.14 SSL/TLS

Encrypted traffic should be ignored by Snort for both performance reasons and to reduce false positives. The SSL Dynamic Preprocessor (SSLPP) decodes SSL and TLS traffic and optionally determines if and when Snort should stop inspection of it.

Typically, SSL is used over port 443 as HTTPS. By enabling the SSLPP to inspect port 443 and enabling the noinspect_encrypted option, only the SSL handshake of each connection will be inspected. Once the traffic is determined to be encrypted, no further inspection of the data on the connection is made.

By default, SSLPP looks for a handshake followed by encrypted traffic traveling to both sides. If one side responds with an indication that something has failed, such as the handshake, the session is not marked as encrypted. Verifying that faultless encrypted traffic is sent from both endpoints ensures two things: the last client-side handshake packet was not crafted to evade Snort, and that the traffic is legitimately encrypted.

In some cases, especially when packets may be missed, the only observed response from one endpoint will be TCP ACKs. Therefore, if a user knows that server-side encrypted data can be trusted to mark the session as encrypted, the user should use the 'trustservers' option, documented below.

2.2.14.1 Configuration

125.

```
ports $\{ <$port$> [<$port$> <...>] \}$
```

This option specifies which ports SSLPP will inspect traffic on.

By default, SSLPP watches the following ports:

- 443 HTTPS
- 465 SMTPS
- 563 NNTPS

- `636` LDAPS
- `989` FTPS
- `992` TelnetS
- `993` IMAPS
- `994` IRCS
- `995` POPS

126.

`noinspect_encrypted`

Disable inspection on traffic that is encrypted. Default is off.

127.

`max_heartbeat_length`

Maximum length of heartbeat record allowed. This config option is used to detect the heartbleed attacks. The allowed range is 0 to 65535. Setting the value to 0 turns off the heartbeat length checks. For heartbeat requests, if the payload size of the request record is greater than the max_heartbeat_length an alert with sid 3 and gid 137 is generated. For heartbeat responses, if the record size itself is greater than the max_heartbeat_length an alert with sid 4 and gid 137 is generated. Default is off.

128.

`trustservers`

Disables the requirement that application (encrypted) data must be observed on both sides of the session before a session is marked encrypted. Use this option for slightly better performance if you trust that your servers are not compromised. This requires the `noinspect_encrypted` option to be useful. Default is off.

2.2.14.2 Examples/Default Configuration from snort.conf

Enables the SSL preprocessor and tells it to disable inspection on encrypted traffic.

```
preprocessor ssl: noinspect_encrypted
```

2.2.14.3 Rule Options

The following rule options are supported by enabling the `ssl` preprocessor:

```
ssl_version
ssl_state
```

`ssl_version`

The `ssl_version` rule option tracks the version negotiated between the endpoints of the SSL encryption. The list of version identifiers are below, and more than one identifier can be specified, via a comma separated list. Lists of identifiers are OR'ed together.

The option will match if any one of the OR'ed versions are used in the SSL connection. To check for two or more SSL versions in use simultaneously, multiple `ssl_version` rule options should be used.

Syntax

```
ssl_version: <version-list>

version-list = version | version , version-list
version      = ["!"] "sslv2" | "sslv3" | "tls1.0" | "tls1.1" | "tls1.2"
```

Examples

```
ssl_version:sslv3;
ssl_version:tls1.0,tls1.1,tls1.2;
ssl_version:!sslv2;
```

`ssl_state`

The `ssl_state` rule option tracks the state of the SSL encryption during the process of hello and key exchange. The list of states are below. More than one state can be specified, via a comma separated list, and are OR'ed together.

The option will match if the connection is currently in any one of the OR'ed states. To ensure the connection has reached each of a set of states, multiple rules using the `ssl_state` rule option should be used.

Syntax

```
ssl_state: <state-list>

state-list = state | state , state-list
state      = ["!"] "client_hello" | "server_hello" | "client_keyx" | "server_keyx"
| "unknown"
```

Examples

```
ssl_state:client_hello;
ssl_state:client_keyx,server_keyx;
ssl_state:!server_hello;
```

2.2.15 ARP Spoof Preprocessor

The ARP spoof preprocessor decodes ARP packets and detects ARP attacks, unicast ARP requests, and inconsistent Ethernet to IP mapping.

When no arguments are specified to arpspoof, the preprocessor inspects Ethernet addresses and the addresses in the ARP packets. When inconsistency occurs, an alert with GID 112 and SID 2 or 3 is generated.

When "`-unicast`" is specified as the argument of arpspoof, the preprocessor checks for unicast ARP requests. An alert with GID 112 and SID 1 will be generated if a unicast ARP request is detected.

Specify a pair of IP and hardware address as the argument to `arpspoof_detect_host`. The host with the IP address should be on the same layer 2 segment as Snort is. Specify one host IP MAC combo per line. The preprocessor will use this list when detecting ARP cache overwrite attacks. Alert SID 4 is used in this case.

2.2.15.1 Format

```
preprocessor arpspoof[: -unicast]
preprocessor arpspoof_detect_host: ip mac
```

Option	Description
`ip`	IP address.
`mac`	The Ethernet address corresponding to the preceding IP.

2.2.15.2 Example Configuration

The first example configuration does neither unicast detection nor ARP mapping monitoring. The preprocessor merely looks for Ethernet address inconsistencies.

```
preprocessor arpspoof
```

The next example configuration does not do unicast detection but monitors ARP mapping for hosts 192.168.40.1 and 192.168.40.2.

```
preprocessor arpspoof
preprocessor arpspoof_detect_host: 192.168.40.1 f0:0f:00:f0:0f:00
preprocessor arpspoof_detect_host: 192.168.40.2 f0:0f:00:f0:0f:01
```

The third example configuration has unicast detection enabled.

```
preprocessor arpspoof: -unicast
preprocessor arpspoof_detect_host: 192.168.40.1 f0:0f:00:f0:0f:00
preprocessor arpspoof_detect_host: 192.168.40.2 f0:0f:00:f0:0f:01
```

2.2.16 DCE/RPC 2 Preprocessor

The main purpose of the preprocessor is to perform SMB desegmentation and DCE/RPC defragmentation to avoid rule evasion using these techniques. SMB desegmentation is performed for the following commands that can be used to transport DCE/RPC requests and responses: `Write`, `Write Block Raw`, `Write and Close`, `Write AndX`, `Transaction`, `Transaction Secondary`, `Read`, `Read Block Raw` and `Read AndX`. The following transports are supported for DCE/RPC: SMB, TCP, UDP and RPC over HTTP v.1 proxy and server. New rule options have been implemented to improve performance, reduce false positives and reduce the count and complexity of DCE/RPC based rules.

2.2.16.1 Dependency Requirements

For proper functioning of the preprocessor:

- Stream session tracking must be enabled, i.e. `stream5`. The preprocessor requires a session tracker to keep its data.
- Stream reassembly must be performed for TCP sessions. If it is decided that a session is SMB or DCE/RPC, either through configured ports, servers or autodetecting, the `dcerpc2` preprocessor will enable stream reassembly for that session if necessary.
- IP defragmentation should be enabled, i.e. the `frag3` preprocessor should be enabled and configured.

2.2.16.2 Target Based

There are enough important differences between Windows and Samba versions that a target based approach has been implemented. Some important differences:

Named pipe instance tracking

A combination of valid login handle or UID, share handle or TID and file/named pipe handle or FID must be used to write data to a named pipe. The binding between these is dependent on OS/software version.

Samba 3.0.22 and earlier

Any valid UID and TID, along with a valid FID can be used to make a request, however, if the TID used in creating the FID is deleted (via a tree disconnect), the FID that was created using this TID becomes invalid, i.e. no more requests can be written to that named pipe instance.

Samba greater than 3.0.22

Any valid TID, along with a valid FID can be used to make a request. However, only the UID used in opening the named pipe can be used to make a request using the FID handle to the named pipe instance. If the TID used to create the FID is deleted (via a tree disconnect), the FID that was created using this TID becomes invalid, i.e. no more requests can be written to that named pipe instance. If the UID used to create the named pipe instance is deleted (via a `Logoff AndX`), since it is necessary in making a request to the named pipe, the FID becomes invalid.

Windows 2003

Windows XP

Windows Vista

These Windows versions require strict binding between the UID, TID and FID used to make a request to a named pipe instance. Both the UID and TID used to open the named pipe instance must be used when writing data to the same named pipe instance. Therefore, deleting either the UID or TID invalidates the FID.

Windows 2000

Windows 2000 is interesting in that the first request to a named pipe must use the same binding as that of the other Windows versions. However, requests after that follow the same binding as Samba 3.0.22 and earlier, i.e. no binding. It also follows Samba greater than 3.0.22 in that deleting the UID or TID used to create the named pipe instance also invalidates it.

Accepted SMB commands

Samba in particular does not recognize certain commands under an `IPC$` tree.

Samba (all versions)

Under an `IPC$` tree, does not accept:

`Open`
`Write And Close`
`Read`
`Read Block Raw`
`Write Block Raw`

Windows (all versions)

Accepts all of the above commands under an `IPC$` tree.

AndX command chaining

Windows is very strict in what command combinations it allows to be chained. Samba, on the other hand, is very lax and allows some nonsensical combinations, e.g. multiple logins and tree connects (only one place to return handles for these), login/logoff and tree connect/tree disconnect. Ultimately, we don't want to keep track of data that the server won't accept. An evasion possibility would be accepting a fragment in a request that the server won't accept that gets sandwiched between an exploit.

Transaction tracking

The differences between a `Transaction` request and using one of the `Write*` commands to write data to a named pipe are that (1) a `Transaction` performs the operations of a write and a read from the named pipe, whereas in using the `Write*` commands, the client has to explicitly send one of the `Read*` requests to tell the server to send the response and (2) a `Transaction` request is not written to the named pipe until all of the data is received (via potential `Transaction Secondary` requests) whereas with the `Write*` commands, data is written to the named pipe as it is received by the server. Multiple Transaction requests can be made simultaneously to the same named pipe. These requests can also be segmented with `Transaction Secondary` commands. What distinguishes them (when the same named pipe is being written to, i.e. having the same FID) are fields in the SMB header representing a process id (PID) and multiplex id (MID). The PID represents the process this request is a part of. An MID represents different sub-processes within a process (or under a PID). Segments for each "thread" are stored separately and written to the named pipe when all segments are received. It is necessary to track this so as not to munge these requests together (which would be a potential evasion opportunity).

Windows (all versions)

Uses a combination of PID and MID to define a "thread".

Samba (all versions)

Uses just the MID to define a "thread".

Multiple Bind Requests

A `Bind` request is the first request that must be made in a connection-oriented DCE/RPC session in order to specify the interface/interfaces that one wants to communicate with.

Windows (all versions)

For all of the Windows versions, only one `Bind` can ever be made on a session whether or not it succeeds or fails. Any binding after that must use the `Alter Context` request. If another `Bind` is made, all previous interface bindings are invalidated.

Samba 3.0.20 and earlier

Any amount of `Bind` requests can be made.

Samba later than 3.0.20

Another `Bind` request can be made if the first failed and no interfaces were successfully bound to. If a `Bind` after a successful `Bind` is made, all previous interface bindings are invalidated.

DCE/RPC Fragmented requests - Context ID

Each fragment in a fragmented request carries the context id of the bound interface it wants to make the request to.

Windows (all versions)

The context id that is ultimately used for the request is contained in the first fragment. The context id field in any other fragment can contain any value.

Samba (all versions)

The context id that is ultimately used for the request is contained in the last fragment. The context id field in any other fragment can contain any value.

DCE/RPC Fragmented requests - Operation number

Each fragment in a fragmented request carries an operation number (opnum) which is more or less a handle to a function offered by the interface.

Samba (all versions)

Windows 2000

Windows 2003

Windows XP

The opnum that is ultimately used for the request is contained in the last fragment. The opnum field in any other fragment can contain any value.

Windows Vista

The opnum that is ultimately used for the request is contained in the first fragment. The opnum field in any other fragment can contain any value.

DCE/RPC Stub data byte order

The byte order of the stub data is determined differently for Windows and Samba.

Windows (all versions)

The byte order of the stub data is that which was used in the `Bind` request.

Samba (all versions)

The byte order of the stub data is that which is used in the request carrying the stub data.

2.2.16.3 Configuration

The `dcerpc2` preprocessor has a global configuration and one or more server configurations. The global preprocessor configuration name is `dcerpc2` and the server preprocessor configuration name is `dcerpc2_server`.

Global Configuration

```
preprocessor dcerpc2
```

The global `dcerpc2` configuration is required. Only one global `dcerpc2` configuration can be specified.

Option syntax

Option	Argument	Required	Default
`memcap`	`<memcap>`	NO	`memcap 102400`
`disable_defrag`	NONE	NO	OFF
`max_frag_len`	`<max-frag-len>`	NO	OFF
`events`	`<events>`	NO	OFF
`reassemble_threshold`	`<re-thresh>`	NO	OFF
`disabled`	NONE	NO	OFF
`smb_fingerprint_policy`	`<fp-policy>`	NO	OFF
`smb_legacy_mode`	NONE	NO	OFF

```
memcap           = 1024-4194303 (kilobytes)
max-frag-len     = 1514-65535
events           = pseudo-event | event | '[' event-list ']'
pseudo-event     = "none" | "all"
event-list       = event | event ',' event-list
event            = "memcap" | "smb" | "co" | "cl"
re-thresh        = 0-65535
fp-policy        = "server" | "client" | "both"
```

Option explanations

`memcap`

Specifies the maximum amount of run-time memory that can be allocated. Run-time memory includes any memory allocated after configuration. Default is 100 MB.

`disabled`

Disables the preprocessor. By default this value is turned off. When the preprocessor is disabled only the memcap option is applied when specified with the configuration.

`disable_defrag`

Tells the preprocessor not to do DCE/RPC defragmentation. Default is to do defragmentation.

`max_frag_len`

Specifies the maximum fragment size that will be added to the defragmentation module. If a fragment is greater than this size, it is truncated before being added to the defragmentation module. The allowed range for this option is 1514 - 65535.

`events`

Specifies the classes of events to enable. (See Events section for an enumeration and explanation of events.)

`memcap`

Only one event. If the memcap is reached or exceeded, alert.

`smb`

Alert on events related to SMB processing.

`co`

Stands for connection-oriented DCE/RPC. Alert on events related to connection-oriented DCE/RPC processing.

`cl`

Stands for connectionless DCE/RPC. Alert on events related to connectionless DCE/RPC processing.

`reassemble_threshold`

Specifies a minimum number of bytes in the DCE/RPC desegmentation and defragmentation buffers before creating a reassembly packet to send to the detection engine. This option is useful in inline mode so as to potentially catch an exploit early before full defragmentation is done. A value of 0 supplied as an argument to this option will, in effect, disable this option. Default is disabled.

`smb_fingerprint_policy`

By default, SMBv1, SMBv2, and SMBv3 files are inspected. If legacy mode is configured, only SMBv1 file inspection is enabled.

`legacy_mode`

In the initial phase of an SMB session, the client needs to authenticate with a SessionSetupAndX. Both the request and response to this command contain OS and version information that can allow the preprocessor to dynamically set the policy for a session which allows for better protection against Windows and Samba specific evasions.

Option examples

```
    memcap 30000
    max_frag_len 16840
    events none
    events all
    events smb
    events co
    events [co]
    events [smb, co]
    events [memcap, smb, co, cl]
    reassemble_threshold 500
    smb_fingerprint_policy both
    smb_fingerprint_policy client
    smb_legacy_mode
```

Configuration examples

```
    preprocessor dcerpc2
    preprocessor dcerpc2: memcap 500000
    preprocessor dcerpc2: max_frag_len 16840, memcap 300000, events smb
    preprocessor dcerpc2: memcap 50000, events [memcap, smb, co, cl], max_frag_len
14440
    preprocessor dcerpc2: disable_defrag, events [memcap, smb]
    preprocessor dcerpc2: reassemble_threshold 500
    preprocessor dcerpc2: memcap 50000, events [memcap, smb, co, cl], max_frag_len
14440, smb_fingerprint_policy both
```

Default global configuration

```
preprocessor dcerpc2: memcap 102400
```

Server Configuration

```
preprocessor dcerpc2_server
```

The `dcerpc2_server` configuration is optional. A `dcerpc2_server` configuration must start with `default` or `net` options. The `default` and `net` options are mutually exclusive. At most one default configuration can be specified. If no `default` configuration is specified, default values will be used for the `default` configuration. Zero or more `net` configurations can be specified. For any `dcerpc2_server` configuration, if non-required options are not specified, the defaults will be used. When processing DCE/RPC traffic, the `default` configuration is used if no net configurations match. If a `net` configuration matches, it will override the `default` configuration. A `net` configuration matches if the packet's server IP address matches an IP address or net specified in the `net` configuration. The `net` option supports IPv6 addresses. Note that port and ip variables defined in `snort.conf` CANNOT be used.

Option syntax

Option	Argument	Required	Default
`default`	NONE	YES	NONE
`net`	`<net>`	YES	NONE
`policy`	`<policy>`	NO	`policy WinXP`
`detect`	`<detect>`	NO	`detect [smb [139,445], tcp 135, udp 135, rpc-over-http-server 593]`
`autodetect`	`<detect>`	NO	`autodetect [tcp 1025:, udp 1025:, rpc-over-http-server 1025:]`
`no_autodetect_http_proxy_ports`	NONE	NO	DISABLED (The preprocessor autodetects on all proxy ports by default)
`smb_invalid_shares`	`<shares>`	NO	NONE
`smb_max_chain`	`<max-chain>`	NO	`smb_max_chain 3`
`smb_file_inspection`	`<file-inspect>`	NO	`smb_file_inspection off`

```
net          = ip | '[' ip-list ']'
ip-list      = ip | ip ',' ip-list
ip           = ip-addr | ip-addr '/' prefix | ip4-addr '/' netmask
ip-addr      = ip4-addr | ip6-addr
ip4-addr     = a valid IPv4 address
ip6-addr     = a valid IPv6 address (can be compressed)
```

```
    prefix       = a valid CIDR
    netmask      = a valid netmask
    policy       = "Win2000" | "Win2003" | "WinXP" | "WinVista" |
                   "Samba" | "Samba-3.0.22" | "Samba-3.0.20"
    detect       = "none" | detect-opt | '[' detect-list ']'
    detect-list  = detect-opt | detect-opt ',' detect-list
    detect-opt   = transport | transport port-item |
                   transport '[' port-list ']'
    transport    = "smb" | "tcp" | "udp" | "rpc-over-http-proxy" |
                   "rpc-over-http-server"
    port-list    = port-item | port-item ',' port-list
    port-item    = port | port-range
    port-range   = ':' port | port ':' | port ':' port
    port         = 0-65535
    shares       = share | '[' share-list ']'
    share-list   = share | share ',' share-list
    share        = word | '"' word '"' | '"' var-word '"'
    word         = graphical ASCII characters except ',' '"' ']' '[' '$'
    var-word     = graphical ASCII characters except ',' '"' ']' '['
    max-chain    = 0-255
    file-inspect = file-arg | '[' file-list ']'
    file-arg     = "off" | "on" | "only"
    file-list    = file-arg [ ',' "file-depth" <int64_t> ]
```

Because the Snort main parser treats '$' as the start of a variable and tries to expand it, shares with '$' must be enclosed quotes.

Option explanations

`default`

Specifies that this configuration is for the default server configuration.

`net`

Specifies that this configuration is an IP or net specific configuration. The configuration will only apply to the IP addresses and nets supplied as an argument.

`policy`

Specifies the target-based policy to use when processing. Default is "WinXP".

`detect`

Specifies the DCE/RPC transport and server ports that should be detected on for the transport. Defaults are ports 139 and 445 for SMB, 135 for TCP and UDP, 593 for RPC over HTTP server and 80 for RPC over HTTP proxy.

`autodetect`

Specifies the DCE/RPC transport and server ports that the preprocessor should attempt to autodetect on for the transport. The autodetect ports are only queried if no detect transport/ports match the packet. The order in which the preprocessor will attempt to autodetect will be - TCP/UDP, RPC over HTTP server, RPC over HTTP proxy and lastly SMB. Note that most dynamic DCE/RPC ports are above 1024 and ride directly over TCP or UDP. It would be very uncommon to see SMB on anything other than ports 139 and 445. Defaults are 1025-65535 for TCP, UDP and RPC over HTTP server.

`no_autodetect_http_proxy_ports`

By default, the preprocessor will always attempt to autodetect for ports specified in the detect configuration for rpc-over-http-proxy. This is because the proxy is likely a web server and the preprocessor should not look at all web traffic. This option is useful if the RPC over HTTP proxy configured with the detect option is only used to proxy DCE/RPC traffic. Default is to autodetect on RPC over HTTP proxy detect ports.

`smb_invalid_shares`

Specifies SMB shares that the preprocessor should alert on if an attempt is made to connect to them via a `Tree Connect` or `Tree Connect AndX`. Default is empty.

`smb_max_chain`

Specifies the maximum amount of AndX command chaining that is allowed before an alert is generated. Default maximum is 3 chained commands. A value of 0 disables this option. This value can be set from 0 to 255.

`smb_file_inspection`

Instructs the preprocessor to do inspection of normal SMB file transfers. This includes doing file type and signature through the file API as well as setting a pointer for the `file_data` rule option. Note that the `file-depth` option only applies to the maximum amount of file data for which it will set the pointer for the `file_data` rule option. For file type and signature it will use the value configured for the file API. If `only` is specified, the preprocessor will only do SMB file inspection, i.e. it will not do any DCE/RPC tracking or inspection. If `on` is specified with no arguments, the default file depth is 16384 bytes. An argument of -1 to `file-depth` disables setting the pointer for `file_data`, effectively disabling SMB file inspection in rules. An argument of 0 to `file-depth` means unlimited. Default is `off`, i.e. no SMB file inspection is done in the preprocessor. SMBv1, SMBv2, and SMBv3 are supported.

Option examples

```
net 192.168.0.10
net 192.168.0.0/24
net [192.168.0.0/24]
net 192.168.0.0/255.255.255.0
net feab:45b3:ab92:8ac4:d322:007f:e5aa:7845
```

```
net feab:45b3:ab92:8ac4:d322:007f:e5aa:7845/128
net feab:45b3::/32
net [192.168.0.10, feab:45b3::/32]
net [192.168.0.0/24, feab:45b3:ab92:8ac4:d322:007f:e5aa:7845]
policy Win2000
policy Samba-3.0.22
detect none
detect smb
detect [smb]
detect smb 445
detect [smb 445]
detect smb [139,445]
detect [smb [139,445]]
detect [smb, tcp]
detect [smb 139, tcp [135,2103]]
detect [smb [139,445], tcp 135, udp 135, rpc-over-http-server [593,6002:6004]]
autodetect none
autodetect tcp
autodetect [tcp]
autodetect tcp 2025:
autodetect [tcp 2025:]
autodetect tcp [2025:3001,3003:]
autodetect [tcp [2025:3001,3003:]]
autodetect [tcp, udp]
autodetect [tcp 2025:, udp 2025:]
autodetect [tcp 2025:, udp, rpc-over-http-server [1025:6001,6005:]]
smb_invalid_shares private
smb_invalid_shares "private"
smb_invalid_shares "C$"
smb_invalid_shares [private, "C$"]
smb_invalid_shares ["private", "C$"]
smb_max_chain 1
smb_file_inspection on
smb_file_inspection off
smb_file_inspection [ on, file-depth -1 ]
smb_file_inspection [ on, file-depth 0 ]
smb_file_inspection [ on, file-depth 4294967296 ]
smb_file_inspection [ only, file-depth -1 ]
```

Configuration examples

```
preprocessor dcerpc2_server: \
    default

preprocessor dcerpc2_server: \
    default, policy Win2000

preprocessor dcerpc2_server: \
    default, policy Win2000, detect [smb, tcp], autodetect tcp 1025:, \
    smb_invalid_shares ["C$", "D$", "ADMIN$"]

preprocessor dcerpc2_server: net 10.4.10.0/24, policy Win2000

preprocessor dcerpc2_server: \
```

```
        net [10.4.10.0/24,feab:45b3::/126], policy WinVista, smb_max_chain 1

    preprocessor dcerpc2_server: \
        net [10.4.10.0/24,feab:45b3::/126], policy WinVista, \
        detect [smb, tcp, rpc-over-http-proxy 8081],
        autodetect [tcp, rpc-over-http-proxy [1025:6001,6005:]], \
        smb_invalid_shares ["C$", "ADMIN$"], no_autodetect_http_proxy_ports

    preprocessor dcerpc2_server: \
        net [10.4.11.56,10.4.11.57], policy Samba, detect smb, autodetect none

    preprocessor dcerpc2_server: default, policy WinXP, \
        smb_file_inspection [ on, file-depth 0 ]
```

Default server configuration

```
    preprocessor dcerpc2_server: default, policy WinXP, \
        detect [smb [139,445], tcp 135, udp 135, rpc-over-http-server 593], \
        autodetect [tcp 1025:, udp 1025:, rpc-over-http-server 1025:], \
        smb_max_chain 3, smb_file_inspection off
```

Complete `dcerpc2` default configuration

```
    preprocessor dcerpc2: memcap 102400

    preprocessor dcerpc2_server: \
        default, policy WinXP, \
        detect [smb [139,445], tcp 135, udp 135, rpc-over-http-server 593], \
        autodetect [tcp 1025:, udp 1025:, rpc-over-http-server 1025:], \
        smb_max_chain 3, smb_file_inspection off
```

2.2.16.4 Events

The preprocessor uses GID 133 to register events.

Memcap events

SID	Description
1	If the memory cap is reached and the preprocessor is configured to alert.

SMB events

SID	Description
2	

	An invalid NetBIOS Session Service type was specified in the header. Valid types are: `Message`, `Request` (only from client), `Positive Response` (only from server), `Negative Response` (only from server), `Retarget Response` (only from server) and `Keep Alive`.
3	An SMB message type was specified in the header. Either a request was made by the server or a response was given by the client.
4	The SMB id does not equal `\xffSMB` or `\xfeSMB`.
5	The word count of the command header is invalid. SMB commands have pretty specific word counts and if the preprocessor sees a command with a word count that doesn't jive with that command, the preprocessor will alert.
6	Some commands require a minimum number of bytes after the command header. If a command requires this and the byte count is less than the minimum required byte count for that command, the preprocessor will alert.
7	Some commands, especially the commands from the SMB Core implementation require a data format field that specifies the kind of data that will be coming next. Some commands require a specific format for the data. The preprocessor will alert if the format is not that which is expected for that command.
8	Many SMB commands have a field containing an offset from the beginning of the SMB header to where the data the command is carrying starts. If this offset puts us before data that has already been processed or after the end of payload, the preprocessor will alert.
9	Some SMB commands, such as `Transaction`, have a field containing the total amount of data to be transmitted. If this field is zero, the preprocessor will alert.
10	The preprocessor will alert if the NetBIOS Session Service length field contains a value less than the size of an SMB header.
11	The preprocessor will alert if the remaining NetBIOS packet length is less than the size of the SMB command header to be decoded.
12	The preprocessor will alert if the remaining NetBIOS packet length is less than the size of the SMB command byte count specified in the command header.
13	The preprocessor will alert if the remaining NetBIOS packet length is less than the size of the SMB command data size specified in the command header.
14	The preprocessor will alert if the total data count specified in the SMB command header is less than the data size specified in the SMB command header. (Total data count must always be greater than or equal to current data size.)
15	The preprocessor will alert if the total amount of data sent in a transaction is greater than the total data count specified in the SMB command header.
16	The preprocessor will alert if the byte count specified in the SMB command header is less than the data size specified in the SMB command. (The byte count must always be greater than or equal to the data size.)
17	Some of the Core Protocol commands (from the initial SMB implementation) require that the byte count be some value greater than the data size exactly. The preprocessor will alert if the byte count minus a predetermined amount based on the SMB command is not equal to the data size.
18	For the `Tree Connect` command (and not the `Tree Connect AndX` command), the preprocessor has to queue the requests up and wait for a server response to determine whether or not an IPC share was successfully connected to (which is what the preprocessor is interested in). Unlike the `Tree Connect AndX` response, there is no indication in the `Tree Connect` response as to whether the share is IPC or

	not. There should be under normal circumstances no more than a few pending tree connects at a time and the preprocessor will alert if this number is excessive.
19	After a client is done writing data using the `Write*` commands, it issues a `Read*` command to the server to tell it to send a response to the data it has written. In this case the preprocessor is concerned with the server response. The `Read*` request contains the file id associated with a named pipe instance that the preprocessor will ultimately send the data to. The server response, however, does not contain this file id, so it need to be queued with the request and dequeued with the response. If multiple `Read*` requests are sent to the server, they are responded to in the order they were sent. There should be under normal circumstances no more than a few pending `Read*` requests at a time and the preprocessor will alert if this number is excessive.
20	The preprocessor will alert if the number of chained commands in a single request is greater than or equal to the configured amount (default is 3).
21	With `AndX` command chaining it is possible to chain multiple `Session Setup AndX` commands within the same request. There is, however, only one place in the SMB header to return a login handle (or Uid). Windows does not allow this behavior, however Samba does. This is anomalous behavior and the preprocessor will alert if it happens.
22	With `AndX` command chaining it is possible to chain multiple `Tree Connect AndX` commands within the same request. There is, however, only one place in the SMB header to return a tree handle (or Tid). Windows does not allow this behavior, however Samba does. This is anomalous behavior and the preprocessor will alert if it happens.
23	When a `Session Setup AndX` request is sent to the server, the server responds (if the client successfully authenticates) which a user id or login handle. This is used by the client in subsequent requests to indicate that it has authenticated. A `Logoff AndX` request is sent by the client to indicate it wants to end the session and invalidate the login handle. With commands that are chained after a `Session Setup AndX` request, the login handle returned by the server is used for the subsequent chained commands. The combination of a `Session Setup AndX` command with a chained `Logoff AndX` command, essentially logins in and logs off in the same request and is anomalous behavior. The preprocessor will alert if it sees this.
24	A `Tree Connect AndX` command is used to connect to a share. The `Tree Disconnect` command is used to disconnect from that share. The combination of a `Tree Connect AndX` command with a chained `Tree Disconnect` command, essentially connects to a share and disconnects from the same share in the same request and is anomalous behavior. The preprocessor will alert if it sees this.
25	An `Open AndX` or `Nt Create AndX` command is used to open/create a file or named pipe. (The preprocessor is only interested in named pipes as this is where DCE/RPC requests are written to.) The `Close` command is used to close that file or named pipe. The combination of a `Open AndX` or `Nt Create AndX` command with a chained `Close` command, essentially opens and closes the named pipe in the same request and is anomalous behavior. The preprocessor will alert if it sees this.
26	The preprocessor will alert if it sees any of the invalid SMB shares configured. It looks for a `Tree Connect` or `Tree Connect AndX` to the share.
48	The preprocessor will alert if a data count for a Core dialect write command is zero.
49	For some of the Core dialect commands such as `Write` and `Read`, there are two data count fields, one in the main command header and one in the data format section. If these aren't the same, the preprocessor will alert.
50	In the initial negotiation phase of an SMB session, the server in a `Negotiate` response and the client in a `SessionSetupAndX` request will advertise the maximum number of outstanding requests supported. The preprocessor will alert if the lesser of the two is exceeded.

51	When a client sends a request it uses a value called the MID (multiplex id) to match a response, which the server is supposed to echo, to a request. If there are multiple outstanding requests with the same MID, the preprocessor will alert.
52	In the `Negotiate` request a client gives a list of SMB dialects it supports, normally in order from least desirable to most desirable and the server responds with the index of the dialect to be used on the SMB session. Anything less than "NT LM 0.12" would be very odd these days (even Windows 98 supports it) and the preprocessor will alert if the client doesn't offer it as a supported dialect or the server chooses a lesser dialect.
53	There are a number of commands that are considered deprecated and/or obsolete by Microsoft (see MS-CIFS and MS-SMB). If the preprocessor detects the use of a deprecated/obsolete command used it will alert.
54	There are some commands that can be used that can be considered unusual in the context they are used. These include some of the transaction commands such as: `SMB_COM_TRANSACTION / TRANS_READ_NMPIPE` `SMB_COM_TRANSACTION / TRANS_WRITE_NMPIPE` `SMB_COM_TRANSACTION2 / TRANS2_OPEN2` `SMB_COM_NT_TRANSACT / NT_TRANSACT_CREATE` The preprocessor will alert if it detects unusual use of a command.
55	Transaction commands have a setup count field that indicates the number of 16bit words in the transaction setup. The preprocessor will alert if the setup count is invalid for the transaction command / sub command.
56	There can be only one Negotiate transaction per session and it is the first thing a client and server do to determine the SMB dialect each supports. The preprocessor will alert if the client attempts multiple dialect negotiations.
57	Malware will often set a file's attributes to ReadOnly/Hidden/System if it is successful in installing itself as a Windows service or is able to write an autorun.inf file since it doesn't want the user to see the file and the default folder options in Windows is not to display Hidden files. The preprocessor will alert if it detects a client attempt to set a file's attributes to ReadOnly/Hidden/System.
58	File offset provided is greater than file size specified. This is applied to read / write requests for file.
59	Nextcommand specified in SMBv2 or SMBv3 header is beyond payload boundary.

Connection-oriented DCE/RPC events

SID	Description
27	The preprocessor will alert if the connection-oriented DCE/RPC major version contained in the header is not equal to 5.
28	The preprocessor will alert if the connection-oriented DCE/RPC minor version contained in the header is not equal to 0.
29	The preprocessor will alert if the connection-oriented DCE/RPC PDU type contained in the header is not a valid PDU type.
30	The preprocessor will alert if the fragment length defined in the header is less than the size of the header.
31	The preprocessor will alert if the remaining fragment length is less than the remaining packet size.
32	

	The preprocessor will alert if in a `Bind` or `Alter Context` request, there are no context items specified.
33	The preprocessor will alert if in a `Bind` or `Alter Context` request, there are no transfer syntaxes to go with the requested interface.
34	The preprocessor will alert if a non-last fragment is less than the size of the negotiated maximum fragment length. Most evasion techniques try to fragment the data as much as possible and usually each fragment comes well below the negotiated transmit size.
35	The preprocessor will alert if a fragment is larger than the maximum negotiated fragment length.
36	The byte order of the request data is determined by the Bind in connection-oriented DCE/RPC for Windows. It is anomalous behavior to attempt to change the byte order mid-session.
37	The call id for a set of fragments in a fragmented request should stay the same (it is incremented for each complete request). The preprocessor will alert if it changes in a fragment mid-request.
38	The operation number specifies which function the request is calling on the bound interface. If a request is fragmented, this number should stay the same for all fragments. The preprocessor will alert if the opnum changes in a fragment mid-request.
39	The context id is a handle to a interface that was bound to. If a request if fragmented, this number should stay the same for all fragments. The preprocessor will alert if the context id changes in a fragment mid-request.

Connectionless DCE/RPC events

SID	Description
40	The preprocessor will alert if the connectionless DCE/RPC major version is not equal to 4.
41	The preprocessor will alert if the connectionless DCE/RPC PDU type is not a valid PDU type.
42	The preprocessor will alert if the packet data length is less than the size of the connectionless header.
43	The preprocessor will alert if the sequence number uses in a request is the same or less than a previously used sequence number on the session. In testing, wrapping the sequence number space produces strange behavior from the server, so this should be considered anomalous behavior.

2.2.16.5 Rule Options

New rule options are supported by enabling the `dcerpc2` preprocessor:

```
dce_iface
dce_opnum
dce_stub_data
```

New modifiers to existing `byte_test` and `byte_jump` rule options:

```
byte_test:dce
byte_jump:dce
```

`dce_iface`

For DCE/RPC based rules it has been necessary to set flow-bits based on a client bind to a service to avoid false positives. It is necessary for a client to bind to a service before being able to make a call to it. When a client sends a bind request to the server, it can, however, specify one or more service interfaces to bind to. Each interface is represented by a UUID. Each interface UUID is paired with a unique index (or context id) that future requests can use to reference the service that the client is making a call to. The server will respond with the interface UUIDs it accepts as valid and will allow the client to make requests to those services. When a client makes a request, it will specify the context id so the server knows what service the client is making a request to. Instead of using flow-bits, a rule can simply ask the preprocessor, using this rule option, whether or not the client has bound to a specific interface UUID and whether or not this client request is making a request to it. This can eliminate false positives where more than one service is bound to successfully since the preprocessor can correlate the bind UUID to the context id used in the request. A DCE/RPC request can specify whether numbers are represented as big endian or little endian. The representation of the interface UUID is different depending on the endianness specified in the DCE/RPC previously requiring two rules - one for big endian and one for little endian. The preprocessor eliminates the need for two rules by normalizing the UUID. An interface contains a version. Some versions of an interface may not be vulnerable to a certain exploit. Also, a DCE/RPC request can be broken up into 1 or more fragments. Flags (and a field in the connectionless header) are set in the DCE/RPC header to indicate whether the fragment is the first, a middle or the last fragment. Many checks for data in the DCE/RPC request are only relevant if the DCE/RPC request is a first fragment (or full request), since subsequent fragments will contain data deeper into the DCE/RPC request. A rule which is looking for data, say 5 bytes into the request (maybe it's a length field), will be looking at the wrong data on a fragment other than the first, since the beginning of subsequent fragments are already offset some length from the beginning of the request. This can be a source of false positives in fragmented DCE/RPC traffic. By default it is reasonable to only evaluate if the request is a first fragment (or full request). However, if the `any_frag` option is used to specify evaluating on all fragments.

Syntax

```
dce_iface:<uuid>[, <operator><version>][, any_frag];

uuid       = hexlong '-' hexshort '-' hexshort '-' 2hexbyte '-' 6hexbyte
hexlong    = 4hexbyte
hexshort   = 2hexbyte
hexbyte    = 2HEXDIGIT
operator   = '<' | '>' | '=' | '!'
version    = 0-65535
```

Examples

```
dce_iface:4b324fc8-1670-01d3-1278-5a47bf6ee188;
dce_iface:4b324fc8-1670-01d3-1278-5a47bf6ee188, <2;
```

```
dce_iface:4b324fc8-1670-01d3-1278-5a47bf6ee188, any_frag;
dce_iface:4b324fc8-1670-01d3-1278-5a47bf6ee188, =1, any_frag;
```

This option is used to specify an interface UUID. Optional arguments are an interface version and operator to specify that the version be less than ('<'), greater than ('>'), equal to ('=') or not equal to ('!') the version specified. Also, by default the rule will only be evaluated for a first fragment (or full request, i.e. not a fragment) since most rules are written to start at the beginning of a request. The `any_frag` argument says to evaluate for middle and last fragments as well. This option requires tracking client `Bind` and `Alter Context` requests as well as server `Bind Ack` and `Alter Context` responses for connection-oriented DCE/RPC in the preprocessor. For each `Bind` and `Alter Context` request, the client specifies a list of interface UUIDs along with a handle (or context id) for each interface UUID that will be used during the DCE/RPC session to reference the interface. The server response indicates which interfaces it will allow the client to make requests to - it either accepts or rejects the client's wish to bind to a certain interface. This tracking is required so that when a request is processed, the context id used in the request can be correlated with the interface UUID it is a handle for.

`hexlong` and `hexshort` will be specified and interpreted to be in big endian order (this is usually the default way an interface UUID will be seen and represented). As an example, the following Messenger interface UUID as taken off the wire from a little endian `Bind` request:

```
|f8 91 7b 5a 00 ff d0 11 a9 b2 00 c0 4f b6 e6 fc|
```

must be written as:

```
5a7b91f8-ff00-11d0-a9b2-00c04fb6e6fc
```

The same UUID taken off the wire from a big endian `Bind` request:

```
|5a 7b 91 f8 ff 00 11 d0 a9 b2 00 c0 4f b6 e6 fc|
```

must be written the same way:

```
5a7b91f8-ff00-11d0-a9b2-00c04fb6e6fc
```

This option matches if the specified interface UUID matches the interface UUID (as referred to by the context id) of the DCE/RPC request and if supplied, the version operation is true. This option will not match if the fragment is not a first fragment (or full request) unless the `any_frag` option is supplied in which case only the interface UUID and version need match. Note that a defragmented DCE/RPC request will be considered a full request.

> **Note:** Using this rule option will automatically insert fast pattern contents into the fast pattern matcher. For UDP rules, the interface UUID, in both big and little endian format will be inserted into the fast pattern matcher. For TCP rules, (1) if the rule option `flow:to_server|from_client` is used, $\vert$05 00 00$\vert$ will be inserted into the fast pattern matcher, (2) if the rule option `flow:from_server|to_client` is used, $ \vert$05 00 02$\vert$ will be inserted into the fast pattern matcher and (3) if the flow isn't known, $\vert$05 00$\vert$ will be inserted into the fast pattern matcher. Note that if the rule already has content rule options in it, the best (meaning longest) pattern will be used. If a content in the rule uses the `fast_pattern` rule option, it will unequivocally be used over the above mentioned patterns.

`dce_opnum`

The opnum represents a specific function call to an interface. After is has been determined that a client has bound to a specific interface and is making a request to it (see above - `dce_iface`) usually we want to know what function call it is making to that service. It is likely that an exploit lies in the particular DCE/RPC function call.

Syntax

```
dce_opnum:<opnum-list>;

opnum-list   = opnum-item | opnum-item ',' opnum-list
opnum-item   = opnum | opnum-range
opnum-range  = opnum '-' opnum
opnum        = 0-65535
```

Examples

```
dce_opnum:15;
dce_opnum:15-18;
dce_opnum:15, 18-20;
dce_opnum:15, 17, 20-22;
```

This option is used to specify an opnum (or operation number), opnum range or list containing either or both opnum and/or opnum-range. The opnum of a DCE/RPC request will be matched against the opnums specified with this option. This option matches if any one of the opnums specified match the opnum of the DCE/RPC request.

`dce_stub_data`

Since most netbios rules were doing protocol decoding only to get to the DCE/RPC stub data, i.e. the remote procedure call or function call data, this option will alleviate this need and place the cursor at the beginning of the DCE/RPC stub data. This reduces the number of rule option checks and the complexity of the rule.

This option takes no arguments.

Example

```
dce_stub_data;
```

This option is used to place the cursor (used to walk the packet payload in rules processing) at the beginning of the DCE/RPC stub data, regardless of preceding rule options. There are no arguments to this option. This option matches if there is DCE/RPC stub data.

The cursor is moved to the beginning of the stub data. All ensuing rule options will be considered "sticky" to this buffer. The first rule option following `dce_stub_data` should use absolute location modifiers if it is position-dependent. Subsequent rule options should use a relative modifier if they are meant to be relative to a previous rule option match in the stub data buffer. Any rule option that does not specify a relative modifier will be

evaluated from the start of the stub data buffer. To leave the stub data buffer and return to the main payload buffer, use the `pkt_data` rule option - see section [*] for details).

`byte_test` and `byte_jump` with `dce`

A DCE/RPC request can specify whether numbers are represented in big or little endian. These rule options will take as a new argument `dce` and will work basically the same as the normal `byte_test`/`byte_jump`, but since the DCE/RPC preprocessor will know the endianness of the request, it will be able to do the correct conversion.

`byte_test`

Syntax

```
byte_test:<convert>, [!]<operator>, <value>, <offset> [, relative], dce;

convert    = 1 | 2 | 4 (only with option "dce")
operator   = '<' | '=' | '>' | '<=' | '>=' | '&' | '^'
value      = 0 - 4294967295
offset     = -65535 to 65535
```

Examples

```
byte_test:4, >, 35000, 0, relative, dce;
byte_test:2, !=, 2280, -10, relative, dce;
```

When using the `dce` argument to a `byte_test`, the following normal `byte_test` arguments will not be allowed: `big`, `little`, `string`, `hex`, `dec` and `oct`.

`byte_jump`

Syntax

```
byte_jump:<convert>, <offset>[, relative][, multiplier <mult_value>] \
    [, align][, post_offset <adjustment_value>], dce;

convert           = 1 | 2 | 4 (only with option "dce")
offset            = -65535 to 65535
mult_value        = 0 - 65535
adjustment_value  = -65535 to 65535
```

Example

```
byte_jump:4,-4,relative,align,multiplier 2,post_offset -4,dce;
```

When using the `dce` argument to a `byte_jump`, the following normal `byte_jump` arguments will not be allowed: `big`, `little`, `string`, `hex`, `dec`, `oct` and `from_beginning`.

Example of rule complexity reduction

The following two rules using the new rule options replace 64 (set and isset flowbit) rules that are necessary if the new rule options are not used:

```
    alert tcp $EXTERNAL_NET any -> $HOME_NET [135,139,445,593,1024:] \
        (msg:"dns R_Dnssrv funcs2 overflow attempt"; flow:established,to_server; \
        dce_iface:50abc2a4-574d-40b3-9d66-ee4fd5fba076; dce_opnum:0-11;
dce_stub_data; \
        pcre:"/^.{12}(\x00\x00\x00\x00|.{12})/s"; byte_jump:4,-4,relative,align,dce;
\
        byte_test:4,>,256,4,relative,dce; reference:bugtraq,23470;
reference:cve,2007-1748; \
        classtype:attempted-admin; sid:1000068;)

    alert udp $EXTERNAL_NET any -> $HOME_NET [135,1024:] \
        (msg:"dns R_Dnssrv funcs2 overflow attempt"; flow:established,to_server; \
        dce_iface:50abc2a4-574d-40b3-9d66-ee4fd5fba076; dce_opnum:0-11;
dce_stub_data; \
        pcre:"/^.{12}(\x00\x00\x00\x00|.{12})/s"; byte_jump:4,-4,relative,align,dce;
\
        byte_test:4,>,256,4,relative,dce; reference:bugtraq,23470;
reference:cve,2007-1748; \
        classtype:attempted-admin; sid:1000069;)
```

2.2.17 Sensitive Data Preprocessor

The Sensitive Data preprocessor is a Snort module that performs detection and filtering of Personally Identifiable Information (PII). This information includes credit card numbers, U.S. Social Security numbers, and email addresses. A limited regular expression syntax is also included for defining your own PII.

2.2.17.1 Dependencies

The Stream preprocessor must be enabled for the Sensitive Data preprocessor to work.

2.2.17.2 Preprocessor Configuration

Sensitive Data configuration is split into two parts: the preprocessor config, and the rule options. The preprocessor config starts with:

```
preprocessor sensitive_data:
```

Option syntax

Option	Argument	Required	Default

alert_threshold	<number>	NO	alert_threshold 25
mask_output	NONE	NO	OFF
ssn_file	<filename>	NO	OFF

```
    alert_threshold     =  1 - 65535
```

Option explanations

`alert_threshold`

The preprocessor will alert when any combination of PII are detected in a session. This option specifies how many need to be detected before alerting. This should be set higher than the highest individual count in your "sd_pattern" rules.

`mask_output`

This option replaces all but the last 4 digits of a detected PII with "X"s. This is only done on credit card & Social Security numbers, where an organization's regulations may prevent them from seeing unencrypted numbers.

`ssn_file`

A Social Security number is broken up into 3 sections: Area (3 digits), Group (2 digits), and Serial (4 digits). On a monthly basis, the Social Security Administration publishes a list of which Group numbers are in use for each Area. These numbers can be updated in Snort by supplying a CSV file with the new maximum Group numbers to use. By default, Snort recognizes Social Security numbers issued up through November 2009.

Example preprocessor config

```
preprocessor sensitive_data: alert_threshold 25 \
                             mask_output \
                             ssn_file ssn_groups_Jan10.csv
```

2.2.17.3 Rule Options

Snort rules are used to specify which PII the preprocessor should look for. A new rule option is provided by the preprocessor:

```
sd_pattern
```

This rule option specifies what type of PII a rule should detect.

Syntax

```
sd_pattern:<count>, <pattern>;
```

```
count   = 1 - 255
pattern = any string
```

Option Explanations

`count`

This dictates how many times a PII pattern must be matched for an alert to be generated. The count is tracked across all packets in a session.

`pattern`

This is where the pattern of the PII gets specified. There are a few built-in patterns to choose from:

`credit_card`

The "credit_card" pattern matches 15- and 16-digit credit card numbers. These numbers may have spaces, dashes, or nothing in between groups. This covers Visa, Mastercard, Discover, and American Express. Credit card numbers matched this way have their check digits verified using the Luhn algorithm.

`us_social`

This pattern matches against 9-digit U.S. Social Security numbers. The SSNs are expected to have dashes between the Area, Group, and Serial sections.

SSNs have no check digits, but the preprocessor will check matches against the list of currently allocated group numbers.

`us_social_nodashes`

This pattern matches U.S. Social Security numbers without dashes separating the Area, Group, and Serial sections.

`email`

This pattern matches against email addresses.

If the pattern specified is not one of the above built-in patterns, then it is the definition of a custom PII pattern. Custom PII types are defined using a limited regex-style syntax. The following special characters and escape sequences are supported:

`\d`	matches any digit

`\D`	matches any non-digit
`\l`	matches any letter
`\L`	matches any non-letter
`\w`	matches any alphanumeric character
`\W`	matches any non-alphanumeric character
`{num}`	used to repeat a character or escape sequence "num" times. example: "{3}" matches 3 digits.
`?`	makes the previous character or escape sequence optional. example: " ?" matches an optional space. This behaves in a greedy manner.
`\\`	matches a backslash
\{, \}	matches { and }
\?	matches a question mark.

Other characters in the pattern will be matched literally.

Note: Unlike PCRE, `\w` in this rule option does NOT match underscores.

Examples

```
sd_pattern: 2,us_social;
```

Alerts when 2 social security numbers (with dashes) appear in a session.

```
sd_pattern: 5,(\d{3})\d{3}-\d{4};
```

Alerts on 5 U.S. phone numbers, following the format (123)456-7890

Whole rule example:

```
alert tcp $HOME_NET $HIGH_PORTS -> $EXTERNAL_NET $SMTP_PORTS \
(msg:"Credit Card numbers sent over email"; gid:138; sid:1000; rev:1; \
sd_pattern:4,credit_card; metadata:service smtp;)
```

Caveats

`sd_pattern` is not compatible with other rule options. Trying to use other rule options with `sd_pattern` will result in an error message.

Rules using `sd_pattern` must use GID 138.

2.2.18 Normalizer

When operating Snort in inline mode, it is helpful to normalize packets to help minimize the chances of evasion.

To enable the normalizer, use the following when configuring Snort:

```
./configure --enable-normalizer
```

The normalize preprocessor is activated via the conf as outlined below. There are also many new preprocessor and decoder rules to alert on or drop packets with "abnormal" encodings.

Note that in the following, fields are cleared only if they are non-zero. Also, normalizations will only be enabled if the selected DAQ supports packet replacement and is operating in inline mode.

If a policy is configured for `inline_test` or passive mode, any normalization statements in the policy config are ignored.

2.2.18.1 IP4 Normalizations

IP4 normalizations are enabled with:

```
preprocessor normalize_ip4: [df], [rf], [tos], [trim]
```

Base normalizations enabled with "preprocessor `normalize_ip4`" include:

- TTL normalization if enabled (explained below).
- Clear the differentiated services field (formerly TOS).
- NOP all options octets.

Optional normalizations include:

- `df` don't fragment: clear this bit on incoming packets.
- `rf` reserved flag: clear this bit on incoming packets.
- `tos` type of service (differentiated services): clear this byte.
- `trim` truncate packets with excess payload to the datagram length specified in the IP header + the layer 2 header (e.g. ethernet), but don't truncate below minimum frame length. This is automatically disabled if the DAQ can't inject packets.

2.2.18.2 IP6 Normalizations

IP6 normalizations are enabled with:

```
preprocessor normalize_ip6
```

Base normalizations enabled with "preprocessor `normalize_ip6`" include:

- Hop limit normalization if enabled (explained below).
- NOP all options octets in hop-by-hop and destination options extension headers.

2.2.18.3 ICMP4/6 Normalizations

ICMP4 and ICMP6 normalizations are enabled with:

```
preprocessor normalize_icmp4
preprocessor normalize_icmp6
```

Base normalizations enabled with the above include:

- Clear the code field in echo requests and replies.

2.2.18.4 TCP Normalizations

TCP normalizations are enabled with:

```
preprocessor normalize_tcp: \
    [block], [rsv], [pad], \
    [req_urg], [req_pay], [req_urp], \
    [ips], [urp], [trim], \
    [trim_syn], [trim_rst], \
    [trim_win], [trim_mss], \
    [ecn <ecn_type>], \
    [opts [allow <allowed_opt>+]]

<ecn_type> ::= stream | packet

<allowed_opt> ::= \
    sack | echo | partial_order | conn_count | alt_checksum | md5 | <num>

<sack> ::= { 4, 5 }
<echo> ::= { 6, 7 }
<partial_order> ::= { 9, 10 }
<conn_count> ::= { 11, 12, 13 }
<alt_checksum> ::= { 14, 15 }
<md5> ::= { 19 }
<num> ::= (3..255)
```

Normalizations include:

- `block` allow packet drops during TCP normalization.
- `rsv` clear the reserved bits in the TCP header.
- `pad` clear any option padding bytes.
- `req_urg` clear the urgent pointer if the urgent flag is not set.

- `req_pay` clear the urgent pointer and the urgent flag if there is no payload.
- `req_urp` clear the urgent flag if the urgent pointer is not set.
- `ips` ensure consistency in retransmitted data (also forces reassembly policy to "first"). Any segments that can't be properly reassembled will be dropped.
- `trim_syn` remove data on SYN.
- `trim_rst` remove any data from RST packet.
- `trim_win` trim data to window.
- `trim_mss` trim data to MSS.
- `trim` enable all of the above trim options.
- `ecn packet`

  clear ECN flags on a per packet basis (regardless of negotiation).
- `ecn stream`

  clear ECN flags if usage wasn't negotiated. Should also enable `require_3whs`.
- `opts`

  NOP all option bytes other than maximum segment size, window scaling, timestamp, and any explicitly allowed with the allow keyword. You can allow options to pass by name or number.
- `opts`

  if timestamp is present but invalid, or valid but not negotiated, NOP the timestamp octets.
- `opts`

  if timestamp was negotiated but not present, block the packet.
- `opts`

  clear TS ECR if ACK flag is not set.
- `opts`

  MSS and window scale options are NOP'd if SYN flag is not set.

2.2.18.5 TTL Normalization

TTL normalization pertains to both IP4 TTL (time-to-live) and IP6 (hop limit) and is only performed if both the relevant base normalization is enabled (as described above) and the minimum and new TTL values are configured, as follows:

```
config min_ttl: <min_ttl>
config new_ttl: <new_ttl>
```

```
<min_ttl> ::= (1..255)
<new_ttl> ::= (<min_ttl>+1..255)
```

If `new_ttl` $>$ `min_ttl`, then if a packet is received with a TTL $<$ `min_ttl`, the TTL will be set to `new_ttl`.

Note that this configuration item was deprecated in 2.8.6:

```
preprocessor stream5_tcp: min_ttl <#>
```

By default `min_ttl` = 1 (TTL normalization is disabled). When TTL normalization is turned on the `new_ttl` is set to 5 by default.

2.2.19 SIP Preprocessor

Session Initiation Protocol (SIP) is an application-layer control (signaling) protocol for creating, modifying, and terminating sessions with one or more participants. These sessions include Internet telephone calls, multimedia distribution, and multimedia conferences. SIP Preprocessor provides ways to tackle Common Vulnerabilities and Exposures (CVEs) related with SIP found over the past few years. It also makes detecting new attacks easier.

2.2.19.1 Dependency Requirements

For proper functioning of the preprocessor:

- Stream session tracking must be enabled, i.e. stream5. Both TCP and UDP must be enabled in stream5. The preprocessor requires a session tracker to keep its data. In addition, Stream API is able to provide correct support for ignoring audio/video data channel.
- IP defragmentation should be enabled, i.e. the frag3 preprocessor should be enabled and configured.

2.2.19.2 Configuration

The preprocessor configuration name is `sip`.

```
preprocessor sip
```

Option syntax

Option	Argument	Required	Default
`disabled`	NONE	NO	OFF
`max_sessions`	`<max_sessions>`	NO	`max_sessions 10000`

max_dialogs	<max_dialogs>	NO	max_dialogs 4
ports	<ports>	NO	ports { 5060 5061 }
methods	<methods>	NO	methods { invite cancel ack bye register options }
max_uri_len	<max_uri_len>	NO	max_uri_len 256
max_call_id_len	<max_call_id_len>	NO	max_call_id_len 256
max_requestName_len	<max_requestName_len>	NO	max_requestName_len 20
max_from_len	<max_from_len>	NO	max_from_len 256
max_to_len	<max_to_len>	NO	max_to_len 256
max_via_len	<max_via_len>	NO	max_via_len 1024
max_contact_len	<max_contact_len>	NO	max_contact_len 256
max_content_len	<max_content_len>	NO	max_content_len 1024
ignore_call_channel	NONE	NO	OFF

```
     max_sessions        = 1024-4194303
     max_dialogs         = 1-4194303
     methods             = "invite"|"cancel"|"ack"|"bye"|"register"| "options"\
                           |"refer"
|"subscribe"|"update"|"join"|"info"|"message"\
                           |"notify"|"prack"
     max_uri_len         = 0-65535
     max_call_id_len     = 0-65535
     max_requestName_len = 0-65535
     max_from_len        = 0-65535
     max_to_len          = 0-65535
     max_via_len         = 0-65535
     max_contact_len     = 0-65535
     max_content_len     = 0-65535
```

Option explanations

`disabled`

SIP dynamic preprocessor can be enabled/disabled through configuration. By default this value is turned off. When the preprocessor is disabled, only the max_sessions option is applied when specified with the configuration.

`max_sessions`

This specifies the maximum number of sessions that can be allocated. Those sessions are stream sessions, so they are bounded by maximum number of stream sessions. Default is 10000.

`max_dialogs`

This specifies the maximum number of dialogs within one stream session. If exceeded, the oldest dialog will be dropped. Default is 4.

`ports`

This specifies on what ports to check for SIP messages. Typically, this will include 5060, 5061.

Syntax

```
    ports { <port> [<port>< ... >] }
```

Examples

```
    ports { 5060 5061 }
```

Note: there are spaces before and after `{' and `}'.

`methods`

This specifies on what methods to check for SIP messages: (1) invite, (2) cancel, (3) ack, (4) bye, (5) register, (6) options, (7) refer, (8) subscribe, (9) update (10) join (11) info (12) message (13) notify (14) prack. Note: those 14 methods are up to date list (Feb. 2011). New methods can be added to the list. Up to 32 methods supported.

Syntax

```
      methods { <method-list> }
      method-list = method|method method-list
      methods     = "invite"|"cancel"|"ack"|"bye"|"register"| "options"\

|"refer"|"subscribe"|"update"|"join"|"info"|"message"\
                    |"notify"|"prack"
```

Examples

```
   methods { invite cancel ack bye register options }
   methods { invite cancel ack bye register options information }
```

Note: there are spaces before and after `{' and `}'.

`max_uri_len`

This specifies the maximum Request URI field size. If the Request URI field is greater than this size, an alert is generated. Default is set to 256. The allowed range for this option is 0 - 65535. “0” means never alert.

`max_call_id_len`

This specifies the maximum Call-ID field size. If the Call-ID field is greater than this size, an alert is generated. Default is set to 256. The allowed range for this option is 0 - 65535. “0” means never alert.

`max_requestName_len`

This specifies the maximum request name size that is part of the CSeq ID. If the request name is greater than this size, an alert is generated. Default is set to 20. The allowed range for this option is 0 - 65535. “0” means never alert.

`max_from_len`

This specifies the maximum From field size. If the From field is greater than this size, an alert is generated. Default is set to 256. The allowed range for this option is 0 - 65535. “0” means never alert.

`max_to_len`

This specifies the maximum To field size. If the To field is greater than this size, an alert is generated. Default is set to 256. The allowed range for this option is 0 - 65535. “0” means never alert.

`max_via_len`

This specifies the maximum Via field size. If the Via field is greater than this size, an alert is generated. Default is set to 1024. The allowed range for this option is 0 - 65535. “0” means never alert.

`max_contact_len`

This specifies the maximum Contact field size. If the Contact field is greater than this size, an alert is generated. Default is set to 256. The allowed range for this option is 0 - 65535. “0” means never alert.

`max_content_len`

This specifies the maximum content length of the message body. If the content length is greater than this number, an alert is generated. Default is set to 1024. The allowed range for this option is 0 - 65535. “0” means never alert.

`ignore_call_channel`

This enables the support for ignoring audio/video data channel (through Stream API). By default, this is disabled.

Option examples

```
max_sessions 30000
disabled
```

```
    ports { 5060 5061 }
    methods { invite cancel ack bye register options }
    methods { invite cancel ack bye register options information }
    max_uri_len 1024
    max_call_id_len 1024
    max_requestName_len 10
    max_from_len 1024
    max_to_len 1024
    max_via_len 1024
    max_contact_len 1024
    max_content_len 1024
    max_content_len
    ignore_call_channel
```

Configuration examples

```
    preprocessor sip
    preprocessor sip: max_sessions 500000
    preprocessor sip: max_contact_len 512, max_sessions 300000, methods { invite \
                      cancel ack bye register options } , ignore_call_channel
    preprocessor sip: ports { 5060 49848 36780 10270 }, max_call_id_len 200, \
                     max_from_len 100, max_to_len 200, max_via_len 1000, \
                     max_requestName_len 50, max_uri_len 100, ignore_call_channel,\
                     max_content_len 1000
    preprocessor sip: disabled
    preprocessor sip: ignore_call_channel
```

Default configuration

```
    preprocessor sip
```

2.2.19.3 Events

The preprocessor uses GID 140 to register events.

SID	Description
1	If the memory cap is reached and the preprocessor is configured to alert, this alert will be created.
2	Request URI is required. When Request URI is empty, this alert will be created.
3	The Request URI is larger than the defined length in configuration.
4	When Call-ID is empty, this alert will be created.
5	The Call-ID is larger than the defined length in configuration.
6	The sequence e number value MUST be expressible as a 32-bit unsigned integer and MUST be less than 2^{31}.
7	The request name in the CSeq is larger than the defined length in configuration.
8	From field is empty.
9	From field is larger than the defined length in configuration.

10	To field is empty.
11	To field is larger than the defined length in configuration.
12	Via filed is empty.
13	Via filed is larger than the defined length in configuration.
14	Contact is empty, but it is required non-empty for the message.
15	The Contact is larger than the defined length in configuration.
16	The content length is larger than the defined length in configuration or is negative.
17	There are multiple requests in a single packet. Old SIP protocol supports multiple sip messages within one packet.
18	There are inconsistencies between Content-Length in SIP header and actual body data.
19	Request name is invalid in response.
20	Authenticated invite message received, but no challenge from server received. This is the case of InviteReplay billing attack.
21	Authenticated invite message received, but session information has been changed. This is different from re-INVITE, where the dialog has been established. and authenticated. This is can prevent FakeBusy billing attack.
22	Response status code is not a 3 digit number.
23	Content type header field is required if the message body is not empty.
24	SIP version other than 2.0, 1.0, and 1.1 is invalid
25	Mismatch in Method of request and the CSEQ header
26	The method is unknown
27	The number of dialogs in the stream session exceeds the maximal value.

2.2.19.4 Rule Options

New rule options are supported by enabling the `sip` preprocessor:

```
sip_method
sip_stat_code
sip_header
sip_body
```

Overload modifiers to existing `pcre` rule options:

H: Match SIP request or SIP response header, Similar to `sip_header`.
P: Match SIP request or SIP response body, Similar to `sip_body`.

`sip_method`

The `sip_method` keyword is used to check for specific SIP request methods. The list of methods is: invite, cancel, ack, bye, register, options, refer, subscribe, update, join, info, message, notify, prack. More than one method can be specified, via a comma separated list, and are OR'ed together. It will be applied in fast pattern match if available. If the method used in this rule is not listed in the preprocessor configuration, it will be added to the preprocessor configuration for the associated policy.

Syntax

```
    sip_method:<method-list>;
    method-list = method|method, method-list
    method      = ["!"] "invite"|"cancel"|"ack"|"bye"|"register"| "options"\

|"refer"|"subscribe"|"update"|"join"|"info"|"message"\
                  |"notify"|"prack"
    Note: if "!" is used, only one method is allowed in sip_method.
```

Examples

```
   sip_method:invite, cancel
   sip_method:!invite

   Note: If a user wants to use "and", they can use something like this:
   sip_method:!invite; sip_method:!bye
```

`sip_stat_code`

The `sip_stat_code` is used to check the SIP response status code. This option matches if any one of the state codes specified matches the status codes of the SIP response.

Syntax

```
   sip_stat_code:<code _list> ;
   code_list = state_code|state_code, code_list
   code      = "100-999"|"1-9"
```

Note: 1,2,3,4,5,6... mean to check for "1xx", "2xx", '3xx', '4xx', '5xx', '6xx'... responses.

Examples

```
   sip_stat_code:200
   sip_stat_code: 2
   sip_stat_code: 200, 180
```

`sip_header`

The `sip_header` keyword restricts the search to the extracted Header fields of a SIP message request or a response. This works similar to `file_data`.

Syntax

```
sip_header;
```

Examples

```
alert udp any any -> any 5060 (sip_header; content:"CSeq"; )
```

`sip_body`

The `sip_body` keyword places the cursor at the beginning of the Body fields of a SIP message. This works similar to `file_data` and `dce_stub_data`. The message body includes channel information using SDP protocol (Session Description Protocol).

Syntax

```
sip_body;
```

Examples

```
alert udp any any -> any 5060 (sip_body; content:"C=IN 0.0.0.0"; within 100;)
```

`pcre`

SIP overloads two options for `pcre`:

- H: Match SIP header for request or response , Similar to `sip_header`.
- P: Match SIP body for request or response , Similar to `sip_body`.

Examples

```
alert udp any any -> any 5060 (pcre:"/INVITE/H"; sid:1000000;)
alert udp any any -> any 5060 (pcre:"/m=/P"; sid:2000000;)
```

2.2.20 Reputation Preprocessor

Reputation preprocessor provides basic IP blacklist/whitelist capabilities, to block/drop/pass traffic from IP addresses listed. In the past, we use standard Snort rules to implement Reputation-based IP blocking.

This preprocessor will address the performance issue and make the IP reputation management easier. This preprocessor runs before other preprocessors.

2.2.20.1 Configuration

The preprocessor configuration name is `reputation`.

```
preprocessor reputation
```

Option syntax

Option	Argument	Required	Default
`memcap`	`<memcap>`	NO	`memcap 500`
`scan_local`	NONE	NO	OFF
`blacklist`	`<list file name>`	NO	NONE
`whitelist`	`<list file name>`	NO	NONE
`priority`	[blacklist whitelist]	NO	`priority whitelist`
`nested_ip`	[inner outer both]	NO	`nested_ip inner`
`white`	[unblack trust]	NO	`white unblack`

```
memcap        = 1-4095 Mbytes
```

Option explanations

`memcap`

Maximum total memory supported. It can be set up to 4095 Mbytes.

`scan_local`

Enable to inspect local address defined in RFC 1918:

10.0.0.0 - 10.255.255.255 (10/8 prefix)
172.16.0.0 - 172.31.255.255 (172.16/12 prefix)
192.168.0.0 - 192.168.255.255 (192.168/16 prefix)

`blacklist/whitelist`

The IP lists are loaded from external files. It supports relative paths for inclusion and $variables for path. Multiple blacklists or whitelists are supported.

Note: if the same IP is redefined later, it will overwrite the previous one. In other words, IP lists always favors the last file or entry processed.

`priority`

Specify either blacklist or whitelist has higher priority when source/destination is on blacklist while destination/source is on whitelist. By default, whitelist has higher priority. In other words, the packet will be passed when either source or destination is whitelisted.

Note: this only defines priority when there is a decision conflict, during run-time. During initialization time, if the same IP address is defined in whitelist and blacklist, whoever the last one defined will be the final one. Priority does not work on this case.

`nested_ip`

Specify which IP address to be used when there is IP encapsulation.

`white`

Specify the meaning of whitelist. When white means unblack, it unblacks IPs that are in blacklists; when white means trust, the packet gets bypassed, without further detection by snort. You can only specify either unblack or trust.

Note: when white means unblack, whitelist always has higher priority than blacklist.

Configuration examples

```
preprocessor reputation:\
               blacklist /etc/snort/default.blacklist, \
               whitelist /etc/snort/default.whitelist

preprocessor reputation: \
               nested_ip both, \
               blacklist /etc/snort/default.blacklist, \
               whitelist /etc/snort/default.whitelist

preprocessor reputation: \
               memcap  4095, scan_local, nested_ip both, \
               priority whitelist,  \
               blacklist /etc/snort/default.blacklist, \
               whitelist /etc/snort/default.whitelist,
               white trust

$REP_BLACK_FILE1 = ../dshield.list
$REP_BLACK_FILE2 = ../snort.org.list
preprocessor reputation: \
```

```
                 blacklist $REP_BLACK_FILE1,\
                 blacklist $REP_BLACK_FILE2
```

IP List File Format

Syntax

The IP list file has 1 entry per line. The entry can be either IP entry or comment.

IP Entry

CIDR notation $<$comments$>$ line break.

Example:

```
172.16.42.32/32
172.33.42.32/16
```

Comment

The comment start with #

$<$comments$>$

Example

```
#  This is a full line comment
172.33.42.32/16    # This is a in-line comment
```

IP List File Example

```
# This is a full line comment
172.16.42.32/32    # This is an inline comment, line with single CIDR block
172.33.42.32/16
```

Use case

A user wants to protect his/her network from unwanted/unknown IPs, only allowing some trusted IPs. Here is the configuration:

```
preprocessor reputation: \
```

```
        blacklist /etc/snort/default.blacklist
        whitelist /etc/snort/default.whitelist

  In file "default.blacklist"
        # These two entries will match all ipv4 addresses
        1.0.0.0/1
        128.0.0.0/1

  In file "default.whitelist"
        68.177.102.22 # sourcefire.com
        74.125.93.104 # google.com
```

2.2.20.2 Events

Reputation preprocessor uses GID 136 to register events.

SID	Description
1	Packet is blacklisted.
2	Packet is whitelisted.
3	Packet is inspected.

2.2.20.3 Shared memory support

In order to minimize memory consumption when multiple Snort instances are running concurrently, we introduce the support of shared memory. After configured, all the snort instances share the same IP tables in shared memory.

System requirement

This feature is supported only in Linux.

Build configuration

A new option, `-enable-shared-rep` is introduced to `./configure` command. This option enables the support for shared memory.

Configuration

`shared_mem`

If the build supports shared memory, this configuration will enable shared memory. If this option isn't set, standard memory is used. This option must specify a path or directory where IP lists will be loaded in shared memory. One snort instance will create and maintain the shared IP lists. We use instance ID 1, specified in the snort `-G` option to be the master snort. All the other snort instances are clients (readers).

Syntax

```
shared_mem: path
```

Examples

```
shared_mem  /user/reputation/iplists
```

`shared_refresh`

This option changes the period of checking new shared memory segment, in the unit of second. By default, the refresh rate is 60 seconds.

Syntax

```
shared_refresh <period>
period = "1 - 4294967295"
```

Examples

```
shared_refresh 60
```

Steps to configure shared memory

- When building Snort, add option `-enable-shared-rep` to `./configure`
  For example:

  ```
  ./configure --enable-gre --enable-sourcefire --enable-flexresp3
  --enable-pthread --enable-linux-smp-stats
  --enable-targetbased --enable-shared-rep --enable-control-socket
  ```

- Put your IP list file into a directory, where snort has full access.
  For example:

  ```
  /user/reputation/iplists
  ```

  In order to separate whitelist with blacklist, you need to specify whitelist with `.wlf` extension and blacklist with `.blf` extension.

- In snort config file, specify shared memory support with the path to IP files.
  For example:

```
shared_mem  /user/reputation/iplists
```

If you want to change the period of checking new IP lists, add refresh period.
For example:

```
shared_refresh 300
```

- Start shared memory master(writer) with `-G 0` option. Note: only one master should be enabled.
- Start shared memory clients (readers) with `-G 1` or other IDs. Note: for one ID, only one snort instance should be enabled.
- You will see the IP lists got loaded and shared across snort instances!

Reload IP lists using control socket

- Run snort using command line with option `-cs-dir <path>` or configure snort with:

```
config cs_dir:<path>
```

- (Optional) you can create a version file named “IPRVersion.dat” in the IP list directory. This file helps managing reloading IP lists, by specifying a version. When the version isn't changed, IP lists will not be reloaded if they are already in shared memory. The version number should be a 32 bit number.
  For example:

```
VERSION=1
```

- In the `<snort root>/src/tools/control` directory, you will find `snort_control` command if built with `-enable-control-socket` option.
- Type the following command to reload IP lists. Before typing this command, make sure to update version file if you are using version file. The `<path>` is the same path in first step.

```
<snort root>/src/tools/control/snort_control  <path> 1361
```

Using manifest file to manage loading (optional)

Using manifest file, you can control the file loading sequence, action taken, and support zone based detection. You can create a manifest file named “zone.info” in the IP list directory.
When Snort is signaled to load new lists, a manifest file is read first to determine which zones the IPs in each list are applicable to and what action to take per list (Block, White, Monitor).
Files listed in manifest are loaded from top to bottom. You should put files that have higher priority first. In manifest file, you can put up to 255 files. Without manifest file, files will be loaded in alphabet order.
Here's the format of the manifest file. Each line of the file has the following format:

```
<filename>, <list id>,<action>[, <zone>]+

<list id> ::= 32 bit integer
<action> ::= "monitor"|"block"|"white"
<zone>  ::= [0-1051]
```

Using manifest file, you can specify a new action called “monitor”, which indicates a packet needs to be inspected, but does not disable detection. This is different from “block” action, which disables further detection. This new action helps users evaluate their IP lists before applying it.

An example manifest file:

```
#ipreputation manifest file
white.wlf, 111 ,white,
black1.blf, 1112, black,  3, 12
black2.blf, 1113, black,  3, 12
monitor.blf,2222, monitor, 0, 2, 8
```

2.2.21 GTP Decoder and Preprocessor

GTP (GPRS Tunneling Protocol) is used in core communication networks to establish a channel between GSNs (GPRS Serving Node). GTP decoding preprocessor provides ways to tackle intrusion attempts to those networks through GTP. It also makes detecting new attacks easier.

Two components are developed: GTP decoder and GTP preprocessor.

- GTP decoder extracts payload inside GTP PDU;
- GTP preprocessor inspects all the signaling messages and provide keywords for further inspection

When the decoder is enabled and configured, the decoder strips the GTP headers and parses the underlying IP/TCP/UDP encapsulated packets. Therefore all rules and detection work as if there was no GTP header.

Example:

Most GTP packets look like this

```
IP -> UDP -> GTP -> IP -> TCP -> HTTP
```

If you had a standard HTTP rule:

```
alert tcp any any ->  any $HTTP_PORTS (msg:"Test HTTP"; flow:to_server,established;
content:"SOMETHINGEVIL"; http_uri;  .... sid:X; rev:Y;)
```

it would alert on the inner HTTP data that is encapsulated in GTP without any changes to the rule other than enabling and configuring the GTP decoder.

2.2.21.1 Dependency Requirements

For proper functioning of the preprocessor:

- Stream session tracking must be enabled, i.e. stream5. UDP must be enabled in stream5. The preprocessor requires a session tracker to keep its data.
- IP defragmentation should be enabled, i.e. the frag3 preprocessor should be enabled and configured.

2.2.21.2 GTP Data Channel Decoder Configuration

GTP decoder extracts payload from GTP PDU. The following configuration sets GTP decoding:

```
config enable_gtp
```

By default, GTP decoder uses port number 2152 (GTPv1) and 3386 (GTPv0). If users want to change those values, they can use `portvar GTP_PORTS`:

```
portvar GTP_PORTS [2152,3386]
```

2.2.21.3 GTP Control Channel Preprocessor Configuration

Different from GTP decoder, GTP preprocessor examines all signaling messages. The preprocessor configuration name is `gtp`.

```
preprocessor gtp
```

Option syntax

Option	Argument	Required	Default
`ports`	`<ports>`	NO	`ports { 2123 3386 }`

Option explanations

`ports`

This specifies on what ports to check for GTP messages. Typically, this will include 2123, 3386.

Syntax

```
ports { <port> [<port>< ... >] }
```

Examples

```
ports { 2123 3386 2152 }
```

Note: there are spaces before and after `{' and `}'.

Default configuration

```
preprocessor gtp
```

2.2.21.4 GTP Decoder Events

SID	Description
297	Two or more GTP encapsulation layers present
298	GTP header length is invalid

2.2.21.5 GTP Preprocessor Events

SID	Description
1	Message length is invalid.
2	Information element length is invalid.
3	Information elements are out of order.

2.2.21.6 Rule Options

New rule options are supported by enabling the `gtp` preprocessor:

```
gtp_type
gtp_info
gtp_version
```

`gtp_type`

The `gtp_type` keyword is used to check for specific GTP types. User can input message type value, an integer in [0, 255], or a string defined in the Table below. More than one type can be specified, via a comma separated list, and are OR'ed together. If the type used in a rule is not listed in the preprocessor configuration, an error will be thrown.

A message type can have different type value in different GTP versions. For example, `sgsn_context_request` has message type value 50 in GTPv0 and GTPv1, but 130 in GTPv2. `gtp_type` will match to a different value depending on the version number in the packet. In this example, evaluating a GTPv0 or GTPv1 packet will check whether the message type value is 50; evaluating a GTPv2 packet will check whether the message type value is 130. When a message type is not defined in a version, any packet in that version will always return “No match”.

If an integer is used to specify message type, every GTP packet is evaluated, no matter what version the packet is. If the message type matches the value in packet, it will return “Match”.

Syntax

```
gtp_type:<type-list>;
type-list = type|type, type-list
type      = "0-255"|
             | "echo_request" | "echo_response" ...
```

Examples

```
gtp_type:10, 11, echo_request;
```

GTP message types

Type	GTPv0	GTPv1	GTPv2	
0	N/A	N/A	N/A	
1	echo_request	echo_request	echo_request	
2	echo_response	echo_response	echo_response	
3	version_not_supported	version_not_supported	version_not_supported	
4	node_alive_request	node_alive_request	N/A	
5	node_alive_response	node_alive_response	N/A	
6	redirection_request	redirection_request	N/A	
7	redirection_response	redirection_response	N/A	
16	create_pdp_context_request	create_pdp_context_request	N/A	
17	create_pdp_context_response	create_pdp_context_response	N/A	
18	update_pdp_context_request	update_pdp_context_request	N/A	
19	update_pdp_context_response	update_pdp_context_response	N/A	
20	delete_pdp_context_request	delete_pdp_context_request	N/A	
21	delete_pdp_context_response	delete_pdp_context_response	N/A	
22	create_aa_pdp_context_request	init_pdp_context_activation_request	N/A	

23	create_aa_pdp_context_response	init_pdp_context_activation_response	N/A	
24	delete_aa_pdp_context_request	N/A	N/A	
25	delete_aa_pdp_context_response	N/A	N/A	
26	error_indication	error_indication	N/A	
27	pdu_notification_request	pdu_notification_request	N/A	
28	pdu_notification_response	pdu_notification_response	N/A	
29	pdu_notification_reject_request	pdu_notification_reject_request	N/A	
30	pdu_notification_reject_response	pdu_notification_reject_response	N/A	
31	N/A	supported_ext_header_notification	N/A	
32	send_routing_info_request	send_routing_info_request	create_session_request	
33	send_routing_info_response	send_routing_info_response	create_session_response	
34	failure_report_request	failure_report_request	modify_bearer_request	
35	failure_report_response	failure_report_response	modify_bearer_response	
36	note_ms_present_request	note_ms_present_request	delete_session_request	
37	note_ms_present_response	note_ms_present_response	delete_session_response	
38	N/A	N/A	change_notification_request	
39	N/A	N/A	change_notification_response	
48	identification_request	identification_request	N/A	
49	identification_response	identification_response	N/A	
50	sgsn_context_request	sgsn_context_request	N/A	
51	sgsn_context_response	sgsn_context_response	N/A	
52	sgsn_context_ack	sgsn_context_ack	N/A	
53	N/A	forward_relocation_request	N/A	
54	N/A	forward_relocation_response	N/A	
55	N/A	forward_relocation_complete	N/A	
56	N/A	relocation_cancel_request	N/A	
57	N/A	relocation_cancel_response	N/A	
58	N/A	forward_srns_contex	N/A	
59	N/A	forward_relocation_complete_ack	N/A	
60	N/A	forward_srns_contex_ack	N/A	
64	N/A	N/A	modify_bearer_command	
65	N/A	N/A	modify_bearer_failure_indication	
66	N/A	N/A	delete_bearer_command	
67	N/A	N/A	delete_bearer_failure_indication	
68	N/A	N/A	bearer_resource_command	

69	N/A	N/A	bearer_resource_failure_indication	
70	N/A	ran_info_relay	downlink_failure_indication	
71	N/A	N/A	trace_session_activation	
72	N/A	N/A	trace_session_deactivation	
73	N/A	N/A	stop_paging_indication	
95	N/A	N/A	create_bearer_request	
96	N/A	mbms_notification_request	create_bearer_response	
97	N/A	mbms_notification_response	update_bearer_request	
98	N/A	mbms_notification_reject_request	update_bearer_response	
99	N/A	mbms_notification_reject_response	delete_bearer_request	
100	N/A	create_mbms_context_request	delete_bearer_response	
101	N/A	create_mbms_context_response	delete_pdn_request	
102	N/A	update_mbms_context_request	delete_pdn_response	
103	N/A	update_mbms_context_response	N/A	
104	N/A	delete_mbms_context_request	N/A	
105	N/A	delete_mbms_context_response	N/A	
112	N/A	mbms_register_request	N/A	
113	N/A	mbms_register_response	N/A	
114	N/A	mbms_deregister_request	N/A	
115	N/A	mbms_deregister_response	N/A	
116	N/A	mbms_session_start_request	N/A	
117	N/A	mbms_session_start_response	N/A	
118	N/A	mbms_session_stop_request	N/A	
119	N/A	mbms_session_stop_response	N/A	
120	N/A	mbms_session_update_request	N/A	
121	N/A	mbms_session_update_response	N/A	
128	N/A	ms_info_change_request	identification_request	
129	N/A	ms_info_change_response	identification_response	
130	N/A	N/A	sgsn_context_request	
131	N/A	N/A	sgsn_context_response	
132	N/A	N/A	sgsn_context_ack	
133	N/A	N/A	forward_relocation_request	
134	N/A	N/A	forward_relocation_response	
135	N/A	N/A	forward_relocation_complete	
136	N/A	N/A	forward_relocation_complete_ack	

137	N/A	N/A	forward_access	
138	N/A	N/A	forward_access_ack	
139	N/A	N/A	relocation_cancel_request	
140	N/A	N/A	relocation_cancel_response	
141	N/A	N/A	configuration_transfer_tunnel	
149	N/A	N/A	detach	
150	N/A	N/A	detach_ack	
151	N/A	N/A	cs_paging	
152	N/A	N/A	ran_info_relay	
153	N/A	N/A	alert_mme	
154	N/A	N/A	alert_mme_ack	
155	N/A	N/A	ue_activity	
156	N/A	N/A	ue_activity_ack	
160	N/A	N/A	create_forward_tunnel_request	
161	N/A	N/A	create_forward_tunnel_response	
162	N/A	N/A	suspend	
163	N/A	N/A	suspend_ack	
164	N/A	N/A	resume	
165	N/A	N/A	resume_ack	
166	N/A	N/A	create_indirect_forward_tunnel_request	
167	N/A	N/A	create_indirect_forward_tunnel_response	
168	N/A	N/A	delete_indirect_forward_tunnel_request	
169	N/A	N/A	delete_indirect_forward_tunnel_response	
170	N/A	N/A	release_access_bearer_request	
171	N/A	N/A	release_access_bearer_response	
176	N/A	N/A	downlink_data	
177	N/A	N/A	downlink_data_ack	
178	N/A	N/A	N/A	
179	N/A	N/A	pgw_restart	
199	N/A	N/A	pgw_restart_ack	
200	N/A	N/A	update_pdn_request	
201	N/A	N/A	update_pdn_response	
211	N/A	N/A	modify_access_bearer_request	
212	N/A	N/A	modify_access_bearer_response	
231	N/A	N/A	mbms_session_start_request	

232	N/A	N/A	mbms_session_start_response	
233	N/A	N/A	mbms_session_update_request	
234	N/A	N/A	mbms_session_update_response	
235	N/A	N/A	mbms_session_stop_request	
236	N/A	N/A	mbms_session_stop_response	
240	data_record_transfer_request	data_record_transfer_request	N/A	
241	data_record_transfer_response	data_record_transfer_response	N/A	
254	N/A	end_marker	N/A	
255	pdu	pdu	N/A	

`gtp_info`

The `gtp_info` keyword is used to check for specific GTP information element. This keyword restricts the search to the information element field. User can input information element value, an integer in $[0, 255]$, or a string defined in the Table below. If the information element used in this rule is not listed in the preprocessor configuration, an error will be thrown.

When there are several information elements with the same type in the message, this keyword restricts the search to the total consecutive buffer. Because the standard requires same types group together, this feature will be available for all valid messages. In the case of “out of order information elements”, this keyword restricts the search to the last buffer.

Similar to message type, same information element might have different information element value in different GTP versions. For example, `cause` has value 1 in GTPv0 and GTPv1, but 2 in GTPv2. `gtp_info` will match to a different value depending on the version number in the packet. When an information element is not defined in a version, any packet in that version will always return “No match”.

If an integer is used to specify information element type, every GTP packet is evaluated, no matter what version the packet is. If the message type matches the value in packet, it will return “Match”.

Syntax

```
gtp_info:<ie>;
ie      = "0-255"|
          "rai" | "tmsi"...
```

Examples

```
gtp_info: 16;
gtp_info: tmsi
```

GTP information elements

Type	GTPv0	GTPv1	GTPv2	

0	N/A	N/A	N/A	
1	cause	cause	imsi	
2	imsi	imsi	cause	
3	rai	rai	recovery	
4	tlli	tlli	N/A	
5	p_tmsi	p_tmsi	N/A	
6	qos	N/A	N/A	
7	N/A	N/A	N/A	
8	recording_required	recording_required	N/A	
9	authentication	authentication	N/A	
10	N/A	N/A	N/A	
11	map_cause	map_cause	N/A	
12	p_tmsi_sig	p_tmsi_sig	N/A	
13	ms_validated	ms_validated	N/A	
14	recovery	recovery	N/A	
15	selection_mode	selection_mode	N/A	
16	flow_label_data_1	teid_1	N/A	
17	flow_label_signalling	teid_control	N/A	
18	flow_label_data_2	teid_2	N/A	
19	ms_unreachable	teardown_ind	N/A	
20	N/A	nsapi	N/A	
21	N/A	ranap	N/A	
22	N/A	rab_context	N/A	
23	N/A	radio_priority_sms	N/A	
24	N/A	radio_priority	N/A	
25	N/A	packet_flow_id	N/A	
26	N/A	charging_char	N/A	
27	N/A	trace_ref	N/A	
28	N/A	trace_type	N/A	
29	N/A	ms_unreachable	N/A	
71	N/A	N/A	apn	
72	N/A	N/A	ambr	
73	N/A	N/A	ebi	
74	N/A	N/A	ip_addr	
75	N/A	N/A	mei	

76	N/A	N/A	msisdn	
77	N/A	N/A	indication	
78	N/A	N/A	pco	
79	N/A	N/A	paa	
80	N/A	N/A	bearer_qos	
81	N/A	N/A	flow_qos	
82	N/A	N/A	rat_type	
83	N/A	N/A	serving_network	
84	N/A	N/A	bearer_tft	
85	N/A	N/A	tad	
86	N/A	N/A	uli	
87	N/A	N/A	f_teid	
88	N/A	N/A	tmsi	
89	N/A	N/A	cn_id	
90	N/A	N/A	s103pdf	
91	N/A	N/A	s1udf	
92	N/A	N/A	delay_value	
93	N/A	N/A	bearer_context	
94	N/A	N/A	charging_id	
95	N/A	N/A	charging_char	
96	N/A	N/A	trace_info	
97	N/A	N/A	bearer_flag	
98	N/A	N/A	N/A	
99	N/A	N/A	pdn_type	
100	N/A	N/A	pti	
101	N/A	N/A	drx_parameter	
102	N/A	N/A	N/A	
103	N/A	N/A	gsm_key_tri	
104	N/A	N/A	umts_key_cipher_quin	
105	N/A	N/A	gsm_key_cipher_quin	
106	N/A	N/A	umts_key_quin	
107	N/A	N/A	eps_quad	
108	N/A	N/A	umts_key_quad_quin	
109	N/A	N/A	pdn_connection	
110	N/A	N/A	pdn_number	

111	N/A	N/A	p_tmsi	
112	N/A	N/A	p_tmsi_sig	
113	N/A	N/A	hop_counter	
114	N/A	N/A	ue_time_zone	
115	N/A	N/A	trace_ref	
116	N/A	N/A	complete_request_msg	
117	N/A	N/A	guti	
118	N/A	N/A	f_container	
119	N/A	N/A	f_cause	
120	N/A	N/A	plmn_id	
121	N/A	N/A	target_id	
122	N/A	N/A	N/A	
123	N/A	N/A	packet_flow_id	
124	N/A	N/A	rab_contex	
125	N/A	N/A	src_rnc_pdcp	
126	N/A	N/A	udp_src_port	
127	charge_id	charge_id	apn_restriction	
128	end_user_address	end_user_address	selection_mode	
129	mm_context	mm_context	src_id	
130	pdp_context	pdp_context	N/A	
131	apn	apn	change_report_action	
132	protocol_config	protocol_config	fq_csid	
133	gsn	gsn	channel	
134	msisdn	msisdn	emlpp_pri	
135	N/A	qos	node_type	
136	N/A	authentication_qu	fqdn	
137	N/A	tft	ti	
138	N/A	target_id	mbms_session_duration	
139	N/A	utran_trans	mbms_service_area	
140	N/A	rab_setup	mbms_session_id	
141	N/A	ext_header	mbms_flow_id	
142	N/A	trigger_id	mbms_ip_multicast	
143	N/A	omc_id	mbms_distribution_ack	
144	N/A	ran_trans	rfsp_index	
145	N/A	pdp_context_pri	uci	

146	N/A	addi_rab_setup	csg_info	
147	N/A	sgsn_number	csg_id	
148	N/A	common_flag	cmi	
149	N/A	apn_restriction	service_indicator	
150	N/A	radio_priority_lcs	detach_type	
151	N/A	rat_type	ldn	
152	N/A	user_loc_info	node_feature	
153	N/A	ms_time_zone	mbms_time_to_transfer	
154	N/A	imei_sv	throttling	
155	N/A	camel	arp	
156	N/A	mbms_ue_context	epc_timer	
157	N/A	tmp_mobile_group_id	signalling_priority_indication	
158	N/A	rim_routing_addr	tmgi	
159	N/A	mbms_config	mm_srvcc	
160	N/A	mbms_service_area	flags_srvcc	
161	N/A	src_rnc_pdcp	mmbr	
162	N/A	addi_trace_info	N/A	
163	N/A	hop_counter	N/A	
164	N/A	plmn_id	N/A	
165	N/A	mbms_session_id	N/A	
166	N/A	mbms_2g3g_indicator	N/A	
167	N/A	enhanced_nsapi	N/A	
168	N/A	mbms_session_duration	N/A	
169	N/A	addi_mbms_trace_info	N/A	
170	N/A	mbms_session_repetition_num	N/A	
171	N/A	mbms_time_to_data	N/A	
173	N/A	bss	N/A	
174	N/A	cell_id	N/A	
175	N/A	pdu_num	N/A	
176	N/A	N/A	N/A	
177	N/A	mbms_bearer_capab	N/A	
178	N/A	rim_routing_disc	N/A	
179	N/A	list_pfc	N/A	
180	N/A	ps_xid	N/A	
181	N/A	ms_info_change_report	N/A	

182	N/A	direct_tunnel_flags	N/A	
183	N/A	correlation_id	N/A	
184	N/A	bearer_control_mode	N/A	
185	N/A	mbms_flow_id	N/A	
186	N/A	mbms_ip_multicast	N/A	
187	N/A	mbms_distribution_ack	N/A	
188	N/A	reliable_inter_rat_handover	N/A	
189	N/A	rfsp_index	N/A	
190	N/A	fqdn	N/A	
191	N/A	evolved_allocation1	N/A	
192	N/A	evolved_allocation2	N/A	
193	N/A	extended_flags	N/A	
194	N/A	uci	N/A	
195	N/A	csg_info	N/A	
196	N/A	csg_id	N/A	
197	N/A	cmi	N/A	
198	N/A	apn_ambr	N/A	
199	N/A	ue_network	N/A	
200	N/A	ue_ambr	N/A	
201	N/A	apn_ambr_nsapi	N/A	
202	N/A	ggsn_backoff_timer	N/A	
203	N/A	signalling_priority_indication	N/A	
204	N/A	signalling_priority_indication_nsapi	N/A	
205	N/A	high_bitrate	N/A	
206	N/A	max_mbr	N/A	
250	N/A	N/A	N/A	
	N/A	N/A	N/A	
251	charging_gateway_addr	charging_gateway_addr	N/A	
255	private_extension	private_extension	private_extension	

`gtp_version`

The `gtp_version` keyword is used to check for specific GTP version.

Because different GTP version defines different message types and information elements, this keyword should combine with `gtp_type` and `gtp_info`.

Syntax

```
gtp_version:<version>;
version   = "0, 1, 2'
```

Examples

```
gtp_version: 1;
```

2.2.22 Modbus Preprocessor

The Modbus preprocessor is a Snort module that decodes the Modbus protocol. It also provides rule options to access certain protocol fields. This allows a user to write rules for Modbus packets without decoding the protocol with a series of "content" and "byte_test" options.

Modbus is a protocol used in SCADA networks. If your network does not contain any Modbus-enabled devices, we recommend leaving this preprocessor turned off.

2.2.22.1 Dependency Requirements

For proper functioning of the preprocessor:

- Stream session tracking must be enabled, i.e. stream5. TCP must be enabled in stream5. The preprocessor requires a session tracker to keep its data.
- Protocol Aware Flushing (PAF) must be enabled.
- IP defragmentation should be enabled, i.e. the frag3 preprocessor should be enabled and configured.

2.2.22.2 Preprocessor Configuration

To get started, the Modbus preprocessor must be enabled. The preprocessor name is `modbus`.

```
preprocessor modbus
```

Option syntax

Option	Argument	Required	Default
`ports`	`<ports>`	NO	`ports { 502 }`

Option explanations

`ports`

This specifies on what ports to check for Modbus messages. Typically, this will include 502.

Syntax

```
ports { <port> [<port>< ... >] }
```

Examples

```
ports { 1237 3945 5067 }
```

Note: there are spaces before and after `{' and `}'.

Default configuration

```
preprocessor modbus
```

2.2.22.3 Rule Options

The Modbus preprocessor adds 3 new rule options. These rule options match on various pieces of the Modbus headers:

```
modbus_func
modbus_unit
modbus_data
```

The preprocessor must be enabled for these rule option to work.

`modbus_func`

This option matches against the Function Code inside of a Modbus header. The code may be a number (in decimal format), or a string from the list provided below.

Syntax

```
modbus_func:<code>

code  = 0-255 |
        "read_coils" |
        "read_discrete_inputs" |
        "read_holding_registers" |
        "read_input_registers" |
```

```
            "write_single_coil" |
            "write_single_register" |
            "read_exception_status" |
            "diagnostics" |
            "get_comm_event_counter" |
            "get_comm_event_log" |
            "write_multiple_coils" |
            "write_multiple_registers" |
            "report_slave_id" |
            "read_file_record" |
            "write_file_record" |
            "mask_write_register" |
            "read_write_multiple_registers" |
            "read_fifo_queue" |
            "encapsulated_interface_transport"
```

Examples

```
    modbus_func:1;
    modbus_func:write_multiple_coils;
```

`modbus_unit`

This option matches against the Unit ID field in a Modbus header.

Syntax

```
    modbus_unit:<unit>

    unit = 0-255
```

Examples

```
    modbus_unit:1;
```

`modbus_data`

This rule option sets the cursor at the beginning of the Data field in a Modbus request/response.

Syntax

```
    modbus_data;
```

Examples

```
    modbus_data; content:"badstuff";
```

2.2.22.4 Preprocessor Events

The Modbus preprocessor uses GID 144 for its preprocessor events.

SID	Description
1	The length in the Modbus header does not match the length needed
	by the Modbus function code.
	Each Modbus function has an expected format for requests and responses.
	If the length of the message does not match the expected format, this
	alert is generated.
2	Modbus protocol ID is non-zero.
	The protocol ID field is used for multiplexing other protocols with
	Modbus. Since the preprocessor cannot handle these other protocols,
	this alert is generated instead.
3	Reserved Modbus function code in use.

2.2.23 DNP3 Preprocessor

The DNP3 preprocessor is a Snort module that decodes the DNP3 protocol. It also provides rule options to access certain protocol fields. This allows a user to write rules for DNP3 packets without decoding the protocol with a series of "content" and "byte_test" options.

DNP3 is a protocol used in SCADA networks. If your network does not contain any DNP3-enabled devices, we recommend leaving this preprocessor turned off.

2.2.23.1 Dependency Requirements

For proper functioning of the preprocessor:

- Stream session tracking must be enabled, i.e. stream5. TCP or UDP must be enabled in stream5. The preprocessor requires a session tracker to keep its data.
- Protocol Aware Flushing (PAF) must be enabled.
- IP defragmentation should be enabled, i.e. the frag3 preprocessor should be enabled and configured.

2.2.23.2 Preprocessor Configuration

To get started, the DNP3 preprocessor must be enabled. The preprocessor name is `dnp3`.

```
preprocessor dnp3
```

Option syntax

Option	Argument	Required	Default
`ports`	`<ports>`	NO	`ports { 20000 }`
`memcap`	`<number`	NO	`memcap 262144`
`check_crc`	NONE	NO	OFF
`disabled`	NONE	NO	OFF

Option explanations

`ports`

This specifies on what ports to check for DNP3 messages. Typically, this will include 20000.

Syntax

```
ports { <port> [<port>< ... >] }
```

Examples

```
ports { 1237 3945 5067 }
```

Note: there are spaces before and after `{' and `}'.

`memcap`

This sets a maximum to the amount of memory allocated to the DNP3 preprocessor for session-tracking purposes. The argument is given in bytes. Each session requires about 4 KB to track, and the default is 256 kB. This gives the preprocessor the ability to track 63 DNP3 sessions simultaneously. Setting the memcap below 4144 bytes will cause a fatal error. When multiple configs are used, the memcap in the non-default configs will be overwritten by the memcap in the default config. If the default config isn't intended to inspect DNP3 traffic, use the "disabled" keyword.

`check_crc`

This option makes the preprocessor validate the checksums contained in DNP3 Link-Layer Frames. Frames with invalid checksums will be ignored. If the corresponding preprocessor rule is enabled, invalid checksums will generate alerts. The corresponding rule is GID 145, SID 1.

`disabled`

This option is used for loading the preprocessor without inspecting any DNP3 traffic. The `disabled` keyword is only useful when the DNP3 preprocessor is turned on in a separate policy.

Default configuration

```
preprocessor dnp3
```

2.2.23.3 Rule Options

The DNP3 preprocessor adds 4 new rule options. These rule options match on various pieces of the DNP3 headers:

```
dnp3_func
dnp3_obj
dnp3_ind
dnp3_data
```

The preprocessor must be enabled for these rule option to work.

`dnp3_func`

This option matches against the Function Code inside of a DNP3 Application-Layer request/response header. The code may be a number (in decimal format), or a string from the list provided below.

Syntax

```
dnp3_func:<code>

code  = 0-255 |
        "confirm" |
        "read" |
        "write" |
        "select" |
        "operate" |
        "direct_operate" |
        "direct_operate_nr" |
        "immed_freeze" |
        "immed_freeze_nr" |
        "freeze_clear" |
        "freeze_clear_nr" |
        "freeze_at_time" |
        "freeze_at_time_nr" |
        "cold_restart" |
        "warm_restart" |
        "initialize_data" |
        "initialize_appl" |
```

```
            "start_appl" |
            "stop_appl" |
            "save_config" |
            "enable_unsolicited" |
            "disable_unsolicited" |
            "assign_class" |
            "delay_measure" |
            "record_current_time" |
            "open_file" |
            "close_file" |
            "delete_file" |
            "get_file_info" |
            "authenticate_file" |
            "abort_file" |
            "activate_config" |
            "authenticate_req" |
            "authenticate_err" |
            "response" |
            "unsolicited_response" |
            "authenticate_resp"
```

Examples

```
    dnp3_func:1;
    dnp3_func:delete_file;
```

`dnp3_ind`

This option matches on the Internal Indicators flags present in a DNP3 Application Response Header. Much like the TCP flags rule option, providing multiple flags in one option will cause the rule to fire if ANY one of the flags is set. To alert on a combination of flags, use multiple rule options.

Syntax

```
    dnp3_ind:<flag>{,<flag>...]

    flag =  "all_stations"
            "class_1_events"
            "class_2_events"
            "class_3_events"
            "need_time"
            "local_control"
            "defice_trouble"
            "device_restart"
            "no_func_code_support"
            "object_unknown"
            "parameter_error"
            "event_buffer_overflow"
            "already_executing"
            "config_corrupt"
            "reserved_2"
            "reserved_1"
```

Examples

```
# Alert on reserved_1 OR reserved_2
dnp3_ind:reserved_1,reserved_2;

# Alert on class_1 AND class_2 AND class_3 events
dnp3_ind:class_1_events; dnp3_ind:class_2_events; dnp3_ind:class_3_events;
```

`dnp3_obj`

This option matches on DNP3 object headers present in a request or response.

Syntax

```
dnp3_obj:<group>,<var>

group = 0 - 255
var   = 0 - 255
```

Examples

```
# Alert on DNP3 "Date and Time" object
dnp3_obj:50,1;
```

`dnp3_data`

As Snort processes DNP3 packets, the DNP3 preprocessor collects Link-Layer Frames and reassembles them back into Application-Layer Fragments. This rule option sets the cursor to the beginning of an Application-Layer Fragment, so that other rule options can work on the reassembled data.

With the dnp3_data rule option, you can write rules based on the data within Fragments without splitting up the data and adding CRCs every 16 bytes.

Syntax

```
dnp3_data;
```

Examples

```
dnp3_data; content:"badstuff_longer_than_16chars";
```

2.2.23.4 Preprocessor Events

The DNP3 preprocessor uses GID 145 for its preprocessor events.

SID	Description
1	A Link-Layer Frame contained an invalid CRC.
	(Enable `check_crc` in the preprocessor config to get this alert.)

2	A DNP3 Link-Layer Frame was dropped, due to an invalid length.
3	A Transport-Layer Segment was dropped during reassembly.
	This happens when segments have invalid sequence numbers.
4	The DNP3 Reassembly buffer was cleared before a complete fragment could
	be reassembled.
	This happens when a segment carrying the "FIR" flag appears after some
	other segments have been queued.
5	A DNP3 Link-Layer Frame is larger than 260 bytes.
6	A DNP3 Link-Layer Frame uses an address that is reserved.
7	A DNP3 request or response uses a reserved function code.

2.2.24 AppId Preprocessor

With increasingly complex networks and growing network traffic, network administrators require application awareness in managing networks. An administrator may allow only applications that are business relevant, low bandwidth and/or deal with certain subject matter.

AppId preprocessor adds application level view to manage networks. It does this by adding the following features

- Network control: The preprocessor provides simplified single point application awareness by making a set of application identifiers (AppId) available to Snort Rule writers.
- Network usage awareness: the preprocessor outputs statistics to show network bandwidth used by each application seen on network. Administrators can monitor bandwidth usage and may decide to block applications that are wasteful.
- Custom applications: The preprocessor enables administrators to create their own application detectors to detect new applications. The detectors are written in Lua and interface with Snort using a well-defined C-Lua API.

2.2.24.1 Dependency Requirements

For proper functioning of the preprocessor:

- Stream session tracking must be enabled, i.e. stream5. TCP or UDP must be enabled in stream5. The preprocessor requires a session tracker to keep its data.
- Protocol Aware Flushing (PAF) must be enabled.
- IP defragmentation should be enabled, i.e. the frag3 preprocessor should be enabled and configured.

- HTTP preprocessor must be enabled and configured. The processor does not require any AppId specific configuration. The preprocessor provides parsed HTTP headers for application determination. Without HTTP preprocessor, AppId preprocessor will identify only non-HTTP applications.
- LuaJIT version 2.0.2 must be installed on host where snort is being compiled and run. Newer versions of LuaJIT are not tested for compatibility.

2.2.24.2 Preprocessor Configuration

AppId dynamic preprocessor is enabled by default(from snort-2.9.12). The preprocessor can be disabled during build time by including the following option in ./configure:

-disable-open-appid

The configuration name is "appid":

The preprocessor name is `appid`.

```
preprocessor appid
```

Option syntax

Option	Argument	Required	Default
app_detector_dir	<directory>	NO	app_detector_dir { /usr/local/etc/appid }
app_stats_filename	<filename>	NO	NULL
app_stats_period	<time in seconds>	NO	300 seconds
app_stats_rollover_size	<disk size in bytes>	NO	20 MB
app_stats_rollover_time	<time in seconds>	NO	1 day
memcap	<memory limit bytes>	NO	256 MB
debug	<"yes">	NO	disabled
dump_ports	No	NO	disabled

Option explanations

app_detector_dir

specifies base path where Cisco provided detectors and application configuration files are installed by ODP (Open Detector Package) package. The package contains Lua detectors and some application metadata. Customer written detectors are stored in subdirectory "custom" under the same base path.

Syntax

```
app_detector_dir <directory name>
```

Examples

```
app_detector_dir /usr/local/cisco/apps
```

`app_stats_filename`

name of file. If this configuration is missing, application stats are disabled.

Syntax

```
app_stats_filename <filename>
```

Examples

```
app_stats_filename appStats.log
```

`app_stats_period`

bucket size in seconds. Default 5 minutes.

Syntax

```
app_stats_period <time in seconds>
```

Examples

```
app_stats_period 15
```

`app_stats_rollover_size`

file size which will cause file rollover. Default 20 MB.

Syntax

```
app_stats_rollover_size <file size in bytes>
```

Examples

```
app_stats_rollover_size 2000000
```

`app_stats_rollover_time >`

time since file creation which will cause rollover. Default 1 day.

Syntax

```
app_stats_rollover_time <time in seconds>
```

Examples

```
app_stats_rollover_time 3600
```

`memcap >`

upper bound for memory used by AppId internal structures. Default 32MB.

Syntax

```
memcap <memory in bytes>
```

Examples

```
memcap 100000000
```

`dump_ports >`

prints port only detectors and information on active detectors. Used for troubleshooting.

Syntax

```
dump_ports <"yes"|"no">
```

Examples

```
dump_ports "yes"
```

`debug`

Used in some old detectors for debugging.

Syntax

```
debug
```

Examples

```
debug
```

Default configuration

```
preprocessor appid
```

2.2.24.3 Rule Options

The AppId preprocessor adds 1 new rule option as follows:

```
appid
```

The preprocessor must be enabled for this rule option to work.

`appid`

The rule option allows users to customize rules to specific application in a simple manner. The option can take up to 10 application names separated by spaces, tabs, or commas. Application names in rules are the names you will see in last column in appMapping.data file. A rule is considered a match if one of the appId in a rule match an appId in a session.

For client side packets, payloadAppId in a session is matched with all AppIds in a rule. Thereafter miscAppId, clientAppId and serviceAppId are matched. Since Alert Events contain one AppId, only the first match is reported. If rule without appId option matches, then the most specific appId (in order of payload, misc, client, server) is reported.

The same logic is followed for server side packets with one exception. Order of matching is changed to make serviceAppId higher then clientAppId.

Syntax

```
    appid:<list of application names>
```

Examples

```
    appid: http;
    appid: ftp, ftp-data;
    appid: cnn.com, zappos;
```

2.2.24.4 Application Rule Events

A new event type is defined for logging application name in Snort Alerts in unified2 format only. These events contain only one application name. The Events can be enabled for unified2 output using 'appid_event_types keyword.

For example, the following configuration will log alert in my.alert file with application name.

```
    output alert\_unified2: filename my.alert, appid\_event\_types
```

u2spewfoo, u2openappid, u2streamer tools can be used to print alerts in new format. Each event will display additional application name at the end of the event.

Examples

```
#> u2spewfoo outputs the following event format
(Event)
        sensor id: 0    event id: 6     event second: 1292962302        event
microsecond: 227323
        sig id: 18763   gen id: 1       revision: 4      classification: 0
        priority: 0     ip source: 98.27.88.56  ip destination: 10.4.10.79
        src port: 80    dest port: 54767        protocol: 6     impact\_flag: 0
blocked: 0
        mpls label: 0   vland id: 0     policy id: 0    appid: zappos
```

2.2.24.5 Application Usage Statistics

AppId preprocessor prints application network usage periodically in snort log directory in unified2 format. File name, time interval for statistic and file rollover are controlled by appId preprocessor configuration. u2spewfoo, u2openappid, u2streamer tools can be used to print contents of these files. An example output from u2openappid tools is as follows:

```
statTime="1292962290",appName="firefox",txBytes="9395",rxBytes="77021"
statTime="1292962290",appName="google\_analytic",txBytes="2024",rxBytes="928"
statTime="1292962290",appName="http",txBytes="28954",rxBytes="238000"
statTime="1292962290",appName="zappos",txBytes="26930",rxBytes="237072"
```

2.2.24.6 Open Detector Package (ODP) Installation

Application detectors from Snort team will be delivered in a separate package called Open Detector Package. ODP is a package that contains the following artifacts:

1. Application detectors in Lua language.
2. Port detectors, which are port only application detectors, in meta-data in YAML format.
3. appMapping.data file containing application metadata. This file should not be modified. The first column contains application identifier and second column contains application name. Other columns contain internal information.
4. Lua library files DetectorCommon.lua, flowTrackerModule.lua and hostServiceTrackerModule.lua

User can install ODP package in any directory of its choosing and configure this directory in app_detector_dir option in appId preprocessor configuration. Installing ODP will not modify any subdirectory named custom, where user-created detectors are located.

When installed, ODP will create following sub-directories:

```
odp/port    //Cisco port-only detectors
odp/lua     //Cisco Lua detectors
odp/libs    //Cisco Lua modules
```

2.2.24.7 User Created Application Detectors

Users can create new applications by coding detectors in Lua language. Users can also copy Snort team provided detectors into custom subdirectory and customize the detector. A document will be posted on Snort Website with details on API usage.

Users must organize their Lua detectors and libraries by creating the following directory structure, under ODP installation directory.

```
custom/port    //port-only detectors
custom/lua     //Lua detectors
custom/libs    //Lua modules
```

End of Navigation Panel Table of Child-Links **Subsections**

- 2.3.1 Configuring
- 2.3.2 Reverting to original behavior

End of Table of Child-Links

2.3 Decoder and Preprocessor Rules

Decoder and preprocessor rules allow one to enable and disable decoder and preprocessor events on a rule by rule basis. They also allow one to specify the rule type or action of a decoder or preprocessor event on a rule by rule basis.

Decoder config options will still determine whether or not to generate decoder events. For example, if `config disable_decode_alerts` is in `snort.conf`, decoder events will not be generated regardless of whether or not there are corresponding rules for the event. Also note that if the decoder is configured to enable drops, e.g. `config enable_decode_drops`, these options will take precedence over the event type of the rule. A packet will be dropped if either a decoder config drop option is in `snort.conf` or the decoder or preprocessor rule type is `drop`. Of course, the drop cases only apply if Snort is running inline. See `doc/README.decode` for config options that control decoder events.

2.3.1 Configuring

The decoder and preprocessor rules are located in the `preproc_rules/` directory in the top level source tree, and have the names `decoder.rules` and `preprocessor.rules` respectively. These files are updated as new decoder and preprocessor events are added to Snort. The `gen-msg.map` under `etc` directory is also updated with new decoder and preprocessor rules.

To enable these rules in `snort.conf`, define the path to where the rules are located and uncomment the `include` lines in `snort.conf` that reference the rules files.

```
var PREPROC_RULE_PATH /path/to/preproc_rules
...
include $PREPROC_RULE_PATH/preprocessor.rules
include $PREPROC_RULE_PATH/decoder.rules
```

To disable any rule, just comment it with a # or remove the rule completely from the file (commenting is recommended).

To change the rule type or action of a decoder/preprocessor rule, just replace `alert` with the desired rule type. Any one of the following rule types can be used:

```
alert
log
pass
drop
sdrop
reject
```

For example one can change:

```
alert ( msg: "DECODE_NOT_IPV4_DGRAM"; sid: 1; gid: 116; rev: 1; \
    metadata: rule-type decode ; classtype:protocol-command-decode;)
```

to

```
drop ( msg: "DECODE_NOT_IPV4_DGRAM"; sid: 1; gid: 116; rev: 1; \
    metadata: rule-type decode ; classtype:protocol-command-decode;)
```

to drop (as well as alert on) packets where the Ethernet protocol is IPv4 but version field in IPv4 header has a value other than 4.

See `README.decode`, `README.gre` and the various preprocessor READMEs for descriptions of the rules in `decoder.rules` and `preprocessor.rules`.

The generator ids (gid) for different preprocessors and the decoder are as follows:

Generator Id	Module
105	Back Orifice preprocessor
106	RPC Decode preprocessor
112	Arpspoof preprocessor
116	Snort Decoder
119	HTTP Inspect preprocessor (Client)
120	HTTP Inspect preprocessor (Server)
122	Portscan preprocessor
123	Frag3 preprocessor
124	SMTP preprocessor
125	FTP (FTP) preprocessor
126	FTP (Telnet) preprocessor
127	ISAKMP preprocessor
128	SSH preprocessor
129	Stream preprocessor
131	DNS preprocessor
132	Skype preprocessor
133	DceRpc2 preprocessor
134	PPM preprocessor
136	Reputation preprocessor
137	SSL preprocessor
139	SDF preprocessor
140	SIP preprocessor
141	IMAP preprocessor
142	POP preprocessor

143	GTP preprocessor

2.3.2 Reverting to original behavior

The following config option in `snort.conf` will make Snort revert to the old behavior:

```
config autogenerate_preprocessor_decoder_rules
```

Note that if you want to revert to the old behavior, you also have to remove the decoder and preprocessor rules and any reference to them from `snort.conf`, otherwise they will be loaded. This option applies to rules not specified and the default behavior is to alert.

End of Navigation Panel Table of Child-Links **Subsections**

- 2.4.1 Rate Filtering
  - 2.4.1.1 Format
  - 2.4.1.2 Examples
- 2.4.2 Event Filtering
  - 2.4.2.1 Format
  - 2.4.2.2 Examples
- 2.4.3 Event Suppression
  - 2.4.3.1 Format
  - 2.4.3.2 Examples
- 2.4.4 Event Logging
  - 2.4.4.1 Event Queue Configuration Options
  - 2.4.4.2 Event Queue Configuration Examples
- 2.4.5 Event Trace
  - 2.4.5.1 Event Trace Examples

End of Table of Child-Links

2.4 Event Processing

Snort provides a variety of mechanisms to tune event processing to suit your needs:

- `Detection Filters`

  You can use detection filters to specify a threshold that must be exceeded before a rule generates an event. This is covered in section [*].

- `Rate Filters`

  You can use rate filters to change a rule action when the number or rate of events indicates a possible attack.

- `Event Filters`

  You can use event filters to reduce the number of logged events for noisy rules. This can be tuned to significantly reduce false alarms.

- `Event Suppression`

  You can completely suppress the logging of uninteresting events.

2.4.1 Rate Filtering

`rate_filter` provides rate based attack prevention by allowing users to configure a new action to take for a specified time when a given rate is exceeded. Multiple rate filters can be defined on the same rule, in which case they are evaluated in the order they appear in the configuration file, and the first applicable action is taken.

2.4.1.1 Format

Rate filters are used as standalone configurations (outside of a rule) and have the following format:

```
rate_filter \
    gen_id <gid>, sig_id <sid>, \
    track <by_src|by_dst|by_rule>, \
    count <c>, seconds <s>, \
    new_action alert|drop|pass|log|sdrop|reject, \
    timeout <seconds> \
    [, apply_to <ip-list>]
```

The options are described in the table below - all are required except `apply_to`, which is optional.

Option	Description

`track by_src | by_dst | by_rule`	rate is tracked either by source IP address, destination IP address, or by rule. This means the match statistics are maintained for each unique source IP address, for each unique destination IP address, or they are aggregated at rule level. For rules related to Stream sessions, source and destination means client and server respectively. `track by_rule` and `apply_to` may not be used together.
`count c`	the maximum number of rule matches in `s` seconds before the rate filter limit to is exceeded. `c` must be nonzero value.
`seconds s`	the time period over which `count` is accrued. 0 seconds means `count` is a total count instead of a specific rate. For example, `rate_filter` may be used to detect if the number of connections to a specific server exceed a specific count. 0 seconds only applies to internal rules (gen_id 135) and other use will produce a fatal error by Snort.
`new_action alert | drop | pass | log | sdrop | reject`	`new_action` replaces rule action for `t` seconds. `drop`, `reject`, and `sdrop` can be used only when snort is used in inline mode. `sdrop` and `reject` are conditionally compiled with GIDS.
`timeout t`	revert to the original rule action after `t` seconds. If `t` is 0, then rule action is never reverted back. An `event_filter` may be used to manage number of alerts after the rule action is enabled by `rate_filter`.
`apply_to <ip-list>`	restrict the configuration to only to source or destination IP address (indicated by track parameter) determined by `<ip-list>`. `track by_rule` and `apply_to` may not be used together. Note that events are generated during the timeout period, even if the rate falls below the configured limit.

2.4.1.2 Examples

Example 1 - allow a maximum of 100 connection attempts per second from any one IP address, and block further connection attempts from that IP address for 10 seconds:

```
rate_filter \
    gen_id 135, sig_id 1, \
    track by_src, \
    count 100, seconds 1, \
    new_action drop, timeout 10
```

Example 2 - allow a maximum of 100 successful simultaneous connections from any one IP address, and block further connections from that IP address for 10 seconds:

```
rate_filter \
    gen_id 135, sig_id 2, \
```

```
        track by_src, \
        count 100, seconds 0, \
        new_action drop, timeout 10
```

2.4.2 Event Filtering

Event filtering can be used to reduce the number of logged alerts for noisy rules by limiting the number of times a particular event is logged during a specified time interval. This can be tuned to significantly reduce false alarms.

There are 3 types of event filters:

- `limit`

  Alerts on the 1st m events during the time interval, then ignores events for the rest of the time interval.

- `threshold`

  Alerts every m times we see this event during the time interval.

- `both`

  Alerts once per time interval after seeing m occurrences of the event, then ignores any additional events during the time interval.

2.4.2.1 Format

```
    event_filter \
        gen_id <gid>, sig_id <sid>, \
        type <limit|threshold|both>, \
        track <by_src|by_dst>, \
        count <c>, seconds <s>

    threshold \
        gen_id <gid>, sig_id <sid>, \
        type <limit|threshold|both>, \
        track <by_src|by_dst>, \
        count <c>, seconds <s>
```

`threshold` is an alias for `event_filter`. Both formats are equivalent and support the options described below - all are required. `threshold` is deprecated and will not be supported in future releases.

Option	Description
`gen_id <gid>`	

	Specify the generator ID of an associated rule. `gen_id 0, sig_id 0` can be used to specify a "global" threshold that applies to all rules.
`sig_id <sid>`	Specify the signature ID of an associated rule. `sig_id 0` specifies a "global" filter because it applies to all `sig_ids` for the given `gen_id`.
`type limit|threshold|both`	type `limit` alerts on the 1st m events during the time interval, then ignores events for the rest of the time interval. Type `threshold` alerts every m times we see this event during the time interval. Type `both` alerts once per time interval after seeing m occurrences of the event, then ignores any additional events during the time interval.
`track by_src|by_dst`	rate is tracked either by source IP address, or destination IP address. This means count is maintained for each unique source IP addresses, or for each unique destination IP addresses. Ports or anything else are not tracked.
`count c`	number of rule matching in s seconds that will cause `event_filter` limit to be exceeded. `c` must be nonzero value. A value of -1 disables the event filter and can be used to override the global `event_filter`.
`seconds s`	time period over which `count` is accrued. `s` must be nonzero value.

> **Note:**
>
> Only one `event_filter` may be defined for a given `gen_id, sig_id`. If more than one `event_filter` is applied to a specific `gen_id, sig_id` pair, Snort will terminate with an error while reading the configuration information.

`event_filters` with `sig_id` 0 are considered "global" because they apply to all rules with the given `gen_id`. If `gen_id` is also 0, then the filter applies to all rules. (`gen_id 0, sig_id != 0` is not allowed). Standard filtering tests are applied first, if they do not block an event from being logged, the global filtering test is applied. Thresholds in a rule (deprecated) will override a global `event_filter`. Global `event_filters` do not override what's in a signature or a more specific stand-alone `event_filter`.

> **Note:** `event_filters` can be used to suppress excessive `rate_filter` alerts, however, the first `new_action` event of the timeout period is never suppressed. Such events indicate a change of state that are significant to the user monitoring the network.

2.4.2.2 Examples

Limit logging to 1 event per 60 seconds:

```
    event_filter \
        gen_id 1, sig_id 1851, \
```

```
        type limit, track by_src,  \
        count 1, seconds 60
```

Limit logging to every 3rd event:

```
    event_filter \
        gen_id 1, sig_id 1852,  \
        type threshold, track by_src, \
        count 3, seconds 60
```

Limit logging to just 1 event per 60 seconds, but only if we exceed 30 events in 60 seconds:

```
    event_filter \
        gen_id 1, sig_id 1853, \
        type both, track by_src, \
        count 30, seconds 60
```

Limit to logging 1 event per 60 seconds per IP triggering each rule (rule gen_id is 1):

```
    event_filter \
        gen_id 1, sig_id 0, \
        type limit, track by_src, \
        count 1, seconds 60
```

Limit to logging 1 event per 60 seconds per IP, triggering each rule for each event generator:

```
    event_filter \
        gen_id 0, sig_id 0, \
        type limit, track by_src, \
        count 1, seconds 60
```

Events in Snort are generated in the usual way, event filters are handled as part of the output system. Read gen-msg.map for details on gen ids.

Users can also configure a memcap for threshold with a “config:” option:

```
    config event_filter: memcap <bytes>

    # this is deprecated:
    config threshold: memcap <bytes>
```

2.4.3 Event Suppression

Event suppression stops specified events from firing without removing the rule from the rule base. Suppression uses an IP list to select specific networks and users for suppression. Suppression tests are performed prior to either standard or global thresholding tests.

Suppression are standalone configurations that reference generators, SIDs, and IP addresses via an IP list . This allows a rule to be completely suppressed, or suppressed when the causative traffic is going to or coming from a specific IP or group of IP addresses.

You may apply multiple suppressions to a non-zero SID. You may also combine one `event_filter` and several suppressions to the same non-zero SID.

2.4.3.1 Format

The suppress configuration has two forms:

```
suppress \
    gen_id <gid>, sig_id <sid>
```

```
suppress \
    gen_id <gid>, sig_id <sid>, \
    track <by_src|by_dst>, ip <ip-list>
```

Option	Description
`gen_id <gid>`	Specify the generator ID of an associated rule. `gen_id 0, sig_id 0` can be used to specify a "global" threshold that applies to all rules.
`sig_id <sid>`	Specify the signature ID of an associated rule. `sig_id 0` specifies a "global" filter because it applies to all `sig_ids` for the given `gen_id`.
`track by_src| by_dst`	Suppress by source IP address or destination IP address. This is optional, but if present, `ip` must be provided as well.
`ip <list>`	Restrict the suppression to only source or destination IP addresses (indicated by `track` parameter) determined by $<$list$>$. If track is provided, ip must be provided as well.

2.4.3.2 Examples

Suppress this event completely:

```
suppress gen_id 1, sig_id 1852:
```

Suppress this event from this IP:

```
suppress gen_id 1, sig_id 1852, track by_src, ip 10.1.1.54
```

Suppress this event to this CIDR block:

```
suppress gen_id 1, sig_id 1852, track by_dst, ip 10.1.1.0/24
```

2.4.4 Event Logging

Snort supports logging multiple events per packet/stream that are prioritized with different insertion methods, such as max content length or event ordering using the event queue.

The general configuration of the event queue is as follows:

```
config event_queue: [max_queue [size]] [log [size]] [order_events [TYPE]]
```

2.4.4.1 Event Queue Configuration Options

There are three configuration options to the configuration parameter 'event_queue'.

129.

`max_queue`

This determines the maximum size of the event queue. For example, if the event queue has a max size of 8, only 8 events will be stored for a single packet or stream.

The default value is 8.

130.

`log`

This determines the number of events to log for a given packet or stream. You can't log more than the max_event number that was specified.

The default value is 3.

131.

`order_events`

This argument determines the way that the incoming events are ordered. We currently have two different methods:

- `priority` - The highest priority (1 being the highest) events are ordered first.
- `content_length` - Rules are ordered before decode or preprocessor alerts, and rules that have a longer content are ordered before rules with shorter contents.

The method in which events are ordered does not affect rule types such as pass, alert, log, etc.

The default value is content_length.

2.4.4.2 Event Queue Configuration Examples

The default configuration:

```
config event_queue: max_queue 8 log 3 order_events content_length
```

Example of a reconfigured event queue:

```
config event_queue: max_queue 10 log 3 order_events content_length
```

Use the default event queue values, but change event order:

```
config event_queue: order_events priority
```

Use the default event queue values but change the number of logged events:

```
config event_queue: log 2
```

2.4.5 Event Trace

Snort supports logging additional information to a file about the events it is generating relative to specific blocks of data that are matching the rule. The blocks of data logged include information about the event, the GID, SID, and other data related to the event itself, plus packet data including sizes, timestamps, raw, normalized, and decompressed buffers extracted from the packet that may have been used in evaluating the rule. The amount of packet data written is limited with each entry. This is useful in debugging rules.

The `config` option `event_trace` to `snort.conf` provides this control.

The general configuration for event tracing is as follows:

```
config event_trace: [file <filename>] [max_data <int>]
```

The are two configuration options for `event_trace`.

132.

`file`

This sets the file name into which the trace data is written, within Snort's log directory (see -l command line option).

The default is event_trace.txt.

133.

`max_data`

This specifies the maximum number of bytes from each buffer of data to write into the file.

The default is 64 bytes and valid values range from 1 to 65535 bytes.

2.4.5.1 Event Trace Examples

The default configuration:

```
config event_trace: file event_trace.txt max_data 64
```

Use the default file, but change the amount of data logged:

```
config event_trace: max_data 128
```

Change the file name to which event traces are logged:

```
config event_trace: file snort_event_trace.out
```

End of Navigation Panel Table of Child-Links **Subsections**

- 2.5.1 Rule Profiling
  - 2.5.1.1 Format
  - 2.5.1.2 Examples
  - 2.5.1.3 Output

- 2.5.2 Preprocessor Profiling
  - 2.5.2.1 Format
  - 2.5.2.2 Examples
  - 2.5.2.3 Output

- 2.5.3 Packet Performance Monitoring (PPM)
  - 2.5.3.1 Configuration
  - 2.5.3.2 Examples
  - 2.5.3.3 Sample Snort Output
  - 2.5.3.4 Implementation Details

End of Table of Child-Links

2.5 Performance Profiling

Snort can provide statistics on rule and preprocessor performance. Each require only a simple `config` option to `snort.conf` and Snort will print statistics on the worst (or all) performers on exit. When a file name is provided in `profile_rules` or `profile_preprocs`, the statistics will be saved in these files. If `append` is not specified, a new file will be created each time Snort is run. The filenames will have timestamps appended to them. These files will be found in the logging directory.

To use this feature, you must build snort with the `–enable–perfprofiling` option to the configure script.

2.5.1 Rule Profiling

2.5.1.1 Format

```
config profile_rules: \
    print [all | <num>], \
    sort <sort_option> \
    [,filename <filename> [append]]
```

- `<num>` is the number of rules to print
- `<sort_option>` is one of:

  `checks` `matches` `nomatches` `avg_ticks` `avg_ticks_per_match` `avg_ticks_per_nomatch` `total_ticks`

- `<filename>` is the output filename
- `[append]` dictates that the output will go to the same file each time (optional)

2.5.1.2 Examples

- Print all rules, sort by avg_ticks (default configuration if option is turned on) `config profile_rules`
- Print all rules, sort by avg_ticks, and append to file `rules_stats.txt` `config profile_rules: filename rules_stats.txt append`
- Print the top 10 rules, based on highest average time `config profile_rules: print 10, sort avg_ticks`
- Print all rules, sorted by number of checks `config profile_rules: print all, sort checks`
- Print top 100 rules, based on total time `config profile_rules: print 100, sort total_ticks`
- Print with default options, save results to performance.txt each time `config profile_rules: filename performance.txt append`

- Print top 20 rules, save results to perf.txt with timestamp in filename `config profile_rules: print 20, filename perf.txt`

2.5.1.3 Output

Snort will print a table much like the following at exit.

Figure: Rule Profiling Example Output

```
\begin{figure}\footnotesize { \begin{verbatim}Rule Profile Statistics (worst 4... ...229.0 4 1734 1 37 2 0 0 90054 45027.0 0.0 45027.0\end{verbatim} }\end{figure}
```

Configuration line used to print the above table:

```
config profile_rules: print 4, sort total_ticks
```

The columns represent:

- Number (rank)
- Sig ID
- Generator ID
- Checks (number of times rule was evaluated after fast pattern match within portgroup or any-$>$any rules)
- Matches (number of times ALL rule options matched, will be high for rules that have no options)
- Alerts (number of alerts generated from this rule)
- CPU Ticks
- Avg Ticks per Check
- Avg Ticks per Match
- Avg Ticks per Nonmatch

Interpreting this info is the key. The Microsecs (or Ticks) column is important because that is the total time spent evaluating a given rule. But, if that rule is causing alerts, it makes sense to leave it alone.

A high Avg/Check is a poor performing rule, that most likely contains PCRE. High Checks and low Avg/Check is usually an any-$>$any rule with few rule options and no content. Quick to check, the few options may or may not match. We are looking at moving some of these into code, especially those with low SIDs.

By default, this information will be printed to the console when Snort exits. You can use the "filename" option in snort.conf to specify a file where this will be written. If "append" is not specified, a new file will be created each time Snort is run. The filenames will have timestamps appended to them. These files will be found in the logging directory.

2.5.2 Preprocessor Profiling

2.5.2.1 Format

```
config profile_preprocs: \
    print [all | <num>], \
    sort <sort_option> \
    [, filename <filename> [append]]
```

- `<num>` is the number of preprocessors to print
- `<sort_option>` is one of: `checks` `avg_ticks` `total_ticks`
- `<filename>` is the output filename
- `[append]` dictates that the output will go to the same file each time (optional)

2.5.2.2 Examples

- Print all preprocessors, sort by avg_ticks (default configuration if option is turned on) `config profile_preprocs`
- Print all preprocessors, sort by avg_ticks, and append to file `preprocs_stats.txt` `config profile_preprocs: filename preprocs_stats.txt append`
- Print the top 10 preprocessors, based on highest average time `config profile_preprocs: print 10, sort avg_ticks`
- Print all preprocessors, sorted by number of checks `config profile_preprocs: print all, sort checks`

2.5.2.3 Output

Snort will print a table much like the following at exit.

Figure: Preprocessor Profiling Example Output

\begin{figure}\footnotesize { \begin{verbatim}Preprocessor Profile Statistics ... ...tal total 0 311202 311202 14011946 45.03 0.00 0.00\end{verbatim} }\end{figure}

Configuration line used to print the above table:

```
config profile_preprocs: \
    print 10, sort total_ticks
```

The columns represent:

- Number (rank) - The number is indented for each layer. Layer 1 preprocessors are listed under their respective caller (and sorted similarly).
- Preprocessor Name
- Layer - When printing a specific number of preprocessors all subtasks info for a particular preprocessor is printed for each layer 0 preprocessor stat.

- Checks (number of times preprocessor decided to look at a packet, ports matched, app layer header was correct, etc)
- Exits (number of corresponding exits - just to verify code is instrumented correctly, should ALWAYS match Checks, unless an exception was trapped)
- CPU Ticks
- Avg Ticks per Check
- Percent of caller - For non layer 0 preprocessors, i.e. subroutines within preprocessors, this identifies the percent of the caller's ticks that is spent for this subtask.

Because of task swapping, non-instrumented code, and other factors, the Pct of Caller field will not add up to 100% of the caller's time. It does give a reasonable indication of how much relative time is spent within each subtask.

By default, this information will be printed to the console when Snort exits. You can use the "filename" option in snort.conf to specify a file where this will be written. If "append" is not specified, a new file will be created each time Snort is run. The filenames will have timestamps appended to them. These files will be found in the logging directory.

2.5.3 Packet Performance Monitoring (PPM)

PPM provides thresholding mechanisms that can be used to provide a basic level of latency control for snort. It does not provide a hard and fast latency guarantee but should in effect provide a good average latency control. Both rules and packets can be checked for latency. The action taken upon detection of excessive latency is configurable. The following sections describe configuration, sample output, and some implementation details worth noting.

To use PPM, you must build with the -enable-ppm or the -enable-sourcefire option to configure.

PPM is configured as follows:

```
# Packet configuration:
config ppm: max-pkt-time <micro-secs>, \
    fastpath-expensive-packets, \
    pkt-log, \
    debug-pkts

# Rule configuration:
config ppm: max-rule-time <micro-secs>, \
    threshold count, \
    suspend-expensive-rules, \
    suspend-timeout <seconds>, \
    rule-log [log] [alert]
```

Packets and rules can be configured separately, as above, or together in just one config ppm statement. Packet and rule monitoring is independent, so one or both or neither may be enabled.

2.5.3.1 Configuration

Packet Configuration Options

`max-pkt-time <micro-secs>`

- enables packet latency thresholding using 'micros-secs' as the limit.
- default is 0 (packet latency thresholding disabled)
- reasonable starting defaults: 100/250/1000 for 1G/100M/5M nets

`fastpath-expensive-packets`

- enables stopping further inspection of a packet if the max time is exceeded
- default is off

`pkt-log`

- enables logging packet event if packet exceeds max-pkt-time
- default is no logging
- if no option is given for 'pkt-log', 'pkt-log log' is implied
- the log option enables output to syslog or console depending upon snort configuration

`debug-pkts`

- must build with the -enable-debug option to configure
- enables per packet timing stats to be printed after each packet
- default is off

Rule Configuration Options

`max-rule-time <micro-secs>`

- enables rule latency thresholding using 'micros-secs' as the limit.
- default is 0 (rule latency thresholding disabled)
- reasonable starting defaults: 100/250/1000 for 1G/100M/5M nets

`threshold <count>`

- sets the number of cumulative rule time excesses before disabling a rule
- default is 5

`suspend-expensive-rules`

- enables suspending rule inspection if the max rule time is exceeded
- default is off

`suspend-timeout <seconds>`

- rule suspension time in seconds
- default is 60 seconds
- set to zero to permanently disable expensive rules

```
rule-log [log] [alert]
```

- enables event logging output for rules
- default is no logging
- one or both of the options 'log' and 'alert' must be used with 'rule-log'
- the log option enables output to syslog or console depending upon snort configuration

2.5.3.2 Examples

Example 1: The following enables packet tracking:

```
config ppm: max-pkt-time 100
```

The following enables rule tracking:

```
config ppm: max-rule-time 50, threshold 5
```

If fastpath-expensive-packets or suspend-expensive-rules is not used, then no action is taken other than to increment the count of the number of packets that should be fastpath'd or the rules that should be suspended. A summary of this information is printed out when snort exits.

Example 2:

The following suspends rules and aborts packet inspection. These rules were used to generate the sample output that follows.

```
config ppm: \
    max-pkt-time 50, fastpath-expensive-packets, \
    pkt-log, debug-pkts

config ppm: \
    max-rule-time 50, threshold 5, suspend-expensive-rules, \
    suspend-timeout 300, rule-log log alert
```

2.5.3.3 Sample Snort Output

Sample Snort Startup Output

```
Packet Performance Monitor Config:
  ticks per usec  : 1600 ticks
  max packet time : 50 usecs
  packet action   : fastpath-expensive-packets
  packet logging  : log
  debug-pkts      : disabled

Rule Performance Monitor Config:
  ticks per usec  : 1600 ticks
  max rule time   : 50 usecs
  rule action     : suspend-expensive-rules
  rule threshold  : 5
```

```
      suspend timeout : 300 secs
      rule logging    : alert log
```

Sample Snort Run-time Output

```
    ...
    PPM: Process-BeginPkt[61] caplen=60
    PPM: Pkt[61] Used= 8.15385 usecs
    PPM: Process-EndPkt[61]

    PPM: Process-BeginPkt[62] caplen=342
    PPM: Pkt[62] Used= 65.3659 usecs
    PPM: Process-EndPkt[62]

    PPM: Pkt-Event Pkt[63] used=56.0438 usecs, 0 rules, 1 nc-rules tested, packet
fastpathed
         (10.4.12.224:0 -> 10.4.14.108:54321).
    PPM: Process-BeginPkt[63] caplen=60
    PPM: Pkt[63] Used= 8.394 usecs
    PPM: Process-EndPkt[63]

    PPM: Process-BeginPkt[64] caplen=60
    PPM: Pkt[64] Used= 8.21764 usecs
    PPM: Process-EndPkt[64]
    ...
```

Sample Snort Exit Output

```
    Packet Performance Summary:
       max packet time       : 50 usecs
       packet events         : 1
       avg pkt time          : 0.633125 usecs
    Rule Performance Summary:
       max rule time         : 50 usecs
       rule events           : 0
       avg nc-rule time      : 0.2675 usecs
```

2.5.3.4 Implementation Details

- Enforcement of packet and rule processing times is done after processing each rule. Latency control is not enforced after each preprocessor.
- This implementation is software based and does not use an interrupt driven timing mechanism and is therefore subject to the granularity of the software based timing tests. Due to the granularity of the timing measurements any individual packet may exceed the user specified packet or rule processing time limit. Therefore this implementation cannot implement a precise latency guarantee with strict timing guarantees. Hence the reason this is considered a best effort approach.
- Since this implementation depends on hardware based high performance frequency counters, latency thresholding is presently only available on Intel and PPC platforms.

- Time checks are made based on the total system time, not processor usage by Snort. This was a conscious design decision because when a system is loaded, the latency for a packet is based on the total system time, not just the processor time the Snort application receives. Therefore, it is recommended that you tune your thresholding to operate optimally when your system is under load.

End of Navigation Panel Table of Child-Links **Subsections**

- 2.6.1 alert_syslog
  - 2.6.1.1 Available Keywords
  - 2.6.1.2 Format
  - 2.6.1.3 Example
- 2.6.2 alert_fast
  - 2.6.2.1 Format
  - 2.6.2.2 Example
- 2.6.3 alert_full
  - 2.6.3.1 Format
  - 2.6.3.2 Example
- 2.6.4 alert_unixsock
  - 2.6.4.1 Format
  - 2.6.4.2 Example
- 2.6.5 log_tcpdump
  - 2.6.5.1 Format
  - 2.6.5.2 Example
- 2.6.6 csv
  - 2.6.6.1 Format
  - 2.6.6.2 Example
- 2.6.7 unified 2
  - 2.6.7.1 Format
  - 2.6.7.2 Example
  - 2.6.7.3 Extra Data Configurations
  - 2.6.7.4 Reading Unified2 Files
  - 2.6.7.5 U2SpewFoo
  - 2.6.7.6 U2Boat
- 2.6.8 log null
  - 2.6.8.1 Format
  - 2.6.8.2 Example
- 2.6.9 Log Limits

End of Table of Child-Links

2.6 Output Modules

Output modules are new as of version 1.6. They allow Snort to be much more flexible in the formatting and presentation of output to its users. The output modules are run when the alert or logging subsystems of Snort are called, after the preprocessors and detection engine. The format of the directives in the config file is very similar to that of the preprocessors.

Multiple output plugins may be specified in the Snort configuration file. When multiple plugins of the same type (log, alert) are specified, they are stacked and called in sequence when an event occurs. As with the standard logging and alerting systems, output plugins send their data to /var/log/snort by default or to a user directed directory (using the -l command line switch).

Output modules are loaded at runtime by specifying the output keyword in the config file:

```
output <name>: <options>
```

```
output alert_syslog: log_auth log_alert
```

2.6.1 alert_syslog

This module sends alerts to the syslog facility (much like the -s command line switch). This module also allows the user to specify the logging facility and priority within the Snort config file, giving users greater flexibility in logging alerts.

2.6.1.1 Available Keywords

2.6.1.1.1 Facilities

- `log_auth`
- `log_authpriv`
- `log_daemon`
- `log_local0`
- `log_local1`
- `log_local2`
- `log_local3`
- `log_local4`
- `log_local5`
- `log_local6`
- `log_local7`
- `log_user`

2.6.1.1.2 Priorities

- `log_emerg`
- `log_alert`
- `log_crit`
- `log_err`
- `log_warning`
- `log_notice`
- `log_info`
- `log_debug`

2.6.1.1.3 Options

- `log_cons`
- `log_ndelay`
- `log_perror`
- `log_pid`

2.6.1.2 Format

```
alert_syslog: \
    <facility> <priority> <options>
```

> **Note:**
>
> As WIN32 does not run syslog servers locally by default, a hostname and port can be passed as options. The default host is 127.0.0.1. The default port is 514.

```
output alert_syslog: \
    [host=<hostname[:<port>],] \
    <facility> <priority> <options>
```

2.6.1.3 Example

```
output alert_syslog: host=10.1.1.1:514, <facility> <priority> <options>
```

2.6.2 alert_fast

This will print Snort alerts in a quick one-line format to a specified output file. It is a faster alerting method than full alerts because it doesn't need to print all of the packet headers to the output file and because it logs to only 1 file.

2.6.2.1 Format

```
output alert_fast: [<filename> ["packet"] [<limit>]]
<limit> ::= <number>[('G'|'M'|K')]
```

- `filename`: the name of the log file. The default name is $<$logdir$>$/alert. You may specify "stdout" for terminal output. The name may include an absolute or relative path.
- `packet`: this option will cause multiline entries with full packet headers to be logged. By default, only brief single-line entries are logged.
- `limit`: an optional limit on file size which defaults to 128 MB. The minimum is 1 KB. See [*] for more information.

2.6.2.2 Example

```
output alert_fast: alert.fast
```

2.6.3 alert_full

This will print Snort alert messages with full packet headers. The alerts will be written in the default logging directory (/var/log/snort) or in the logging directory specified at the command line.

Inside the logging directory, a directory will be created per IP. These files will be decoded packet dumps of the packets that triggered the alerts. The creation of these files slows Snort down considerably. This output method is discouraged for all but the lightest traffic situations.

2.6.3.1 Format

```
output alert_full: [<filename> [<limit>]]
<limit> ::= <number>[('G'|'M'|K')]
```

- `filename`: the name of the log file. The default name is $<$logdir$>$/alert. You may specify "stdout" for terminal output. The name may include an absolute or relative path.
- `limit`: an optional limit on file size which defaults to 128 MB. The minimum is 1 KB. See [*] for more information.

2.6.3.2 Example

```
output alert_full: alert.full
```

2.6.4 alert_unixsock

Sets up a UNIX domain socket and sends alert reports to it. External programs/processes can listen in on this socket and receive Snort alert and packet data in real time.

2.6.4.1 Format

```
alert_unixsock
```

2.6.4.2 Example

```
output alert_unixsock
```

Note: On FreeBSD, the default `sysctl` value for `net.local.dgram.recvspace` is too low for `alert_unixsock` datagrams and you will likely not receive any data. You can change this value after booting by running:

```
$ sudo sysctl net.local.dgram.recvspace=100000
```

To have this value set on each boot automatically, add the following to `/etc/sysctl.conf`:

```
net.local.dgram.recvspace=100000
```

Note that the value of 100000 may be slightly generous, but the value should be at least 65864.

2.6.5 log_tcpdump

The log_tcpdump module logs packets to a tcpdump-formatted file. This is useful for performing post-process analysis on collected traffic with the vast number of tools that are available for examining tcpdump-formatted files.

2.6.5.1 Format

```
output log_tcpdump: [<filename> [<limit>]]
<limit> ::= <number>[('G'|'M'|K')]
```

- `filename`: the name of the log file. The default name is $<$logdir$>$/snort.log. The name may include an absolute or relative path. A UNIX timestamp is appended to the filename.
- `limit`: an optional limit on file size which defaults to 128 MB. When a sequence of packets is to be logged, the aggregate size is used to test the rollover condition. See [*] for more information.

2.6.5.2 Example

```
output log_tcpdump: snort.log
```

2.6.6 csv

The csv output plugin allows alert data to be written in a format easily importable to a database. The output fields and their order may be customized.

2.6.6.1 Format

```
output alert_csv: [<filename> [<format> [<limit>]]]
<format> ::= "default"|<list>
<list> ::= <field>(,<field>)*
<field> ::= "dst"|"src"|"ttl" ...
<limit> ::= <number>[('G'|'M'|K')]
```

- `filename`: the name of the log file. The default name is $<$logdir$>$/alert.csv. You may specify "stdout" for terminal output. The name may include an absolute or relative path.
- `format`: The list of formatting options is below. If the formatting option is "default", the output is in the order of the formatting options listed.
    - `timestamp`
    - `sig_generator`
    - `sig_id`
    - `sig_rev`
    - `msg`
    - `proto`
    - `src`
    - `srcport`
    - `dst`
    - `dstport`
    - `ethsrc`
    - `ethdst`
    - `ethlen`
    - `tcpflags`
    - `tcpseq`
    - `tcpack`
    - `tcplen`
    - `tcpwindow`
    - `ttl`
    - `tos`
    - `id`
    - `dgmlen`
    - `iplen`
    - `icmptype`
    - `icmpcode`
    - `icmpid`
    - `icmpseq`

- `limit`: an optional limit on file size which defaults to 128 MB. The minimum is 1 KB. See [*] for more information.

2.6.6.2 Example

```
    output alert_csv: /var/log/alert.csv default

    output alert_csv: /var/log/alert.csv timestamp, msg
```

2.6.7 unified 2

Unified2 can work in one of three modes, packet logging, alert logging, or true unified logging. Packet logging includes a capture of the entire packet and is specified with `log_unified2`. Likewise, alert logging will only log events and is specified with `alert_unified2`. To include both logging styles in a single, unified file, simply specify `unified2`.

When MPLS support is turned on, MPLS labels can be included in unified2 events. Use option `mpls_event_types` to enable this. If option `mpls_event_types` is not used, then MPLS labels will be not be included in unified2 events.

> **Note:**
>
> By default, unified 2 files have the file creation time (in Unix Epoch format) appended to each file when it is created.

2.6.7.1 Format

```
    output alert_unified2: \
        filename <base filename> [, <limit <size in MB>] [, nostamp] [,
mpls_event_types] \
        [, vlan_event_types]

    output log_unified2: \
        filename <base filename> [, <limit <size in MB>] [, nostamp]

    output unified2: \
        filename <base file name> [, <limit <size in MB>] [, nostamp] [,
mpls_event_types] \
        [, vlan_event_types]
```

2.6.7.2 Example

```
    output alert_unified2: filename snort.alert, limit 128, nostamp
    output log_unified2: filename snort.log, limit 128, nostamp
    output unified2: filename merged.log, limit 128, nostamp
    output unified2: filename merged.log, limit 128, nostamp, mpls_event_types
    output unified2: filename merged.log, limit 128, nostamp, vlan_event_types
```

2.6.7.3 Extra Data Configurations

Unified2 also has logging support for various extra data. The following configuration items will enable these extra data logging facilities.

```
config log_ipv6_extra_data
```

This option enables Snort to log IPv6 source and destination address as unified2 extra data events.

See section [*] for more information

```
enable_xff
```

This option enables HTTP Inspect to parse and log the original client IP present in the X-Forwarded-For, True-Client-IP, or custom HTTP request headers along with the generated events.

See section [*] for more information

```
log_uri
```

This option enables HTTP Inspect to parse and log the URI data from the HTTP request and log it along with all the generated events for that session.

See section [*] for more information

```
log_hostname
```

This option enables HTTP Inspect to parse and log the Host header data from the HTTP request and log it along with all the generated events for that session.

See section [*] for more information

```
log_hostname
```

This option enables HTTP Inspect to parse and log the Host header data from the HTTP request and log it along with all the generated events for that session.

See section [*] for more information

```
log_mailfrom
```

This option enables SMTP preprocessor to parse and log the senders email address extracted from the "MAIL FROM" command along with all the generated events for that session.

See section [*] for more information

```
log_rcptto
```

This option enables SMTP preprocessor to parse and log the recipients email address extracted from the "RCPT TO" command along with all the generated events for that session.

See section [*] for more information

```
log_filename
```

This option enables SMTP preprocessor to parse and log the MIME attachment filenames extracted from the Content-Disposition header within the MIME body along with all the generated events for that session.

See section [*] for more information

```
log_email_hdrs
```

This option enables SMTP preprocessor to parse and log the SMTP email headers extracted from the SMTP data along with all the generated events for that session.

See section [*] for more information

2.6.7.4 Reading Unified2 Files

2.6.7.5 U2SpewFoo

U2SpewFoo is a lightweight tool for dumping the contents of unified2 files to stdout.

Example usage:

```

    u2spewfoo snort.log
```

Example Output:

```
(Event)
    sensor id: 0    event id: 4 event second: 1299698138    event microsecond: 146591
    sig id: 1   gen id: 1   revision: 0  classification: 0
    priority: 0 ip source: 10.1.2.3 ip destination: 10.9.8.7
    src port: 60710 dest port: 80   protocol: 6 impact_flag: 0  blocked: 0

Packet
    sensor id: 0    event id: 4 event second: 1299698138
    packet second: 1299698138   packet microsecond: 146591
    linktype: 1 packet_length: 54
[    0] 02 09 08 07 06 05 02 01 02 03 04 05 08 00 45 00  ..............E.
[   16] 00 28 00 06 00 00 40 06 5C B7 0A 01 02 03 0A 09  .(....@.\.......
[   32] 08 07 ED 26 00 50 00 00 00 62 00 00 00 2D 50 10  ...&.P...b...-P.
[   48] 01 00 A2 BB 00 00                                ......

(ExtraDataHdr)
    event type: 4   event length: 33
```

```
(ExtraData)
    sensor id: 0    event id: 2 event second: 1299698138
    type: 9 datatype: 1 bloblength: 9   HTTP URI: /

(ExtraDataHdr)
    event type: 4   event length: 78

(ExtraData)
    sensor id: 0    event id: 2 event second: 1299698138
    type: 10    datatype: 1 bloblength: 12  HTTP Hostname: example.com
```

2.6.7.6 U2Boat

U2boat is a tool for converting unified2 files into different formats.

Currently supported conversion formats are: pcap

Example usage:

```
    u2boat -t pcap <infile> <outfile>
```

2.6.8 log null

Sometimes it is useful to be able to create rules that will alert to certain types of traffic but will not cause packet log entries. In Snort 1.8.2, the log_null plugin was introduced. This is equivalent to using the -n command line option but it is able to work within a ruletype.

2.6.8.1 Format

```
    output log_null
```

2.6.8.2 Example

```
    output log_null  # like using snort -n

    ruletype info {
        type alert
        output alert_fast: info.alert
        output log_null
    }
```

2.6.9 Log Limits

This section pertains to logs produced by `alert_fast`, `alert_full`, `alert_csv`, and `log_tcpdump`. `unified2` also may be given limits. Those limits are described in the respective sections.

When a configured limit is reached, the current log is closed and a new log is opened with a UNIX timestamp appended to the configured log name.

Limits are configured as follows:

```
<limit> ::= <number>[(<gb>|<mb>|<kb>)]
<gb> ::= 'G'|'g'
<mb> ::= 'M'|'m'
<kb> ::= 'K'|'k'
```

Rollover will occur at most once per second so if limit is too small for logging rate, limit will be exceeded. Rollover works correctly if snort is stopped/restarted.

End of Navigation Panel Table of Child-Links **Subsections**

- 2.7.1 Rule evaluation
- 2.7.2 Snort Configuration
- 2.7.3 Host Attribute Table File Format
- 2.7.4 Attribute Table Example
- 2.7.5 Attribute Table Affect on preprocessors
    - 2.7.5.1 Attribute Table Affect on rules

End of Table of Child-Links

2.7 Host Attribute Table

Snort can use information from an external source(s) to more accurately inspect network traffic. This information saved to a file is known as a host attributes table. Host attributes are used in IPS rule evalution, IP fragment (see section [*])/TCP stream (see section [*]) reassembly policy selection, and by the application preprocessors. Host attribute table is loaded during initialization and supports runtime reloads; Snort reloads the host attributes table when it receives the signal SIGNAL_SNORT_READ_ATTR_TBL (See [*] to learn more).

> **Note:** To use a host attribute table and service information, Snort must be configured with the -enable-targetbased flag.

2.7.1 Rule evaluation

In rule evaluation, service information can be used instead of the ports when the metadata service(s) in the rule matches the service corresponding to the traffic. If the rule does not have metadata service(s), or the packet service was not matched then the port checks are used exclusively.

2.7.2 Snort Configuration

```
attribute_table filename <PATH/TO/HOSTS.XML>
```

2.7.3 Host Attribute Table File Format

The attribute table uses an XML format and consists of two sections, a mapping section, used to reduce the size of the file for common data elements, and the host attribute section. The mapping section is optional.

Provided below is an example attribute table

```
<SNORT_ATTRIBUTES>
    <ATTRIBUTE_MAP>
        <ENTRY>
            <ID>1</ID>
            <VALUE>Linux</VALUE>
        </ENTRY>
        <ENTRY>
            <ID>2</ID>
            <VALUE>ssh</VALUE>
        </ENTRY>
    </ATTRIBUTE_MAP>
    <ATTRIBUTE_TABLE>
        <HOST>
```

```
            <IP>192.168.1.234</IP>
            <OPERATING_SYSTEM>
                <NAME>
                    <ATTRIBUTE_ID>1</ATTRIBUTE_ID>
                    <CONFIDENCE>100</CONFIDENCE>
                </NAME>
                <VENDOR>
                    <ATTRIBUTE_VALUE>Red Hat</ATTRIBUTE_VALUE>
                    <CONFIDENCE>99</CONFIDENCE>
                </VENDOR>
                <VERSION>
                    <ATTRIBUTE_VALUE>2.6</ATTRIBUTE_VALUE>
                    <CONFIDENCE>98</CONFIDENCE>
                </VERSION>
                <FRAG_POLICY>linux</FRAG_POLICY>
                <STREAM_POLICY>linux</STREAM_POLICY>
            </OPERATING_SYSTEM>
            <SERVICES>
                <SERVICE>
                    <PORT>
                        <ATTRIBUTE_VALUE>22</ATTRIBUTE_VALUE>
                        <CONFIDENCE>100</CONFIDENCE>
                    </PORT>
                    <IPPROTO>
                        <ATTRIBUTE_VALUE>tcp</ATTRIBUTE_VALUE>
                        <CONFIDENCE>100</CONFIDENCE>
                    </IPPROTO>
                    <PROTOCOL>
                        <ATTRIBUTE_ID>2</ATTRIBUTE_ID>
                        <CONFIDENCE>100</CONFIDENCE>
                    </PROTOCOL>
                    <APPLICATION>
                        <ATTRIBUTE_VALUE>OpenSSH</ATTRIBUTE_VALUE>
                        <CONFIDENCE>100</CONFIDENCE>
                        <VERSION>
                            <ATTRIBUTE_VALUE>3.9p1</ATTRIBUTE_VALUE>
                            <CONFIDENCE>93</CONFIDENCE>
                        </VERSION>
                    </APPLICATION>
                </SERVICE>
                <SERVICE>
                    <PORT>
                        <ATTRIBUTE_VALUE>2300</ATTRIBUTE_VALUE>
                        <CONFIDENCE>100</CONFIDENCE>
                    </PORT>
                    <IPPROTO>
                        <ATTRIBUTE_VALUE>tcp</ATTRIBUTE_VALUE>
                        <CONFIDENCE>100</CONFIDENCE>
                    </IPPROTO>
                    <PROTOCOL>
                        <ATTRIBUTE_VALUE>telnet</ATTRIBUTE_VALUE>
                        <CONFIDENCE>100</CONFIDENCE>
                    </PROTOCOL>
                    <APPLICATION>
```

```
                        <ATTRIBUTE_VALUE>telnet</ATTRIBUTE_VALUE>
                        <CONFIDENCE>50</CONFIDENCE>
                    </APPLICATION>
                </SERVICE>
            </SERVICES>
            <CLIENTS>
                <CLIENT>
                    <IPPROTO>
                        <ATTRIBUTE_VALUE>tcp</ATTRIBUTE_VALUE>
                        <CONFIDENCE>100</CONFIDENCE>
                    </IPPROTO>
                    <PROTOCOL>
                        <ATTRIBUTE_VALUE>http</ATTRIBUTE_VALUE>
                        <CONFIDENCE>91</CONFIDENCE>
                    </PROTOCOL>
                    <APPLICATION>
                        <ATTRIBUTE_VALUE>IE Http Browser</ATTRIBUTE_VALUE>
                        <CONFIDENCE>90</CONFIDENCE>
                        <VERSION>
                            <ATTRIBUTE_VALUE>6.0</ATTRIBUTE_VALUE>
                            <CONFIDENCE>89</CONFIDENCE>
                        </VERSION>
                    </APPLICATION>
                </CLIENT>
            </CLIENTS>
        </HOST>
    </ATTRIBUTE_TABLE>
</SNORT_ATTRIBUTES>
```

A DTD for verification of the Host Attribute Table XML file is provided with the snort packages.

The confidence metric may be used to indicate the validity of a given service or client application and its respective elements. That field is not currently used by Snort, but may be in future releases.

> **Note:**
>
> Snort requires that the file be a properly formatted schema of the format defined above. Empty or incorrectly formatted files will likely cause a Fatal Error.

2.7.4 Attribute Table Example

In the example above, a host running Red Hat 2.6 is described. This host has an IP address of 192.168.1.234. On that host, TCP port 22 is ssh (running Open SSH), and TCP port 2300 is telnet.

The IP fragmentation and TCP stream reassembly is mimicked by the "linux" configuration (see sections [*] and [*]).

2.7.5 Attribute Table Affect on preprocessors

- Network Layer Preprocessors

Each of the network layer preprocessors (frag3 and stream5) make use of the respective `FRAG_POLICY` and `STREAM_POLICY` in terms of how data is handled for reassembly for packets being received by that host.

- Application Layer Preprocessors

  The application layer preprocessors (HTTP, SMTP, FTP, Telnet, etc) make use of the `SERVICE` information for connections destined to that host on that port.

  For example, even if the telnet portion of the FTP/Telnet preprocessor is only configured to inspect port 23, Snort will inspect packets for a connection to 192.168.1.234 port 2300 as telnet.

  Conversely, if, for example, HTTP Inspect is configured to inspect traffic on port 2300, HTTP Inspect will NOT process the packets on a connection to 192.168.1.234 port 2300 because it is identified as telnet.

Below is a list of the common services used by Snort's application layer preprocessors and Snort rules (see below).

http	ftp	ftp-data	telnet	smtp	ssh	tftp
dcerpc	netbios-dgm	netbios-ns	netbios-ssn	nntp	finger	sunrpc
dns	isakmp	mysql	oracle	cvs	shell	x11
imap	pop2	pop3	snmp			

2.7.5.1 Attribute Table Affect on rules

Snort uses service information in two ways; initialization of detection engine and as a detection criteria. To take advantage of this, Snort rules must contain the `metadata: service SERVICE` convention specified. During rule evaluation the default behavior will check first that the packet has been matched to a service, and then check that the packet's service matches the service(s) specified in the rule; if both these checks passed then Snort will disable source and destination port checks for the rule.

Snort 2.9.8 provides new functionality to control how rules use service information known as "service overrides". Service overrides are specified in the same way that services are specified in the rule, or more simply as a "reserved" `SERVICE`. The table below defines the new reserved service override names.

Metadata	Description
service and-ports	• Packet service must be set. • Packet service must match one of the rule services. • Packet must match the ports specified in the rule header.
service or-ports	• Packet service is set and it matches one of the rule services (skipping port checks).

	• Packet service is set but it DOES NOT match one of the rule services; packet must match the ports specified in the rule header. • Packet service is NOT set, only perform port checks.
service else-ports	• Packet service is set, packet must match one of the rule services. • Packet service is NOT set; then packet must match the ports specified in the rule header. This is the default behavior of a rule containing metadata service information.
service unknown	The keyword "unknown" is an alias for "service else-ports" above. It is intended for use in rules that do not contain metadata service information (port only rules). When used in a rule that doesn't otherwise contain metadata (rule only specifies ports), the evaluation is: • Packet service must NOT be set; then match ports.

End of Navigation Panel Table of Child-Links **Subsections**

- 2.8.1 Format
- 2.8.2 Directives

End of Table of Child-Links

2.8 Dynamic Modules

Dynamically loadable modules were introduced with Snort 2.6. They can be loaded via directives in `snort.conf` or via command-line options.

2.8.1 Format

```
<directive> <parameters>
```

2.8.2 Directives

Syntax	Description
`dynamicpreprocessor $[$ file $<$shared library path$>$ $ \vert$ directory $<$directory of shared libraries$>$ $]$`	Tells snort to load the dynamic preprocessor shared library (if file is used) or all dynamic preprocessor shared libraries (if directory is used). Specify `file`, followed by the full or relative path to the shared library. Or, specify `directory`, followed by the full or relative path to a directory of preprocessor shared libraries. (Same effect as `-dynamic-preprocessor-lib` or `-dynamic-preprocessor-lib-dir` options). See chapter [*] for more information on dynamic preprocessor libraries.
`dynamicengine $[$ file $<$shared library path$>$ $ \vert$ directory $<$directory of shared libraries$>$ $]$`	Tells snort to load the dynamic engine shared library (if file is used) or all dynamic engine shared libraries (if directory is used). Specify `file`, followed by the full or relative path to the shared library. Or, specify `directory`, followed by the full or relative path to a directory of preprocessor shared libraries. (Same effect as `-dynamic-engine-lib` or `-dynamic-preprocessor-lib-dir` options). See chapter [*] for more information on dynamic engine libraries.
`dynamicdetection $[$ file $<$shared library path$>$ $ \vert$ directory $<$directory of shared libraries$>$ $]$`	Tells snort to load the dynamic detection rules shared library (if file is used) or all dynamic detection rules shared libraries (if directory is used). Specify `file`, followed by the full or relative path to the shared library. Or, specify `directory`, followed by the full or relative path to a directory of detection rules shared libraries. (Same effect as `-dynamic-detection-lib` or `-dynamic-detection-lib-dir` options). See chapter [*] for more information on dynamic detection rules libraries.

End of Navigation Panel Table of Child-Links **Subsections**

- 2.9.1 Enabling support
- 2.9.2 Reloading a configuration
- 2.9.3 Non-reloadable configuration options

End of Table of Child-Links

2.9 Reloading a Snort Configuration

Snort now supports reloading a configuration in lieu of restarting Snort in so as to provide seamless traffic inspection during a configuration change. A separate thread will parse and create a swappable configuration object while the main Snort packet processing thread continues inspecting traffic under the current configuration. When a swappable configuration object is ready for use, the main Snort packet processing thread will swap in the new configuration to use and will continue processing under the new configuration. Note that for some preprocessors, existing session data will continue to use the configuration under which they were created in order to continue with proper state for that session. All newly created sessions will, however, use the new configuration.

2.9.1 Enabling support

To enable support for reloading a configuration, add `-enable-reload` to configure when compiling.

There is also an ancillary option that determines how Snort should behave if any non-reloadable options are changed (see section [*] below). This option is enabled by default and the behavior is for Snort to restart if any non-reloadable options are added/modified/removed. To disable this behavior and have Snort exit instead of restart, add `-disable-reload-error-restart` in addition to `-enable-reload` to configure when compiling.

> **Note:** This functionality is not currently supported in Windows.
>
> Caveat : When Snort is run on the primary network interface of an OpenBSD system, the reload and failopen operations may not function as expected.

2.9.2 Reloading a configuration

First modify your snort.conf (the file passed to the `-c` option on the command line).

Then, to initiate a reload, send Snort a `SIGHUP` signal, e.g.

```
$ kill -SIGHUP <snort pid>
```

> **Note:** If reload support is not enabled, Snort will restart (as it always has) upon receipt of a SIGHUP.

> **Note:** An invalid configuration will still result in a fatal error, so you should test your new configuration before issuing a reload, e.g. `$ snort -c snort.conf -T`

2.9.3 Non-reloadable configuration options

There are a number of option changes that are currently non-reloadable because they require changes to output, startup memory allocations, etc. Modifying any of these options will cause Snort to restart (as a `SIGHUP` previously did) or exit (if `-disable-reload-error-restart` was used to configure Snort).

Reloadable configuration options of note:

- Adding/modifying/removing text rules and variables are reloadable.
- Adding/modifying/removing preprocessor configurations are reloadable (except as noted below).

Non-reloadable configuration options of note:

- Adding/modifying/removing shared objects via dynamicdetection, dynamicengine and dynamicpreprocessor are not reloadable, i.e. any new/modified/removed shared objects will require a restart.
- Any changes to output will require a restart.

Changes to the following options are not reloadable:

```
attribute_table
config alertfile
config asn1
config chroot
config daemon
config detection_filter
config flowbits_size
config interface
config logdir
config max_attribute_hosts
config max_attribute_services_per_host
config nolog
config no_promisc
config pkt_count
config rate_filter
config response
config set_gid
config set_uid
config snaplen
config threshold
dynamicdetection
dynamicengine
dynamicpreprocessor
output
```

In certain cases, only some of the parameters to a config option or preprocessor configuration are not reloadable. Those parameters are listed below the relevant config option or preprocessor.

```
config ppm: max-rule-time <int>
 rule-log
```

```
config profile_rules
 filename
 print
 sort
config profile_preprocs
 filename
 print
 sort
preprocessor dcerpc2
 memcap
preprocessor frag3_global
 max_frags
 memcap
 prealloc_frags
 prealloc_memcap
 disabled
preprocessor perfmonitor
 file
 snortfile
preprocessor sfportscan
 memcap
 logfile
 disabled
preprocessor stream5_global
 memcap
 max_tcp
 max_udp
 max_icmp
 track_tcp
 track_udp
 track_icmp
```

End of Navigation Panel Table of Child-Links **Subsections**

- 2.10.1 Creating Multiple Configurations
- 2.10.2 Configuration Specific Elements
  - 2.10.2.1 Config Options
  - 2.10.2.2 Rules
  - 2.10.2.3 Variables
  - 2.10.2.4 Preprocessors
  - 2.10.2.5 Events and Output

- 2.10.3 How Configuration is applied?

End of Table of Child-Links

2.10 Multiple Configurations

Snort now supports multiple configurations based on VLAN Id or IP subnet within a single instance of Snort. This will allow administrators to specify multiple snort configuration files and bind each configuration to one or more VLANs or subnets rather than running one Snort for each configuration required. Each unique snort configuration file will create a new configuration instance within snort. VLANs/Subnets not bound to any specific configuration will use the default configuration. Each configuration can have different preprocessor settings and detection rules.

2.10.1 Creating Multiple Configurations

Default configuration for snort is specified using the existing -c option. A default configuration binds multiple vlans or networks to non-default configurations, using the following configuration line:

```
config binding: <path_to_snort.conf> vlan <vlanIdList>
config binding: <path_to_snort.conf> net <ipList>
config binding: <path_to_snort.conf> policy_id <id>
```

`path_to_snort.conf`
: - Refers to the absolute or relative path to the snort.conf for specific configuration.

`vlanIdList`
: - Refers to the comma separated list of vlandIds and vlanId ranges. The format for ranges is two vlanId separated by a "-". Spaces are allowed within ranges. Valid vlanId is any number in 0-4095 range. Negative vland Ids and alphanumeric are not supported.

`ipList`
: - Refers to ip subnets. Subnets can be CIDR blocks for IPV6 or IPv4. A maximum of 512 individual IPv4 or IPv6 addresses or CIDRs can be specified.

`policy_id`
: - Refers to the specific policyi_id to be applied. Valid policyi_id is any number in 0-4095 range.

> **Note:** Vlan and Subnets can not be used in the same line. Configurations can be applied based on either Vlans or Subnets not both.

> **Note:** Even though Vlan Ids 0 and 4095 are reserved, they are included as valid in terms of configuring Snort.

2.10.2 Configuration Specific Elements

2.10.2.1 Config Options

Generally config options defined within the default configuration are global by default i.e. their value applies to all other configurations. The following config options are specific to each configuration.

```
policy_id
policy_mode
policy_version
```

The following config options are specific to each configuration. If not defined in a configuration, the default values of the option (not the default configuration values) take effect.

```
config checksum_drop
config disable_decode_alerts
config disable_decode_drops
config disable_ipopt_alerts
config disable_ipopt_drops
config disable_tcpopt_alerts
config disable_tcpopt_drops
config disable_tcpopt_experimental_alerts
config disable_tcpopt_experimental_drops
config disable_tcpopt_obsolete_alerts
config disable_tcpopt_obsolete_drops
config disable_ttcp_alerts
config disable_tcpopt_ttcp_alerts
config disable_ttcp_drops
```

2.10.2.2 Rules

Rules are specific to configurations but only some parts of a rule can be customized for performance reasons. If a rule is not specified in a configuration then the rule will never raise an event for the configuration. A rule shares all parts of the rule options, including the general options, payload detection options, non-payload detection options, and post-detection options. Parts of the rule header can be specified differently across configurations, limited to:

```
Source IP address and port
Destination IP address and port
Action
```

A higher revision of a rule in one configuration will override other revisions of the same rule in other configurations.

2.10.2.3 Variables

Variables defined using "var", "portvar" and "ipvar" are specific to configurations. If the rules in a configuration use variables, those variables must be defined in that configuration.

2.10.2.4 Preprocessors

Preprocessors configurations can be defined within each vlan or subnet specific configuration. Options controlling specific preprocessor memory usage, through specific limit on memory usage or number of instances, are processed only in default policy. The options control total memory usage for a preprocessor across all policies. These options are ignored in non-default policies without raising an

error. A preprocessor must be configured in default configuration before it can be configured in non-default configuration. This is required as some mandatory preprocessor configuration options are processed only in default configuration.

2.10.2.5 Events and Output

An unique policy id can be assigned by user, to each configuration using the following config line:

```
config policy_id: <id>
```

`id`

- Refers to a 16-bit unsigned value. This policy id will be used to identify alerts from a specific configuration in the unified2 records.

> **Note:** If no policy id is specified, snort assigns 0 (zero) value to the configuration.

To enable vlanId logging in unified2 records the following option can be used.

```
output alert_unified2: vlan_event_types (alert logging only)
output unified2: filename <filename>, vlan_event_types (true unified logging)
```

`filename`

- Refers to the absolute or relative filename.

`vlan_event_types`

- When this option is set, snort will use unified2 event type 104 and 105 for IPv4 and IPv6 respectively.

> **Note:** Each event logged will have the vlanId from the packet if vlan headers are present otherwise 0 will be used.

2.10.3 How Configuration is applied?

Snort assigns every incoming packet to a unique configuration based on the following criteria. If VLANID is present, then the innermost VLANID is used to find bound configuration. If the bound configuration is the default configuration, then destination IP address is searched to the most specific subnet that is bound to a non-default configuration. The packet is assigned non-default configuration if found otherwise the check is repeated using source IP address. In the end, default configuration is used if no other matching configuration is found.

For addressed based configuration binding, this can lead to conflicts between configurations if source address is bound to one configuration and destination address is bound to another. In this case, snort will use the first configuration in the order of definition, that can be applied to the packet.

End of Navigation Panel Table of Child-Links **Subsections**

- 2.11.1 Enabling Active Response
- 2.11.2 Configure Sniping
- 2.11.3 Flexresp
- 2.11.4 React
- 2.11.5 Rule Actions

End of Table of Child-Links

2.11 Active Response

Snort 2.9 includes a number of changes to better handle inline operation, including:

- a single mechanism for all responses
- fully encoded reset or icmp unreachable packets
- updated flexible response rule option
- updated react rule option
- added block and sblock rule actions

These changes are outlined below.

2.11.1 Enabling Active Response

This enables active responses (snort will send TCP RST or ICMP unreachable/port) when dropping a session.

```
./configure --enable-active-response / -DACTIVE_RESPONSE

preprocessor stream5_global: \
    max_active_responses <max_rsp>, \
    min_response_seconds <min_sec>

<max_rsp> ::= (0..25)
<min_sec> ::= (1..300)
```

Active responses will be encoded based on the triggering packet. TTL will be set to the value captured at session pickup.

2.11.2 Configure Sniping

Configure the number of attempts to land a TCP RST within the session's current window (so that it is accepted by the receiving TCP). This sequence "strafing" is really only useful in passive mode. In inline mode the reset is put straight into the stream in lieu of the triggering packet so strafing is not necessary.

Each attempt (sent in rapid succession) has a different sequence number. Each active response will actually cause this number of TCP resets to be sent. TCP data (sent for react) is multiplied similarly. At most 1 ICMP unreachable is sent, if and only if attempts $>$ 0.

```
./configure --enable-active-response

config response: [device <dev>] [dst_mac <MAC address>] attempts <att>

<dev> ::= ip | eth0 | etc.
<att> ::= (1..20)
<MAC address> ::= nn:nn:nn:nn:nn:nn
 (n is a hex number from 0-F)
```

device ip will perform network layer injection. It is probably a better choice to specify an interface and avoid kernel routing tables, etc.

dst_mac will change response destination MAC address, if the device is eth0, eth1, eth2 etc. Otherwise, response destination MAC address is derived from packet. Example:

```
    config response: device eth0 dst_mac 00:06:76:DD:5F:E3 attempts 2
```

2.11.3 Flexresp

Flexresp and flexresp2 are replaced with flexresp3.

* Flexresp is deleted; these features are no longer available:

```
    ./configure --enable-flexresp / -DENABLE_RESPOND -DENABLE_RESPONSE
    config flexresp: attempts 1
```

* Flexresp2 is deleted; these features are deprecated, non-functional, and will be deleted in a future release:

```
    ./configure --enable-flexresp2 / -DENABLE_RESPOND -DENABLE_RESPONSE2

    config flexresp2_interface: eth0
    config flexresp2_attempts: 4
    config flexresp2_memcap: 1000000
    config flexresp2_rows: 1000
```

* Flexresp3 is new: the resp rule option keyword is used to configure active responses for rules that fire.

```
    ./configure --enable-flexresp3 / -DENABLE_RESPOND -DENABLE_RESPONSE3

    alert tcp any any -> any 80 (content:"a"; resp:<resp_t>; sid:1;)
```

* `resp_t` includes all flexresp and flexresp2 options:

```
    <resp_t> ::= \
        rst_snd | rst_rcv | rst_all | \
        reset_source | reset_dest | reset_both | icmp_net | \
        icmp_host | icmp_port | icmp_all
```

2.11.4 React

react is a rule option keyword that enables sending an HTML page on a session and then resetting it. This is built with:

```
./configure --enable-react / -DENABLE_REACT
```

The page to be sent can be read from a file:

```
config react: <block.html>
```

or else the default is used:

```
<default_page> ::= \
    "HTTP/1.1 403 Forbidden\r\n"
    "Connection: close\r\n"
    "Content-Type: text/html; charset=utf-8\r\n"
    "\r\n"
    "<!DOCTYPE html PUBLIC \"-//W3C//DTD XHTML 1.1//EN\"\r\n" \
    "    \"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd\">\r\n" \
    "<html xmlns=\"http://www.w3.org/1999/xhtml\" xml:lang=\"en\">\r\n" \
    "<head>\r\n" \
    "<meta http-equiv=\"Content-Type\" content=\"text/html; charset=UTF-8\" />\r\n"
\
    "<title>Access Denied</title>\r\n" \
    "</head>\r\n" \
    "<body>\r\n" \
    "<h1>Access Denied</h1>\r\n" \
    "<p>%s</p>\r\n" \
    "</body>\r\n" \
    "</html>\r\n";
```

Note that the file must contain the entire response, including any HTTP headers. In fact, the response isn't strictly limited to HTTP. You could craft a binary payload of arbitrary content.

Be aware of size when creating such responses. While it may be possible to respond with arbitrarily large responses, responses for TCP sessions will need to take into account that the receiver's window may only accept up to a certain amount of data. Sending past this limit will result in truncated data. In general, the smaller the response, the more likely it will be successful.

When the rule is configured, the page is loaded and the selected message, which defaults to:

```
<default_msg> ::= \
    "You are attempting to access a forbidden site.<br />" \
    "Consult your system administrator for details.";
```

Additional formatting operators beyond a single within a reference URL.

This is an example rule:

```
drop tcp any any -> any $HTTP_PORTS ( \
    content: "d"; msg:"Unauthorized Access Prohibited!"; \
    react: <react_opts>; sid:4;)

<react_opts> ::= [msg] [, <dep_opts>]
```

These options are deprecated:

```
    <dep_opts> ::= [block|warn], [proxy <port#>]
```

The original version sent the web page to one end of the session only if the other end of the session was port 80 or the optional proxy port. The new version always sends the page to the client. If no page should be sent, a resp option can be used instead. The deprecated options are ignored.

2.11.5 Rule Actions

The block and sblock actions have been introduced as synonyms for drop and sdrop to help avoid confusion between packets dropped due to load (e.g. lack of available buffers for incoming packets) and packets blocked due to Snort's analysis.

End of Navigation Panel

3. Writing Snort Rules

Subsections

- 3.1 The Basics
- 3.2 Rules Headers
  - 3.2.1 Rule Actions
  - 3.2.2 Protocols
  - 3.2.3 IP Addresses
  - 3.2.4 Port Numbers
  - 3.2.5 The Direction Operator
  - 3.2.6 Activate/Dynamic Rules
- 3.3 Rule Options
- 3.4 General Rule Options
  - 3.4.1 msg
  - 3.4.2 reference
  - 3.4.3 gid
  - 3.4.4 sid
  - 3.4.5 rev
  - 3.4.6 classtype
  - 3.4.7 priority
  - 3.4.8 metadata
  - 3.4.9 General Rule Quick Reference
- 3.5 Payload Detection Rule Options
  - 3.5.1 content
  - 3.5.2 protected_content
  - 3.5.3 hash
  - 3.5.4 length
  - 3.5.5 nocase
  - 3.5.6 rawbytes
  - 3.5.7 depth
  - 3.5.8 offset
  - 3.5.9 distance
  - 3.5.10 within
  - 3.5.11 http_client_body
  - 3.5.12 http_cookie
  - 3.5.13 http_raw_cookie
  - 3.5.14 http_header
  - 3.5.15 http_raw_header

- 3.5.16 http_method
- 3.5.17 http_uri
- 3.5.18 http_raw_uri
- 3.5.19 http_stat_code
- 3.5.20 http_stat_msg
- 3.5.21 http_encode
- 3.5.22 fast_pattern
- 3.5.23 uricontent
- 3.5.24 urilen
- 3.5.25 isdataat
- 3.5.26 pcre
- 3.5.27 pkt_data
- 3.5.28 file_data
- 3.5.29 base64_decode
- 3.5.30 base64_data
- 3.5.31 byte_test
- 3.5.32 byte_jump
- 3.5.33 byte_extract
- 3.5.34 byte_math
- 3.5.35 ftpbounce
- 3.5.36 asn1
- 3.5.37 cvs
- 3.5.38 dce_iface
- 3.5.39 dce_opnum
- 3.5.40 dce_stub_data
- 3.5.41 sip_method
- 3.5.42 sip_stat_code
- 3.5.43 sip_header
- 3.5.44 sip_body
- 3.5.45 gtp_type
- 3.5.46 gtp_info
- 3.5.47 gtp_version
- 3.5.48 ssl_version
- 3.5.49 ssl_state
- 3.5.50 Payload Detection Quick Reference

- 3.6 Non-Payload Detection Rule Options
  - 3.6.1 fragoffset
  - 3.6.2 ttl
  - 3.6.3 tos
  - 3.6.4 id
  - 3.6.5 ipopts
  - 3.6.6 fragbits
  - 3.6.7 dsize
  - 3.6.8 flags
  - 3.6.9 flow

- 3.6.10 flowbits
- 3.6.11 seq
- 3.6.12 ack
- 3.6.13 window
- 3.6.14 itype
- 3.6.15 icode
- 3.6.16 icmp_id
- 3.6.17 icmp_seq
- 3.6.18 rpc
- 3.6.19 ip_proto
- 3.6.20 sameip
- 3.6.21 stream_reassemble
- 3.6.22 stream_size
- 3.6.23 Non-Payload Detection Quick Reference

- 3.7 Post-Detection Rule Options
  - 3.7.1 logto
  - 3.7.2 session
  - 3.7.3 resp
  - 3.7.4 react
  - 3.7.5 tag
  - 3.7.6 replace
  - 3.7.7 detection_filter
  - 3.7.8 Post-Detection Quick Reference

- 3.8 Rule Thresholds
- 3.9 Writing Good Rules
  - 3.9.1 Content Matching
  - 3.9.2 Catch the Vulnerability, Not the Exploit
  - 3.9.3 Catch the Oddities of the Protocol in the Rule
  - 3.9.4 Optimizing Rules
  - 3.9.5 Testing Numerical Values

End of Navigation Panel

3.1 The Basics

Snort uses a simple, lightweight rules description language that is flexible and quite powerful. There are a number of simple guidelines to remember when developing Snort rules that will help safeguard your sanity.

Most Snort rules are written in a single line. This was required in versions prior to 1.8. In current versions of Snort, rules may span multiple lines by adding a backslash \ to the end of the line.

Snort rules are divided into two logical sections, the rule header and the rule options. The rule header contains the rule's action, protocol, source and destination IP addresses and netmasks, and the source and destination ports information. The rule option section contains alert messages and information on which parts of the packet should be inspected to determine if the rule action should be taken.

Figure [*] illustrates a sample Snort rule.

Figure: Sample Snort Rule

```
\begin{figure}\begin{verbatim}alert tcp any any -> 192.168.1.0/24 111 \ (con... ...\vert0 01 86 a5\vert'';
msg:''mountd access'';)\end{verbatim} \par\end{figure}
```

The text up to the first parenthesis is the rule header and the section enclosed in parenthesis contains the rule options. The words before the colons in the rule options section are called option keywords.

> **Note:**
>
> Note that the rule options section is not specifically required by any rule, they are just used for the sake of making tighter definitions of packets to collect or alert on (or drop, for that matter).

All of the elements in that make up a rule must be true for the indicated rule action to be taken. When taken together, the elements can be considered to form a logical AND statement. At the same time, the various rules in a Snort rules library file can be considered to form a large logical OR statement.

End of Navigation Panel Table of Child-Links **Subsections**

- 3.2.1 Rule Actions
- 3.2.2 Protocols
- 3.2.3 IP Addresses
- 3.2.4 Port Numbers
- 3.2.5 The Direction Operator
- 3.2.6 Activate/Dynamic Rules

End of Table of Child-Links

3.2 Rules Headers

3.2.1 Rule Actions

The rule header contains the information that defines the who, where, and what of a packet, as well as what to do in the event that a packet with all the attributes indicated in the rule should show up. The first item in a rule is the rule action. The rule action tells Snort what to do when it finds a packet that matches the rule criteria. There are 3 available default actions in Snort, alert, log, pass. In addition, if you are running Snort in inline mode, you have additional options which include drop, reject, and sdrop.

1. alert - generate an alert using the selected alert method, and then log the packet
2. log - log the packet
3. pass - ignore the packet
4. drop - block and log the packet
5. reject - block the packet, log it, and then send a TCP reset if the protocol is TCP or an ICMP port unreachable message if the protocol is UDP.
6. sdrop - block the packet but do not log it.

You can also define your own rule types and associate one or more output plugins with them. You can then use the rule types as actions in Snort rules.

This example will create a type that will log to just tcpdump:

```
ruletype suspicious
{
    type log
    output log_tcpdump: suspicious.log
}
```

This example will create a rule type that will log to syslog and tcpdump: database:

```
ruletype redalert
{
      type alert
      output alert_syslog: LOG_AUTH LOG_ALERT
      output log_tcpdump: suspicious.log
}
```

3.2.2 Protocols

The next field in a rule is the protocol. There are four protocols that Snort currently analyzes for suspicious behavior - TCP, UDP, ICMP, and IP. In the future there may be more, such as ARP, IGRP, GRE, OSPF, RIP, IPX, etc.

3.2.3 IP Addresses

The next portion of the rule header deals with the IP address and port information for a given rule. The keyword any may be used to define any address. Snort does not have a mechanism to provide host name lookup for the IP address fields in the config file. The addresses are formed by a straight numeric IP address and a CIDR[3] block. The CIDR block indicates the netmask that should be applied to the rule's address and any incoming packets that are tested against the rule. A CIDR block mask of /24 indicates a Class C network, /16 a Class B network, and /32 indicates a specific machine address. For example, the address/CIDR combination 192.168.1.0/24 would signify the block of addresses from 192.168.1.1 to 192.168.1.255. Any rule that used this designation for, say, the destination address would match on any address in that range. The CIDR designations give us a nice short-hand way to designate large address spaces with just a few characters.

In Figure [*], the source IP address was set to match for any computer talking, and the destination address was set to match on the 192.168.1.0 Class C network.

There is an operator that can be applied to IP addresses, the negation operator. This operator tells Snort to match any IP address except the one indicated by the listed IP address. The negation operator is indicated with a !. For example, an easy modification to the initial example is to make it alert on any traffic that originates outside of the local net with the negation operator as shown in Figure [*].

Figure: Example IP Address Negation Rule

```
\begin{figure}\begin{verbatim}alert tcp !192.168.1.0/24 any -> 192.168.1.0/24... ... 86 a5\vert''; msg:''external
mountd access'';)\end{verbatim} \par\end{figure}
```

This rule's IP addresses indicate any tcp packet with a source IP address not originating from the internal network and a destination address on the internal network.

You may also specify lists of IP addresses. An IP list is specified by enclosing a comma separated list of IP addresses and CIDR blocks within square brackets. For the time being, the IP list may not include spaces between the addresses. See Figure [*] for an example of an IP list in action.

Figure: IP Address Lists

```
\begin{figure}\begin{verbatim}alert tcp ![192.168.1.0/24,10.1.1.0/24] any -> ... ... a5\vert''; \ msg:''external
mountd access'';)\end{verbatim} \par\end{figure}
```

3.2.4 Port Numbers

Port numbers may be specified in a number of ways, including any ports, static port definitions, ranges, and by negation. Any ports are a wildcard value, meaning literally any port. Static ports are indicated by

a single port number, such as 111 for portmapper, 23 for telnet, or 80 for http, etc. Port ranges are indicated with the range operator :. The range operator may be applied in a number of ways to take on different meanings, such as in Figure [*].

Figure: Port Range Examples

```
\begin{figure}\begin{verbatim}log udp any any -> 192.168.1.0/24 1:1024\end{ve... ... or equal to 1024 going to ports greater than or equal to 500 \par\end{figure}
```

Port negation is indicated by using the negation operator !. The negation operator may be applied against any of the other rule types (except any, which would translate to none, how Zen...). For example, if for some twisted reason you wanted to log everything except the X Windows ports, you could do something like the rule in Figure [*].

Figure: Example of Port Negation

```
\begin{figure}\begin{verbatim}log tcp any any -> 192.168.1.0/24 !6000:6010\end{verbatim} \par\end{figure}
```

3.2.5 The Direction Operator

The direction operator -$>$ indicates the orientation, or direction, of the traffic that the rule applies to. The IP address and port numbers on the left side of the direction operator is considered to be the traffic coming from the source host, and the address and port information on the right side of the operator is the destination host. There is also a bidirectional operator, which is indicated with a $<>$ symbol. This tells Snort to consider the address/port pairs in either the source or destination orientation. This is handy for recording/analyzing both sides of a conversation, such as telnet or POP3 sessions. An example of the bidirectional operator being used to record both sides of a telnet session is shown in Figure [*].

Also, note that there is no $<$- operator. In Snort versions before 1.8.7, the direction operator did not have proper error checking and many people used an invalid token. The reason the $<$- does not exist is so that rules always read consistently.

Figure: Snort rules using the Bidirectional Operator

```
\begin{figure}\begin{verbatim}log tcp !192.168.1.0/24 any <> 192.168.1.0/24 23\end{verbatim} \par\end{figure}
```

3.2.6 Activate/Dynamic Rules

Activate and Dynamic rules are phased out in favor of a combination of tagging ([*]) and flowbits ([*]).

End of Navigation Panel

3.3 Rule Options

Rule options form the heart of Snort's intrusion detection engine, combining ease of use with power and flexibility. All Snort rule options are separated from each other using the semicolon (;) character. Rule option keywords are separated from their arguments with a colon (:) character.

There are four major categories of rule options.

general
: These options provide information about the rule but do not have any affect during detection

payload
: These options all look for data inside the packet payload and can be inter-related

non-payload
: These options look for non-payload data

post-detection
: These options are rule specific triggers that happen after a rule has “fired.”

End of Navigation Panel Table of Child-Links **Subsections**

- 3.4.1 msg
  - 3.4.1.1 Format
- 3.4.2 reference
  - 3.4.2.1 Format
  - 3.4.2.2 Examples
- 3.4.3 gid
  - 3.4.3.1 Format
  - 3.4.3.2 Example
- 3.4.4 sid
  - 3.4.4.1 Format
  - 3.4.4.2 Example
- 3.4.5 rev
  - 3.4.5.1 Format
  - 3.4.5.2 Example
- 3.4.6 classtype
  - 3.4.6.1 Format
  - 3.4.6.2 Example
  - 3.4.6.3 Warnings
- 3.4.7 priority
  - 3.4.7.1 Format
  - 3.4.7.2 Examples
- 3.4.8 metadata
  - 3.4.8.1 Format
  - 3.4.8.2 Examples
- 3.4.9 General Rule Quick Reference

End of Table of Child-Links

3.4 General Rule Options

3.4.1 msg

The msg rule option tells the logging and alerting engine the message to print along with a packet dump or to an alert. It is a simple text string that utilizes the \ as an escape character to indicate a discrete character that might otherwise confuse Snort's rules parser (such as the semi-colon ; character).

3.4.1.1 Format

```
msg:"<message text>";
```

3.4.2 reference

The reference keyword allows rules to include references to external attack identification systems. The plugin currently supports several specific systems as well as unique URLs. This plugin is to be used by output plugins to provide a link to additional information about the alert produced.

Make sure to also take a look at `http://www.snort.org/pub-bin/sigs-search.cgi/` for a system that is indexing descriptions of alerts based on of the sid (See Section [*]).

Table: Supported Systems

System	URL Prefix
bugtraq	http://www.securityfocus.com/bid/
cve	http://cve.mitre.org/cgi-bin/cvename.cgi?name=
nessus	http://cgi.nessus.org/plugins/dump.php3?id=
arachnids	(currently down) http://www.whitehats.com/info/IDS
mcafee	http://vil.nai.com/vil/content/v_
osvdb	http://osvdb.org/show/osvdb/
msb	http://technet.microsoft.com/en-us/security/bulletin/
url	http://

3.4.2.1 Format

```
reference:<id system>, <id>; [reference:<id system>, <id>;]
```

3.4.2.2 Examples

```
alert tcp any any -> any 7070 (msg:"IDS411/dos-realaudio"; \
    flags:AP; content:"|fff4 fffd 06|"; reference:arachnids,IDS411;)

alert tcp any any -> any 21 (msg:"IDS287/ftp-wuftp260-venglin-linux"; \
    flags:AP; content:"|31c031db 31c9b046 cd80 31c031db|"; \
    reference:arachnids,IDS287; reference:bugtraq,1387; \
    reference:cve,CAN-2000-1574;)
```

3.4.3 gid

The `gid` keyword (generator id) is used to identify what part of Snort generates the event when a particular rule fires. For example gid 1 is associated with the rules subsystem and various gids over 100 are designated for specific preprocessors and the decoder. See etc/generators in the source tree for the current generator ids in use. Note that the gid keyword is optional and if it is not specified in a rule, it will default to 1 and the rule will be part of the general rule subsystem. To avoid potential conflict with gids defined in Snort (that for some reason aren't noted it etc/generators), it is recommended that values starting at 1,000,000 be used. For general rule writing, it is not recommended that the `gid` keyword be used. This option should be used with the `sid` keyword. (See section [*])

The file etc/gen-msg.map contains contains more information on preprocessor and decoder gids.

3.4.3.1 Format

```
gid:<generator id>;
```

3.4.3.2 Example

This example is a rule with a generator id of 1000001.

```
alert tcp any any -> any 80 (content:"BOB"; gid:1000001; sid:1; rev:1;)
```

3.4.4 sid

The `sid` keyword is used to uniquely identify Snort rules. This information allows output plugins to identify rules easily. This option should be used with the `rev` keyword. (See section [*])

- $<$100 Reserved for future use
- 100-999,999 Rules included with the Snort distribution
- $>=$1,000,000 Used for local rules

The file sid-msg.map contains a mapping of alert messages to Snort rule IDs. This information is useful when post-processing alert to map an ID to an alert message.

3.4.4.1 Format

```
sid:<snort rules id>;
```

3.4.4.2 Example

This example is a rule with the Snort Rule ID of 1000983.

```
alert tcp any any -> any 80 (content:"BOB"; sid:1000983; rev:1;)
```

3.4.5 rev

The `rev` keyword is used to uniquely identify revisions of Snort rules. Revisions, along with Snort rule id's, allow signatures and descriptions to be refined and replaced with updated information. This option should be used with the `sid` keyword. (See section [*])

3.4.5.1 Format

```
rev:<revision integer>;
```

3.4.5.2 Example

This example is a rule with the Snort Rule Revision of 1.

```
alert tcp any any -> any 80 (content:"BOB"; sid:1000983; rev:1;)
```

3.4.6 classtype

The `classtype` keyword is used to categorize a rule as detecting an attack that is part of a more general type of attack class. Snort provides a default set of attack classes that are used by the default set of rules it provides. Defining classifications for rules provides a way to better organize the event data Snort produces.

3.4.6.1 Format

```
classtype:<class name>;
```

3.4.6.2 Example

```
alert tcp any any -> any 25 (msg:"SMTP expn root"; flags:A+; \
    content:"expn root"; nocase; classtype:attempted-recon;)
```

Attack classifications defined by Snort reside in the `classification.config` file. The file uses the following syntax:

```
config classification:  <class name>,<class description>,<default priority>
```

These attack classifications are listed in Table [*]. They are currently ordered with 4 default priorities. A priority of 1 (high) is the most severe and 4 (very low) is the least severe.

Table: Snort Default Classifications

Classtype	Description	Priority
attempted-admin	Attempted Administrator Privilege Gain	high
attempted-user	Attempted User Privilege Gain	high
inappropriate-content	Inappropriate Content was Detected	high
policy-violation	Potential Corporate Privacy Violation	high
shellcode-detect	Executable code was detected	high
successful-admin	Successful Administrator Privilege Gain	high
successful-user	Successful User Privilege Gain	high
trojan-activity	A Network Trojan was detected	high
unsuccessful-user	Unsuccessful User Privilege Gain	high
web-application-attack	Web Application Attack	high
attempted-dos	Attempted Denial of Service	medium
attempted-recon	Attempted Information Leak	medium
bad-unknown	Potentially Bad Traffic	medium
default-login-attempt	Attempt to login by a default username and password	medium
denial-of-service	Detection of a Denial of Service Attack	medium
misc-attack	Misc Attack	medium
non-standard-protocol	Detection of a non-standard protocol or event	medium
rpc-portmap-decode	Decode of an RPC Query	medium
successful-dos	Denial of Service	medium
successful-recon-largescale	Large Scale Information Leak	medium
successful-recon-limited	Information Leak	medium
suspicious-filename-detect	A suspicious filename was detected	medium
suspicious-login	An attempted login using a suspicious username was detected	medium
system-call-detect	A system call was detected	medium
unusual-client-port-connection	A client was using an unusual port	medium
web-application-activity	Access to a potentially vulnerable web application	medium

icmp-event	Generic ICMP event	low
misc-activity	Misc activity	low
network-scan	Detection of a Network Scan	low
not-suspicious	Not Suspicious Traffic	low
protocol-command-decode	Generic Protocol Command Decode	low
string-detect	A suspicious string was detected	low
unknown	Unknown Traffic	low
tcp-connection	A TCP connection was detected	very low

3.4.6.3 Warnings

The `classtype` option can only use classifications that have been defined in `snort.conf` by using the `config classification` option. Snort provides a default set of classifications in `classification.config` that are used by the rules it provides.

3.4.7 priority

The `priority` tag assigns a severity level to rules. A `classtype` rule assigns a default priority (defined by the `config classification` option) that may be overridden with a priority rule. Examples of each case are given below.

3.4.7.1 Format

```
priority:<priority integer>;
```

3.4.7.2 Examples

```
alert tcp any any -> any 80 (msg:"WEB-MISC phf attempt"; flags:A+; \
    content:"/cgi-bin/phf"; priority:10;)

alert tcp any any -> any 80 (msg:"EXPLOIT ntpdx overflow"; \
    dsize:>128; classtype:attempted-admin; priority:10 );
```

3.4.8 metadata

The `metadata` tag allows a rule writer to embed additional information about the rule, typically in a key-value format. Certain metadata keys and values have meaning to Snort and are listed in Table [*]. Keys other than those listed in the table are effectively ignored by Snort and can be free-form, with a key and a value. Multiple keys are separated by a comma, while keys and values are separated by a space.

Table: Snort Metadata Keys

Key	Description	Value Format
`engine`	Indicate a Shared Library Rule	"shared"
`soid`	Shared Library Rule Generator and SID	gid$\vert$sid
`service`	Target-Based Service Identifier	"http"

> **Note:**
>
> The `service` Metadata Key is only meaningful when a Host Attribute Table is provided. See Section [*] for details on the Host Attribute Table.

.

3.4.8.1 Format

The examples below show an stub rule from a shared library rule. The first uses multiple metadata keywords, the second a single metadata keyword, with keys separated by commas.

```
metadata:key1 value1;
metadata:key1 value1, key2 value2;
```

3.4.8.2 Examples

```
alert tcp any any -> any 80 (msg:"Shared Library Rule Example"; \
    metadata:engine shared; metadata:soid 3|12345;)

alert tcp any any -> any 80 (msg:"Shared Library Rule Example"; \
    metadata:engine shared, soid 3|12345;)

alert tcp any any -> any 80 (msg:"HTTP Service Rule Example"; \
    metadata:service http;)
```

3.4.9 General Rule Quick Reference

Table: General rule option keywords

Keyword	Description
`msg`	The msg keyword tells the logging and alerting engine the message to print with the packet dump or alert.
`reference`	The reference keyword allows rules to include references to external attack identification systems.
`gid`	

	The gid keyword (generator id) is used to identify what part of Snort generates the event when a particular rule fires.
`sid`	The sid keyword is used to uniquely identify Snort rules.
`rev`	The rev keyword is used to uniquely identify revisions of Snort rules.
`classtype`	The classtype keyword is used to categorize a rule as detecting an attack that is part of a more general type of attack class.
`priority`	The priority keyword assigns a severity level to rules.
`metadata`	The metadata keyword allows a rule writer to embed additional information about the rule, typically in a key-value format.

End of Navigation Panel Table of Child-Links **Subsections**

- 3.5.1 content
  - 3.5.1.1 Format
  - 3.5.1.2 Examples
  - 3.5.1.3 Changing content behavior
- 3.5.2 protected_content
  - 3.5.2.1 Format
  - 3.5.2.2 Examples
- 3.5.3 hash
  - 3.5.3.1 Format
- 3.5.4 length
  - 3.5.4.1 Format
- 3.5.5 nocase
  - 3.5.5.1 Format
  - 3.5.5.2 Example
- 3.5.6 rawbytes
  - 3.5.6.1 format
  - 3.5.6.2 Example
- 3.5.7 depth
  - 3.5.7.1 Format
- 3.5.8 offset
  - 3.5.8.1 Format
  - 3.5.8.2 Example
- 3.5.9 distance
  - 3.5.9.1 Format
  - 3.5.9.2 Example
- 3.5.10 within
  - 3.5.10.1 Format
  - 3.5.10.2 Examples
- 3.5.11 http_client_body
  - 3.5.11.1 Format
  - 3.5.11.2 Examples
- 3.5.12 http_cookie
  - 3.5.12.1 Format
  - 3.5.12.2 Examples

- 3.5.13 http_raw_cookie
  - 3.5.13.1 Format
  - 3.5.13.2 Examples
- 3.5.14 http_header
  - 3.5.14.1 Format
  - 3.5.14.2 Examples
- 3.5.15 http_raw_header
  - 3.5.15.1 Format
  - 3.5.15.2 Examples
- 3.5.16 http_method
  - 3.5.16.1 Format
  - 3.5.16.2 Examples
- 3.5.17 http_uri
  - 3.5.17.1 Format
  - 3.5.17.2 Examples
- 3.5.18 http_raw_uri
  - 3.5.18.1 Format
  - 3.5.18.2 Examples
- 3.5.19 http_stat_code
  - 3.5.19.1 Format
  - 3.5.19.2 Examples
- 3.5.20 http_stat_msg
  - 3.5.20.1 Format
  - 3.5.20.2 Examples
- 3.5.21 http_encode
  - 3.5.21.1 Format
  - 3.5.21.2 Examples
- 3.5.22 fast_pattern
  - 3.5.22.1 Format
  - 3.5.22.2 Examples
- 3.5.23 uricontent
  - 3.5.23.1 Format
- 3.5.24 urilen
  - 3.5.24.1 Format

- 3.5.25 isdataat
  - 3.5.25.1 Format
  - 3.5.25.2 Example
- 3.5.26 pcre
  - 3.5.26.1 Format
  - 3.5.26.2 Example
- 3.5.27 pkt_data
  - 3.5.27.1 Format
  - 3.5.27.2 Example
- 3.5.28 file_data
  - 3.5.28.1 Format
  - 3.5.28.2 Example
- 3.5.29 base64_decode
  - 3.5.29.1 Format
  - 3.5.29.2 Examples
- 3.5.30 base64_data
  - 3.5.30.1 Format
  - 3.5.30.2 Example
- 3.5.31 byte_test
  - 3.5.31.1 Format
  - 3.5.31.2 Examples
- 3.5.32 byte_jump
  - 3.5.32.1 Format
  - 3.5.32.2 Example
- 3.5.33 byte_extract
  - 3.5.33.1 Format
  - 3.5.33.2 Other options which use byte_extract variables
  - 3.5.33.3 Examples
- 3.5.34 byte_math
  - 3.5.34.1 Format
  - 3.5.34.2 Other rule options which use byte_math result variable
  - 3.5.34.3 Examples
- 3.5.35 ftpbounce
  - 3.5.35.1 Format
  - 3.5.35.2 Example

- 3.5.36 asn1
  - 3.5.36.1 Format
  - 3.5.36.2 Examples

- 3.5.37 cvs
  - 3.5.37.1 Format
  - 3.5.37.2 Examples

- 3.5.38 dce_iface
- 3.5.39 dce_opnum
- 3.5.40 dce_stub_data
- 3.5.41 sip_method
- 3.5.42 sip_stat_code
- 3.5.43 sip_header
- 3.5.44 sip_body
- 3.5.45 gtp_type
- 3.5.46 gtp_info
- 3.5.47 gtp_version
- 3.5.48 ssl_version
- 3.5.49 ssl_state
- 3.5.50 Payload Detection Quick Reference

End of Table of Child-Links

3.5 Payload Detection Rule Options

3.5.1 content

The content keyword is one of the more important features of Snort. It allows the user to set rules that search for specific content in the packet payload and trigger response based on that data. Whenever a content option pattern match is performed, the Boyer-Moore pattern match function is called and the (rather computationally expensive) test is performed against the packet contents. If data exactly matching the argument data string is contained anywhere within the packet's payload, the test is successful and the remainder of the rule option tests are performed. Be aware that this test is case sensitive.

The option data for the content keyword is somewhat complex; it can contain mixed text and binary data. The binary data is generally enclosed within the pipe ($\vert$) character and represented as bytecode. Bytecode represents binary data as hexadecimal numbers and is a good shorthand method for describing complex binary data. The example below shows use of mixed text and binary data in a Snort rule.

Note that multiple content rules can be specified in one rule. This allows rules to be tailored for less false positives.

If the rule is preceded by a `!`, the alert will be triggered on packets that do not contain this content. This is useful when writing rules that want to alert on packets that do not match a certain pattern

Note:

Also note that the following characters must be escaped inside a content rule:

```
; \ "
```

3.5.1.1 Format

```
content:[!]"<content string>";
```

3.5.1.2 Examples

```
alert tcp any any -> any 139 (content:"|5c 00|P|00|I|00|P|00|E|00 5c|";)
```

```
alert tcp any any -> any 80 (content:!"GET";)
```

Note:

A `!` modifier negates the results of the entire content search, modifiers included. For example, if using `content:!"A"; within:50;` and there are only 5 bytes of payload and there is no "A" in those 5 bytes, the result will return a match. If there must be 50 bytes for a valid match, use `isdataat` as a pre-cursor to the content.

3.5.1.3 Changing content behavior

The `content` keyword has a number of modifier keywords. The modifier keywords change how the previously specified content works. These modifier keywords are:

Table: Content Modifiers

Modifier	Section
nocase	[*]
rawbytes	[*]
depth	[*]
offset	[*]
distance	[*]
within	[*]
http_client_body	[*]
http_cookie	[*]
http_raw_cookie	[*]
http_header	[*]
http_raw_header	[*]
http_method	[*]
http_uri	[*]
http_raw_uri	[*]
http_stat_code	[*]
http_stat_msg	[*]
fast_pattern	[*]

3.5.2 protected_content

The protected_content keyword provides much of the functionality of the content keyword, however it performs and is utilized in a very different manner. The primary advantage protected_content has over

content is that protected allows one to hide the target contents by only revealing secure hash digests of said content. As with the content keyword, its primary purpose is to match strings of specific bytes. The search is performed by hashing portions of incoming packets and comparing the results against the hash provided, and as such, it is computationally expensive.

Currently, it is possible to utilize the MD5, SHA256, and SHA512 hash algorithms with the protected_content keyword. A hashing algorithm must be specified in the rule using `hash` if a default has not be set in the Snort configuration. Additionally, a `length` modifier must be specified with protected to indicate the length of the raw data.

As with content, it is possible to use multiple protected_content rules can in one rule. Additionally, it is possible to mix multiple protected_content rules with multiple content rules.

If the rule is preceded by a `!`, the alert will be triggered on packets that do not contain the target content. This is useful when writing rules that want to alert on packets that do not match a certain pattern

Note:

The protected_content keyword can be used with some (but not all) of the content modifiers. Those not supported include:

```
        nocase
        fast_pattern
        depth
        within
```

3.5.2.1 Format

```
    protected_content:[!]"<content hash>", length:orig_len[, hash:md5|sha256|sha512];
```

3.5.2.2 Examples

The following alert on the string "HTTP":

```
    alert tcp any any <> any 80 (msg:"MD5 Alert";
    protected_content:"293C9EA246FF9985DC6F62A650F78986"; hash:md5; offset:0;
length:4;)
```

```
    alert tcp any any <> any 80 (msg:"SHA256 Alert";

protected_content:"56D6F32151AD8474F40D7B939C2161EE2BBF10023F4AF1DBB3E13260EBDC6342";
    hash:sha256; offset:0; length:4;)
```

Note:

A `!` modifier negates the results of the entire content search, modifiers included. For example, if using `content:!"A"; within:50;` and there are only 5 bytes of payload and there is no "A" in those 5 bytes, the

result will return a match. If there must be 50 bytes for a valid match, use `isdataat` as a pre-cursor to the content.

3.5.3 hash

The hash keyword is used to specify the hashing algorithm to use when matching a protected_content rule. If a default algorithm is not specified in the Snort configuration, a protected_content rule must specify the algorithm used. Currently, MD5, SHA256, and SHA512 are supported.

3.5.3.1 Format

```
hash:[md5|sha256|sha512];
```

3.5.4 length

The length keyword is used to specify the original length of the content specified in a protected_content rule digest. The value provided must be greater than 0 and less than 65536.

3.5.4.1 Format

```
length:[<original_length>];
```

3.5.5 nocase

The nocase keyword allows the rule writer to specify that the Snort should look for the specific pattern, ignoring case. nocase modifies the previous `content` keyword in the rule.

3.5.5.1 Format

```
nocase;
```

3.5.5.2 Example

```
alert tcp any any -> any 21 (msg:"FTP ROOT"; content:"USER root"; nocase;)
```

3.5.6 rawbytes

The rawbytes keyword allows rules to look at the raw packet data, ignoring any decoding that was done by preprocessors. This acts as a modifier to the previous content [*] option.

HTTP Inspect has a set of keywords to use raw data, such as `http_raw_cookie`, `http_raw_header`, `http_raw_uri` etc that match on specific portions of the raw HTTP requests and responses.

Most other preprocessors use decoded/normalized data for content match by default, if `rawbytes` is not specified explicitly. Therefore, `rawbytes` should be specified in order to inspect arbitrary raw data from the packet.

3.5.6.1 format

```
rawbytes;
```

3.5.6.2 Example

This example tells the content pattern matcher to look at the raw traffic, instead of the decoded traffic provided by the Telnet decoder.

```
alert tcp any any -> any 21 (msg:"Telnet NOP"; content:"|FF F1|"; rawbytes;)
```

3.5.7 depth

The depth keyword allows the rule writer to specify how far into a packet Snort should search for the specified pattern. depth modifies the previous `content' keyword in the rule.

A depth of 5 would tell Snort to only look for the specified pattern within the first 5 bytes of the payload.

As the depth keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `depth` is specified.

This keyword allows values greater than or equal to the pattern length being searched. The minimum allowed value is 1. The maximum allowed value for this keyword is 65535.

The value can also be set to a string value referencing a variable extracted by the `byte_extract` keyword in the same rule.

3.5.7.1 Format

```
depth:[<number>|<var_name>];
```

3.5.8 offset

The offset keyword allows the rule writer to specify where to start searching for a pattern within a packet. offset modifies the previous 'content' keyword in the rule.

An offset of 5 would tell Snort to start looking for the specified pattern after the first 5 bytes of the payload.

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `offset` is specified.

This keyword allows values from -65535 to 65535.

The value can also be set to a string value referencing a variable extracted by the `byte_extract` keyword in the same rule.

3.5.8.1 Format

```
offset:[<number>|<var_name>];
```

3.5.8.2 Example

The following example shows use of a combined content, offset, and depth search rule.

```
alert tcp any any -> any 80 (content:"cgi-bin/phf"; offset:4; depth:20;)
```

3.5.9 distance

The distance keyword allows the rule writer to specify how far into a packet Snort should ignore before starting to search for the specified pattern relative to the end of the previous pattern match.

This can be thought of as exactly the same thing as offset (See Section [*]), except it is relative to the end of the last pattern match instead of the beginning of the packet.

This keyword allows values from -65535 to 65535.

The value can also be set to a string value referencing a variable extracted by the `byte_extract` keyword in the same rule.

3.5.9.1 Format

```
distance:[<byte_count>|<var_name>];
```

3.5.9.2 Example

The rule below maps to a regular expression of /ABC.{1,}DEF/.

```
alert tcp any any -> any any (content:"ABC"; content:"DEF"; distance:1;)
```

3.5.10 within

The within keyword is a content modifier that makes sure that at most N bytes are between pattern matches using the content keyword (See Section [*]). It's designed to be used in conjunction with the distance (Section [*]) rule option.

This keyword allows values greater than or equal to pattern length being searched. The maximum allowed value for this keyword is 65535.

The value can also be set to a string value referencing a variable extracted by the `byte_extract` keyword in the same rule.

3.5.10.1 Format

```
within:[<byte_count>|<var_name>];
```

3.5.10.2 Examples

This rule constrains the search of EFG to not go past 10 bytes past the ABC match.

```
alert tcp any any -> any any (content:"ABC"; content:"EFG"; within:10;)
```

3.5.11 http_client_body

The http_client_body keyword is a content modifier that restricts the search to the body of an HTTP client request.

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before 'http_client_body' is specified.

The amount of data that is inspected with this option depends on the `post_depth` config option of HttpInspect. Pattern matches with this keyword wont work when `post_depth` is set to -1.

3.5.11.1 Format

```
http_client_body;
```

3.5.11.2 Examples

This rule constrains the search for the pattern "EFG" to the raw body of an HTTP client request.

```
alert tcp any any -> any 80 (content:"ABC"; content:"EFG"; http_client_body;)
```

> **Note:** The `http_client_body` modifier is not allowed to be used with the `rawbytes` modifier for the same content.

3.5.12 http_cookie

The http_cookie keyword is a content modifier that restricts the search to the extracted Cookie Header field (excluding the header name itself and the CRLF terminating the header line) of a HTTP client request or a HTTP server response (per the configuration of HttpInspect [*]). The Cookie buffer does not include the header names (`Cookie:` for HTTP requests or `Set-Cookie:` for HTTP responses) or leading spaces and the CRLF terminating the header line. These are included in the HTTP header buffer.

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_cookie` is specified. This keyword is dependent on the `enable_cookie` config option. The Cookie Header field will be extracted only when this option is configured. If `enable_cookie` is not specified, the cookie still ends up in HTTP header. When `enable_cookie` is not specified, using `http_cookie` is the same as using `http_header`.

The extracted Cookie Header field may be NORMALIZED, per the configuration of HttpInspect (see [*]).

3.5.12.1 Format

```
http_cookie;
```

3.5.12.2 Examples

This rule constrains the search for the pattern "EFG" to the extracted Cookie Header field of a HTTP client request.

```
alert tcp any any -> any 80 (content:"ABC"; content:"EFG"; http_cookie;)
```

> **Note:**
>
> The `http_cookie` modifier is not allowed to be used with the `rawbytes` or `fast_pattern` modifiers for the same content.

3.5.13 http_raw_cookie

The http_raw_cookie keyword is a content modifier that restricts the search to the extracted UNNORMALIZED Cookie Header field of a HTTP client request or a HTTP server response (per the configuration of HttpInspect [*]).

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_raw_cookie` is specified. This keyword is dependent on the `enable_cookie` config option. The Cookie Header field will be extracted only when this option is configured.

3.5.13.1 Format

```
http_raw_cookie;
```

3.5.13.2 Examples

This rule constrains the search for the pattern "EFG" to the extracted Unnormalized Cookie Header field of a HTTP client request.

```
alert tcp any any -> any 80 (content:"ABC"; content:"EFG"; http_raw_cookie;)
```

Note:

The `http_raw_cookie` modifier is not allowed to be used with the `rawbytes`, `http_cookie` or `fast_pattern` modifiers for the same content.

3.5.14 http_header

The http_header keyword is a content modifier that restricts the search to the extracted Header fields of a HTTP client request or a HTTP server response (per the configuration of HttpInspect [*]).

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_header` is specified.

The extracted Header fields may be NORMALIZED, per the configuration of HttpInspect (see [*]).

3.5.14.1 Format

```
http_header;
```

3.5.14.2 Examples

This rule constrains the search for the pattern "EFG" to the extracted Header fields of a HTTP client request or a HTTP server response.

```
alert tcp any any -> any 80 (content:"ABC"; content:"EFG"; http_header;)
```

> **Note:**
>
> The `http_header` modifier is not allowed to be used with the `rawbytes` modifier for the same content.

3.5.15 http_raw_header

The http_raw_header keyword is a content modifier that restricts the search to the extracted UNNORMALIZED Header fields of a HTTP client request or a HTTP server response (per the configuration of HttpInspect [*]).

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_raw_header` is specified.

3.5.15.1 Format

```
http_raw_header;
```

3.5.15.2 Examples

This rule constrains the search for the pattern "EFG" to the extracted Header fields of a HTTP client request or a HTTP server response.

```
alert tcp any any -> any 80 (content:"ABC"; content:"EFG"; http_raw_header;)
```

> **Note:**
>
> The `http_raw_header` modifier is not allowed to be used with the `rawbytes`, `http_header` or `fast_pattern` modifiers for the same content.

3.5.16 http_method

The http_method keyword is a content modifier that restricts the search to the extracted Method from a HTTP client request.

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_method` is specified.

3.5.16.1 Format

```
http_method;
```

3.5.16.2 Examples

This rule constrains the search for the pattern "GET" to the extracted Method from a HTTP client request.

```
alert tcp any any -> any 80 (content:"ABC"; content:"GET"; http_method;)
```

> **Note:**
>
> The `http_method` modifier is not allowed to be used with the `rawbytes` or `fast_pattern` modifiers for the same content.

3.5.17 http_uri

The http_uri keyword is a content modifier that restricts the search to the NORMALIZED request URI field . Using a content rule option followed by a http_uri modifier is the same as using a uricontent by itself (see: [*]).

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_uri` is specified.

3.5.17.1 Format

```
http_uri;
```

3.5.17.2 Examples

This rule constrains the search for the pattern "EFG" to the NORMALIZED URI.

```
alert tcp any any -> any 80 (content:"ABC"; content:"EFG"; http_uri;)
```

> **Note:**
>
> The `http_uri` modifier is not allowed to be used with the `rawbytes` modifier for the same content.

3.5.18 http_raw_uri

The http_raw_uri keyword is a content modifier that restricts the search to the UNNORMALIZED request URI field .

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_raw_uri` is specified.

3.5.18.1 Format

```
http_raw_uri;
```

3.5.18.2 Examples

This rule constrains the search for the pattern "EFG" to the UNNORMALIZED URI.

```
alert tcp any any -> any 80 (content:"ABC"; content:"EFG"; http_raw_uri;)
```

> **Note:**
>
> The `http_raw_uri` modifier is not allowed to be used with the `rawbytes`, `http_uri` or `fast_pattern` modifiers for the same content.

3.5.19 http_stat_code

The http_stat_code keyword is a content modifier that restricts the search to the extracted Status code field from a HTTP server response.

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_stat_code` is specified.

The Status Code field will be extracted only if the extended_response_inspection is configured for the HttpInspect (see [*]).

3.5.19.1 Format

```
http_stat_code;
```

3.5.19.2 Examples

This rule constrains the search for the pattern "200" to the extracted Status Code field of a HTTP server response.

```
alert tcp any any -> any 80 (content:"ABC"; content:"200"; http_stat_code;)
```

Note:

The `http_stat_code` modifier is not allowed to be used with the `rawbytes` or `fast_pattern` modifiers for the same content.

3.5.20 http_stat_msg

The http_stat_msg keyword is a content modifier that restricts the search to the extracted Status Message field from a HTTP server response.

As this keyword is a modifier to the previous `content` keyword, there must be a content in the rule before `http_stat_msg` is specified.

The Status Message field will be extracted only if the extended_response_inspection is configured for the HttpInspect (see [*]).

3.5.20.1 Format

```
http_stat_msg;
```

3.5.20.2 Examples

This rule constrains the search for the pattern "Not Found" to the extracted Status Message field of a HTTP server response.

```
alert tcp any any -> any 80 (content:"ABC"; content:"Not Found"; http_stat_msg;)
```

Note:

The `http_stat_msg` modifier is not allowed to be used with the `rawbytes` or `fast_pattern` modifiers for the same content.

3.5.21 http_encode

The `http_encode` keyword will enable alerting based on encoding type present in a HTTP client request or a HTTP server response (per the configuration of HttpInspect [*]).

There are several keywords associated with `http_encode`. The keywords 'uri', 'header' and 'cookie' determine the HTTP fields used to search for a particular encoding type. The keywords 'utf8', 'double_encode', 'non_ascii', 'uencode', 'iis_encode', 'ascii' and 'bare_byte' determine the encoding

type which would trigger the alert. These keywords can be combined using a OR operation. Negation is allowed on these keywords.

The config option 'normalize_headers' needs to be turned on for rules to work with the keyword 'header'. The keyword 'cookie' is dependent on config options 'enable_cookie' and 'normalize_cookies' (see [*]). This rule option will not be able to detect encodings if the specified HTTP fields are not NORMALIZED.

Option	Description
`uri`	Check for the specified encoding type in HTTP client request URI field.
`header`	Check for the specified encoding type in HTTP request or HTTP response header fields (depending on the packet flow)
`cookie`	Check for the specified encoding type in HTTP request or HTTP response cookie header fields (depending on the packet flow)
`utf8`	Check for utf8 encoding in the specified buffer
`double_encode`	Check for double encoding in the specified buffer
`non_ascii`	Check for non-ASCII encoding in the specified buffer
`uencode`	Check for u-encoding in the specified buffer
`bare_byte`	Check for bare byte encoding in the specified buffer
`ascii`	Check for ascii encoding in the specified buffer
`iis_encode`	Check for IIS Unicode encoding in the specified buffer

3.5.21.1 Format

```
    http_encode:<http buffer type>, [!]<encoding type>
    http_encode:[uri|header|cookie], [!][<utf8|double_encode|non_ascii|uencode|
bare_byte|ascii|iis_encode>];
```

3.5.21.2 Examples

```
    alert tcp any any -> any any (msg:"UTF8/UEncode Encoding present";
http_encode:uri,utf8|uencode;)
    alert tcp any any -> any any (msg:"No UTF8"; http_encode:uri,!utf8;)
```

Note:

Negation(!) and OR(|) operations cannot be used in conjunction with each other for the `http_encode` keyword. The OR and negation operations work only on the encoding type field and not on http buffer type field.

3.5.22 fast_pattern

The `fast_pattern` keyword is a content modifier that sets the content within a rule to be used with the fast pattern matcher. The default behavior of fast pattern determination is to use the longest HTTP buffer content. If no HTTP buffer is present, then the fast pattern is the longest content. Given this behavior, it is useful if a shorter content is more "unique" than the longer content, meaning the shorter content is less likely to be found in a packet than the longer content.

The fast pattern matcher is used to select only those rules that have a chance of matching by using a content in the rule for selection and only evaluating that rule if the content is found in the payload. Though this may seem to be overhead, it can significantly reduce the number of rules that need to be evaluated and thus increases performance. The better the content used for the fast pattern matcher, the less likely the rule will needlessly be evaluated.

As this keyword is a modifier to the previous `content` keyword, there must be a `content` rule option in the rule before `fast_pattern` is specified. The `fast_pattern` option may be specified only once per rule.

Note: The `fast_pattern` modifier cannot be used with the following http content modifiers: `http_cookie`, `http_raw_uri`, `http_raw_header`, `http_raw_cookie`, `http_method`, `http_stat_code`, `http_stat_msg`.

Note: The `fast_pattern` modifier can be used with negated contents only if those contents are not modified with `offset`, `depth`, `distance` or `within`.

Note: The fast pattern matcher is always case insensitive.

3.5.22.1 Format

The `fast_pattern` option can be used alone or optionally take arguments. When used alone, the meaning is simply to use the specified content as the fast pattern content for the rule.

```
fast_pattern;
```

The optional argument `only` can be used to specify that the content should only be used for the fast pattern matcher and should not be evaluated as a rule option. This is useful, for example, if a known content must be located in the payload independent of location in the payload, as it saves the time necessary to evaluate the rule option. Note that (1) the modified content must be case insensitive since patterns are inserted into the pattern matcher in a case insensitive manner, (2) negated contents cannot be used and (3) contents cannot have any positional modifiers such as `offset`, `depth`, `distance` or `within`.

```
fast_pattern:only;
```

The optional argument `<offset>,<length>` can be used to specify that only a portion of the content should be used for the fast pattern matcher. This is useful if the pattern is very long and only a portion of

the pattern is necessary to satisfy "uniqueness" thus reducing the memory required to store the entire pattern in the fast pattern matcher.

```
fast_pattern:<offset>,<length>;
```

Note: The optional arguments `only` and `<offset>,<length>` are mutually exclusive.

3.5.22.2 Examples

This rule causes the pattern "IJKLMNO" to be used with the fast pattern matcher, even though it is shorter than the earlier pattern "ABCDEFGH".

```
alert tcp any any -> any 80 (content:"ABCDEFGH"; content:"IJKLMNO"; fast_pattern;)
```

This rule says to use the content "IJKLMNO" for the fast pattern matcher and that the content should only be used for the fast pattern matcher and not evaluated as a `content` rule option.

```
alert tcp any any -> any 80 (content:"ABCDEFGH"; content:"IJKLMNO"; nocase;
fast_pattern:only;)
```

This rule says to use "JKLMN" as the fast pattern content, but still evaluate the `content` rule option as "IJKLMNO".

```
alert tcp any any -> any 80 (content:"ABCDEFGH"; content:"IJKLMNO"; fast_pattern:1,5;)
```

3.5.23 uricontent

The `uricontent` keyword in the Snort rule language searches the NORMALIZED request URI field. This is equivalent to using the `http_uri` modifier to a `content` keyword. As such if you are writing rules that include things that are normalized, such as %2f or directory traversals, these rules will not alert. The reason is that the things you are looking for are normalized out of the URI buffer.

For example, the URI:

```
    /scripts/..%c0%af../winnt/system32/cmd.exe?/c+ver
```

will get normalized into:

```
    /winnt/system32/cmd.exe?/c+ver
```

Another example, the URI:

```
    /cgi-bin/aaaaaaaaaaaaaaaaaaaaaaaaa/..%252fp%68f?
```

will get normalized into:

```
/cgi-bin/phf?
```

When writing a `uricontent` rule, write the content that you want to find in the context that the URI will be normalized. For example, if Snort normalizes directory traversals, do not include directory traversals.

You can write rules that look for the non-normalized content by using the content option. (See Section [*])

`uricontent` can be used with several of the modifiers available to the `content` keyword. These include:

Table: Uricontent Modifiers

Modifier	Section
nocase	[*]
depth	[*]
offset	[*]
distance	[*]
within	[*]
fast_pattern	[*]

This option works in conjunction with the HTTP Inspect preprocessor specified in Section [*].

3.5.23.1 Format

```
uricontent:[!]"<content string>";
```

> **Note:**
>
> `uricontent` cannot be modified by a `rawbytes` modifier or any of the other HTTP modifiers. If you wish to search the UNNORMALIZED request URI field, use the `http_raw_uri` modifier with a `content` option.

3.5.24 urilen

The `urilen` keyword in the Snort rule language specifies the exact length, the minimum length, the maximum length, or range of URI lengths to match. By default the raw uri buffer will be used. With the optional `<uribuf>` argument, you can specify whether the raw or normalized buffer are used.

3.5.24.1 Format

```
urilen:min<>max[,<uribuf>];
urilen:[<|>]<number>[,<uribuf>];

<uribuf> : "norm" | "raw"
```

The following example will match URIs that are 5 bytes long:

```
urilen:5;
```

The following example will match URIs that are shorter than 5 bytes:

```
urilen:<5;
```

The following example will match URIs that are greater than 5 bytes and less than 10 bytes (inclusive):

```
urilen:5<>10;
```

The following example will match URIs that are greater than 500 bytes using the normalized URI buffer:

```
urilen:>500,norm;
```

The following example will match URIs that are greater than 500 bytes explicitly stating to use the raw URI buffer:

```
urilen:>500,raw;
```

This option works in conjunction with the HTTP Inspect preprocessor specified in Section [*].

3.5.25 isdataat

Verify that the payload has data at a specified location, optionally looking for data relative to the end of the previous content match.

3.5.25.1 Format

```
isdataat:[!]<int>[, relative|rawbytes];
```

3.5.25.2 Example

```
alert tcp any any -> any 111 (content:"PASS"; isdataat:50,relative; \
    content:!"|0a|"; within:50;)
```

This rule looks for the string PASS exists in the packet, then verifies there is at least 50 bytes after the end of the string PASS, then verifies that there is not a newline character within 50 bytes of the end of the PASS string.

When the `rawbytes` modifier is specified with `isdataat`, it looks at the raw packet data, ignoring any decoding that was done by the preprocessors. This modifier will work with the `relative` modifier as long as the previous content match was in the raw packet data.

A `!` modifier negates the results of the isdataat test. It will alert if a certain amount of data is not present within the payload. For example, the rule with modifiers `content:"foo"; isdataat:!10,relative;` would alert if there were not 10 bytes after "foo" before the payload ended.

3.5.26 pcre

The pcre keyword allows rules to be written using perl compatible regular expressions. For more detail on what can be done via a pcre regular expression, check out the PCRE web site `http://www.pcre.org`

3.5.26.1 Format

```
pcre:[!]"(/<regex>/|m<delim><regex><delim>)[ismxAEGRUBPHMCOIDKYS]";
```

The post-re modifiers set compile time flags for the regular expression. See tables [*], [*], and [*] for descriptions of each modifier.

Table: Perl compatible modifiers for `pcre`

i	case insensitive
s	include newlines in the dot metacharacter
m	By default, the string is treated as one big line of characters. ^ and $ match at the beginning and ending of the string. When m is set, ^ and $ match immediately following or immediately before any newline in the buffer, as well as the very start and very end of the buffer.
x	whitespace data characters in the pattern are ignored except when escaped or inside a character class

Table: PCRE compatible modifiers for `pcre`

A	the pattern must match only at the start of the buffer (same as ^)
E	Set $ to match only at the end of the subject string. Without E, $ also matches immediately before the final character if it is a newline (but not before any other newlines).
G	

	Inverts the "greediness" of the quantifiers so that they are not greedy by default, but become greedy if followed by "?".

Table: Snort specific modifiers for `pcre`

R	Match relative to the end of the last pattern match. (Similar to distance:0;)
U	Match the decoded URI buffers (Similar to `uricontent` and `http_uri`). This modifier is not allowed with the unnormalized HTTP request uri buffer modifier(I) for the same content.
I	Match the unnormalized HTTP request uri buffer (Similar to `http_raw_uri`). This modifier is not allowed with the HTTP request uri buffer modifier(U) for the same content.
P	Match unnormalized HTTP request body (Similar to `http_client_body`).
	For SIP message, match SIP body for request or response (Similar to `sip_body`).
H	Match normalized HTTP request or HTTP response header (Similar to `http_header`). This modifier is not allowed with the unnormalized HTTP request or HTTP response header modifier(D) for the same content.
	For SIP message, match SIP header for request or response (Similar to `sip_header`).
D	Match unnormalized HTTP request or HTTP response header (Similar to `http_raw_header`). This modifier is not allowed with the normalized HTTP request or HTTP response header modifier(H) for the same content.
M	Match normalized HTTP request method (Similar to `http_method`)
C	Match normalized HTTP request or HTTP response cookie (Similar to `http_cookie`). This modifier is not allowed with the unnormalized HTTP request or HTTP response cookie modifier(K) for the same content.
K	Match unnormalized HTTP request or HTTP response cookie (Similar to `http_raw_cookie`). This modifier is not allowed with the normalized HTTP request or HTTP response cookie modifier(C) for the same content.
S	Match HTTP response status code (Similar to `http_stat_code`)
Y	Match HTTP response status message (Similar to `http_stat_msg`)
B	Do not use the decoded buffers (Similar to rawbytes)
O	Override the configured pcre match limit and pcre match limit recursion for this expression (See section [*]). It completely ignores the limits while evaluating the pcre pattern specified.

Note: The modifiers R (relative) and B (rawbytes) are not allowed with any of the HTTP modifiers such as U, I, P, H, D, M, C, K, S and Y.

3.5.26.2 Example

This example performs a case-insensitive search for the HTTP URI `foo.php?id=<some numbers>`

```
    alert tcp any any -> any 80 (content:"/foo.php?id="; pcre:"/\/foo.php?id=[0-9]
{1,10}/iU";)
```

Note: It is wise to have at least one `content` keyword in a rule that uses `pcre`. This allows the fast-pattern matcher to filter out non-matching packets so that the pcre evaluation is not performed on each and every packet coming across the wire.

Note:

Snort's handling of multiple URIs with PCRE does not work as expected. PCRE when used without a `uricontent` only evaluates the first URI. In order to use pcre to inspect all URIs, you must use either a content or a uricontent.

3.5.27 pkt_data

This option sets the cursor used for detection to the raw transport payload.

Any relative or absolute content matches (without HTTP modifiers or rawbytes) and other payload detecting rule options that follow `pkt_data` in a rule will apply to the raw TCP/UDP payload or the normalized buffers (in case of telnet, smtp normalization) until the cursor (used for detection) is set again.

This rule option can be used several times in a rule.

3.5.27.1 Format

```
        pkt_data;
```

3.5.27.2 Example

```
        alert tcp any any -> any any(msg:"Absolute Match"; pkt_data; content:"BLAH";
offset:0; depth:10;)
        alert tcp any any -> any any(msg:"PKT DATA"; pkt_data; content:"foo";
within:10;)
        alert tcp any any -> any any(msg:"PKT DATA"; pkt_data; content:"foo";)
        alert tcp any any -> any any(msg:"PKT DATA"; pkt_data; pcre:"/foo/i";)
```

3.5.28 file_data

This option sets the cursor used for detection to one of the following buffers: 1. When the traffic being detected is HTTP it sets the buffer to, a. HTTP response body (without chunking/compression/normalization) b. HTTP de-chunked response body c. HTTP decompressed response body (when

`inspect_gzip` is turned on) d. HTTP normalized response body (when `normalized_javascript` is turned on) e. HTTP UTF normalized response body (when `normalize_utf` is turned on) f. All of the above 2. When the traffic being detected is SMTP/POP/IMAP it sets the buffer to, a. SMTP/POP/IMAP data body (including Email headers and MIME when decoding is turned off) b. Base64 decoded MIME attachment (when `b64_decode_depth` is greater than -1) c. Non-Encoded MIME attachment (when `bitenc_decode_depth` is greater than -1) d. Quoted-Printable decoded MIME attachment (when `qp_decode_depth` is greater than -1) e. Unix-to-Unix decoded attachment (when `uu_decode_depth` is greater than -1) 3. If it is not set by 1 and 2, it will be set to the payload.

Any relative or absolute content matches (without HTTP modifiers or rawbytes) and payload detecting rule options that follow `file_data` in a rule will apply to this buffer until explicitly reset by other rule options.

This rule option can be used several time in a rule.

The argument `mime` to `file_data` is deprecated. The rule options `file_data` will itself point to the decoded MIME attachment.

3.5.28.1 Format

```
        file_data;
```

3.5.28.2 Example

```
        alert tcp any any -> any any(msg:"Absolute Match"; file_data; content:"BLAH";
offset:0; depth:10;)
        alert tcp any any -> any any(msg:"FILE DATA"; file_data; content:"foo";
within:10;)
        alert tcp any any -> any any(msg:"FILE DATA"; file_data; content:"foo";)
        alert tcp any any -> any any(msg:"FILE DATA"; file_data; pcre:"/foo/i";)

        The following rule searches for content "foo" within the file_data buffer and
content "bar" within the
        entire packet payload. The rule option pkt_data will reset the cursor used for
detection to the
        TCP payload.
        alert tcp any any -> any any(msg:"FILE DATA"; file_data; content:"foo";
pkt_data; content:"bar";)
```

3.5.29 base64_decode

This option is used to decode the base64 encoded data. This option is particularly useful in case of HTTP headers such as HTTP authorization headers. This option unfolds the data before decoding it.

3.5.29.1 Format

```
base64_decode[:[bytes <bytes_to_decode>][, ][offset <offset>[, relative]]];
```

Option	Description
`bytes`	Number of base64 encoded bytes to decode. This argument takes positive and non-zero values only. When this option is not specified we look for base64 encoded data till either the end of header line is reached or end of packet payload is reached.
`offset`	Determines the offset relative to the doe_ptr when the option `relative` is specified or relative to the start of the packet payload to begin inspection of base64 encoded data. This argument takes positive and non-zero values only.
`relative`	Specifies the inspection for base64 encoded data is relative to the doe_ptr.

The above arguments to `base64_decode` are optional.

Note:

This option can be extended to protocols with folding similar to HTTP. If folding is not present the search for base64 encoded data will end when we see a carriage return or line feed or both without a following space or tab.

This option needs to be used in conjunction with `base64_data` for any other payload detecting rule options to work on base64 decoded buffer.

3.5.29.2 Examples

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any \
    (msg:"Base64 Encoded Data"; base64_decode; base64_data; \
    content:"foo bar"; within:20;)

alert tcp $EXTERNAL_NET any -> $HOME_NET any \
    (msg:"Authorization NTLM"; content:"Authorization: NTLM";
    base64_decode:relative; base64_data; content:"NTLMSSP"; )

alert tcp any any -> any any (msg:"Authorization NTLM"; \
    content:"Authorization:"; http_header; \
    base64_decode:bytes 12, offset 6, relative; base64_data; \
    content:"NTLMSSP"; within:8;)
```

3.5.30 base64_data

This option is similar to the rule option `file_data` and is used to set the cursor used for detection to the beginning of the base64 decoded buffer if present.

This option does not take any arguments. The rule option `base64_decode` needs to be specified before the `base64_data` option.

3.5.30.1 Format

```
        base64_data;
```

This option matches if there is base64 decoded buffer.

> **Note:**
>
> Fast pattern content matches are not allowed with this buffer.

3.5.30.2 Example

```
    alert tcp any any -> any any (msg:"Authorization NTLM"; \
        content:"Authorization:"; http_header; \
        base64_decode:bytes 12, offset 6, relative; base64_data; \
        content:"NTLMSSP"; within:8;)
```

3.5.31 byte_test

Test a byte field against a specific value (with operator). Capable of testing binary values or converting representative byte strings to their binary equivalent and testing them.

For a more detailed explanation, please read Section [*].

3.5.31.1 Format

```
    byte_test:<bytes to convert>, [!]<operator>, <value>, <offset> \
        [, relative][, <endian>][, string, <number type>][, dce] \
        [, bitmask <bitmask_value>];

    bytes            = 1 - 10
    operator         = '<' | '=' | '>' | '<=' | '>=' | '&' | '^'
    value            = 0 - 4294967295
    offset           = -65535 to 65535
    bitmask_value    = 1 to 4 byte hexadecimal value
```

Option	Description
`bytes_to_convert`	Number of bytes to pick up from the packet. The allowed values are 1 to 10 when used without `dce`. If used with `dce` allowed values are 1, 2 and 4.
`operator`	

	Operation to perform to test the value: • < - less than • > - greater than • <= - less than or equal • >= - greater than or equal • = - equal • & - bitwise AND • ^ - bitwise OR
`value`	Value to test the converted value against
`offset`	Number of bytes into the payload to start processing
`relative`	Use an offset relative to last pattern match
`endian`	Endian type of the number being read: • `big` - Process data as big endian (default) • `little` - Process data as little endian
`string`	Data is stored in string format in packet
`number type`	Type of number being read: • `hex` - Converted string data is represented in hexadecimal • `dec` - Converted string data is represented in decimal • `oct` - Converted string data is represented in octal
`dce`	Let the DCE/RPC 2 preprocessor determine the byte order of the value to be converted. See section [*] for a description and examples ([*] for quick reference).
`bitmask`	Applies the AND operator on the bytes converted. The result will be right-shifted by the number of bits equal to the number of trailing zeros in the mask.

Any of the operators can also include ! to check if the operator is not true. If ! is specified without an operator, then the operator is set to =.

Note:

Snort uses the C operators for each of these operators. If the & operator is used, then it would be the same as using if (data & value) { do_something();}

3.5.31.2 Examples

```
alert udp $EXTERNAL_NET any -> $HOME_NET any \
    (msg:"AMD procedure 7 plog overflow"; \
    content:"|00 04 93 F3|"; \
    content:"|00 00 00 07|"; distance:4; within:4; \
    byte_test:4, >, 1000, 20, relative;)

alert tcp $EXTERNAL_NET any -> $HOME_NET any \
```

```
        (msg:"AMD procedure 7 plog overflow"; \
        content:"|00 04 93 F3|"; \
        content:"|00 00 00 07|"; distance:4; within:4; \
        byte_test:4, >, 1000, 20, relative;)

    alert udp any any -> any 1234 \
        (byte_test:4, =, 1234, 0, string, dec; \
        msg:"got 1234!";)

    alert udp any any -> any 1235 \
        (byte_test:3, =, 123, 0, string, dec; \
        msg:"got 123!";)

    alert udp any any -> any 1236 \
        (byte_test:2, =, 12, 0, string, dec; \
        msg:"got 12!";)

    alert udp any any -> any 1237 \
        (byte_test:10, =, 1234567890, 0, string, dec; \
        msg:"got 1234567890!";)

    alert udp any any -> any 1238 \
        (byte_test:8, =, 0xdeadbeef, 0, string, hex; \
        msg:"got DEADBEEF!";)

    alert tcp any any -> any any \
        (byte_test:2, =, 568, 0, bitmask 0x3FF0; \
        msg:"got 568 after applying bitmask 0x3FF0 on 2 bytes extracted";)
```

3.5.32 byte_jump

The `byte_jump` keyword allows rules to be written for length encoded protocols trivially. By having an option that reads the length of a portion of data, then skips that far forward in the packet, rules can be written that skip over specific portions of length-encoded protocols and perform detection in very specific locations.

The `byte_jump` option does this by reading some number of bytes, convert them to their numeric representation, move that many bytes forward and set a pointer for later detection. This pointer is known as the detect offset end pointer, or doe_ptr.

For a more detailed explanation, please read Section [*].

3.5.32.1 Format

```
    byte_jump:<bytes_to_convert>, <offset> [, relative][, multiplier <mult_value>] \
        [, <endian>][, string, <number_type>][, align][, from_beginning][, from_end] \
        [, post_offset <adjustment value>][, dce][, bitmask <bitmask_value>];

    bytes             = 1 - 10
```

```
    offset            = -65535 to 65535
    mult_value        = 0 - 65535
    post_offset       = -65535 to 65535
    bitmask_value     = 1 to 4 bytes hexadecimal value
```

Option	Description
`bytes_to_convert`	Number of bytes to pick up from the packet. The allowed values are 1 to 10 when used without `dce`. If used with `dce` allowed values are 1, 2 and 4. If used with `from_end` argument, bytes_to_convert can be 0. If bytes_to_convert is 0, the extracted value is 0.
`offset`	Number of bytes into the payload to start processing
`relative`	Use an offset relative to last pattern match
`multiplier $<$value$>$`	Multiply the number of calculated bytes by `$<$value$>$` and skip forward that number of bytes.
`big`	Process data as big endian (default)
`little`	Process data as little endian
`string`	Data is stored in string format in packet
`hex`	Converted string data is represented in hexadecimal
`dec`	Converted string data is represented in decimal
`oct`	Converted string data is represented in octal
`align`	Round the number of converted bytes up to the next 32-bit boundary
`from_beginning`	Skip forward from the beginning of the packet payload instead of from the current position in the packet.
`from_end`	The jump will originate from the end of payload
`post_offset $<$value$>$`	Skip forward or backwards (positive of negative value) by `$<$value$>$` number of bytes after the other jump options have been applied.
`dce`	Let the DCE/RPC 2 preprocessor determine the byte order of the value to be converted. See section [*] for a description and examples ([*] for quick reference).
`bitmask`	Applies the AND operator on the `bytes_to_convert` argument. The result will be right-shifted by the number of bits equal to the number of trailing zeros in the mask.

3.5.32.2 Example

```
    alert udp any any -> any 32770:34000 (content:"|00 01 86 B8|"; \
        content:"|00 00 00 01|"; distance:4; within:4; \
        byte_jump:4, 12, relative, align; \
        byte_test:4, >, 900, 20, relative; \
        msg:"statd format string buffer overflow";)

    alert tcp any any -> any any (content:"Begin"; \
        byte_jump:0, 0, from_end, post_offset -6; \
```

```
        content:"end.."; distance:0; within:5; \
        msg:"Content match from end of the payload";)

    alert tcp any any -> any any (content:"catalog"; \
        byte_jump:2, 1, relative, post_offset 2, bitmask 0x03f0; \
        byte_test:2, =, 968, 0, relative; \
        msg:"Bitmask applied on the 2 bytes extracted for byte_jump";)

    alert tcp any any -> any any (content:"catalog"; \
        byte_jump:1, 2, from_end, post_offset -5, bitmask 0x3c; \
        byte_test:1, =, 106, 0, relative; \
        msg:"Byte jump calculated from end of payload after bitmask applied";)
```

3.5.33 byte_extract

The `byte_extract` keyword is another useful option for writing rules against length-encoded protocols. It reads in some number of bytes from the packet payload and saves it to a variable. These variables can be referenced later in the rule, instead of using hard-coded values.

> **Note:**
>
> Only two `byte_extract` variables may be created per rule. They can be re-used in the same rule any number of times.

3.5.33.1 Format

```
    byte_extract:<bytes_to_extract>, <offset>, <name> [, relative] \
        [, multiplier <multiplier value>][, <endian>][, string][, hex][, dec][, oct] \
        [, align <align value>][, dce][, bitmask <bitmask>];

    bytes_to_extract = 1 - 10
    operator         = '<' | '=' | '>' | '<=' | '>=' | '&' | '^'
    value            = 0 - 4294967295
    offset           = -65535 to 65535
    bitmask_value    = 1 to 4 byte hexadecimal value
```

Option	Description
`bytes_to_extract`	Number of bytes to pick up from the packet
`offset`	Number of bytes into the payload to start processing
`name`	Name of the variable. This will be used to reference the variable in other rule options.
`relative`	Use an offset relative to last pattern match
`multiplier $<$value$>$`	Multiply the bytes read from the packet by `$<$value$>$` and save that number into the variable.
`big`	Process data as big endian (default)

little	Process data as little endian
dce	Use the DCE/RPC 2 preprocessor to determine the byte-ordering. The DCE/RPC 2 preprocessor must be enabled for this option to work.
string	Data is stored in string format in packet
hex	Converted string data is represented in hexadecimal
dec	Converted string data is represented in decimal
oct	Converted string data is represented in octal
align $<$value$>$	Round the number of converted bytes up to the next $<$value$>$-byte boundary. $<$value$>$ may be 2 or 4.
bitmask	Applies the AND operator on the value of bytes_to_extract argument. The result will be right-shifted by the number of bits equal to the number of trailing zeros in the mask.

3.5.33.2 Other options which use byte_extract variables

A byte_extract rule option detects nothing by itself. Its use is in extracting packet data for use in other rule options. Here is a list of places where byte_extract variables can be used:

Rule Option	Arguments that Take Variables
content/uricontent	offset, depth, distance, within
byte_test	offset, value
byte_jump	offset
isdataat	offset

3.5.33.3 Examples

This example uses two variables to:

- Read the offset of a string from a byte at offset 0.
- Read the depth of a string from a byte at offset 1.
- Use these values to constrain a pattern match to a smaller area.

```
alert tcp any any -> any any (byte_extract:1, 0, str_offset; \
    byte_extract:1, 1, str_depth; \
    content:"bad stuff"; offset:str_offset; depth:str_depth; \
    msg:"Bad Stuff detected within field";)

alert tcp any any -> any any (content:"|04 63 34 35|"; offset:4; depth:4; \
    byte_extract: 2, 0, var_match, relative, bitmask 0x03ff; \
    byte_test: 2, =, var_match, 2, relative; \
    msg:"Byte test value matches bitmask applied on bytes extracted";)
```

3.5.34 byte_math

Perform a mathematical operation on an extracted value and a specified value or existing variable, and store the outcome in a new resulting variable. These resulting variables can be referenced later in the rule, instead of using hard-coded values.

3.5.34.1 Format

```
byte_math:bytes <bytes_to_extract>, offset <offset_value>, oper <operator>,
    rvalue <r_value>, result <result_variable> [, relative]
    [, endian <endian>] [, string <number type>][, dce]
    [, bitmask <bitmask_value>];

bytes_to_extract = 1 - 10
operator         = '+' | '-' | '*' | '/' | '<<' | '>>'
r_value          = 0 - 4294967295 | byte extract variable
offset_value     = -65535 to 65535
bitmask_value    = 1 to 4 byte hexadecimal value
result_variable  = Result Variable name
```

Option	Description
`bytes_to_extract`	Number of bytes to pick up from the packet. The allowed values are 1 to 10 when used without `dce`. If used with `dce` allowed values are 1, 2 and 4. If used with $<<$ or $>>$ operator, allowed values are 1 to 4.
`oper`	Mathematical Operation to perform on the extracted value Operations allowed: +, -, *, /, $<<$, $>>$
`rvalue`	Value to use mathematical operation against
`offset`	Number of bytes into the payload to start processing
`relative`	Use an offset relative to last pattern match
`endian`	Endian type of the number being read: • `big` - Process data as big endian (default) • `little` - Process data as little endian
`string`	Data is stored in string format in packet
`number type`	Type of number being read: • `hex` - Converted string data is represented in hexadecimal • `dec` - Converted string data is represented in decimal • `oct` - Converted string data is represented in octal

dce	Let the DCE/RPC 2 preprocessor determine the byte order of the value to be converted. See section [*] for a description and examples ([*] for quick reference).
bitmask	Applies the AND operator on the bytes extracted. The result will be right-shifted by the number of bits equal to the number of trailing zeros in the mask.

3.5.34.2 Other rule options which use byte_math result variable

Rule Option	Arguments that take result variable
content	offset, depth, distance, within
byte_test	offset, value
byte_jump	offset
isdataat	offset

3.5.34.3 Examples

```
    alert udp $EXTERNAL_NET any -> $HOME_NET any \
        (msg:"Perform Arithmetic Operation on the extracted bytes"; \
        content:"|00 04 93 F3|"; \
        content:"|00 00 00 07|"; distance:4; within:4; \
        byte_math:bytes 4, offset 0, oper +, rvalue 248, result var, relative; \
        byte_test:4, >, var, 2, relative;)

    alert tcp $EXTERNAL_NET any -> $HOME_NET any \
        (msg:"Bitwise shift operator"; \
        content:"|00 00 00 07|"; distance:4; within:4; \
        byte_extract: 1, 0, extracted_val, relative; \
        byte_math: bytes 1, offset 2, oper >>, rvalue extracted_val, result var,
relative; \
        byte_test:2, =, var, 0, relative;)

    alert udp any any -> any 1234 \
        (content: "Packets start"; \
        byte_math: bytes 2, offset 0, oper -, rvalue 100, result var, relative, bitmask
0x7FF0; \
        content: "Packets end"; distance: 2; within var; \
        msg:"Content match with bitmask applied to the bytes extracted";)

    alert udp any any -> any 1235 \
        (byte_extract: 4, 3,  extracted_val, relative; \
        byte_math: bytes 5, offset 0, oper +, rvalue extracted_val, result var, string
hex; \
        byte_test:5, =, var, 4, string, hex; \
        msg:"String operator used with math rule option";)
```

3.5.35 ftpbounce

The ftpbounce keyword detects FTP bounce attacks.

3.5.35.1 Format

```
    ftpbounce;
```

3.5.35.2 Example

```
    alert tcp $EXTERNAL_NET any -> $HOME_NET 21 (msg:"FTP PORT bounce attempt"; \
        flow:to_server,established; content:"PORT"; nocase; ftpbounce; pcre:"/^PORT/
smi";\
        classtype:misc-attack; sid:3441; rev:1;)
```

3.5.36 asn1

The ASN.1 detection plugin decodes a packet or a portion of a packet, and looks for various malicious encodings.

Multiple options can be used in an 'asn1' option and the implied logic is boolean OR. So if any of the arguments evaluate as true, the whole option evaluates as true.

The ASN.1 options provide programmatic detection capabilities as well as some more dynamic type detection. If an option has an argument, the option and the argument are separated by a space or a comma. The preferred usage is to use a space between option and argument.

3.5.36.1 Format

```
    asn1:[bitstring_overflow][, double_overflow][, oversize_length <value>][,
absolute_offset <value>|relative_offset <value>];
```

Option	Description
`bitstring_overflow`	Detects invalid bitstring encodings that are known to be remotely exploitable.
`double_overflow`	Detects a double ASCII encoding that is larger than a standard buffer. This is known to be an exploitable function in Microsoft, but it is unknown at this time which services may be exploitable.
`oversize_length $<$value$>$`	Compares ASN.1 type lengths with the supplied argument. The syntax looks like, “oversize_length 500”. This means that if an ASN.1 type is greater than 500, then this

	keyword is evaluated as true. This keyword must have one argument which specifies the length to compare against.
`absolute_offset $<$value$>$`	This is the absolute offset from the beginning of the packet. For example, if you wanted to decode snmp packets, you would say “absolute_offset 0”. `absolute_offset` has one argument, the offset value. Offset may be positive or negative.
`relative_offset $<$value$>$`	This is the relative offset from the last content match, pcre or byte_jump. `relative_offset` has one argument, the offset number. So if you wanted to start decoding an ASN.1 sequence right after the content “foo”, you would specify `'content:"foo"; asn1:bitstring_overflow, relative_offset 0'`. Offset values may be positive or negative.

3.5.36.2 Examples

```
alert udp any any -> any 161 (msg:"Oversize SNMP Length"; \
    asn1:oversize_length 10000, absolute_offset 0;)

alert tcp any any -> any 80 (msg:"ASN1 Relative Foo"; content:"foo"; \
    asn1:bitstring_overflow, relative_offset 0;)
```

3.5.37 cvs

The CVS detection plugin aids in the detection of: Bugtraq-10384, CVE-2004-0396: "Malformed Entry Modified and Unchanged flag insertion". Default CVS server ports are 2401 and 514 and are included in the default ports for stream reassembly.

> **Note:**
>
> This plugin cannot do detection over encrypted sessions, e.g. SSH (usually port 22).

3.5.37.1 Format

```
cvs:<option>;
```

Option	Description
`invalid-entry`	Looks for an invalid Entry string, which is a way of causing a heap overflow (see CVE-2004-0396) and bad pointer dereference in versions of CVS 1.11.15 and before.

3.5.37.2 Examples

```
alert tcp any any -> any 2401 (msg:"CVS Invalid-entry"; \
    flow:to_server,established; cvs:invalid-entry;)
```

3.5.38 dce_iface

See the DCE/RPC 2 Preprocessor section [*] for a description and examples of using this rule option.

3.5.39 dce_opnum

See the DCE/RPC 2 Preprocessor section [*] for a description and examples of using this rule option.

3.5.40 dce_stub_data

See the DCE/RPC 2 Preprocessor section [*] for a description and examples of using this rule option.

3.5.41 sip_method

See the SIP Preprocessor section [*] for a description and examples of using this rule option.

3.5.42 sip_stat_code

See the SIP Preprocessor section [*] for a description and examples of using this rule option.

3.5.43 sip_header

See the SIP Preprocessor section [*] for a description and examples of using this rule option.

3.5.44 sip_body

See the SIP Preprocessor section [*] for a description and examples of using this rule option.

3.5.45 gtp_type

See the GTP Preprocessor section [*] for a description and examples of using this rule option.

3.5.46 gtp_info

See the GTP Preprocessor section [*] for a description and examples of using this rule option.

3.5.47 gtp_version

See the GTP Preprocessor section [*] for a description and examples of using this rule option.

3.5.48 ssl_version

See the SSL/TLS Preprocessor section [*] for a description and examples of using this rule option.

3.5.49 ssl_state

See the SSL/TLS Preprocessor section [*] for a description and examples of using this rule option.

3.5.50 Payload Detection Quick Reference

Table: Payload detection rule option keywords

Keyword	Description
`content`	The content keyword allows the user to set rules that search for specific content in the packet payload and trigger response based on that data.
`rawbytes`	The rawbytes keyword allows rules to look at the raw packet data, ignoring any decoding that was done by preprocessors.
`depth`	The depth keyword allows the rule writer to specify how far into a packet Snort should search for the specified pattern.
`offset`	The offset keyword allows the rule writer to specify where to start searching for a pattern within a packet.
`distance`	The distance keyword allows the rule writer to specify how far into a packet Snort should ignore before starting to search for the specified pattern relative to the end of the previous pattern match.
`within`	The within keyword is a content modifier that makes sure that at most N bytes are between pattern matches using the content keyword.
`uricontent`	The uricontent keyword in the Snort rule language searches the normalized request URI field.
`isdataat`	The isdataat keyword verifies that the payload has data at a specified location.
`pcre`	

	The pcre keyword allows rules to be written using perl compatible regular expressions.
`byte_test`	The byte_test keyword tests a byte field against a specific value (with operator).
`byte_jump`	The byte_jump keyword allows rules to read the length of a portion of data, then skip that far forward in the packet.
`ftpbounce`	The ftpbounce keyword detects FTP bounce attacks.
`asn1`	The asn1 detection plugin decodes a packet or a portion of a packet, and looks for various malicious encodings.
`cvs`	The cvs keyword detects invalid entry strings.
`dce_iface`	See the DCE/RPC 2 Preprocessor section [*].
`dce_opnum`	See the DCE/RPC 2 Preprocessor section [*].
`dce_stub_data`	See the DCE/RPC 2 Preprocessor section [*].
`sip_method`	See the SIP Preprocessor section [*].
`sip_stat_code`	See the SIP Preprocessor section [*].
`sip_header`	See the SIP Preprocessor section [*].
`sip_body`	See the SIP Preprocessor section [*].
`gtp_type`	See the GTP Preprocessor section [*].
`gtp_info`	See the GTP Preprocessor section [*].
`gtp_version`	See the GTP Preprocessor section [*].

End of Navigation Panel Table of Child-Links **Subsections**

- 3.6.1 fragoffset
  - 3.6.1.1 Format
  - 3.6.1.2 Example
- 3.6.2 ttl
  - 3.6.2.1 Format
  - 3.6.2.2 Example
- 3.6.3 tos
  - 3.6.3.1 Format
  - 3.6.3.2 Example
- 3.6.4 id
  - 3.6.4.1 Format
  - 3.6.4.2 Example
- 3.6.5 ipopts
  - 3.6.5.1 Format
  - 3.6.5.2 Example
  - 3.6.5.3 Warning
- 3.6.6 fragbits
  - 3.6.6.1 Format
  - 3.6.6.2 Example
- 3.6.7 dsize
  - 3.6.7.1 Format
  - 3.6.7.2 Example
  - 3.6.7.3 Warning
- 3.6.8 flags
  - 3.6.8.1 Format
  - 3.6.8.2 Example
- 3.6.9 flow
  - Options
  - 3.6.9.1 Format
  - 3.6.9.2 Examples
- 3.6.10 flowbits
  - 3.6.10.1 General Format
  - 3.6.10.2 set
  - 3.6.10.3 setx
  - 3.6.10.4 unset
  - 3.6.10.5 toggle

- 3.6.10.6 isset
- 3.6.10.7 isnotset
- 3.6.10.8 noalert
- 3.6.10.9 reset
- 3.6.10.10 Examples

- 3.6.11 seq
  - 3.6.11.1 Format
  - 3.6.11.2 Example

- 3.6.12 ack
  - 3.6.12.1 Format
  - 3.6.12.2 Example

- 3.6.13 window
  - 3.6.13.1 Format
  - 3.6.13.2 Example

- 3.6.14 itype
  - 3.6.14.1 Format
  - 3.6.14.2 Example

- 3.6.15 icode
  - 3.6.15.1 Format
  - 3.6.15.2 Examples

- 3.6.16 icmp_id
  - 3.6.16.1 Format
  - 3.6.16.2 Example

- 3.6.17 icmp_seq
  - 3.6.17.1 Format
  - 3.6.17.2 Example

- 3.6.18 rpc
  - 3.6.18.1 Format
  - 3.6.18.2 Example
  - 3.6.18.3 Warning

- 3.6.19 ip_proto
  - 3.6.19.1 Format
  - 3.6.19.2 Example

- 3.6.20 sameip
  - 3.6.20.1 Format
  - 3.6.20.2 Example

- 3.6.21 stream_reassemble
  - 3.6.21.1 Format
  - 3.6.21.2 Example

- 3.6.22 stream_size
  - 3.6.22.1 Format
  - 3.6.22.2 Example

- 3.6.23 Non-Payload Detection Quick Reference

End of Table of Child-Links

3.6 Non-Payload Detection Rule Options

3.6.1 fragoffset

The fragoffset keyword allows one to compare the IP fragment offset field against a decimal value. To catch all the first fragments of an IP session, you could use the fragbits keyword and look for the More fragments option in conjunction with a fragoffset of 0.

3.6.1.1 Format

```
fragoffset:[!|<|>]<number>;
```

3.6.1.2 Example

```
alert ip any any -> any any \
    (msg:"First Fragment"; fragbits:M; fragoffset:0;)
```

3.6.2 ttl

The ttl keyword is used to check the IP time-to-live value. This option keyword was intended for use in the detection of traceroute attempts. This keyword takes numbers from 0 to 255.

3.6.2.1 Format

```
ttl:[<, >, =, <=, >=]<number>;
ttl:[<number>]-[<number>];
```

3.6.2.2 Example

This example checks for a time-to-live value that is less than 3.

```
ttl:<3;
```

This example checks for a time-to-live value that between 3 and 5.

```
ttl:3-5;
```

This example checks for a time-to-live value that between 0 and 5.

```
ttl:-5;
```

This example checks for a time-to-live value that between 5 and 255.

```
ttl:5-;
```

Few other examples are as follows:

```
ttl:<=5;
ttl:>=5;
ttl:=5;
```

The following examples are NOT allowed by ttl keyword:

```
ttl:=>5;
ttl:=<5;
ttl:5-3;
```

3.6.3 tos

The tos keyword is used to check the IP TOS field for a specific value.

3.6.3.1 Format

```
tos:[!]<number>;
```

3.6.3.2 Example

This example looks for a tos value that is not 4

```
tos:!4;
```

3.6.4 id

The id keyword is used to check the IP ID field for a specific value. Some tools (exploits, scanners and other odd programs) set this field specifically for various purposes, for example, the value 31337 is very popular with some hackers.

3.6.4.1 Format

```
id:<number>;
```

3.6.4.2 Example

This example looks for the IP ID of 31337.

```
id:31337;
```

3.6.5 ipopts

The ipopts keyword is used to check if a specific IP option is present.

The following options may be checked:

rr
- Record Route

eol
- End of list

nop
- No Op

ts
- Time Stamp

sec
- IP Security

esec
- IP Extended Security

lsrr
- Loose Source Routing

lsrre
- Loose Source Routing (For MS99-038 and CVE-1999-0909)

ssrr
- Strict Source Routing

satid
- Stream identifier

any
- any IP options are set

The most frequently watched for IP options are strict and loose source routing which aren't used in any widespread internet applications.

3.6.5.1 Format

```
ipopts:<rr|eol|nop|ts|sec|esec|lsrr|lsrre|ssrr|satid|any>;
```

3.6.5.2 Example

This example looks for the IP Option of Loose Source Routing.

```
ipopts:lsrr;
```

3.6.5.3 Warning

Only a single ipopts keyword may be specified per rule.

3.6.6 fragbits

The `fragbits` keyword is used to check if fragmentation and reserved bits are set in the IP header.

The following bits may be checked:

M
: - More Fragments

D
: - Don't Fragment

R
: - Reserved Bit

The following modifiers can be set to change the match criteria:

+
: match on the specified bits, plus any others

*****
: match if any of the specified bits are set

!
: match if the specified bits are not set

3.6.6.1 Format

```
fragbits:[+*!]<[MDR]>;
```

3.6.6.2 Example

This example checks if the More Fragments bit and the Do not Fragment bit are set.

```
fragbits:MD+;
```

3.6.7 dsize

The dsize keyword is used to test the packet payload size. This may be used to check for abnormally sized packets that might cause buffer overflows.

3.6.7.1 Format

```
dsize:min<>max;
dsize:[<|>]<number>;
```

3.6.7.2 Example

This example looks for a dsize that is between 300 and 400 bytes (inclusive).

```
dsize:300<>400;
```

3.6.7.3 Warning

Note that segmentation makes dsize less reliable for TCP based protocols such as HTTP. Furthermore, dsize will fail on stream rebuilt packets, regardless of the size of the payload, unless protocol aware flushing (PAF) marks this packet as the start of a message.

3.6.8 flags

The flags keyword is used to check if specific TCP flag bits are present.

The following bits may be checked:

F
- FIN - Finish (LSB in TCP Flags byte)

S
- SYN - Synchronize sequence numbers

R
- RST - Reset

P
- PSH - Push

A
- ACK - Acknowledgment

U
- URG - Urgent

C
- CWR - Congestion Window Reduced (MSB in TCP Flags byte)

E
- ECE - ECN-Echo (If SYN, then ECN capable. Else, CE flag in IP header is set)

0
- No TCP Flags Set

The following modifiers can be set to change the match criteria:

+
- match on the specified bits, plus any others

*****
- match if any of the specified bits are set

!
- match if the specified bits are not set

To handle writing rules for session initiation packets such as ECN where a SYN packet is sent with CWR and ECE set, an option mask may be specified by preceding the mask with a comma. A rule could check for a flags value of S,CE if one wishes to find packets with just the syn bit, regardless of the values of the reserved bits.

3.6.8.1 Format

```
flags:[!|*|+]<FSRPAUCE0>[,<FSRPAUCE>];
```

3.6.8.2 Example

This example checks if just the SYN and the FIN bits are set, ignoring CWR (reserved bit 1) and ECN (reserved bit 2).

```
alert tcp any any -> any any (flags:SF,CE;)
```

> **Note:**
>
> The reserved bits '1' and '2' have been replaced with 'C' and 'E', respectively, to match RFC 3168, "The Addition of Explicit Congestion Notification (ECN) to IP". The old values of '1' and '2' are still valid for the `flag` keyword, but are now deprecated.

3.6.9 flow

The flow keyword is used in conjunction with session tracking (see Section [*]). It allows rules to only apply to certain directions of the traffic flow.

This allows rules to only apply to clients or servers. This allows packets related to $HOME_NET clients viewing web pages to be distinguished from servers running in the $HOME_NET.

The established keyword will replace the `flags:+A` used in many places to show established TCP connections.

Options

Option	Description
`to_client`	Trigger on server responses from A to B
`to_server`	Trigger on client requests from A to B
`from_client`	Trigger on client requests from A to B
`from_server`	Trigger on server responses from A to B
`established`	Trigger only on established TCP connections
`not_established`	Trigger only when no TCP connection is established

`stateless`	Trigger regardless of the state of the stream processor (useful for packets that are designed to cause machines to crash)
`no_stream`	Do not trigger on rebuilt stream packets (useful for dsize and stream5)
`only_stream`	Only trigger on rebuilt stream packets
`no_frag`	Do not trigger on rebuilt frag packets
`only_frag`	Only trigger on rebuilt frag packets

3.6.9.1 Format

```
    flow:[(established|not_established|stateless)]
        [,(to_client|to_server|from_client|from_server)]
        [,(no_stream|only_stream)]
        [,(no_frag|only_frag)];
```

3.6.9.2 Examples

```
    alert tcp !$HOME_NET any -> $HOME_NET 21 (msg:"cd incoming detected"; \
        flow:from_client; content:"CWD incoming"; nocase;)

    alert tcp !$HOME_NET 0 -> $HOME_NET 0 (msg:"Port 0 TCP traffic"; \
        flow:stateless;)
```

3.6.10 flowbits

The `flowbits` keyword is used in conjunction with conversation tracking from the Session preprocessor (see Section[*]). It allows rules to track states during a transport protocol session. The flowbits option is most useful for TCP sessions, as it allows rules to generically track the state of an application protocol.

There are several keywords associated with flowbits. Most of the options need a user-defined name for the specific state that is being checked. Some keyword uses group name. When no group name is specified the flowbits will belong to a default group. A particular flowbit can belong to more than one group. Flowbit name and group name should be limited to any alphanumeric string including periods, dashes, and underscores.

3.6.10.1 General Format

```
    flowbits:[set|setx|unset|toggle|isset|isnotset|noalert|reset][, <bits/bats>][,
<GROUP_NAME>];
    bits ::= bit[|bits]
    bats ::= bit[&bats]
```

Option	Description

`set`	Sets the specified states for the current flow and assign them to a group when a GROUP_NAME is specified.
`setx`	Sets the specified states for the current flow and clear other states in the group
`unset`	Unsets the specified states for the current flow.
`toggle`	For every state specified, sets the specified state if the state is unset and unsets it if the state is set.
`isset`	Checks if the specified states are set.
`isnotset`	Checks if the specified states are not set.
`noalert`	Cause the rule to not generate an alert, regardless of the rest of the detection options.
`reset`	Reset all states on a given flow.

3.6.10.2 set

This keyword sets bits to group for a particular flow. When no group specified, set the default group. This keyword always returns true.

```
Syntax:
    flowbits:set,bats[,group]
Usage:
    flowbits:set,bit1,doc;
    flowbits:set,bit2&bit3,doc;
    First rule sets bit1 in doc group, second rule sets bit2 and bit3 in doc group.
    So doc group has bit 1, bit2 and bit3 set
```

3.6.10.3 setx

This keyword sets bits to group exclusively. This clears other bits in group. Group must present.This keyword always returns true.

```
Syntax:
    flowbits:setx,bats,group
Usage:
    flowbits: setx, bit1, doc
    flowbits: setx, bit2&bit3, doc
    First rule sets bit1 in doc group, second rule sets bit2 and bit3 in doc group.
    So doc group has bit2 and bit3 set, because bit1 is cleared by rule 2.
```

3.6.10.4 unset

This keyword clears bits specified for a particular flow or clears all bits in the group (Group must present). This keyword always returns true.

```
Syntax:
   flowbits:unset,bats
   flowbits:unset,all,group

Usage:
```

```
   flowbits: unset, bit1
   Clear bit1.

   flowbits: unset, bit1&bit2
   Clear bit1 and bit2

   flowbits: unset, all, doc
   This clears all bits in the doc group.
```

3.6.10.5 toggle

If flowbit is set, unset it. If it is unset, set it. Toggle every bit specified or toggle all the bits in group (Group must be present). This keyword always returns true.

```
Syntax:
   flowbits:toggle,bats
   flowbits:toggle,all,group

Usage:
   flowbits: toggle, bit1&bit2
   If bit1 is 0 and bit2 is 1 before, after this rule, bit1 is 1 and bit2 is 0.

   flowbits:toggle,all,doc
   Toggle all the bits in group doc as described above.
```

3.6.10.6 isset

This keyword checks a bit or several bits to see if it is set. It returns true or false based on the following syntax.

```
Syntax:
   flowbits:isset, bits => Check whether any bit is set
   flowbits:isset, bats => Check whether all bits are set
   flowbits:isset, any, group => Check whether any bit in the group is set.
   flowbits:isset, all, group => Check whether all bits in the group are set.

Usage
   flowbits:isset, bit1|bit2 => If either bit1 or bit2 is set, return true
   flowbits:isset, bit1&bit2 => If both bit1 and bit2 are set, return true, otherwise
false
   flowbits:isset, any, doc => If any bit in group doc is set, return true
   flowbits:isset, all, doc => If all the bits in doc group are set, return true
```

3.6.10.7 isnotset

This keyword is the reverse of isset. It returns true if isset is false, it returns false if isset is true. Isnotset works on the final result, not on individual bits.

```
Syntax:
   flowbits:isnotset, bits => Check whether not any bit is set
```

```
   flowbits:isnotset, bats => Check whether not all bits are set
   flowbits:isnotset, any, group => Check whether not bit in the group is set.
   flowbits:isnotset, all, group => Check whether not all bits in the group are set.

Usage
   flowbits:isnotset, bit1|bit2 => If either bit1 or bit2 is set, return true
   flowbits:isnotset, bit1&bit2 => If both bit1 and bit2 are set, return true,
otherwise false
   flowbits:isnotset, any, doc => If any bit in group doc is set, return true
   flowbits:isnotset, all, doc => If all the bits in doc group are set, return true
```

3.6.10.8 noalert

This keyword always returns false. It allows users to write rules that set, unset, and toggle bit without generating an alert. This is most useful for writing flowbit rules that set bit on normal traffic and significantly reduces unwanted alerts. There is no bit specified with this keyword.

```
  flowbits:noalert;
```

3.6.10.9 reset

This keyword resets all of the states on a given flow if no group specified, otherwise, reset all the bits in a group. This always returns true. There is no bit specified with this keyword.

```
Syntax:
   flowbits:reset[,group]

Usage:
   flowbits:reset => reset all the bits in the flow
   flowbits: reset, doc => reset all the bits in the doc group
```

3.6.10.10 Examples

```
    alert tcp any 143 -> any any (msg:"IMAP login";
        content:"OK LOGIN"; flowbits:set,logged_in;
        flowbits:noalert;)

    alert tcp any any -> any 143 (msg:"IMAP LIST"; content:"LIST";
        flowbits:isset,logged_in;)
```

3.6.11 seq

The seq keyword is used to check for a specific TCP sequence number.

3.6.11.1 Format

```
seq:<number>;
```

3.6.11.2 Example

This example looks for a TCP sequence number of 0.

```
seq:0;
```

3.6.12 ack

The ack keyword is used to check for a specific TCP acknowledge number.

3.6.12.1 Format

```
ack:<number>;
```

3.6.12.2 Example

This example looks for a TCP acknowledge number of 0.

```
ack:0;
```

3.6.13 window

The window keyword is used to check for a specific TCP window size.

3.6.13.1 Format

```
window:[!]<number>;
```

3.6.13.2 Example

This example looks for a TCP window size of 55808.

```
window:55808;
```

3.6.14 itype

The itype keyword is used to check for a specific ICMP type value.

3.6.14.1 Format

```
itype:min<>max;
itype:[<|>]<number>;
```

3.6.14.2 Example

This example looks for an ICMP type greater than 30.

```
itype:>30;
```

3.6.15 icode

The icode keyword is used to check for a specific ICMP code value.

3.6.15.1 Format

```
icode:min<>max;
icode:[<|>]<number>;
```

The <> operator in the first format checks for an ICMP code within a specified range (exclusive). That is, strictly greater than the min value and strictly less than the max value. Note that the min value can a –1 allowing an ICMP code of zero to be included in the range.

Numerical values are validated with respect to permissible ICMP code values between 0 and 255 and other criteria.

```
icode:min<>max
    -1 <= min <= 254
     1 <= max <= 256
     (max - min) > 1

icode:number
    0 <= number <= 255

icode:<number
    1 <= number <= 256

icode:>number
    0 <= number <= 254
```

3.6.15.2 Examples

This example looks for an ICMP code greater than 30.

```
icode:>30;
```

This example looks for an ICMP code greater than zero and less than 30.

```
icode:-1<>30;
```

3.6.16 icmp_id

The icmp_id keyword is used to check for a specific ICMP ID value.

This is useful because some covert channel programs use static ICMP fields when they communicate. This particular plugin was developed to detect the stacheldraht DDoS agent.

3.6.16.1 Format

```
icmp_id:<number>;
```

3.6.16.2 Example

This example looks for an ICMP ID of 0.

```
icmp_id:0;
```

3.6.17 icmp_seq

The icmp_seq keyword is used to check for a specific ICMP sequence value.

This is useful because some covert channel programs use static ICMP fields when they communicate. This particular plugin was developed to detect the stacheldraht DDoS agent.

3.6.17.1 Format

```
icmp_seq:<number>;
```

3.6.17.2 Example

This example looks for an ICMP Sequence of 0.

```
icmp_seq:0;
```

3.6.18 rpc

The rpc keyword is used to check for a RPC application, version, and procedure numbers in SUNRPC CALL requests.

Wildcards are valid for both version and procedure numbers by using '*';

3.6.18.1 Format

```
rpc:<application number>, [<version number>|*], [<procedure number>|*]>;
```

3.6.18.2 Example

The following example looks for an RPC portmap GETPORT request.

```
alert tcp any any -> any 111 (rpc:100000, *, 3;);
```

3.6.18.3 Warning

Because of the fast pattern matching engine, the RPC keyword is slower than looking for the RPC values by using normal content matching.

3.6.19 ip_proto

The ip_proto keyword allows checks against the IP protocol header. For a list of protocols that may be specified by name, see /etc/protocols.

3.6.19.1 Format

```
ip_proto:[!|>|<] <name or number>;
```

3.6.19.2 Example

This example looks for IGMP traffic.

```
alert ip any any -> any any (ip_proto:igmp;)
```

3.6.20 sameip

The sameip keyword allows rules to check if the source ip is the same as the destination IP.

3.6.20.1 Format

```
sameip;
```

3.6.20.2 Example

This example looks for any traffic where the Source IP and the Destination IP is the same.

```
alert ip any any -> any any (sameip;)
```

3.6.21 stream_reassemble

The stream_reassemble keyword allows a rule to enable or disable TCP stream reassembly on matching traffic.

> **Note:**
>
> The stream_reassemble option is only available when the Stream preprocessor is enabled.

3.6.21.1 Format

```
stream_reassemble:<enable|disable>, <server|client|both>[, noalert][, fastpath];
```

- The optional `noalert` parameter causes the rule to not generate an alert when it matches.
- The optional `fastpath` parameter causes Snort to ignore the rest of the connection.

3.6.21.2 Example

For example, to disable TCP reassembly for client traffic when we see a HTTP 200 Ok Response message, use:

```
alert tcp any 80 -> any any (flow:to_client, established; content:"200 OK";
   stream_reassemble:disable,client,noalert;)
```

3.6.22 stream_size

The stream_size keyword allows a rule to match traffic according to the number of bytes observed, as determined by the TCP sequence numbers.

> **Note:**
>
> The stream_size option is only available when the Stream preprocessor is enabled.

3.6.22.1 Format

```
stream_size:<server|client|both|either>, <operator>, <number>;
```

Where the operator is one of the following:

- $<$ - less than
- $>$ - greater than
- = - equal
- != - not equal
- $<$= - less than or equal
- $>$= - greater than or equal

3.6.22.2 Example

For example, to look for a session that is less that 6 bytes from the client side, use:

```
alert tcp any any -> any any (stream_size:client,<,6;)
```

3.6.23 Non-Payload Detection Quick Reference

Table: Non-payload detection rule option keywords

Keyword	Description
`fragoffset`	The fragoffset keyword allows one to compare the IP fragment offset field against a decimal value.
`ttl`	The ttl keyword is used to check the IP time-to-live value.
`tos`	The tos keyword is used to check the IP TOS field for a specific value.
`id`	The id keyword is used to check the IP ID field for a specific value.
`ipopts`	The ipopts keyword is used to check if a specific IP option is present.
`fragbits`	The fragbits keyword is used to check if fragmentation and reserved bits are set in the IP header.
`dsize`	The dsize keyword is used to test the packet payload size.
`flags`	The flags keyword is used to check if specific TCP flag bits are present.
`flow`	The flow keyword allows rules to only apply to certain directions of the traffic flow.
`flowbits`	The flowbits keyword allows rules to track states during a transport protocol session.
`seq`	The seq keyword is used to check for a specific TCP sequence number.
`ack`	The ack keyword is used to check for a specific TCP acknowledge number.
`window`	The window keyword is used to check for a specific TCP window size.
`itype`	

	The itype keyword is used to check for a specific ICMP type value.
`icode`	The icode keyword is used to check for a specific ICMP code value.
`icmp_id`	The icmp_id keyword is used to check for a specific ICMP ID value.
`icmp_seq`	The icmp_seq keyword is used to check for a specific ICMP sequence value.
`rpc`	The rpc keyword is used to check for a RPC application, version, and procedure numbers in SUNRPC CALL requests.
`ip_proto`	The ip_proto keyword allows checks against the IP protocol header.
`sameip`	The sameip keyword allows rules to check if the source ip is the same as the destination IP.

End of Navigation Panel Table of Child-Links **Subsections**

- 3.7.1 logto
  - 3.7.1.1 Format
- 3.7.2 session
  - 3.7.2.1 Format
  - 3.7.2.2 Example
  - 3.7.2.3 Warnings
- 3.7.3 resp
- 3.7.4 react
- 3.7.5 tag
  - 3.7.5.1 Format
  - 3.7.5.2 Example
- 3.7.6 replace
- 3.7.7 detection_filter
- 3.7.8 Post-Detection Quick Reference

End of Table of Child-Links

3.7 Post-Detection Rule Options

3.7.1 logto

The logto keyword tells Snort to log all packets that trigger this rule to a special output log file. This is especially handy for combining data from things like NMAP activity, HTTP CGI scans, etc. It should be noted that this option does not work when Snort is in binary logging mode.

3.7.1.1 Format

```
logto:"filename";
```

3.7.2 session

The session keyword is built to extract user data from TCP Sessions. There are many cases where seeing what users are typing in telnet, rlogin, ftp, or even web sessions is very useful.

There are three available argument keywords for the session rule option: `printable`, `binary`, or `all`.

The `printable` keyword only prints out data that the user would normally see or be able to type. The `binary` keyword prints out data in a binary format. The `all` keyword substitutes non-printable characters with their hexadecimal equivalents.

3.7.2.1 Format

```
session:[printable|binary|all];
```

3.7.2.2 Example

The following example logs all printable strings in a telnet packet.

```
log tcp any any <> any 23 (session:printable;)
```

Given an FTP data session on port 12345, this example logs the payload bytes in binary form.

```
log tcp any any <> any 12345 (metadata:service ftp-data; session:binary;)
```

3.7.2.3 Warnings

Using the session keyword can slow Snort down considerably, so it should not be used in heavy load situations. The session keyword is best suited for post-processing binary (pcap) log files.

The `binary` keyword does not log any protocol headers below the application layer, and Stream reassembly will cause duplicate data when the reassembled packets are logged.

3.7.3 resp

The resp keyword enables an active response that kills the offending session. Resp can be used in both passive or inline modes. See [*] for details.

3.7.4 react

The react keyword enables an active response that includes sending a web page or other content to the client and then closing the connection. React can be used in both passive and inline modes. See [*] for details.

3.7.5 tag

The tag keyword allow rules to log more than just the single packet that triggered the rule. Once a rule is triggered, additional traffic involving the source and/or destination host is tagged. Tagged traffic is logged to allow analysis of response codes and post-attack traffic. tagged alerts will be sent to the same output plugins as the original alert, but it is the responsibility of the output plugin to properly handle these special alerts.

3.7.5.1 Format

```
tag:host, <count>, <metric>, <direction>;
tag:session[, <count>, <metric>][, exclusive];
```

`type`

- `session` - Log packets in the session that set off the rule
- `host` - Log packets from the host that caused the tag to activate (uses [direction] modifier)

`count`

- `<integer>` - Count is specified as a number of units. Units are specified in the $<$metric$>$ field.

`metric`

- `packets` - Tag the host/session for $<$count$>$ packets
- `seconds` - Tag the host/session for $<$count$>$ seconds
- `bytes` - Tag the host/session for $<$count$>$ bytes

`other`

- `src` - Tag packets containing the source IP address of the packet that generated the initial event. Only relevant if host type is used.
- `dst` - Tag packets containing the destination IP address of the packet that generated the initial event. Only relevant if host type is used.
- `exclusive` - Tag packets only in the first matching session. Only relevant if session type is used.

Note that neither subsequent alerts nor event filters will prevent a tagged packet from being logged. Subsequent tagged alerts will cause the limit to reset.

```
alert tcp any any <> 10.1.1.1 any \
    (flowbits:isnotset,tagged; content:"foobar"; nocase; \
     flowbits:set,tagged; tag:host,600,seconds,src;)
```

Also note that if you have a tag option in a rule that uses a metric other than `packets`, a `tagged_packet_limit` will be used to limit the number of tagged packets regardless of whether the `seconds` or `bytes` count has been reached. The default `tagged_packet_limit` value is 256 and can be modified by using a config option in your snort.conf file (see Section [*] on how to use the `tagged_packet_limit` config option). You can disable this packet limit for a particular rule by adding a `packets` metric to your tag option and setting its count to 0 (This can be done on a global scale by setting the `tagged_packet_limit` option in snort.conf to 0). Doing this will ensure that packets are tagged for the full amount of `seconds` or `bytes` and will not be cut off by the `tagged_packet_limit`. (Note that the `tagged_packet_limit` was introduced to avoid DoS situations on high bandwidth sensors for tag rules with a high `seconds` or `bytes` counts.)

```
alert tcp 10.1.1.4 any -> 10.1.1.1 any \
    (content:"TAGMYPACKETS"; tag:host,0,packets,600,seconds,src;)
```

3.7.5.2 Example

This example logs the first 10 seconds or the `tagged_packet_limit` (whichever comes first) of any telnet session.

```
alert tcp any any -> any 23 (flags:S,CE; tag:session,10,seconds;)
```

While at least one count and metric is required for tag:host, tag:session with exclusive without any metrics can be used to get a full session like this:

```
pass tcp any any -> 192.168.1.1 80 (flags:S; tag:session,exclusive;)
```

3.7.6 replace

The `replace` keyword is a feature available in inline mode which will cause Snort to replace the prior matching content with the given string. Both the new string and the content it is to replace must have the same length. You can have multiple replacements within a rule, one per content.

```
    replace:"<string>";
```

3.7.7 detection_filter

detection_filter defines a rate which must be exceeded by a source or destination host before a rule can generate an event. detection_filter has the following format:

```
    detection_filter: \
        track <by_src|by_dst>, \
        count <c>, seconds <s>;
```

Option	Description
`track by_src| by_dst`	Rate is tracked either by source IP address or destination IP address. This means count is maintained for each unique source IP address or each unique destination IP address.
`count c`	The maximum number of rule matches in s seconds allowed before the detection filter limit to be exceeded. C must be nonzero.
`seconds s`	Time period over which count is accrued. The value must be nonzero.

Snort evaluates a `detection_filter` as the last step of the detection phase, after evaluating all other rule options (regardless of the position of the filter within the rule source). At most one `detection_filter` is permitted per rule.

Example - this rule will fire on every failed login attempt from 10.1.2.100 during one sampling period of 60 seconds, after the first 30 failed login attempts:

```
    drop tcp 10.1.2.100 any > 10.1.1.100 22 ( \
        msg:"SSH Brute Force Attempt";
        flow:established,to_server; \
        content:"SSH"; nocase; offset:0; depth:4; \
```

```
        detection_filter:track by_src, count 30, seconds 60; \
        sid:1000001; rev:1;)
```

Since potentially many events will be generated, a `detection_filter` would normally be used in conjunction with an `event_filter` to reduce the number of logged events.

> **Note:** As mentioned above, Snort evaluates `detection_filter` as the last step of the detection and not in post-detection.

3.7.8 Post-Detection Quick Reference

Table: Post-detection rule option keywords

Keyword	Description
`logto`	The logto keyword tells Snort to log all packets that trigger this rule to a special output log file.
`session`	The session keyword is built to extract user data from TCP Sessions.
`resp`	The resp keyword is used attempt to close sessions when an alert is triggered.
`react`	This keyword implements an ability for users to react to traffic that matches a Snort rule by closing connection and sending a notice.
`tag`	The tag keyword allow rules to log more than just the single packet that triggered the rule.
`replace`	Replace the prior matching content with the given string of the same length. Available in inline mode only.
`detection_filter`	Track by source or destination IP address and if the rule otherwise matches more than the configured rate it will fire.

End of Navigation Panel Table of Child-Links **Subsections**

- ◦ 3.8.0.1 Examples

End of Table of Child-Links

3.8 Rule Thresholds

Note: Rule thresholds are deprecated and will not be supported in a future release. Use `detection_filters` ([*]) within rules, or `event_filters` ([*]) as standalone configurations instead.

`threshold` can be included as part of a rule, or you can use standalone thresholds that reference the generator and SID they are applied to. There is no functional difference between adding a threshold to a rule, or using a standalone threshold applied to the same rule. There is a logical difference. Some rules may only make sense with a threshold. These should incorporate the threshold into the rule. For instance, a rule for detecting a too many login password attempts may require more than 5 attempts. This can be done using the `limit' type of threshold. It makes sense that the threshold feature is an integral part of this rule.

3.8.0.0.1 Format

```
threshold: \
    type <limit|threshold|both>, \
    track <by_src|by_dst>, \
    count <c>, seconds <s>;
```

Option	Description
`type limit|threshold|both`	type `limit` alerts on the 1st m events during the time interval, then ignores events for the rest of the time interval. Type `threshold` alerts every m times we see this event during the time interval. Type `both` alerts once per time interval after seeing m occurrences of the event, then ignores any additional events during the time interval.
`track by_src|by_dst`	rate is tracked either by source IP address, or destination IP address. This means count is maintained for each unique source IP addresses, or for each unique destination IP addresses. Ports or anything else are not tracked.
`count c`	number of rule matching in s seconds that will cause `event_filter` limit to be exceeded. `c` must be nonzero value.
`seconds s`	time period over which `count` is accrued. `s` must be nonzero value.

3.8.0.1 Examples

This rule logs the first event of this SID every 60 seconds.

```
alert tcp $external_net any -> $http_servers $http_ports \
    (msg:"web-misc robots.txt access"; flow:to_server, established; \
    uricontent:"/robots.txt"; nocase; reference:nessus,10302; \
    classtype:web-application-activity; threshold:type limit, track \
    by_src, count 1 , seconds 60; sid:1000852; rev:1;)
```

This rule logs every 10th event on this SID during a 60 second interval. So if less than 10 events occur in 60 seconds, nothing gets logged. Once an event is logged, a new time period starts for type=threshold.

```
alert tcp $external_net any -> $http_servers $http_ports \
    (msg:"web-misc robots.txt access"; flow:to_server, established; \
    uricontent:"/robots.txt"; nocase; reference:nessus,10302; \
    classtype:web-application-activity; threshold:type threshold, \
    track by_dst, count 10 , seconds 60 ; sid:1000852; rev:1;)
```

This rule logs at most one event every 60 seconds if at least 10 events on this SID are fired.

```
alert tcp $external_net any -> $http_servers $http_ports \
    (msg:"web-misc robots.txt access"; flow:to_server, established; \
    uricontent:"/robots.txt"; nocase; reference:nessus,10302; \
    classtype:web-application-activity; threshold:type both, track \
    by_dst, count 10, seconds 60; sid:1000852; rev:1;)
```

End of Navigation Panel Table of Child-Links **Subsections**

- 3.9.1 Content Matching
- 3.9.2 Catch the Vulnerability, Not the Exploit
- 3.9.3 Catch the Oddities of the Protocol in the Rule
- 3.9.4 Optimizing Rules
- 3.9.5 Testing Numerical Values

End of Table of Child-Links

3.9 Writing Good Rules

There are some general concepts to keep in mind when developing Snort rules to maximize efficiency and speed.

3.9.1 Content Matching

Snort groups rules by protocol (ip, tcp, udp, icmp), then by ports (ip and icmp use slightly different logic), then by those with `content` and those without. For rules with `content`, a multi-pattern matcher is used to select rules that have a chance at matching based on a single content. Selecting rules for evaluation via this "fast" pattern matcher was found to increase performance, especially when applied to large rule groups like HTTP. The longer and more unique a `content` is, the less likely that rule and all of its rule options will be evaluated unnecessarily - it's safe to say there is generally more "good" traffic than "bad". Rules without `content` are always evaluated (relative to the protocol and port group in which they reside), potentially putting a drag on performance. While some detection options, such as `pcre` and `byte_test`, perform detection in the payload section of the packet, they are not used by the fast pattern matching engine. If at all possible, try and have at least one `content` (or `uricontent`) rule option in your rule.

3.9.2 Catch the Vulnerability, Not the Exploit

Try to write rules that target the vulnerability, instead of a specific exploit.

For example, look for a the vulnerable command with an argument that is too large, instead of shellcode that binds a shell.

By writing rules for the vulnerability, the rule is less vulnerable to evasion when an attacker changes the exploit slightly.

3.9.3 Catch the Oddities of the Protocol in the Rule

Many services typically send the commands in upper case letters. FTP is a good example. In FTP, to send the username, the client sends:

```
user username_here
```

A simple rule to look for FTP root login attempts could be:

```
alert tcp any any -> any any 21 (content:"user root";)
```

While it may seem trivial to write a rule that looks for the username root, a good rule will handle all of the odd things that the protocol might handle when accepting the user command.

For example, each of the following are accepted by most FTP servers:

```
user root
user root
user root
user root
user<tab>root
```

To handle all of the cases that the FTP server might handle, the rule needs more smarts than a simple string match.

A good rule that looks for root login on ftp would be:

```
alert tcp any any -> any 21 (flow:to_server,established; \
    content:"root"; pcre:"/user\s+root/i";)
```

There are a few important things to note in this rule:

- The rule has a flow option, verifying this is traffic going to the server on an established session.
- The rule has a content option, looking for root, which is the longest, most unique string in the attack. This option is added to allow the fast pattern matcher to select this rule for evaluation only if the content root is found in the payload.
- The rule has a pcre option, looking for user, followed at least one space character (which includes tab), followed by root, ignoring case.

3.9.4 Optimizing Rules

The content matching portion of the detection engine has recursion to handle a few evasion cases. Rules that are not properly written can cause Snort to waste time duplicating checks.

The way the recursion works now is if a pattern matches, and if any of the detection options after that pattern fail, then look for the pattern again after where it was found the previous time. Repeat until the pattern is not found again or the opt functions all succeed.

On first read, that may not sound like a smart idea, but it is needed. For example, take the following rule:

```
alert ip any any -> any any (content:"a"; content:"b"; within:1;)
```

This rule would look for “a”, immediately followed by “b”. Without recursion, the payload “aab” would fail, even though it is obvious that the payload “aab” has “a” immediately followed by “b”, because the first "a" is not immediately followed by “b”.

While recursion is important for detection, the recursion implementation is not very smart.

For example, the following rule options are not optimized:

```
content:"|13|"; dsize:1;
```

By looking at this rule snippet, it is obvious the rule looks for a packet with a single byte of 0x13. However, because of recursion, a packet with 1024 bytes of 0x13 could cause 1023 too many pattern match attempts and 1023 too many dsize checks. Why? The content 0x13 would be found in the first byte, then the dsize option would fail, and because of recursion, the content 0x13 would be found again starting after where the previous 0x13 was found, once it is found, then check the dsize again, repeating until 0x13 is not found in the payload again.

Reordering the rule options so that discrete checks (such as dsize) are moved to the beginning of the rule speed up Snort.

The optimized rule snipping would be:

```
    dsize:1; content:"|13|";
```

A packet of 1024 bytes of 0x13 would fail immediately, as the dsize check is the first option checked and dsize is a discrete check without recursion.

The following rule options are discrete and should generally be placed at the beginning of any rule:

- `dsize`
- `flags`
- `flow`
- `fragbits`
- `icmp_id`
- `icmp_seq`
- `icode`
- `id`
- `ipopts`
- `ip_proto`
- `itype`
- `seq`
- `session`
- `tos`
- `ttl`
- `ack`
- `window`
- `resp`
- `sameip`

3.9.5 Testing Numerical Values

The rule options byte_test and byte_jump were written to support writing rules for protocols that have length encoded data. RPC was the protocol that spawned the requirement for these two rule options, as RPC uses simple length based encoding for passing data.

In order to understand why byte_test and byte_jump are useful, let's go through an exploit attempt against the sadmind service.

This is the payload of the exploit:

```
89 09 9c e2 00 00 00 00 00 00 00 02 00 01 87 88  ................
00 00 00 0a 00 00 00 01 00 00 00 01 00 00 00 20  ................
40 28 3a 10 00 00 00 0a 4d 45 54 41 53 50 4c 4f  @(:.....metasplo
49 54 00 00 00 00 00 00 00 00 00 00 00 00 00 00  it..............
00 00 00 00 00 00 00 00 40 28 3a 14 00 07 45 df  ........@(:...e.
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 00  ................
00 00 00 00 00 00 00 04 00 00 00 00 00 00 00 04  ................
7f 00 00 01 00 01 87 88 00 00 00 0a 00 00 00 04  ................
7f 00 00 01 00 01 87 88 00 00 00 0a 00 00 00 11  ................
00 00 00 1e 00 00 00 00 00 00 00 00 00 00 00 00  ................
00 00 00 00 00 00 00 3b 4d 45 54 41 53 50 4c 4f  .......;metasplo
49 54 00 00 00 00 00 00 00 00 00 00 00 00 00 00  it..............
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00 00 00 00 00 00 00 06 73 79 73 74 65 6d 00 00  ........system..
00 00 00 15 2e 2e 2f 2e 2e 2f 2e 2e 2f 2e 2e 2f  ....../../../../
2e 2e 2f 62 69 6e 2f 73 68 00 00 00 00 00 04 1e  ../bin/sh.......
<snip>
```

Let's break this up, describe each of the fields, and figure out how to write a rule to catch this exploit.

There are a few things to note with RPC:

- Numbers are written as uint32s, taking four bytes. The number 26 would show up as 0x0000001a.
- Strings are written as a uint32 specifying the length of the string, the string, and then null bytes to pad the length of the string to end on a 4 byte boundary. The string “bob” would show up as 0x00000003626f6200.

```
89 09 9c e2     - the request id, a random uint32, unique to each request
00 00 00 00     - rpc type (call = 0, response = 1)
00 00 00 02     - rpc version (2)
00 01 87 88     - rpc program (0x00018788 = 100232 = sadmind)
00 00 00 0a     - rpc program version (0x0000000a = 10)
00 00 00 01     - rpc procedure (0x00000001 = 1)
00 00 00 01     - credential flavor (1 = auth\_unix)
00 00 00 20     - length of auth\_unix data (0x20 = 32

## the next 32 bytes are the auth\_unix data
40 28 3a 10 - unix timestamp (0x40283a10 = 1076378128 = feb 10 01:55:28 2004 gmt)
00 00 00 0a - length of the client machine name (0x0a = 10)
4d 45 54 41 53 50 4c 4f 49 54 00 00  - metasploit

00 00 00 00 - uid of requesting user (0)
00 00 00 00 - gid of requesting user (0)
00 00 00 00 - extra group ids (0)
```

```
00 00 00 00     - verifier flavor (0 = auth\_null, aka none)
00 00 00 00     - length of verifier (0, aka none)
```

The rest of the packet is the request that gets passed to procedure 1 of sadmind.

However, we know the vulnerability is that sadmind trusts the uid coming from the client. sadmind runs any request where the client's uid is 0 as root. As such, we have decoded enough of the request to write our rule.

First, we need to make sure that our packet is an RPC call.

```
    content:"|00 00 00 00|"; offset:4; depth:4;
```

Then, we need to make sure that our packet is a call to sadmind.

```
    content:"|00 01 87 88|"; offset:12; depth:4;
```

Then, we need to make sure that our packet is a call to the procedure 1, the vulnerable procedure.

```
    content:"|00 00 00 01|"; offset:20; depth:4;
```

Then, we need to make sure that our packet has auth_unix credentials.

```
    content:"|00 00 00 01|"; offset:24; depth:4;
```

We don't care about the hostname, but we want to skip over it and check a number value after the hostname. This is where byte_test is useful. Starting at the length of the hostname, the data we have is:

```
00 00 00 0a 4d 45 54 41 53 50 4c 4f 49 54 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00
```

We want to read 4 bytes, turn it into a number, and jump that many bytes forward, making sure to account for the padding that RPC requires on strings. If we do that, we are now at:

```
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00
```

which happens to be the exact location of the uid, the value we want to check.

In English, we want to read 4 bytes, 36 bytes from the beginning of the packet, and turn those 4 bytes into an integer and jump that many bytes forward, aligning on the 4 byte boundary. To do that in a Snort rule, we use:

```
    byte_jump:4,36,align;
```

then we want to look for the uid of 0.

```
content:"|00 00 00 00|"; within:4;
```

Now that we have all the detection capabilities for our rule, let's put them all together.

```
content:"|00 00 00 00|"; offset:4; depth:4;
content:"|00 01 87 88|"; offset:12; depth:4;
content:"|00 00 00 01|"; offset:20; depth:4;
content:"|00 00 00 01|"; offset:24; depth:4;
byte_jump:4,36,align;
content:"|00 00 00 00|"; within:4;
```

The 3rd and fourth string match are right next to each other, so we should combine those patterns. We end up with:

```
content:"|00 00 00 00|"; offset:4; depth:4;
content:"|00 01 87 88|"; offset:12; depth:4;
content:"|00 00 00 01 00 00 00 01|"; offset:20; depth:8;
byte_jump:4,36,align;
content:"|00 00 00 00|"; within:4;
```

If the sadmind service was vulnerable to a buffer overflow when reading the client's hostname, instead of reading the length of the hostname and jumping that many bytes forward, we would check the length of the hostname to make sure it is not too large.

To do that, we would read 4 bytes, starting 36 bytes into the packet, turn it into a number, and then make sure it is not too large (let's say bigger than 200 bytes). In Snort, we do:

```
byte_test:4,>,200,36;
```

Our full rule would be:

```
content:"|00 00 00 00|"; offset:4; depth:4;
content:"|00 01 87 88|"; offset:12; depth:4;
content:"|00 00 00 01 00 00 00 01|"; offset:20; depth:8;
byte_test:4,>,200,36;
```

End of Navigation Panel

4. Dynamic Modules

Preprocessors, detection capabilities, and rules can now be developed as dynamically loadable modules to snort. The dynamic API presents a means for loading dynamic libraries and allowing the module to utilize certain functions within the main snort code.

The remainder of this chapter will highlight the data structures and API functions used in developing preprocessors, detection engines, and rules as a dynamic plugin to snort.

Beware: the definitions herein may be out of date; check the appropriate header files for the current definitions.

Subsections

- 4.1 Data Structures
  - 4.1.1 DynamicPluginMeta
  - 4.1.2 DynamicPreprocessorData
  - 4.1.3 DynamicEngineData
  - 4.1.4 SFSnortPacket
  - 4.1.5 Dynamic Rules
- 4.2 Required Functions
  - 4.2.1 Preprocessors
  - 4.2.2 Detection Engine
  - 4.2.3 Rules
- 4.3 Examples
  - 4.3.1 Preprocessor Example
  - 4.3.2 Rules

End of Navigation Panel Table of Child-Links **Subsections**

- 4.1.1 DynamicPluginMeta
- 4.1.2 DynamicPreprocessorData
- 4.1.3 DynamicEngineData
- 4.1.4 SFSnortPacket
- 4.1.5 Dynamic Rules
    - 4.1.5.1 Rule
    - 4.1.5.2 RuleInformation
    - 4.1.5.3 RuleReference
    - 4.1.5.4 IPInfo
    - 4.1.5.5 RuleOption

End of Table of Child-Links

4.1 Data Structures

A number of data structures are central to the API. The definition of each is defined in the following sections.

4.1.1 DynamicPluginMeta

The *DynamicPluginMeta* structure defines the type of dynamic module (preprocessor, rules, or detection engine), the version information, and path to the shared library. A shared library can implement all three types, but typically is limited to a single functionality such as a preprocessor. It is defined in `sf_dynamic_meta.h` as:

```
#define MAX_NAME_LEN 1024

#define TYPE_ENGINE 0x01
#define TYPE_DETECTION 0x02
#define TYPE_PREPROCESSOR 0x04

typedef struct _DynamicPluginMeta
{
    int type;
    int major;
    int minor;
    int build;
    char uniqueName[MAX_NAME_LEN];
    char *libraryPath;
} DynamicPluginMeta;
```

4.1.2 DynamicPreprocessorData

The *DynamicPreprocessorData* structure defines the interface the preprocessor uses to interact with snort itself. This includes functions to register the preprocessor's configuration parsing, restart, exit, and processing functions. It includes function to log messages, errors, fatal errors, and debugging info. It also includes information for setting alerts, handling Inline drops, access to the StreamAPI, and it provides access to the normalized http and alternate data buffers. This data structure should be initialized when the preprocessor shared library is loaded. It is defined in `sf_dynamic_preprocessor.h`. Check the header file for the current definition.

4.1.3 DynamicEngineData

The *DynamicEngineData* structure defines the interface a detection engine uses to interact with snort itself. This includes functions for logging messages, errors, fatal errors, and debugging info as well as a means to register and check flowbits. It also includes a location to store rule-stubs for dynamic rules that are loaded, and it provides access to the normalized http and alternate data buffers. It is defined in `sf_dynamic_engine.h` as:

```
typedef struct _DynamicEngineData
{
    int version;
    u_int8_t *altBuffer;
    UriInfo *uriBuffers[MAX_URIINFOS];
    RegisterRule ruleRegister;
    RegisterBit flowbitRegister;
    CheckFlowbit flowbitCheck;
    DetectAsn1 asn1Detect;
    LogMsgFunc logMsg;
    LogMsgFunc errMsg;
    LogMsgFunc fatalMsg;
    char *dataDumpDirectory;

    GetPreprocRuleOptFuncs getPreprocOptFuncs;

    SetRuleData setRuleData;
    GetRuleData getRuleData;

    DebugMsgFunc debugMsg;
#ifdef HAVE_WCHAR_H
    DebugWideMsgFunc debugWideMsg;
#endif

    char **debugMsgFile;
    int *debugMsgLine;

    PCRECompileFunc pcreCompile;
    PCREStudyFunc pcreStudy;
    PCREExecFunc pcreExec;

} DynamicEngineData;
```

4.1.4 SFSnortPacket

The *SFSnortPacket* structure mirrors the snort Packet structure and provides access to all of the data contained in a given packet.

It and the data structures it incorporates are defined in `sf_snort_packet.h`. Additional data structures may be defined to reference other protocol fields. Check the header file for the current definitions.

4.1.5 Dynamic Rules

A dynamic rule should use any of the following data structures. The following structures are defined in `sf_snort_plugin_api.h`.

4.1.5.1 Rule

The *Rule* structure defines the basic outline of a rule and contains the same set of information that is seen in a text rule. That includes protocol, address and port information and rule information

(classification, generator and signature IDs, revision, priority, classification, and a list of references). It also includes a list of rule options and an optional evaluation function.

```
#define RULE_MATCH 1
#define RULE_NOMATCH 0

typedef struct _Rule
{
    IPInfo ip;
    RuleInformation info;

    RuleOption **options; /* NULL terminated array of RuleOption union */

    ruleEvalFunc evalFunc;

    char initialized;     /* Rule Initialized, used internally */
    u_int32_t numOptions; /* Rule option count, used internally */
    char noAlert;         /* Flag with no alert, used internally */
    void *ruleData;    /* Hash table for dynamic data pointers */
} Rule;
```

The rule evaluation function is defined as

```
typedef int (*ruleEvalFunc)(void *);
```

where the parameter is a pointer to the SFSnortPacket structure.

4.1.5.2 RuleInformation

The *RuleInformation* structure defines the meta data for a rule and includes generator ID, signature ID, revision, classification, priority, message text, and a list of references.

```
typedef struct _RuleInformation
{
    u_int32_t genID;
    u_int32_t sigID;
    u_int32_t revision;
    char     *classification; /* String format of classification name */
    u_int32_t priority;
    char     *message;
    RuleReference **references; /* NULL terminated array of references */
    RuleMetaData **meta; /* NULL terminated array of references */
} RuleInformation;
```

4.1.5.3 RuleReference

The *RuleReference* structure defines a single rule reference, including the system name and rereference identifier.

```
typedef struct _RuleReference
{
    char *systemName;
    char *refIdentifier;
} RuleReference;
```

4.1.5.4 IPInfo

The *IPInfo* structure defines the initial matching criteria for a rule and includes the protocol, src address and port, destination address and port, and direction. Some of the standard strings and variables are predefined - any, HOME_NET, HTTP_SERVERS, HTTP_PORTS, etc.

```
typedef struct _IPInfo
{
    u_int8_t protocol;
    char *   src_addr;
    char *   src_port; /* 0 for non TCP/UDP */
    char     direction;     /* non-zero is bi-directional */
    char *   dst_addr;
    char *   dst_port; /* 0 for non TCP/UDP */
} IPInfo;

#define ANY_NET         "any"
#define HOME_NET        "$HOME_NET"
#define EXTERNAL_NET    "$EXTERNAL_NET"
#define ANY_PORT        "any"
#define HTTP_SERVERS    "$HTTP_SERVERS"
#define HTTP_PORTS      "$HTTP_PORTS"
#define SMTP_SERVERS    "$SMTP_SERVERS"
```

4.1.5.5 RuleOption

The *RuleOption* structure defines a single rule option as an option type and a reference to the data specific to that option. Each option has a flags field that contains specific flags for that option as well as a "Not" flag. The "Not" flag is used to negate the results of evaluating that option.

```
typedef enum DynamicOptionType {
     OPTION_TYPE_PREPROCESSOR,
     OPTION_TYPE_CONTENT,
     OPTION_TYPE_PCRE,
     OPTION_TYPE_FLOWBIT,
     OPTION_TYPE_FLOWFLAGS,
     OPTION_TYPE_ASN1,
     OPTION_TYPE_CURSOR,
     OPTION_TYPE_HDR_CHECK,
     OPTION_TYPE_BYTE_TEST,
     OPTION_TYPE_BYTE_JUMP,
     OPTION_TYPE_BYTE_EXTRACT,
     OPTION_TYPE_SET_CURSOR,
     OPTION_TYPE_LOOP,
     OPTION_TYPE_MAX
```

```
};

typedef struct _RuleOption
{
    int optionType;
    union
    {
        void *ptr;
        ContentInfo *content;
        CursorInfo *cursor;
        PCREInfo *pcre;
        FlowBitsInfo *flowBit;
        ByteData *byte;
        ByteExtract *byteExtract;
        FlowFlags *flowFlags;
        Asn1Context *asn1;
        HdrOptCheck *hdrData;
        LoopInfo    *loop;
        PreprocessorOption *preprocOpt;
    } option_u;
} RuleOption;

#define NOT_FLAG                0x10000000
```

Some options also contain information that is initialized at run time, such as the compiled PCRE information, Boyer-Moore content information, the integer ID for a flowbit, etc.

The option types and related structures are listed below.

- OptionType: Content & Structure: *ContentInfo*

  The *ContentInfo* structure defines an option for a content search. It includes the pattern, depth and offset, and flags (one of which must specify the buffer - raw, URI or normalized - to search). Additional flags include nocase, relative, unicode, and a designation that this content is to be used for snorts fast pattern evaluation. The most unique content, that which distinguishes this rule as a possible match to a packet, should be marked for fast pattern evaluation. In the dynamic detection engine provided with Snort, if no *ContentInfo* structure in a given rules uses that flag, the one with the longest content length will be used.

  ```
  typedef struct _ContentInfo
  {
      u_int8_t *pattern;
      u_int32_t depth;
      int32_t   offset;
      u_int32_t flags;        /* must include a CONTENT_BUF_X */
      void     *boyer_ptr;
      u_int8_t *patternByteForm;
      u_int32_t patternByteFormLength;
      u_int32_t incrementLength;
  } ContentInfo;

  #define CONTENT_NOCASE          0x01
  #define CONTENT_RELATIVE        0x02
  ```

```
#define CONTENT_UNICODE2BYTE    0x04
#define CONTENT_UNICODE4BYTE    0x08
#define CONTENT_FAST_PATTERN    0x10
#define CONTENT_END_BUFFER      0x20

#define CONTENT_BUF_NORMALIZED  0x100
#define CONTENT_BUF_RAW         0x200
#define CONTENT_BUF_URI         0x400
```

- OptionType: PCRE & Structure: *PCREInfo*

  The *PCREInfo* structure defines an option for a PCRE search. It includes the PCRE expression, pcre_flags such as caseless, as defined in PCRE.h, and flags to specify the buffer.

```
/*
pcre.h provides flags:

PCRE_CASELESS
PCRE_MULTILINE
PCRE_DOTALL
PCRE_EXTENDED
PCRE_ANCHORED
PCRE_DOLLAR_ENDONLY
PCRE_UNGREEDY
*/

typedef struct _PCREInfo
{
    char     *expr;
    void     *compiled_expr;
    void     *compiled_extra;
    u_int32_t compile_flags;
    u_int32_t flags; /* must include a CONTENT_BUF_X */
} PCREInfo;
```

- OptionType: Flowbit & Structure: *FlowBitsInfo*

  The *FlowBitsInfo* structure defines a flowbits option. It includes the name of the flowbit and the operation (set, setx, unset, toggle, isset, isnotset).

```
#define FLOWBIT_SET       0x01
#define FLOWBIT_UNSET     0x02
#define FLOWBIT_TOGGLE    0x04
#define FLOWBIT_ISSET     0x08
#define FLOWBIT_ISNOTSET  0x10
#define FLOWBIT_RESET     0x20
#define FLOWBIT_NOALERT   0x40
#define FLOWBIT_SETX      0x80

typedef struct _FlowBitsInfo
{
    char     *flowBitsName;
    uint8_t   operation;
```

```
    uint16_t  id;
    uint32_t  flags;
    char     *groupName;
    uint8_t   eval;
    uint16_t *ids;
    uint8_t   num_ids;

} FlowBitsInfo;
```

- OptionType: Flow Flags & Structure: *FlowFlags*

  The *FlowFlags* structure defines a flow option. It includes the flags, which specify the direction (from_server, to_server), established session, etc.

```
#define FLOW_ESTABLISHED 0x10
#define FLOW_IGNORE_REASSEMBLED 0x1000
#define FLOW_ONLY_REASSMBLED    0x2000
#define FLOW_FR_SERVER   0x40
#define FLOW_TO_CLIENT   0x40 /* Just for redundancy */
#define FLOW_TO_SERVER   0x80
#define FLOW_FR_CLIENT   0x80 /* Just for redundancy */

typedef struct _FlowFlags
{
    u_int32_t   flags;
} FlowFlags;
```

- OptionType: ASN.1 & Structure: *Asn1Context*

  The *Asn1Context* structure defines the information for an ASN1 option. It mirrors the ASN1 rule option and also includes a flags field.

```
#define ASN1_ABS_OFFSET 1
#define ASN1_REL_OFFSET 2

typedef struct _Asn1Context
{
    int bs_overflow;
    int double_overflow;
    int print;
    int length;
    unsigned int max_length;
    int offset;
    int offset_type;
    u_int32_t  flags;
} Asn1Context;
```

- OptionType: Cursor Check & Structure: *CursorInfo*

  The *CursorInfo* structure defines an option for a cursor evaluation. The cursor is the current position within the evaluation buffer, as related to content and PCRE searches, as well as byte

tests and byte jumps. It includes an offset and flags that specify the buffer. This can be used to verify there is sufficient data to continue evaluation, similar to the isdataat rule option.

```
typedef struct _CursorInfo
{
    int32_t   offset;
    u_int32_t flags;        /* specify one of CONTENT_BUF_X */
} CursorInfo;
```

- OptionType: Protocol Header & Structure: *HdrOptCheck*

The *HdrOptCheck* structure defines an option to check a protocol header for a specific value. It includes the header field, the operation ($<$,$>$,=,etc), a value, a mask to ignore that part of the header field, and flags.

```
#define IP_HDR_ID           0x0001  /* IP Header ID */
#define IP_HDR_PROTO        0x0002  /* IP Protocol */
#define IP_HDR_FRAGBITS     0x0003  /* Frag Flags set in IP Header */
#define IP_HDR_FRAGOFFSET   0x0004  /* Frag Offset set in IP Header */
#define IP_HDR_OPTIONS      0x0005  /* IP Options -- is option xx included */
#define IP_HDR_TTL          0x0006  /* IP Time to live */
#define IP_HDR_TOS          0x0007  /* IP Type of Service */
#define IP_HDR_OPTCHECK_MASK 0x000f

#define TCP_HDR_ACK         0x0010  /* TCP Ack Value */
#define TCP_HDR_SEQ         0x0020  /* TCP Seq Value */
#define TCP_HDR_FLAGS       0x0030  /* Flags set in TCP Header */
#define TCP_HDR_OPTIONS     0x0040  /* TCP Options -- is option xx included */
#define TCP_HDR_WIN         0x0050  /* TCP Window */
#define TCP_HDR_OPTCHECK_MASK 0x00f0

#define ICMP_HDR_CODE       0x1000  /* ICMP Header Code */
#define ICMP_HDR_TYPE       0x2000  /* ICMP Header Type */
#define ICMP_HDR_ID         0x3000  /* ICMP ID for ICMP_ECHO/ICMP_ECHO_REPLY */
#define ICMP_HDR_SEQ        0x4000  /* ICMP ID for ICMP_ECHO/ICMP_ECHO_REPLY */
#define ICMP_HDR_OPTCHECK_MASK 0xf000

typedef struct _HdrOptCheck
{
    u_int16_t hdrField;   /* Field to check */
    u_int32_t op;         /* Type of comparison */
    u_int32_t value;      /* Value to compare value against */
    u_int32_t mask_value; /* bits of value to ignore */
    u_int32_t flags;
} HdrOptCheck;
```

- OptionType: Byte Test & Structure: *ByteData*

The *ByteData* structure defines the information for both ByteTest and ByteJump operations. It includes the number of bytes, an operation (for ByteTest, $<$,$>$,=,etc), a value, an offset, multiplier, and flags. The flags must specify the buffer.

```
#define CHECK_EQ            0
#define CHECK_NEQ           1
#define CHECK_LT            2
#define CHECK_GT            3
#define CHECK_LTE           4
#define CHECK_GTE           5
#define CHECK_AND           6
#define CHECK_XOR           7
#define CHECK_ALL           8
#define CHECK_ATLEASTONE    9
#define CHECK_NONE          10

typedef struct _ByteData
{
    u_int32_t bytes;      /* Number of bytes to extract */
    u_int32_t op;         /* Type of byte comparison, for checkValue */
    u_int32_t value;      /* Value to compare value against, for checkValue, or
extracted value */
    int32_t   offset;     /* Offset from cursor */
    u_int32_t multiplier; /* Used for byte jump -- 32bits is MORE than enough */
    u_int32_t flags;      /* must include a CONTENT_BUF_X */
} ByteData;
```

- OptionType: Byte Jump & Structure: *ByteData*

  See *Byte Test* above.

- OptionType: Set Cursor & Structure: *CursorInfo*

  See *Cursor Check* above.

- OptionType: Loop & Structures: *LoopInfo,ByteExtract,DynamicElement*

  The *LoopInfo* structure defines the information for a set of options that are to be evaluated repeatedly. The loop option acts like a FOR loop and includes start, end, and increment values as well as the comparison operation for termination. It includes a cursor adjust that happens through each iteration of the loop, a reference to a RuleInfo structure that defines the RuleOptions are to be evaluated through each iteration. One of those options may be a ByteExtract.

```
typedef struct _LoopInfo
{
    DynamicElement *start;      /* Starting value of FOR loop (i=start) */
    DynamicElement *end;        /* Ending value of FOR loop (i OP end) */
    DynamicElement *increment;  /* Increment value of FOR loop (i+= increment) */
    u_int32_t op;               /* Type of comparison for loop termination */
    CursorInfo *cursorAdjust;   /* How to move cursor each iteration of loop */
    struct _Rule *subRule;      /* Pointer to SubRule & options to evaluate within
                                 * the loop */
    u_int8_t initialized;       /* Loop initialized properly (safeguard) */
    u_int32_t flags;            /* can be used to negate loop results,
specifies                                 * relative. */
} LoopInfo;
```

The *ByteExtract* structure defines the information to use when extracting bytes for a DynamicElement used a in Loop evaluation. It includes the number of bytes, an offset, multiplier, flags specifying the buffer, and a reference to the DynamicElement.

```
typedef struct _ByteExtract
{
    u_int32_t bytes;      /* Number of bytes to extract */
    int32_t   offset;     /* Offset from cursor */
    u_int32_t multiplier; /* Multiply value by this (similar to byte jump) */
    u_int32_t flags;      /* must include a CONTENT_BUF_X */
    char *refId;          /* To match up with a DynamicElement refId */
    void *memoryLocation; /* Location to store the data extracted */
} ByteExtract;
```

The *DynamicElement* structure is used to define the values for a looping evaluation. It includes whether the element is static (an integer) or dynamic (extracted from a buffer in the packet) and the value. For a dynamic element, the value is filled by a related ByteExtract option that is part of the loop.

```
#define DYNAMIC_TYPE_INT_STATIC 1
#define DYNAMIC_TYPE_INT_REF    2

typedef struct _DynamicElement
{
    char dynamicType;           /* type of this field - static or reference */
    char *refId;                /* reference ID (NULL if static) */
    union
    {
        void *voidPtr;          /* Holder */
        int32_t staticInt;        /* Value of static */
        int32_t *dynamicInt;  /* Pointer to value of dynamic */
    } data;
} DynamicElement;
```

End of Navigation Panel Table of Child-Links **Subsections**

- 4.2.1 Preprocessors
- 4.2.2 Detection Engine
- 4.2.3 Rules

End of Table of Child-Links

4.2 Required Functions

Each dynamic module must define a set of functions and data objects to work within this framework.

4.2.1 Preprocessors

Each dynamic preprocessor must define the following items. These must be defined in the global scope of a source file (e.g. spp_example.c).

- *const int MAJOR_VERSION*

  This specifies the major version of the preprocessor.

- *const int MINOR_VERSION*

  This specifies the minor version of the preprocessor.

- *const int BUILD_VERSION*

  This specifies the build version of the preprocessor.

- *const char *PREPROC_NAME*

  This specifies the display name of the preprocessor.

- *void DYNAMIC_PREPROC_SETUP(void)*

  This function is called to register the preprocessor to be called with packets data.

The preprocessor must be built with the same macros defined as the Snort binary and linked with the dynamic preprocessor library that was created during the Snort build. A package configuration file is exported as part of the Snort build and can be accessed using the following commands with PKG_CONFIG_PATH=$<$snort build prefix/lib/pkgconfig$>$:

- *pkg-config -cflags snort_preproc*

  Returns the macros and include path needed to compile the dynamic preprocessor.

- *pkg-config -libs snort_preproc*

  Returns the library and library path needed to link the dynamic preprocessor.

4.2.2 Detection Engine

Each dynamic detection engine library must define the following functions.

- *int LibVersion(DynamicPluginMeta *)*

  This function returns the metadata for the shared library.

- *int InitializeEngineLib(DynamicEngineData *)*

  This function initializes the data structure for use by the engine.

The sample code provided with Snort predefines those functions and defines the following APIs to be used by a dynamic rules library.

- *int RegisterRules(Rule **)*

  This is the function to iterate through each rule in the list, initialize it to setup content searches, PCRE evaluation data, and register flowbits.

- *int DumpRules(char *,Rule **)*

  This is the function to iterate through each rule in the list and write a rule-stop to be used by snort to control the action of the rule (alert, log, drop, etc).

- *int ruleMatch(void *p, Rule *rule)*

  This is the function to evaluate a rule if the rule does not have its own Rule Evaluation Function. This uses the individual functions outlined below for each of the rule options and handles repetitive content issues.

  Each of the functions below returns RULE_MATCH if the option matches based on the current criteria (cursor position, etc).

  - *int contentMatch(void *p, ContentInfo* content, u_int8_t **cursor)*

    This function evaluates a single content for a given packet, checking for the existence of that content as delimited by ContentInfo and cursor. Cursor position is updated and returned in *cursor.

    With a text rule, the with option corresponds to depth, and the distance option corresponds to offset.

  - *int checkFlow(void *p, FlowFlags *flowflags)*

    This function evaluates the flow for a given packet.

  - *int extractValue(void *p, ByteExtract *byteExtract, u_int8_t *cursor)*

    This function extracts the bytes from a given packet, as specified by ByteExtract and delimited by cursor. Value extracted is stored in ByteExtract memoryLocation parameter.

  - *int processFlowbits(void *p, FlowBitsInfo *flowbits)*

    This function evaluates the flowbits for a given packet, as specified by FlowBitsInfo. It will interact with flowbits used by text-based rules.

  - *int setCursor(void *p, CursorInfo *cursorInfo, u_int8_t **cursor)*

This function adjusts the cursor as delimited by CursorInfo. New cursor position is returned in *cursor. It handles bounds checking for the specified buffer and returns RULE_NOMATCH if the cursor is moved out of bounds.

It is also used by contentMatch, byteJump, and pcreMatch to adjust the cursor position after a successful match.

- *int checkCursor(void *p, CursorInfo *cursorInfo, u_int8_t *cursor)*

  This function validates that the cursor is within bounds of the specified buffer.

- *int checkValue(void *p, ByteData *byteData, u_int32_t value, u_int8_t *cursor)*

  This function compares the *value* to the value stored in ByteData.

- *int byteTest(void *p, ByteData *byteData, u_int8_t *cursor)*

  This is a wrapper for extractValue() followed by checkValue().

- *int byteJump(void *p, ByteData *byteData, u_int8_t **cursor)*

  This is a wrapper for extractValue() followed by setCursor().

- *int pcreMatch(void *p, PCREInfo *pcre, u_int8_t **cursor)*

  This function evaluates a single pcre for a given packet, checking for the existence of the expression as delimited by PCREInfo and cursor. Cursor position is updated and returned in *cursor.

- *int detectAsn1(void *p, Asn1Context *asn1, u_int8_t *cursor)*

  This function evaluates an ASN.1 check for a given packet, as delimited by Asn1Context and cursor.

- *int checkHdrOpt(void *p, HdrOptCheck *optData)*

  This function evaluates the given packet's protocol headers, as specified by HdrOptCheck.

- *int loopEval(void *p, LoopInfo *loop, u_int8_t **cursor)*

  This function iterates through the SubRule of LoopInfo, as delimited by LoopInfo and cursor. Cursor position is updated and returned in *cursor.

- *int preprocOptionEval(void *p, PreprocessorOption *preprocOpt, u_int8_t **cursor)*

  This function evaluates the preprocessor defined option, as specified by PreprocessorOption. Cursor position is updated and returned in *cursor.

- *void setTempCursor(u_int8_t **temp_cursor, u_int8_t **cursor)*

  This function is used to handled repetitive contents to save off a cursor position temporarily to be reset at later point.

- *void revertTempCursor(u_int8_t **temp_cursor, u_int8_t **cursor)*

This function is used to revert to a previously saved temporary cursor position.

> **Note:**
>
> If you decide to write your own rule evaluation function, patterns that occur more than once may result in false negatives. Take extra care to handle this situation and search for the matched pattern again if subsequent rule options fail to match. This should be done for both content and PCRE options.

4.2.3 Rules

Each dynamic rules library must define the following functions. Examples are defined in the file `sfnort_dynamic_detection_lib.c`. The metadata and setup function for the preprocessor should be defined in `sfsnort_dynamic_detection_lib.h`.

- *int LibVersion(DynamicPluginMeta *)*

  This function returns the metadata for the shared library.

- *int EngineVersion(DynamicPluginMeta *)*

  This function defines the version requirements for the corresponding detection engine library.

- *int DumpSkeletonRules()*

  This functions writes out the rule-stubs for rules that are loaded.

- *int InitializeDetection()*

  This function registers each rule in the rules library. It should set up fast pattern-matcher content, register flowbits, etc.

The sample code provided with Snort predefines those functions and uses the following data within the dynamic rules library.

- *Rule *rules[]*

  A NULL terminated list of Rule structures that this library defines.

End of Navigation Panel Table of Child-Links **Subsections**

- 4.3.1 Preprocessor Example
- 4.3.2 Rules

End of Table of Child-Links

4.3 Examples

This section provides a simple example of a dynamic preprocessor and a dynamic rule.

4.3.1 Preprocessor Example

The following is an example of a simple preprocessor. This preprocessor always alerts on a packet if the TCP port matches the one configured.

The following code is defined in *spp_example.c* and is compiled together with *libsf_dynamic_preproc.a*, using pkg-config, into lib_sfdynamic_preprocessor_example.so.

Define the required meta data variables.

```
#define GENERATOR_EXAMPLE 256
extern DynamicPreprocessorData _dpd;

const int MAJOR_VERSION = 1;
const int MINOR_VERSION = 0;
const int BUILD_VERSION = 0;
const char *PREPROC_NAME = "SF_Dynamic_Example_Preprocessor";

#define ExampleSetup DYNAMIC_PREPROC_SETUP
```

Define the Setup function to register the initialization function.

```
void ExampleInit(unsigned char *);
void ExampleProcess(void *, void *);

void ExampleSetup()
{
    _dpd.registerPreproc("dynamic_example", ExampleInit);

    DEBUG_WRAP(_dpd.debugMsg(DEBUG_PLUGIN, "Preprocessor: Example is setup\n"););
}
```

The initialization function to parse the keywords from `snort.conf`.

```
u_int16_t portToCheck;

void ExampleInit(unsigned char *args)
{
    char *arg;
    char *argEnd;
    unsigned long port;

    _dpd.logMsg("Example dynamic preprocessor configuration\n");

    arg = strtok(args, " \t\n\r");
```

```
    if(!strcasecmp("port", arg))
    {
        arg = strtok(NULL, "\t\n\r");
        if (!arg)
        {
            _dpd.fatalMsg("ExamplePreproc: Missing port\n");
        }

        port = strtoul(arg, &argEnd, 10);
        if (port < 0 || port > 65535)
        {
            _dpd.fatalMsg("ExamplePreproc: Invalid port %d\n", port);
        }
        portToCheck = port;

        _dpd.logMsg("    Port: %d\n", portToCheck);
    }
    else
    {
        _dpd.fatalMsg("ExamplePreproc: Invalid option %s\n", arg);
    }

    /* Register the preprocessor function, Transport layer, ID 10000 */
    _dpd.addPreproc(ExampleProcess, PRIORITY_TRANSPORT, 10000);

    DEBUG_WRAP(_dpd.debugMsg(DEBUG_PLUGIN, "Preprocessor: Example is initialized\n"););
}
```

The function to process the packet and log an alert if the either port matches.

```
#define SRC_PORT_MATCH  1
#define SRC_PORT_MATCH_STR "example_preprocessor: src port match"
#define DST_PORT_MATCH  2
#define DST_PORT_MATCH_STR "example_preprocessor: dest port match"
void ExampleProcess(void *pkt, void *context)
{
    SFSnortPacket *p = (SFSnortPacket *)pkt;
    if (!p->ip4_header || p->ip4_header->proto != IPPROTO_TCP || !p->tcp_header)
    {
        /* Not for me, return */
        return;
    }

    if (p->src_port == portToCheck)
    {
        /* Source port matched, log alert */
        _dpd.alertAdd(GENERATOR_EXAMPLE, SRC_PORT_MATCH,
                      1, 0, 3, SRC_PORT_MATCH_STR, 0);
        return;
    }

    if (p->dst_port == portToCheck)
    {
```

```
        /* Destination port matched, log alert */
        _dpd.alertAdd(GENERATOR_EXAMPLE, DST_PORT_MATCH,
                      1, 0, 3, DST_PORT_MATCH_STR, 0);
        return;
    }
}
```

4.3.2 Rules

The following is an example of a simple rule, take from the current rule set, SID 109. It is implemented to work with the detection engine provided with snort.

The snort rule in normal format:

```
alert tcp $HOME_NET 12345:12346 -> $EXTERNAL_NET any \
(msg:"BACKDOOR netbus active"; flow:from_server,established; \
content:"NetBus"; reference:arachnids,401; classtype:misc-activity; \
sid:109; rev:5;)
```

This is the metadata for this rule library, defined in *detection_lib_meta.h*.

```
/* Version for this rule library */
#define DETECTION_LIB_MAJOR_VERSION 1
#define DETECTION_LIB_MINOR_VERSION 0
#define DETECTION_LIB_BUILD_VERSION 1
#define DETECTION_LIB_NAME "Snort_Dynamic_Rule_Example"

/* Required version and name of the engine */
#define REQ_ENGINE_LIB_MAJOR_VERSION 1
#define REQ_ENGINE_LIB_MINOR_VERSION 0
#define REQ_ENGINE_LIB_NAME "SF_SNORT_DETECTION_ENGINE"
```

The definition of each data structure for this rule is in *sid109.c*.

Declaration of the data structures.

- Flow option

  Define the *FlowFlags* structure and its corresponding *RuleOption*. Per the text version, flow is from_server,established.

  ```
  static FlowFlags sid109flow =
  {
      FLOW_ESTABLISHED|FLOW_TO_CLIENT
  };

  static RuleOption sid109option1 =
  {
      OPTION_TYPE_FLOWFLAGS,
      {
          &sid109flow
  ```

```
    }
};
```

- Content Option

Define the *ContentInfo* structure and its corresponding *RuleOption*. Per the text version, content is "NetBus", no depth or offset, case sensitive, and non-relative. Search on the normalized buffer by default. **NOTE**: This content will be used for the fast pattern matcher since it is the longest content option for this rule and no contents have a flag of *CONTENT_FAST_PATTERN*.

```
static ContentInfo sid109content =
{
    "NetBus",               /* pattern to search for */
    0,                      /* depth */
    0,                      /* offset */
    CONTENT_BUF_NORMALIZED, /* flags */
    NULL,                   /* holder for boyer/moore info */
    NULL,                   /* holder for byte representation of "NetBus" */
    0,                      /* holder for length of byte representation */
    0                       /* holder for increment length */
};

static RuleOption sid109option2 =
{
    OPTION_TYPE_CONTENT,
    {
        &sid109content
    }
};
```

- Rule and Meta Data

Define the references.

```
static RuleReference sid109ref_arachnids =
{
    "arachnids",    /* Type */
    "401"           /* value */
};

static RuleReference *sid109refs[] =
{
    &sid109ref_arachnids,
    NULL
};
```

The list of rule options. Rule options are evaluated in the order specified.

```
RuleOption *sid109options[] =
{
    &sid109option1,
    &sid109option2,
```

```
    NULL
};
```

The rule itself, with the protocol header, meta data (sid, classification, message, etc).

```
Rule sid109 =
{
    /* protocol header, akin to => tcp any any -> any any               */
    {
        IPPROTO_TCP,        /* proto */
        HOME_NET,           /* source IP     */
        "12345:12346",      /* source port(s) */
        0,                  /* Direction */
        EXTERNAL_NET,       /* destination IP     */
        ANY_PORT,           /* destination port   */
    },
    /* metadata */
    {
        3,                  /* genid -- use 3 to distinguish a C rule */
        109,                /* sigid */
        5,                  /* revision */
        "misc-activity",    /* classification */
        0,                  /* priority */
        "BACKDOOR netbus active",     /* message */
        sid109refs          /* ptr to references */
    },
    sid109options,          /* ptr to rule options */
    NULL,                   /* Use internal eval func */
    0,                      /* Holder, not yet initialized, used internally */
    0,                      /* Holder, option count, used internally */
    0,                      /* Holder, no alert, used internally for flowbits */
    NULL                    /* Holder, rule data, used internally */
```

- The List of rules defined by this rules library

The NULL terminated list of rules. The InitializeDetection iterates through each Rule in the list and initializes the content, flowbits, pcre, etc.

```
extern Rule sid109;
extern Rule sid637;

Rule *rules[] =
{
    &sid109,
    &sid637,
    NULL
};
```

End of Navigation Panel

5. Snort Development

Currently, this chapter is here as a place holder. It will someday contain references on how to create new detection plugins and preprocessors. End users don't really need to be reading this section. This is intended to help developers get a basic understanding of whats going on quickly.

If you are going to be helping out with Snort development, please use the HEAD branch of cvs. We've had problems in the past of people submitting patches only to the stable branch (since they are likely writing this stuff for their own IDS purposes). Bug fixes are what goes into STABLE. Features go into HEAD.

Subsections

- 5.1 Submitting Patches
- 5.2 Snort Data Flow
  - 5.2.1 Preprocessors
  - 5.2.2 Detection Plugins
  - 5.2.3 Output Plugins
- 5.3 Unified2 File Format
  - 5.3.1 Serial Unified2 Header
  - 5.3.2 Unified2 Packet
  - 5.3.3 Unified2 IDS Event
  - 5.3.4 Unified2 IDS Event IP6
  - 5.3.5 Unified2 IDS Event (Version 2)
  - 5.3.6 Unified2 IDS Event IP6 (Version 2)
  - 5.3.7 Unified2 Extra Data
  - 5.3.8 Description of Fields
- 5.4 Buffer dump utility
  - 5.4.1 Example Buffer Dump output
- 5.5 The Snort Team

End of Navigation Panel

5.1 Submitting Patches

Patches to Snort should be sent to the `snort-devel@lists.snort.org` mailing list. Patches should done with the command `diff -nu snort-orig snort-new`.

End of Navigation Panel Table of Child-Links **Subsections**

- 5.2.1 Preprocessors
- 5.2.2 Detection Plugins
- 5.2.3 Output Plugins

End of Table of Child-Links

5.2 Snort Data Flow

First, traffic is acquired from the network link via libpcap. Packets are passed through a series of decoder routines that first fill out the packet structure for link level protocols then are further decoded for things like TCP and UDP ports.

Packets are then sent through the registered set of preprocessors. Each preprocessor checks to see if this packet is something it should look at.

Packets are then sent through the detection engine. The detection engine checks each packet against the various options listed in the Snort config files. Each of the keyword options is a plugin. This allows this to be easily extensible.

5.2.1 Preprocessors

For example, a TCP analysis preprocessor could simply return if the packet does not have a TCP header. It can do this by checking:

```
if (p->tcph==null)
   return;
```

Similarly, there are a lot of packet_flags available that can be used to mark a packet as “reassembled” or logged. Check out src/decode.h for the list of pkt_* constants.

5.2.2 Detection Plugins

Basically, look at an existing output plugin and copy it to a new item and change a few things. Later, we'll document what these few things are.

5.2.3 Output Plugins

Generally, new output plugins should go into the barnyard project rather than the Snort project. We are currently cleaning house on the available output options.

End of Navigation Panel Table of Child-Links **Subsections**

- 5.3.1 Serial Unified2 Header
- 5.3.2 Unified2 Packet
- 5.3.3 Unified2 IDS Event
- 5.3.4 Unified2 IDS Event IP6
- 5.3.5 Unified2 IDS Event (Version 2)
- 5.3.6 Unified2 IDS Event IP6 (Version 2)
- 5.3.7 Unified2 Extra Data
- 5.3.8 Description of Fields

End of Table of Child-Links

5.3 Unified2 File Format

Unified 2 records should not be assumed to be in any order. All values are stored in network byte order.

An example structure of unified2 files

```
[ Serial Unified2 Header    ]
[ Unified2 IDS Event        ]
[ Unified2 Packet           ]
[ Unified2 Extra Data       ]
.
.
.
[ Serial Unified2 Header    ]
[ Unified2 IDS Event        ]
[ Unified2 Packet           ]
[ Unified2 Extra Data       ]
```

5.3.1 Serial Unified2 Header

```
record type             4 bytes
record length           4 bytes
```

All unified2 records are preceded by a Serial Unified2 header. This unified2 record allows an interpreting application to skip past and apply simple heuristics against records.

The Record Type indicates one of the following unified2 records follows the Serial Unified2 Header:

```
Value           Record Type
----------      -----------
2               Unified2 Packet
7               Unified2 IDS Event
72              Unified2 IDS Event IP6
104             Unified2 IDS Event      (Version 2)
105             Unified2 IDS Event IP6  (Version 2)
110             Unified2 Extra Data
```

The record length field specifies the entire length of the record (not including the Serial Unified2 Header itself) up to the next Serial Unified2 Header or EOF.

5.3.2 Unified2 Packet

```
sensor id               4 bytes
event id                4 bytes
event seconds           4 bytes
packet seconds          4 bytes
packet microseconds     4 bytes
```

```
    linktype                4 bytes
    packet length           4 bytes
    packet data             <variable length>
```

A Unified2 Packet is provided with each Unified2 Event record. This packet is the `alerting' packet that caused a given event.

Unified2 Packet records contain contain a copy of the packet that caused an alert (Packet Data) and is packet length octets long.

5.3.3 Unified2 IDS Event

```
    sensor id               4 bytes
    event id                4 bytes
    event second            4 bytes
    event microsecond       4 bytes
    signature id            4 bytes
    generator id            4 bytes
    signature revision      4 bytes
    classification id       4 bytes
    priority id             4 bytes
    ip source               4 bytes
    ip destination          4 bytes
    source port/icmp type   2 bytes
    dest. port/icmp code    2 bytes
    protocol                1 byte
    impact flag             1 byte
    impact                  1 byte
    blocked                 1 byte
```

Unified2 IDS Event is logged for IPv4 Events without VLAN or MPLS tagging.

5.3.4 Unified2 IDS Event IP6

```
    sensor id               4 bytes
    event id                4 bytes
    event second            4 bytes
    event microsecond       4 bytes
    signature id            4 bytes
    generator id            4 bytes
    signature revision      4 bytes
    classification id       4 bytes
    priority id             4 bytes
    ip source               16 bytes
    ip destination          16 bytes
    source port/icmp type   2 bytes
    dest. port/icmp code    2 bytes
    protocol                1 byte
    impact flag             1 byte
```

```
    impact                  1 byte
    blocked                 1 byte
```

Unified2 IDS Event IP6 is logged for IPv6 Events without VLAN or MPLS tagging.

5.3.5 Unified2 IDS Event (Version 2)

```
    sensor id               4 bytes
    event id                4 bytes
    event second            4 bytes
    event microsecond       4 bytes
    signature id            4 bytes
    generator id            4 bytes
    signature revision      4 bytes
    classification id       4 bytes
    priority id             4 bytes
    ip source               4 bytes
    ip destination          4 bytes
    source port/icmp type   2 bytes
    dest. port/icmp code    2 bytes
    protocol                1 byte
    impact flag             1 byte
    impact                  1 byte
    blocked                 1 byte
    mpls label              4 bytes
    vlan id                 2 bytes
    padding                 2 bytes
```

Unified2 IDS Event (Version 2) are logged for IPv4 packets which contain either MPLS or VLAN headers. Otherwise a Unified2 IDS Event is logged.

Note:

- Note that you'll need to pass -enable-mpls to configure in order to have Snort fill in the mpls label field.
- Note that you'll need to configure unified2 logging with either mpls_event_types or vlan_event_types to get this record type.

5.3.6 Unified2 IDS Event IP6 (Version 2)

```
    sensor id               4 bytes
    event id                4 bytes
    event second            4 bytes
    event microsecond       4 bytes
    signature id            4 bytes
    generator id            4 bytes
    signature revision      4 bytes
    classification id       4 bytes
    priority id             4 bytes
```

```
    ip source                16 bytes
    ip destination           16 bytes
    source port/icmp type    2 bytes
    dest. port/icmp code     2 bytes
    protocol                 1 byte
    impact flag              1 byte
    impact                   1 byte
    blocked                  1 byte
    mpls label               4 bytes
    vlan id                  2 bytes
    padding                  2 bytes
```

Unified2 IDS Event IP6 (Version 2) are logged for IPv6 packets which contain either MPLS or VLAN headers. Otherwise a Unified2 IDS Event IP6 is logged.

Note:

- Note that you'll need to pass -enable-mpls to configure in order to have Snort fill in the mpls label field.
- Note that you'll need to configure unified2 logging with either mpls_event_types or vlan_event_types to get this record type.

5.3.7 Unified2 Extra Data

```
    sensor id                4 bytes
    event id                 4 bytes
    event second             4 bytes
    type                     4 bytes
    data type                4 bytes
    data length              4 bytes
    data                     <variable length>
```

5.3.8 Description of Fields

- Sensor ID

  Unused

- Event ID

  The upper 2 bytes represent the snort instance, if specified by passing the -G option to Snort.

  The lower 2 bytes indicate the unique id of the event.

  The Event ID field is used to facilitate the task of coalescing events with packet data.

- Event Seconds and Event Microseconds

  Timestamp represented as seconds since the epoch of when the alert was generated.

- Link Type (Unified2 Packet)

  The Datalink type of the packet, typically EN10M but could be any of the values as returned by pcap_datalink that Snort handles.

- Packet Length (Unified2 Packet)

  Length of the Packet Data.

- Packet Data (Unified2 Packet)

  The alerting packet, of Packet Length bytes long.

- Type (Unified2 Extra Data)

  Type specifies the type of extra data that was logged, the valid types are:

```
Value           Description
----------      -----------
1               Original Client IPv4
2               Original Client IPv6
3               UNUSED
4               GZIP Decompressed Data
5               SMTP Filename
6               SMTP Mail From
7               SMTP RCPT To
8               SMTP Email Headers
9               HTTP URI
10              HTTP Hostname
11              IPv6 Source Address
12              IPv6 Destination Address
13              Normalized Javascript Data
```

- Data Type (Unified2 Extra Data)

  The type of extra data in the record.

```
Value           Description
----------      -----------
1               Blob
```

- Data Length (Unified2 Extra Data)

  Length of the data stored in the extra data record

- Data (Unified2 Extra Data)

  Raw extra event data up to Data Length bytes in size.

  All of these Extra data types, with the exception of 1, 2, 11, and 12 (IP Addresses) are stored in plain-text. The IP Address types need to be interpreted as if they were coming off the wire.

- Signature ID

The Signature ID of the alerting rule, as specified by the sid keyword.

- Generator ID

  The Generator ID of the alerting rule, as specified by the gid keyword.

- Signature Revision

  Revision of the rule as specified by the rev keyword.

- Classification ID

  Classification ID as mapped in the file classifications.conf

- Priority ID

  Priority of the rule as mapped in the file classifications.conf or overridden by the priority keyword for text rules.

- IP Source

  Source IP of the packet that generated the event.

- IP Destination

  Destination IP of the packet that generated the event.

- Source Port/ICMP Type

  If Protocol is TCP or UDP than this field contains the source port of the alerting packet.

  If Protocol is ICMP than this field contains the ICMP type of the alerting packet.

- Destination Port/ICMP Code

  If protocol is TCP or UDP than this field contains the source port of the alerting packet.

  If protocol is icmp than this field contains the icmp code of the alerting packet.

- Protocol

  Transport protocol of the alerting packet. One of: ip, tcp, udp, or icmp.

- Impact flag

  Legacy field, specifies whether a packet was dropped or not.

```
Value           Description
----------      -----------
32              Blocked
```

- Impact

  UNUSED; deprecated.

- Blocked

  Whether the packet was not dropped, was dropped or would have been dropped.

```
Value           Description
----------      -----------
0               Was NOT Dropped
1               Was Dropped
2               Would Have Dropped*
```

> **Note:** Note that you'll obtain Would Have Dropped on rules which are set to drop while Snort is running in inline-test mode. Would Have Dropped is also obtained when a drop rule fires while pruning sessions or during shutdown.

- MPLS Label (4 bytes)

  The extracted mpls label from the mpls header in the alerting packet.

- VLAN ID

  The extracted vlan id from the vlan header in the alerting packet.

- Padding

  Padding is used to keep the event structures aligned on a 4 byte boundary.

End of Navigation Panel Table of Child-Links **Subsections**

- 5.4.1 Example Buffer Dump output

End of Table of Child-Links

5.4 Buffer dump utility

Buffer dump option will dump the buffers used by snort during different stages of packet processing path.

```
    ./configure --enable-buffer-dump / -DDUMP_BUFFER
```

Two options are provided to dump buffers. '-buffer-dump-alert' will dump buffers only when there is an alert.

'-buffer-dump' will dump buffers for every packet.

```
   ./snort -A cmg -k none -Q --daq-dir=<dir> --daq dump -r <pcap> -c snort.conf --
buffer-dump-alert=<file>
     or
   ./snort -A cmg -k none -Q --daq-dir=<dir> --daq dump -r <pcap> -c snort.conf --
buffer-dump=<file>

Note: If <file> parameter is not used, buffers are dumped on the console
```

5.4.1 Example Buffer Dump output

```
METHOD_DUMP, 3

00000000  47 45 54                                          |GET             |

URI_DUMP, 340

00000000  2F 70 68 70 42 42 33 2F  76 69 65 77 74 6F 70 69  |/phpBB3/viewtopi|
00000010  63 2E 70 68 70 3F 70 3D  39 30 30 32 26 73 69 64  |c.php?p=9002&sid|
00000020  3D 66 35 33 39 39 61 32  64 32 34 33 63 65 61 64  |=f5399a2d243cead|
00000030  33 61 35 65 61 37 61 64  66 31 35 62 66 63 38 37  |3a5ea7adf15bfc87|
00000040  32 26 68 69 67 68 6C 69  67 68 74 3D 27 2E 66 77  |2&highlight='.fw|
00000050  72 69 74 65 28 66 6F 70  65 6E 28 63 68 72 28 31  |rite(fopen(chr(1|
00000060  30 39 29 2E 63 68 72 28  34 39 29 2E 63 68 72 28  |09).chr(49).chr(|
00000070  31 30 34 29 2E 63 68 72  28 31 31 31 29 2E 63 68  |104).chr(111).ch|
00000080  72 28 35 30 29 2E 63 68  72 28 31 31 31 29 2E 63  |r(50).chr(111).c|
00000090  68 72 28 31 30 32 29 2C  63 68 72 28 39 37 29 29  |hr(102),chr(97))|
000000a0  2C 63 68 72 28 33 35 29  2E 63 68 72 28 33 33 29  |,chr(35).chr(33)|
000000b0  2E 63 68 72 28 34 37 29  2E 63 68 72 28 31 31 37  |.chr(47).chr(117|
000000c0  29 2E 63 68 72 28 31 31  35 29 2E 63 68 72 28 31  |).chr(115).chr(1|
000000d0  31 34 29 2E 63 68 72 28  34 37 29 2E 63 68 72 28  |14).chr(47).chr(|
000000e0  39 38 29 2E 63 68 72 28  31 30 35 29 2E 63 68 72  |98).chr(105).chr|
000000f0  28 31 31 30 29 2E 63 68  72 28 34 37 29 2E 63 68  |(110).chr(47).ch|
00000100  72 28 31 31 32 29 2E 63  68 72 28 31 30 31 29 2E  |r(112).chr(101).|
00000110  63 68 72 28 31 31 34 29  2E 63 68 72 28 31 30 38  |chr(114).chr(108|
00000120  29 2E 63 68 72 28 31 30  29 2E 63 68 72 28 31 31  |).chr(10).chr(11|
00000130  37 29 2E 63 68 72 28 31  31 35 29 2E 63 68 72 28  |7).chr(115).chr(|
```

```
00000140  31 30 31 29 2E 63 68 72  28 33 32 29 29 2C 65 78  |101).chr(32)),ex|
00000150  69 74 2E FF                                       |it..            |
```

End of Navigation Panel

5.5 The Snort Team

Creator and Lead Architect	Marty Roesch
Lead Snort Developers	Steve Sturges
	Bhagyashree Bantwal
	Ed Borgoyn
	Hui Cao
	Russ Combs
	Victor Roemer
	Charles Summers
	Josh Rosenbaum
	Carter Waxman
	Tom Peters
	A V K Nageswara Rao (ANR)
	Rahul Burman
	Seshaiah Erugu
	Krishnakanth K
Snort QA Team	Chris Spencer
	Jigeshwar Patel
	Albert Lewis
	Nihal Desai
Vulnerability Research Team	Matt Watchinski
	Aaron Benson
	Nathan Benson
	Andrew Blunck
	Christoph Cordes
	Joel Esler
	Douglas Goddard
	Ethan Gulla
	Nigel Houghton
	Pawel Janic

	Richard Johnson
	Tom Judge
	Alex Kambis
	Alex Kirk
	Chris Marshall
	Christopher McBee
	Alex McDonnell
	Kevin Miklavcic
	Steve Morgan
	Patrick Mullen
	Katie Nolan
	Matt Olney
	Carlos Pacho
	Ryan Pentney
	Nick Randolph
	Dave Raynor
	Marcos Rodriguez
	Ryan Steinmetz
	Brandon Stultz
	Andy Walker
	Shawn Webb
	Carl Wu
	Yves Younan
	Alain Zidouemba

Win32 Maintainer	Snort Team
Snort Community Manager	Joel Esler
Snort Web Team	Aaron Norling
	Mike Verbeck
Major Contributors	Erek Adams
	Michael Altizer

	Ayushi Agarwal
	Andrew Baker
	Scott Campbell
	Brian Caswell
	Dilbagh Chahal
	JJ Cummings
	Scott Czajkowski
	Roman D.
	Michael Davis
	Ron Dempster
	Matt Donnan
	Chris Green
	Lurene Grenier
	Mike Guiterman
	Jed Haile
	Justin Heath
	Patrick Harper
	Jeremy Hewlett
	Ryan Jordan
	Victor Julien
	Glenn Mansfield Keeni
	Adam Keeton
	Keith Konecnik
	Veronica Kovah
	Chad Kreimendahl
	Kevin Liu
	Rob McMillen
	William Metcalf
	Andrew Mullican
	Jeff Nathan
	Marc Norton
	Judy Novak
	Andreas Ostling
	William Parker
	Chris Reid

	Daniel Roelker
	Dragos Ruiu
	Chris Sherwin
	Matt Smith
	Jennifer Steffens
	Todd Wease
	JP Vossen
	Leon Ward
	Daniel Wittenberg
	Phil Wood
	Fyodor Yarochkin

End of Navigation Panel

Bibliography

1 http://packetstorm.securify.com/mag/phrack/phrack49/p49-06

2 http://www.nmap.org

3 http://public.pacbell.net/dedicated/cidr.html

4 http://www.whitehats.com

5 http://www.incident.org/snortdb

6 http://www.pcre.org